Appendix to: Forecasting Care Load and Placement Demand in the HHS Unaccompanied Children Program

This volume supports the research paper. It holds definitions, detailed tables, tuning logs, full model outputs, additional charts and all raw source code. Section, table and figure numbers without a letter refer to the main paper, and its reference list applies here too.

Contents

Appendix A. KPI Definitions and Length-of-Stay Reconciliation 2

A.1 Forecast KPIs 2

A.2 Descriptive pipeline KPIs 2

A.3 Alert rules 3

A.4 Implied length of stay: two definitions 3

Appendix B. Data-Quality Detail 3

B.1 Source fields and the population balance 3

B.2 Input validation 4

B.3 Reporting cadence 4

B.4 Stale reports 4

B.5 Copied CBP records 4

B.6 What the counts mean, and whether interpolating them is justified 5

B.7 Decisions 6

Appendix C. Additional Exploratory Analysis 6

C.1 Structural break and length of stay 7

C.2 Net pressure 7

C.3 Weekly structure and the weekend ratchet 8

C.4 Monthly pattern, outliers, surge periods and correlations 9

C.5 Decomposition and stationarity 9

Appendix D. Feature and Leakage Audit 11

D.1 Time-series preparation 11

D.2 Feature groups, targets and perturbation test 11

D.3 The month feature 11

D.4 Full feature audit 11

Appendix E. Model Specifications and Tuning Logs 13

E.1 Train–test design 14

E.2 Model specifications and fallbacks 14

E.3 SARIMA order-selection log 15

E.4 Evaluation metrics 15

E.5 Uncertainty quantification 15

E.6 Recursive against direct forecasting 15

E.7 Early-warning design 16

E.8 Scenario engine 16

Appendix F. Full Model Outputs 16
F.1 Accuracy of every model 16
F.2 Error by horizon 17
F.3 Statistical significance 18
F.4 Robustness 18
F.5 Interval coverage 19
F.6 Early warning 21
F.7 Production forecast and scenarios 23
F.8 Pipeline analytics 24
Appendix G. Software Validation and Reproducibility 26
G.1 Software validation 26
G.2 Where everything is 27
Appendix H. Raw Source Code 28
H.1 Analysis notebook source: notebook_source/UAC_Care_Load_Forecasting_v2.py 28
H.2 Streamlit dashboard: uac_forecast_dashboard.py 72
H.3 Data loading: src/data_loader.py 86
H.4 KPI calculations: src/kpi_calculations.py 89
H.5 Bottleneck detection: src/bottleneck_detection.py 92
H.6 Chart helpers: src/visualizations.py 95
H.7 Pipeline analysis script: tools/pipeline_analysis.py 97
H.8 Paper figure script: tools/paper_figures.py 100

Appendix A. KPI Definitions and Length-of-Stay Reconciliation

This appendix defines every indicator used in the paper and in the dashboard.

A.1 Forecast KPIs

Table A1. Forecast KPIs

KPI	Definition
Forecast Accuracy (%)	100 − WAPE, where WAPE = Σ|error| / Σ|actual| on the test window; undefined when all actuals are zero
Surge Lead Time	Days between the start of the unbroken alert streak and the threshold crossing
Capacity Breach Probability	P(care load > threshold), from the point forecast plus walk-forward errors for that horizon (last 26 weeks of observable errors); threshold supplied by the user
Forecast Stability Index	100 / (1 + mean revision for the same target date between consecutive origins ÷ mean actual daily change)
Model robustness	Win-rate vs naive across test weeks; error growth from week 1 to week 4; interval coverage

A.2 Descriptive pipeline KPIs

All computed on reported days only. Stocks: children in CBP custody, children in HHS care. Flows: apprehensions, transfers, discharges. These definitions are the ones shown in the dashboard tooltips (`src/kpi_calculations.py`, `KPI_DEFINITIONS`).

Table A2. Descriptive pipeline KPIs

KPI	Definition
HHS census at period end	Children in HHS care on the last reported day of the period (stock)
HHS census change	Last minus first reported HHS census in the period (stock change, not a sum)
Peak HHS census	Highest reported HHS census in the period, with its date
CBP custody at period end	Children in CBP custody on the last trusted report of the period (stock)
Apprehensions per reported day	Mean of daily CBP apprehensions over reported days
Transfers into HHS per reported day	Mean of daily transfers out of CBP into HHS over reported days
Discharges per reported day	Mean of daily HHS discharges (mostly to sponsors) over reported days
Recorded net intake per reported day	Mean of (transfers - discharges) on days with both reported. Recorded flows do not close the census balance (about +3 children/day unexplained), so use the census change for backlog
CBP transfer rate (%/day)	100 x sum(transfers) / sum(children in CBP custody): share of the CBP population moved to HHS per day
HHS discharge rate (%/day)	100 x sum(discharges) / sum(children in HHS care): share of the HHS population discharged per day
Recorded throughput (exits per entry)	sum(discharges) / sum(transfers): above 1 = recorded exits exceed entries. Not used for alerts: it misses unreported Fri/Sat flows and non-CBP admissions
Implied average stay in HHS care (days)	Little's law, ratio of period means: sum(children in HHS care) / sum(discharges) = 100 / discharge rate
Implied CBP holding time (days)	Little's law: sum(children in CBP custody) / sum(transfers). An approximation of the average time in CBP custody before transfer (an upper bound if children also leave CBP by other routes); compare with the 72-hour transfer standard
CBP pass-through (transfers per apprehension)	sum(transfers) / sum(apprehensions) on days with both reported

A.3 Alert rules

Table A3. Alert rules in full

Indicator	Value	Watch	Alert	Window and data requirement
HHS census growth	100 × (census / census at the last report ≥ 28 days earlier − 1)	≥ 5%	≥ 10%	Lagged report must be within 7 days of the 28-day mark
Discharge slowdown	100 × (1 − 7-day discharge rate / median 7-day rate of the previous 8 weeks)	≥ 15%	≥ 30%	≥ 3 reports in the 7-day window; ≥ 15 rates in the baseline
CBP holding time	Σ custody / Σ transfers over 7 days (∞ if no transfers while children are held)	≥ 2 days	≥ 3 days	≥ 3 reports with both values; trusted CBP data only

Overall status = worst indicator level (Normal, Watch, Alert); indicators without a complete window are ignored ("insufficient data" if none is available).

A.4 Implied length of stay: two definitions

Section 3.3 of the paper averages a 28-day smoothed daily ratio (calendar grid) over fixed windows. The dashboard and Section 8 of the paper use the ratio of period means over reported days, so the value equals 100 ÷ the discharge rate for any selected period. On the same windows the two definitions differ by 4–5%:

Table A4. Implied length of stay under the two definitions

Window	Definition in Section 3.3 of the paper	Ratio of reported-day means	Difference
22 Aug 2024 – 17 Feb 2025 ("before the break" in Section 3.3 of the paper)	35.9 days	37.2 days	+3.7%
17 Mar – 21 Dec 2025 (current regime)	203.2 days	192.3 days	−5.4%

Appendix B. Data-Quality Detail

This appendix gives the full evidence behind Section 2 of the paper.

B.1 Source fields and the population balance

The source file contains 1,170 rows below its header line, of which 720 are daily reports and 450 are blank padding rows. Each report has six fields:

Table B1. Source fields and their role in the HHS population balance

Field	Meaning	Type	Role in the HHS population balance
Date	Reporting date (text, newest first)	index	
Children apprehended and placed in CBP custody	CBP intake	daily count	upstream, not part of the HHS balance
Children in CBP custody	CBP care load	stock (snapshot)	upstream pipeline
Children transferred out of CBP custody	Flow into HHS	daily count	admissions from CBP (one movement, counted once)
Children in HHS care	HHS care load	stock (snapshot)	closing population
Children discharged from HHS care	Sponsor placements	daily count	exits

The distinction between **stocks** (levels) and **flows** (daily counts) matters throughout: a stock must reconcile with its flows, and that identity becomes the main data-quality test. The testable balance is *closing = opening + transfers in − discharges + (unrecorded other admissions − other exits)*. The file contains no columns for non-CBP admissions, other exits or bed capacity; these gaps are measured and reported (Appendix B.6) rather than invented, and every capacity threshold in this study is a user input or an illustration.

B.2 Input validation

Two validation layers run before any analysis. **Schema validation** stops a malformed file with a message naming the column, file line and value: six columns in the documented order, at least one data row and at least 365 reports, dates in the documented format (ISO dates accepted and reported; time-stamped or other text rejected), numbers with optional thousands separators (blank and `N/A` mean missing; other text is rejected), no empty or constant target columns, and a daily cadence (median gap ≤ 3 days). **Plausibility validation** quarantines impossible values (negative counts; a care load more than 1.5× or less than 1/1.5× the median of the three reports on either side) by setting them to missing and listing them (never replacing them with zero), and flags unusual but possible values (non-integer counts; flows more than 5× their local median) for review.

On the real file every check passes and nothing is quarantined or flagged; the largest care-load deviation from neighbouring reports is ×1.11 and the largest flow ratio 3.2×, far inside the limits. The negative cases are exercised on synthetic files in the automated test suite (Appendix G).

B.3 Reporting cadence

Reports arrive Sunday to Thursday. In three years there are only 2 Friday reports and none on a Saturday (Figure B1). Gaps of 3 days (Thursday→Sunday) are the second most common interval, and about a third of calendar days (355 of 1,075) have no report. Missing days are therefore **structural, not random**, and Friday/Saturday flows are never observed.

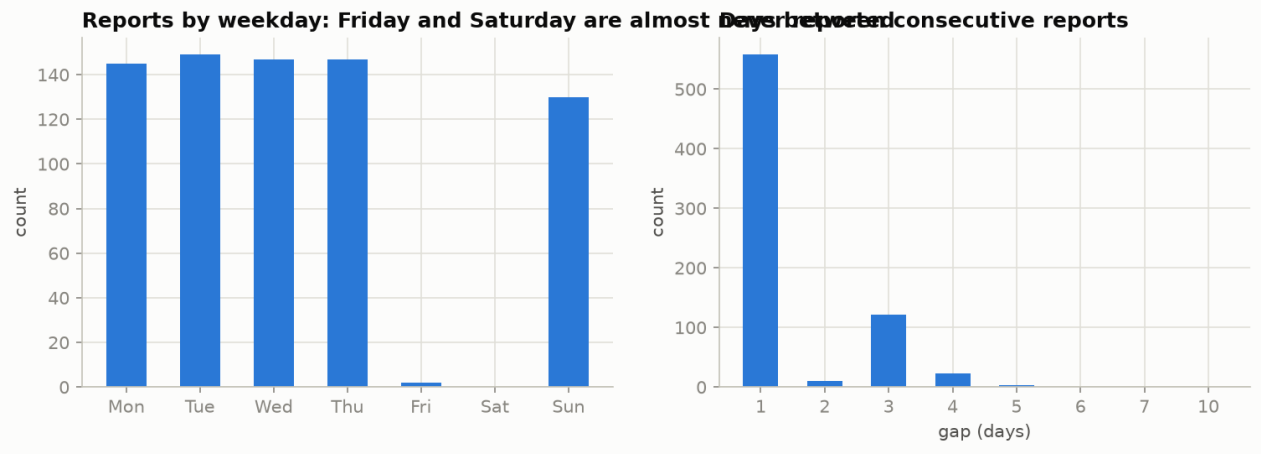


Figure B1. Reports by weekday and gaps between reports

B.4 Stale reports

Five reports repeat the previous report's care load *and* discharges exactly, while their CBP fields change: 8 Jan 2024, 26 Feb 2024, 25 Jun 2024, 27 Mar 2025 and 14 May 2025. These are carry-forwards. They were set to missing, re-estimated by interpolation, and never used as a scored target.

B.5 Copied CBP records

A CBP report is a triple (apprehended, in custody, transferred), and identical triples on different dates should be rare. In this file, 374 of 720 rows share their triple with another date. The earlier copies form one **contiguous block of 251 reports from 12 Jan 2023 to 30 Jan 2024**; 100% of 2023 CBP rows are copies. For example, the triple (56, 82, 68) appears on 1 Jun 2023, 15 Oct 2023, 27 Nov 2023, 26 Dec 2023 and 9 Jan 2025, each time alongside a completely different HHS care load. (The detector requires a run of at least 14 consecutive copies before declaring a corrupted block, so isolated chance repeats at low counts on a future refresh are not masked.)

Proof via the stock-flow identity. On consecutive days, the change in HHS care should equal transfers in minus discharges out. Figure B2 tests this:

Table B2. Stock-flow identity test

Period	Day pairs	corr(ΔHHS, net flow)	Mean residual per day
Copied CBP block (2023 – Jan 2024)	192	**0.51**	**+190 children**
Trusted (Feb 2024 →)	357	**0.90**	+2.6 children

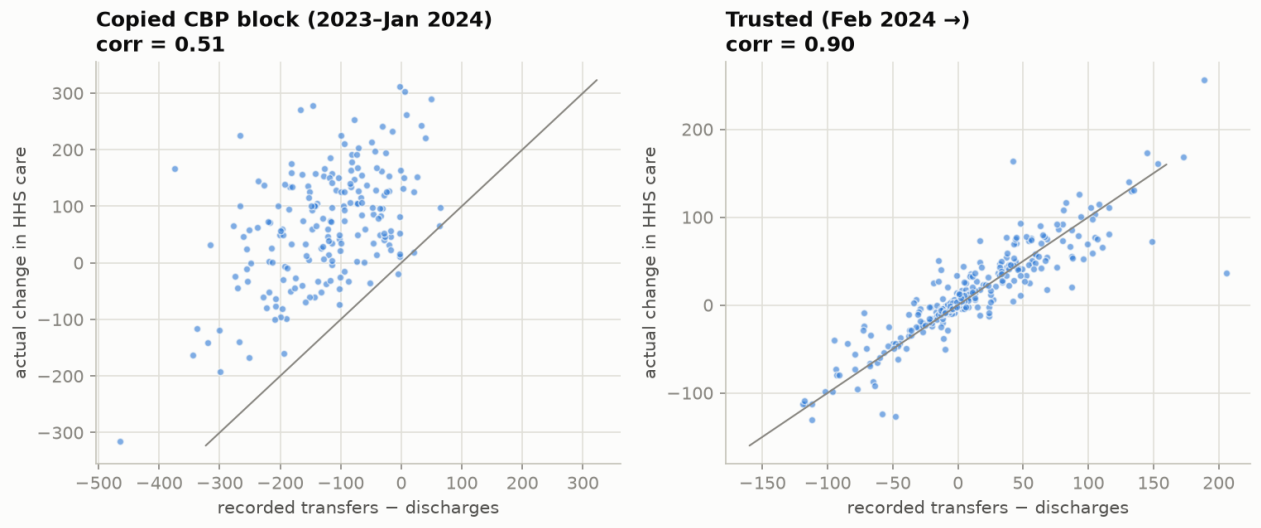


Figure B2. Stock-flow identity: copied block vs trusted period

Admissions implied by the HHS columns alone (ΔHHS + discharges) ran at 280–400 per day in 2023, while recorded transfers captured only 34–57% of that. In Q2–Q4 2024 recorded transfers match implied admissions (97–100%); at the very low 2025 volumes the ratio is 67–92%, consistent with the roughly 3 children a day that enter care outside recorded transfers (Appendix B.6). **Conclusion:** the CBP columns for 2023 – January 2024 are corrupted; the HHS columns reconcile wherever they can be checked (February 2024 onward) and are kept for the whole history, although they cannot be cross-checked inside the copied block itself.

Origin-time check. Copy detection is retrospective: a copy is recognised only once its later original exists. All originals of the copied rows are dated 26 June 2024 – 12 February 2025, before the first forecast origin (22 June 2025). Re-running the detection, and the change-point search of Section 3.2 of the paper, with data up to that origin only gives the identical block and identical break dates, so the masks and training windows used in the backtest would have been available in real time.

B.6 What the counts mean, and whether interpolating them is justified

Interpolating a stock between snapshots is natural; interpolating daily **counts** needs evidence. Three checks were run (current regime unless stated):

1. **Sunday counts are single-day counts, not 3-day totals.** Sunday discharges average 7.2 against 12.8 on Monday–Thursday; a weekend total would be about three times a weekday figure.
2. **Interpolated weekend flows reconcile with the stock.** The care-load change not explained by recorded flows is 2.97 children/day between consecutive weekday reports and 3.42/day across Thursday→Sunday gaps with interpolated Friday/Saturday flows, a difference of about 0.45/day. Across the trusted period from February 2024 onward, which the high-volume 2024 reports dominate, the weekend gap was large (−18.8/day vs +2.6/day), so 2024 weekend flow features are biased; they enter only the ML training rows, which a 180-day recency half-life down-weights.
3. **The choice barely changes model inputs.** 14-day flow means from the interpolated grid differ by 3.2% (transfers) and 4.8% (discharges) from means built on reported days only, with correlation ≥ 0.999.

Zero-filling was rejected because the stock keeps moving over weekends. The constant ~3 children/day that recorded flows never explain is consistent with admissions from outside CBP or a definitional difference; it is carried as a measured reconciliation term (+3.35 children/day over the last 60 days), not as a modelled flow.

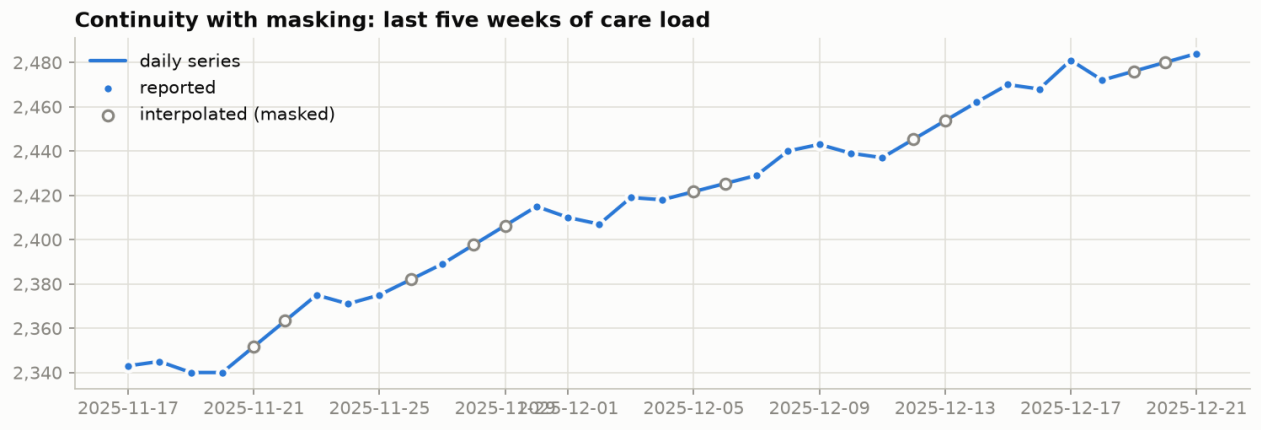


Figure B3. Continuity with masking: last five weeks of care load

B.7 Decisions

Table B3. Data-quality decisions with their evidence

Issue	Evidence	Treatment
450 blank rows; text dates; comma numbers; newest-first order	Raw audit	Dropped, parsed, typed, sorted (stable sort)
Duplicate dates (none in this file)	Stable sort + conflict listing	Identical repeats dropped; for conflicting repeats the first record in file order is kept and printed (the file carries no revision timestamp)
Malformed or implausible values on a refresh (none in this file)	Validation rules (Appendix B.2)	Stop with a clear message, or quarantine to missing and list
Saturday never reported, Friday almost never (2 reports)	Weekday counts	Daily calendar + time interpolation, with an `is_reported` mask
5 stale HHS reports	Exact repeats	Set to missing, interpolated, never scored
CBP block copied (Jan 2023 – Jan 2024)	Duplicate triples + broken identity	CBP fields masked; ML trained on the trusted period only
HHS columns	Reconcile where CBP is trusted	Kept for the full history (EDA, surge replay)

Leakage guard. An interpolated Friday value is computed from the following Sunday. A forecast issued "as of Friday" would therefore secretly know Sunday. Every forecast is issued from a reported day and scored only on reported days with a genuine value for that series, so interpolated values fill rolling-window features *between* reports but never carry future information.

Appendix C. Additional Exploratory Analysis

This appendix holds exploratory results and notebook charts that the paper summarises.

C.1 Structural break and length of stay

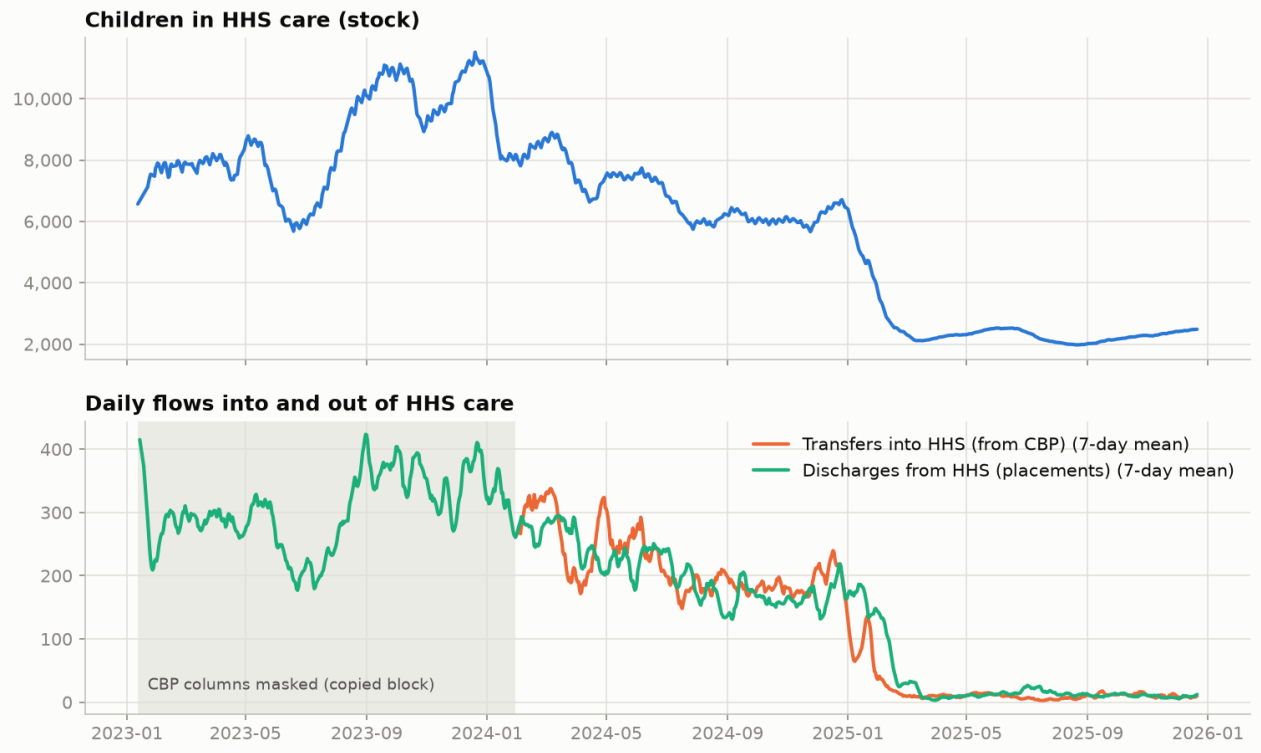


Figure C1. Care load and daily flows, 2023–2025 (notebook version)

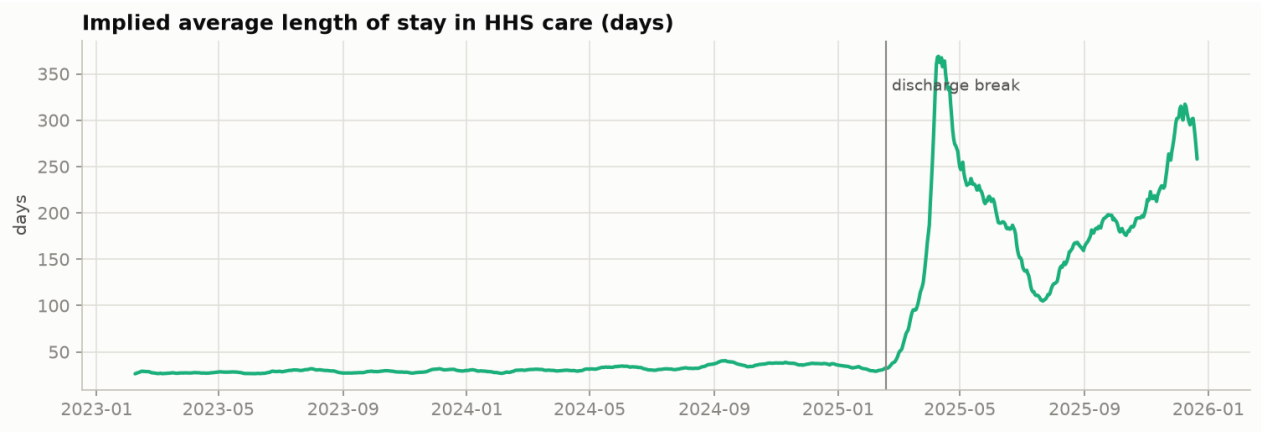


Figure C2. Implied average length of stay (notebook version)

C.2 Net pressure

Net pressure (transfers − discharges) is the flow signal that drives the stock. In the current regime it explains the shape of the care-load curve (Figure C3): net outflow (down to −19 children/day in mid-July) drove the June–August 2025 drawdown (about 2,500 → 1,972). Small recorded net inflows since September (about +1/day) coincide with the climb, but explain only part of it; the reconciliation term and the weekend pattern below account for the rest.

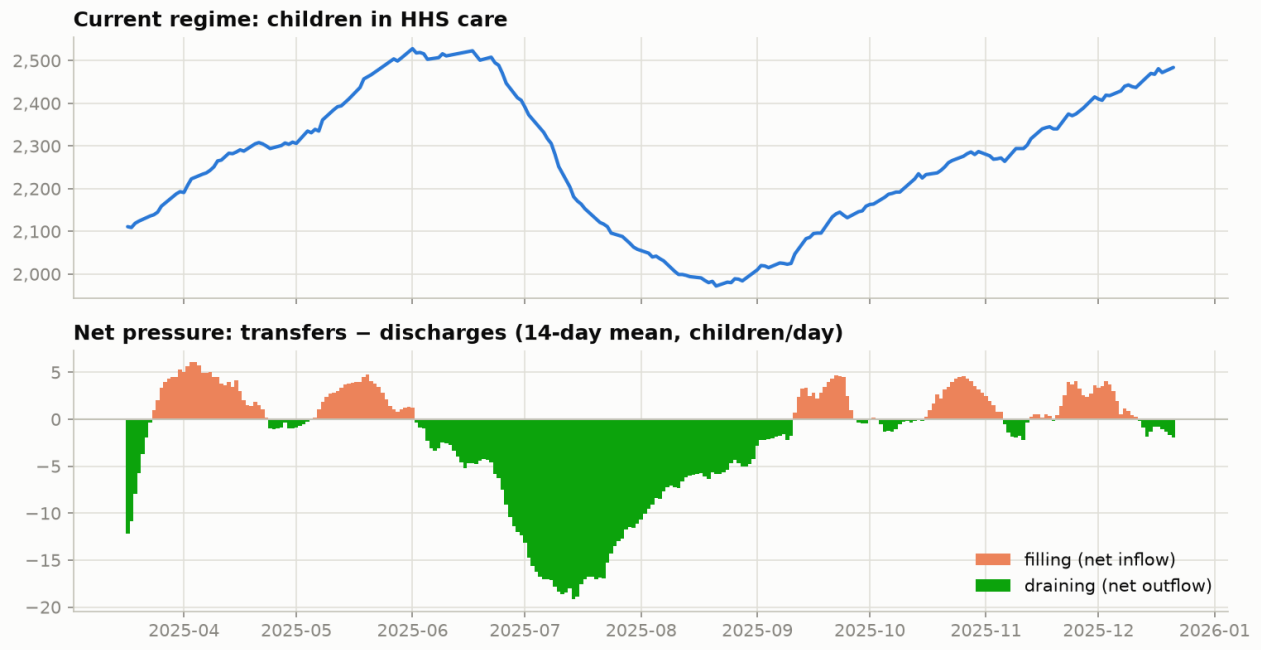


Figure C3. Current-regime care load and net pressure

C.3 Weekly structure and the weekend ratchet

On reported days in the current regime, discharges peak mid-week (Thursday 14.9, Wednesday 13.5) and are lowest on Sundays (7.2). Transfers are flatter (8.7–10.8). Comparing care-load changes on weekday transitions with changes across the unreported Thursday→Sunday gap reveals a striking pattern:

Table C1. Care-load change per day: weekdays against the Thursday-to-Sunday gap

Period	Weekday change / day	Weekend (Thu→Sun) change / day
2023 – Jan 2024	+66.8	−76.6
Feb 2024 – break	+19.4	−64.5
Current regime	**−0.3**	**+3.4**

Over the current regime, care load rose by 373. Weekday transitions netted **−40**, Thursday→Sunday gaps **+323** and other multi-day reporting gaps (holidays, a June outage and removed stale reports) **+90**. In other words, **net growth has come across weekends**, when arrivals continue but placements slow. The pattern is the reverse of the high-volume era, which is another reason to model the current regime separately.

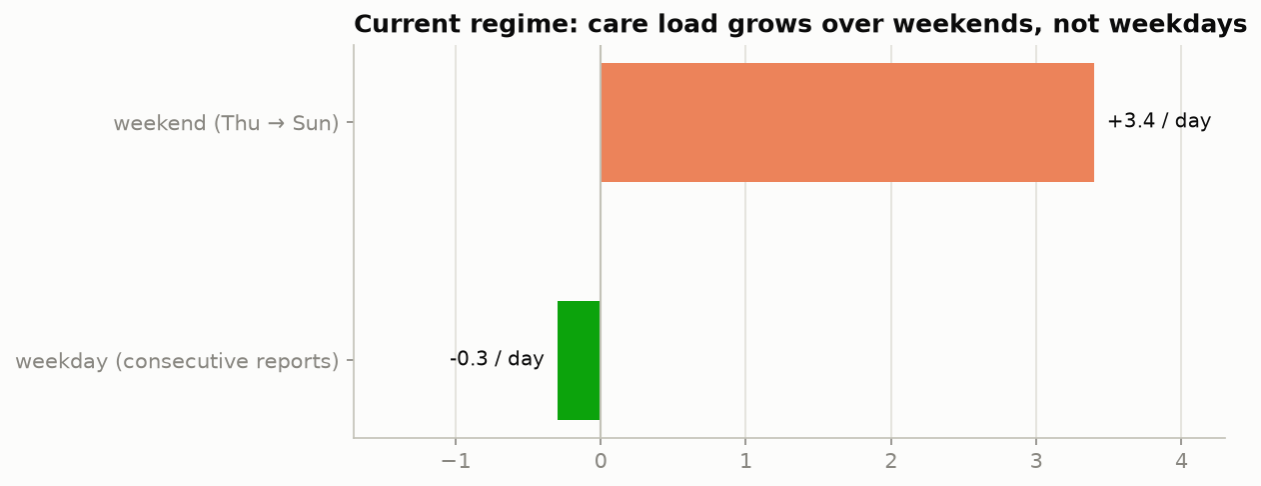


Figure C4. The weekend ratchet

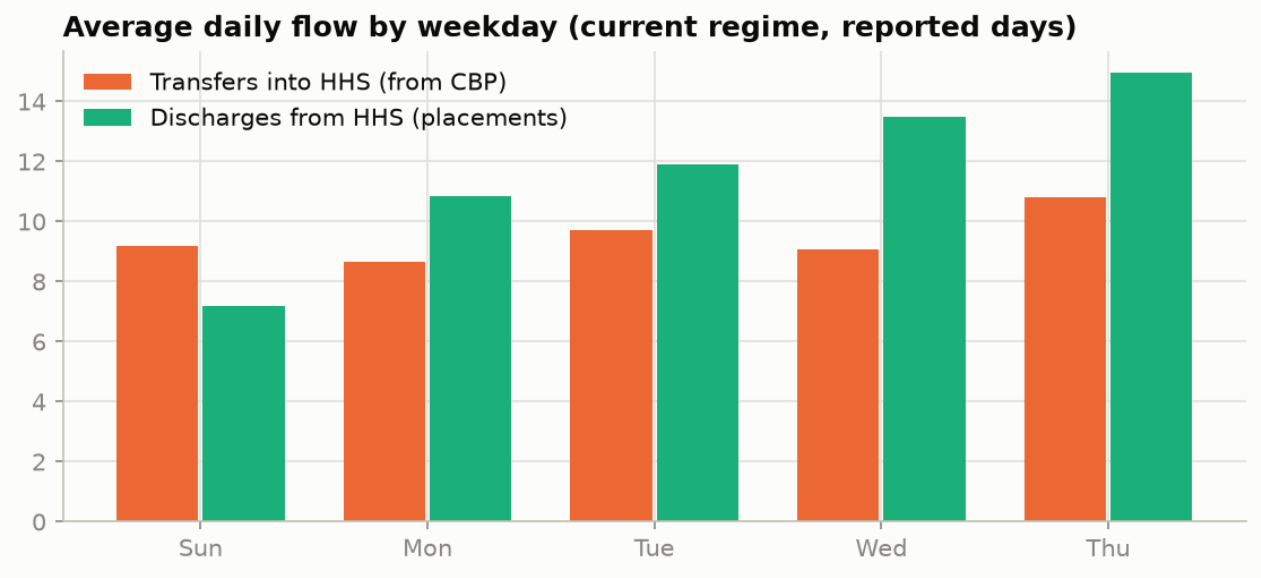


Figure C5. Weekday profile of flows

C.4 Monthly pattern, outliers, surge periods and correlations

- **No stable month-of-year pattern.** Month-end care load followed a different path every year (peak 11,516 on 20 Dec 2023; month-end values of about 6,000–8,700 through 2024; collapse in early 2025), and the month-to-month direction agreed between 2023 and 2024 in only 5 of 11 months (Figure C6). Each calendar month occurs at most three times, once on each side of the break, so month effects cannot be estimated.
- **Outliers.** The largest robust deviations (MAD z-scores against a centred 29-day median) occur at the low volumes of 2025: 29 transfers on 11 Sep 2025 against a local median of 11 (z = 12.1) and 39 discharges on 10 Jul 2025 against 17 (z = 4.9). They pass the plausibility rules and are kept.
- **Surge periods.** The fastest 28-day rises in care load were all in 2023: +2,806 (+43%) to 13 Aug 2023 (on the interpolated grid, because no reports exist for 13–15 Aug 2023; on reported days the largest rise is +37.6% to 16 Aug 2023, Section 8.4 of the paper), +1,342 (+20%) to 9 Feb 2023 and +1,695 (+17%) to the 11,516 peak on 20 Dec 2023.
- **Correlations (associations, not causal effects).** In the current regime the daily care-load change rises with transfers (+0.47) and falls with discharges (−0.61). Transfers correlate 0.38 with same-day apprehensions but 0.59 and 0.58 with apprehensions one and two days earlier, so CBP intake leads HHS transfers by one to two days.

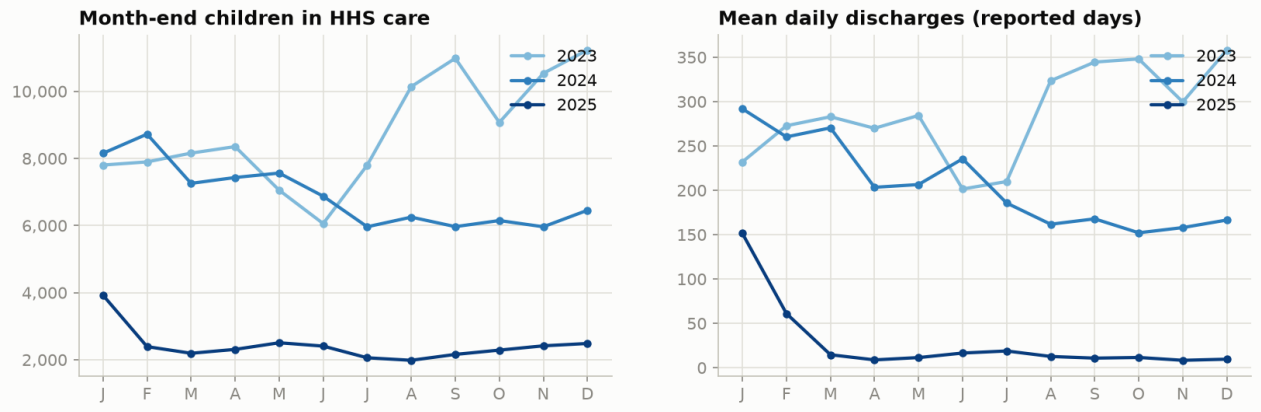


Figure C6. Monthly pattern by year

C.5 Decomposition and stationarity

STL decomposition (weekly period, robust fitting) gives a weekly seasonal strength of 0.42 for discharges but only 0.14 for care load (Figure C7). With fewer than ten months in the current regime and an irregular reporting cadence, the decomposition is descriptive; no annual component can be estimated.

- **Care load.** ADF and KPSS are inconclusive on the short regime window (low test power). Because a stock is integrated by construction (care_t = care_t−1 + in − out) and shows multi-month trends, care load is modelled with one difference (d = 1).
- **Flows.** Transfers test stationary. Discharges test non-stationary in level because of a temporary mid-2025 placement push, and are modelled in logs with a constant (d = 0), so forecasts revert to the regime's typical level instead of extrapolating a temporary surge.

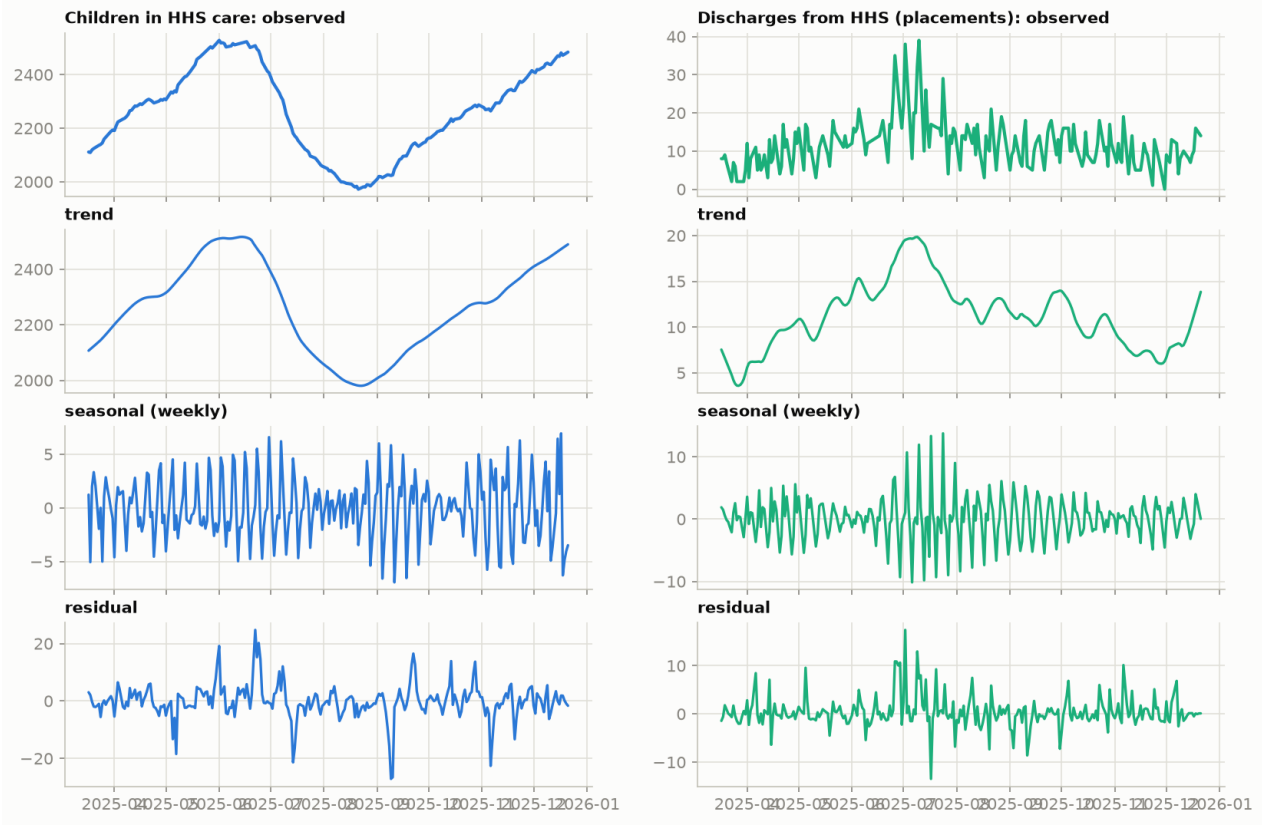


Figure C7. STL decomposition, current regime

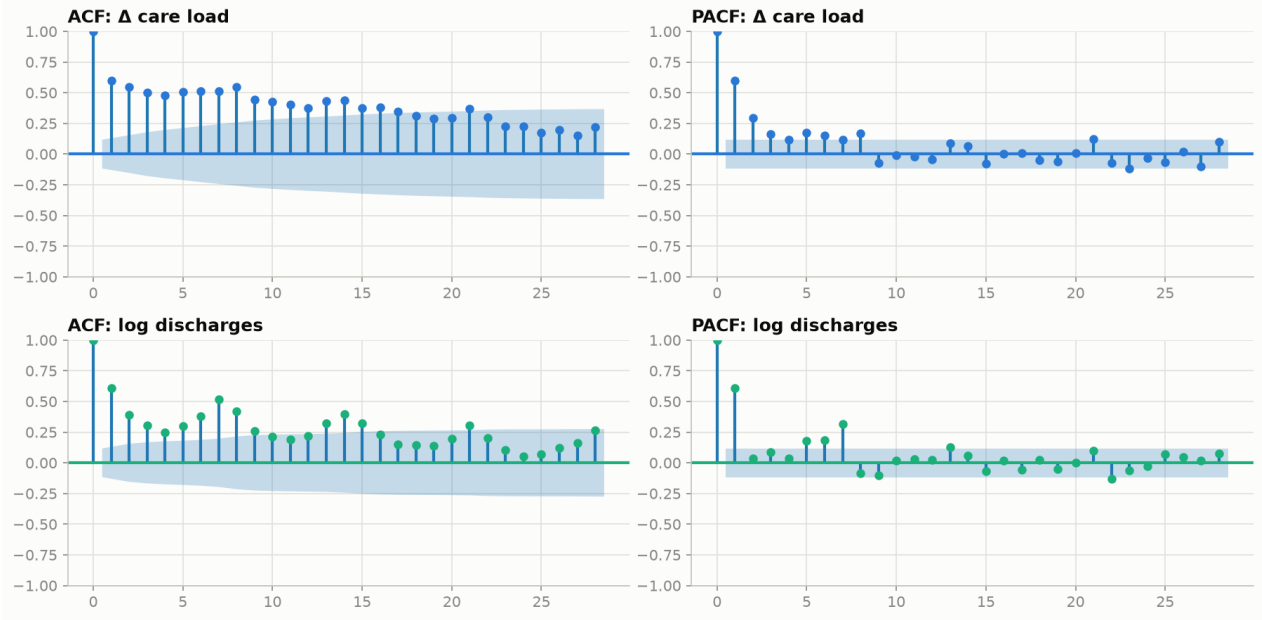


Figure C8. Autocorrelation and partial autocorrelation

Stationarity tests (executed notebook output).

	series	form	ADF p	KPSS p	verdict
0	hhs_care	level	0.106	0.086	inconclusive
1	hhs_care	1st difference	0.350	0.079	inconclusive
2	discharged	level	0.171	0.050	non-stationary
3	discharged	1st difference	0.000	0.100	stationary

```
4  transferred          level  0.035   0.100      stationary
5  transferred  1st difference  0.000   0.100      stationary
```

Appendix D. Feature and Leakage Audit

This appendix documents every model input and the tests that show none of them sees the future.

D.1 Time-series preparation

1. Re-index onto a continuous daily calendar (1,075 days).
2. Time-weighted linear interpolation of interior gaps (justified in Appendix B.6).
3. An `is_reported` mask that restricts forecast origins to genuine, non-stale reports, and a per-series "observed" mask that restricts scored and training targets to genuine values of that series.
4. CBP fields masked in the copied block.

D.2 Feature groups, targets and perturbation test

All features describe information available at the end of origin day *t*. The ML models use a direct multi-horizon design: one row per (origin, horizon h = 1…28), so a single model learns all horizons and never needs predicted or actual future values as inputs.

Table D1. Feature groups in full

Group	Features
Lags (t−1, t−7, t−14)	care-load log-changes; flow lags relative to their 14-day mean
Rolling mean & variance (7 / 14 days)	care-load deviation from rolling mean, rolling volatility; flow rolling means and log-variances
Flow-based signals	net pressure per 100 children (origin day, 7- and 14-day means); CBP pipeline per 100 children; flow rates per 100 children
Calendar	target weekday, US federal-holiday window (±1 day), origin weekday, horizon
Seasonal anchor	same weekday last week relative to today's level

Scale-free targets. The 2024 and 2025 regimes differ twentyfold in volume. ML targets are therefore log-ratios: log(care_t+h / care_t) for the stock, and log1p(flow_t+h) − log1p(14-day mean) for flows.

Leakage audit. Each of the 40 features is documented with its definition, source columns, look-back window, availability at the origin and horizon suitability (`artifacts/feature_leakage_audit.csv`). An executed perturbation test then scaled every report after a cut-off (18 Oct 2025) by 1.5, rebuilt the whole preparation and all features, and compared values at the 410 reported origins up to the cut-off: **0** of the feature values changed (40 features × 3 series × horizons 1/7/14/28). Negative controls confirm that the test detects look-ahead: the target changed in 99 rows, and 2 of the 3 interpolated days just before the cut-off changed. The net-pressure feature is labelled as an incomplete balance (Appendix B.6).

D.3 The month feature

The figures in the first two paragraphs of this subsection come from a development run that is not kept in the delivered notebook, so they cannot be reproduced from the delivered outputs. The exceptions are the post-removal test errors (38 and 31), which are. The first full run included the target month as a calendar feature. It became the **single most important care-load feature** (about 30% of Random Forest importance), and the production forecast showed Random Forest falling from 2,525 on 31 December to 2,282 on 1 January, a **10% cliff at the month boundary**. The trusted history contains only one January, January 2025, which is the crash at the early-2025 structural break. The model had learned a regime change disguised as seasonality.

The month feature was removed because of that cliff, not because of a test score. For transparency, removing it raised Random Forest's test error (MAE 25 → 38) but lowered Gradient Boosting's (40 → 31), and the ensemble stayed the best test model. Week of year has the same flaw and was never added. The holiday-window feature changes forecasts by at most about 4 children and was kept.

D.4 Full feature audit

Each row is one of the 40 features. The last column counts values that changed in the perturbation test.

Table D2. Feature leakage audit

feature	definition	source columns	look-back window	available at origin t?	horizon suitability	changed under perturbation
care_logchg_1	log(care[t] / care[t−1])	hhs_care	t−1 … t	yes (≤ t only)	all h; information ages with h	0

feature	definition	source columns	look-back window	available at origin t?	horizon suitability	changed under perturbation
care_logchg_7	log(care[t] / care[t−7])	hhs_care	t−7 … t	yes (≤ t only)	all h; information ages with h	0
care_logchg_14	log(care[t] / care[t−14])	hhs_care	t−14 … t	yes (≤ t only)	all h; information ages with h	0
care_dev_ma7	log(care[t] / 7-day mean of care)	hhs_care	t−6 … t	yes (≤ t only)	all h; information ages with h	0
care_vol7	std of daily care change over 7 days ÷ care[t] × 100	hhs_care	t−7 … t	yes (≤ t only)	all h; information ages with h	0
net_rate_ma7	7-day mean of (transfers − discharges) ÷ care[t] × 100	transferred, discharged, hhs_care	t−6 … t	yes (≤ t only)	all h; information ages with h	0
care_dev_ma14	log(care[t] / 14-day mean of care)	hhs_care	t−13 … t	yes (≤ t only)	all h; information ages with h	0
care_vol14	std of daily care change over 14 days ÷ care[t] × 100	hhs_care	t−14 … t	yes (≤ t only)	all h; information ages with h	0
net_rate_ma14	14-day mean of (transfers − discharges) ÷ care[t] × 100	transferred, discharged, hhs_care	t−13 … t	yes (≤ t only)	all h; information ages with h	0
net_rate_lag1	(transfers − discharges) on the origin day ÷ care[t] × 100	transferred, discharged, hhs_care	t	yes (≤ t only)	all h; information ages with h	0
cbp_pipeline_rate	children in CBP custody ÷ care[t] × 100	cbp_custody, hhs_care	t	yes (≤ t only)	all h; information ages with h	0
apprehended_lag0	log1p(apprehended[t−0]) − log1p(14-day mean)	apprehended	t−13 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_lag1	log1p(apprehended[t−1]) − log1p(14-day mean)	apprehended	t−13 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_lag7	log1p(apprehended[t−7]) − log1p(14-day mean)	apprehended	t−13 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_lag14	log1p(apprehended[t−14]) − log1p(14-day mean)	apprehended	t−14 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_ma7	log1p(7-day mean) − log1p(14-day mean) of apprehended	apprehended	t−13 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_logvar7	variance of log1p(apprehended) over 7 days	apprehended	t−6 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_logvar14	variance of log1p(apprehended) over 14 days	apprehended	t−13 … t	yes (≤ t only)	all h; information ages with h	0
apprehended_rate14	14-day mean of apprehended ÷ care[t] × 100	apprehended, hhs_care	t−13 … t	yes (≤ t only)	all h; information ages with h	0
transferred_lag0	log1p(transferred[t−0]) − log1p(14-day mean)	transferred	t−13 … t	yes (≤ t only)	all h; information ages with h	0
transferred_lag1	log1p(transferred[t−1]) − log1p(14-day mean)	transferred	t−13 … t	yes (≤ t only)	all h; information ages with h	0
transferred_lag7	log1p(transferred[t−7]) − log1p(14-day mean)	transferred	t−13 … t	yes (≤ t only)	all h; information ages with h	0
transferred_lag14	log1p(transferred[t−14]) − log1p(14-day mean)	transferred	t−14 … t	yes (≤ t only)	all h; information ages with h	0
transferred_ma7	log1p(7-day mean) − log1p(14-day mean) of transferred	transferred	t−13 … t	yes (≤ t only)	all h; information ages with h	0
transferred_logvar7	variance of log1p(transferred) over 7 days	transferred	t−6 … t	yes (≤ t only)	all h; information ages with h	0
transferred_logvar14	variance of log1p(transferred) over 14 days	transferred	t−13 … t	yes (≤ t only)	all h; information ages with h	0

feature	definition	source columns	look-back window	available at origin t?	horizon suitability	changed under perturbation
transferred_rate14	14-day mean of transferred ÷ care[t] × 100	transferred, hhs_care	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_lag0	log1p(discharged[t−0]) − log1p(14-day mean)	discharged	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_lag1	log1p(discharged[t−1]) − log1p(14-day mean)	discharged	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_lag7	log1p(discharged[t−7]) − log1p(14-day mean)	discharged	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_lag14	log1p(discharged[t−14]) − log1p(14-day mean)	discharged	t−14 … t	yes (≤ t only)	all h; information ages with h	0
discharged_ma7	log1p(7-day mean) − log1p(14-day mean) of discharged	discharged	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_logvar7	variance of log1p(discharged) over 7 days	discharged	t−6 … t	yes (≤ t only)	all h; information ages with h	0
discharged_logvar14	variance of log1p(discharged) over 14 days	discharged	t−13 … t	yes (≤ t only)	all h; information ages with h	0
discharged_rate14	14-day mean of discharged ÷ care[t] × 100	discharged, hhs_care	t−13 … t	yes (≤ t only)	all h; information ages with h	0
origin_dow	weekday of the origin day	calendar	t	yes (≤ t only)	all h; information ages with h	0
h	forecast horizon in days	design	n/a	yes (deterministic)	built per horizon (direct strategy)	0
target_dow	weekday of the target day t+h	calendar	known in advance	yes (deterministic)	built per horizon (direct strategy)	0
target_holiday	target day within ±1 day of a US federal holiday	calendar (USFederalHolidayCalendar)	known in advance	yes (deterministic)	built per horizon (direct strategy)	0
seasonal_anchor	same-weekday value at t+h−7⌈h/7⌉ (≤ t) relative to today's level (flows: 14-day mean)	series being forecast	t−13 … t	yes (≤ t only)	built per horizon (direct strategy)	0

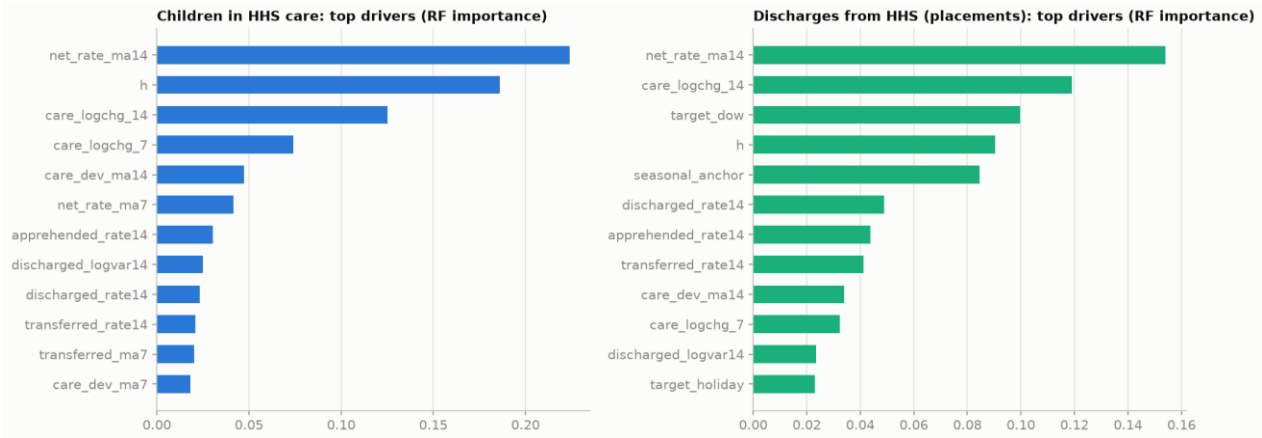


Figure D1. Feature importance

Appendix E. Model Specifications and Tuning Logs

This appendix gives full model settings, the order-selection logs and the design of each experiment. SARIMA orders were searched by AIC. The machine-learning models use fixed settings, so no grid-search log exists for them.

E.1 Train–test design

Table E1. Data used by each component

Component	Data used
Statistical models (SARIMA, ETS)	current regime, 17 Mar 2025 → origin
ML models (RF, GB)	trusted period, 14 Feb 2024 → origin, recency-weighted (180-day half-life)
Calibration window (champion selection, interval calibration)	22 Jun – 20 Sep 2025
Test window (reporting only)	21 Sep – 21 Dec 2025

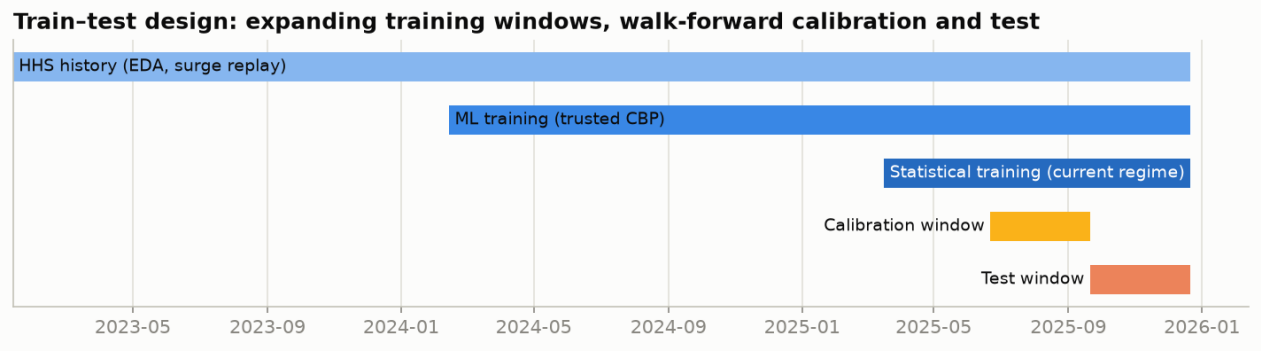


Figure E1. Train–test design

Forecasts are issued from **every reported day** (124 origins) for horizons 1–28. Models are re-estimated every 14 days and absorb each new observation in between, mirroring production use. The backtest produced 83,328 forecasts (7 models plus the ensemble), of which 52,464 had reported targets. A model whose fit or prediction fails is logged and replaced by naive persistence for that origin (flagged); in the delivered run no fit failed. The 28-day horizon covers four complete weekly cycles; 30-day forecasts were not validated.

Training-window ablation. A development experiment, not re-run in the delivered notebook, compared three ML training designs: the trusted window with recency weights, the trusted window without weights, and the current regime only. The decision rests on the **calibration** window alone: regime-only training was far worse early in the regime (calibration MAE 125 vs 89 for Random Forest, 126 vs 97 for Gradient Boosting, 5.8 vs 4.2 for discharges), when only about three months of history existed, so the recency-weighted trusted window was adopted.

E.2 Model specifications and fallbacks

The statistical models and the STL, ADF and KPSS diagnostics use statsmodels (Seabold & Perktold, 2010); the machine-learning models use scikit-learn (Pedregosa et al., 2011).

Table E2. Models, specifications and failure fallbacks

Family	Model	Specification	Failure condition → fallback
Baseline	Naive persistence	$\hat{y}_{t+h} = y_t$	none
Baseline	Seasonal naive	$\hat{y}_{t+h}$ = y at the same weekday of the last week (≤ t)	none
Baseline	Moving average	7-day mean, flat	none
Statistical	SARIMA	orders by AIC before the calibration window: care (1,1,1); discharges $(1,0,1)(1,0,1)_7$ + constant on log1p; transfers (1,0,0) + constant on log1p	< 56 days of history or non-convergence → previous fit, else naive
Statistical	ETS	care: additive error, damped additive trend, weekly season; flows: log1p, weekly season	< 28 days → previous fit, else naive
ML	Random Forest	200 trees, min leaf 5, 50% features, 70% bootstrap sample	< 200 training rows → previous fit, else naive
ML	Gradient Boosting	150 trees, depth 3, learning rate 0.07, Huber loss	as RF
Hybrid	Ensemble	equal-weight mean of SARIMA, ETS, RF, GB	components that fail contribute their flagged fallback

ARIMA vs SARIMA. For each series the best non-seasonal and the best weekly-seasonal specification were compared (18 specifications fitted per series on pre-calibration data). Weekly terms earn their place only for discharges (ΔAIC −11.5: 74.9 vs 86.4); for care load and transfers the seasonal versions are worse (+3.4 and +3.7). Walk-forward errors on the calibration window confirm every AIC choice (care 55.1 vs 55.3, discharges 4.09 vs 4.76, transfers 4.47 vs 4.60, selected vs other family).

E.3 SARIMA order-selection log

Best specification per series, chosen by AIC on data before the calibration window (executed notebook output).

```
hhs_care     best SARIMA(1, 1, 1)x(0, 0, 0, 7) trend=n  AIC=617.9
discharged   best SARIMA(1, 0, 1)x(1, 0, 1, 7) trend=c  AIC=74.9
transferred  best SARIMA(1, 0, 0)x(0, 0, 0, 7) trend=c  AIC=84.6
```

Best non-seasonal against best weekly-seasonal specification (executed notebook output).

series	best ARIMA	AIC ARIMA	best SARIMA	AIC SARIMA	ΔAIC (SARIMA − ARIMA)	specs fitted	selected
hhs_care	(1, 1, 1)	617.9	(1, 1, 1)x(1, 0, 1, 7)	621.3	3.4	18	ARIMA
discharged	(1, 0, 2)	86.4	(1, 0, 1)x(1, 0, 1, 7)	74.9	-11.5	18	SARIMA
transferred	(1, 0, 0)	84.6	(1, 0, 0)x(1, 0, 1, 7)	88.4	3.7	18	ARIMA

Selected specification against the other family, walked forward (executed notebook output).

Alternative specifications walked forward in 24s

series	period	selected (lowest AIC)	MAE selected	other family	MAE other
hhs_care	calibration	ARIMA(1, 1, 1)	55.10	SARIMA(1, 1, 1)x(1, 0, 1, 7)	55.27
	test	ARIMA(1, 1, 1)	32.00	SARIMA(1, 1, 1)x(1, 0, 1, 7)	31.31
discharged	calibration	SARIMA(1, 0, 1)x(1, 0, 1, 7)	4.09	ARIMA(1, 0, 2)	4.76
	test	SARIMA(1, 0, 1)x(1, 0, 1, 7)	3.33	ARIMA(1, 0, 2)	3.38
transferred	calibration	ARIMA(1, 0, 0)	4.47	SARIMA(1, 0, 0)x(1, 0, 1, 7)	4.60
	test	ARIMA(1, 0, 0)	4.04	SARIMA(1, 0, 0)x(1, 0, 1, 7)	4.07

E.4 Evaluation metrics

MAE, RMSE, MAPE (non-zero actuals only; the number of excluded zero actuals is reported, e.g. 18 discharge forecasts on test), WAPE = Σ|error| / Σ|actual|, and **Forecast Accuracy = 100 – WAPE** = 100 × (1 − Σ|y − ŷ| / Σ|y|). Accuracy is a volume-weighted complement of relative error, not a transformation of MAE or MAPE; it is 100 for a perfect forecast, can be negative when errors exceed volumes, and is undefined (reported as missing, never as a number) when all actuals are zero. Bias = mean(ŷ − y). Errors are analysed by horizon (1–28 days, weekly buckets, and a scorecard at 1/7/14/28 days). Diebold–Mariano tests with Bartlett variance and the Harvey correction test whether gains over naive are significant.

E.5 Uncertainty quantification

For each model, series and horizon, intervals add the empirical quantiles of out-of-sample errors (pooled over h ± 2) to the point forecast. Two calibration rules are evaluated on the test window: the **fixed calibration window** (split conformal), and the **rolling-origin rule** that the production forecast uses: at each origin, the error quantiles of all forecasts from the previous 26 weeks of origins whose target date has already been observed (a 13-week variant as sensitivity check). The rolling rule was fixed before its test results were inspected. Flow bounds are floored at zero; flooring cannot change coverage for non-negative actuals but is a truncation, not a count model.

E.6 Recursive against direct forecasting

To isolate the multi-step strategy, two Gradient Boosting models (100 trees, depth 3, Huber loss, recency weights) were given identical target-only inputs (the last 14 transformed values, their 7- and 14-day means, and the target weekday) and walked forward over the same 124 origins (refit every 28 days): a **direct** stacked model told the horizon, and a **recursive** one-step model fed its own predictions. The recursive model must step through every calendar day, so it trains on interpolated Friday/Saturday values; both are scored on reported days only.

Results.

Table E3. Recursive against direct, target-only Gradient Boosting (MAE)

Recursive vs direct (target-only GB, MAE)	1–7 d	8–14 d	15–21 d	22–28 d
Care load, test: direct / recursive	24.3 / **13.0**	27.0 / **23.9**	41.1 / **33.1**	64.7 / **40.6**
Care load, calibration: direct / recursive	41.7 / **21.5**	67.8 / **58.6**	108.0 / **102.3**	144.1 / **142.5**
Discharges, test: direct / recursive	3.47 / **3.16**	**3.64** / 3.77	**3.61** / 4.13	**3.68** / 4.17

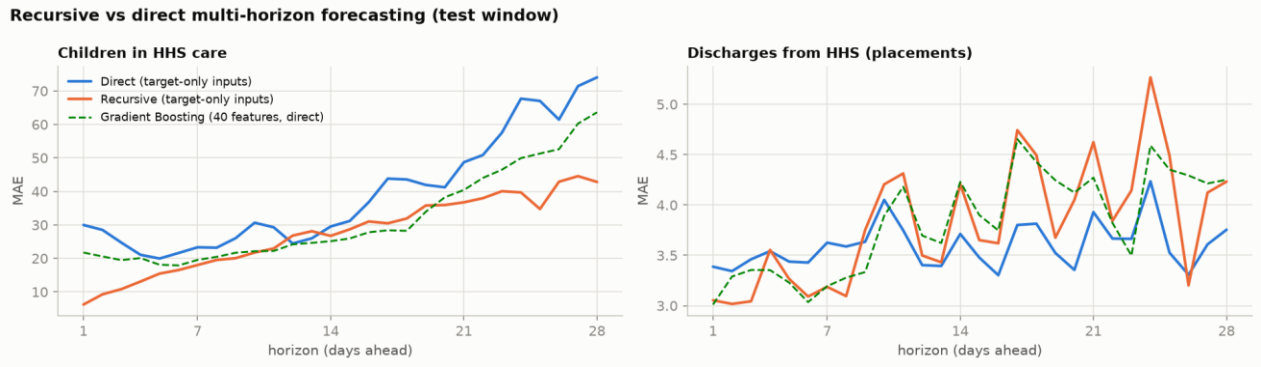


Figure E2. Recursive against direct forecasting on the test window

With identical inputs, the recursive strategy is better for the smooth care-load stock at every horizon in both windows, while for the noisy discharge flow it is better only in week 1 and worse from week 2 on the test window (from week 3 on the calibration window) as its own errors feed back. On the calibration window (the decision window) neither variant approaches the SARIMA champion (week averages of about 81 vs 55 children), although the recursive model would have beaten it on the test window (about 28 vs 32). The production design is unchanged; a recursive care-load learner is a candidate for the next model review.

E.7 Early-warning design

A **surge event** is the first reported day care load crosses threshold *C* after at least 14 days below it. An **alert** fires at origin *t* when the probability of exceeding *C* within 28 days reaches *p**. **Lead time** is the length of the unbroken alert streak before the event. Sweeping *C* over many thresholds produces many events. The system is evaluated on the 2025 test window (champion and its calibrated errors) and in a **real-time replay of 2023–24** with a rolling 120-day damped-trend ETS model and analytic intervals (only the HHS columns are trustworthy then, so the replay model is univariate). The operating point p* = 30% was chosen on the replay curve; the test window played no part in the choice.

E.8 Scenario engine

Scenarios modify forecast flows and propagate the change through the stock-flow identity:

care^scenario_t+h = care^baseline_t+h + Σ (Δtransfers − Δdischarges)

The baseline path is a model forecast; every other path is a **hypothetical assumption**, not a prediction, and carries no probability of its own. The same engine runs in the dashboard's Scenario lab.

Appendix F. Full Model Outputs

This appendix holds the complete result tables and the notebook charts behind Sections 6 to 8.

F.1 Accuracy of every model

Table F1. Accuracy by series, window and model

target	period	model	MAE	RMSE	MAPE %	WAPE %	Accuracy %	Bias	n	zero y excluded
discharged	calibration	ETS	4.9	6.43	43.33	36.09	63.91	0.57	1187	0
discharged	calibration	Ensemble	3.91	5.67	34.11	28.78	71.22	-1.04	1187	0
discharged	calibration	Gradient Boosting	4.56	6.57	36.57	33.56	66.44	-1.86	1187	0
discharged	calibration	Moving Avg (7d)	5.6	7.17	58.07	41.21	58.79	1.21	1187	0
discharged	calibration	Naive	6.87	9.15	67.87	50.53	49.47	1.29	1187	0
discharged	calibration	Random Forest	4.97	7.03	40.32	36.61	63.39	-2.11	1187	0
discharged	calibration	SARIMA	4.09	5.83	36.97	30.13	69.87	-0.75	1187	0
discharged	calibration	Seasonal Naive (7d)	5.61	7.62	50.75	41.3	58.7	1.62	1187	0
discharged	test	ETS	3.69	4.6	48.57	37.68	62.32	0.15	999	18
discharged	test	Ensemble	3.46	4.19	50.87	35.42	64.58	1.01	999	18
discharged	test	Gradient Boosting	3.78	4.59	56.28	38.64	61.36	1.68	999	18
discharged	test	Moving Avg (7d)	3.63	4.44	55.91	37.14	62.86	0.41	999	18
discharged	test	Naive	4.6	5.73	65.79	46.98	53.02	0.42	999	18
discharged	test	Random Forest	3.8	4.61	57.45	38.85	61.15	1.71	999	18
discharged	test	SARIMA	3.33	4.01	48.77	34.09	65.91	0.5	999	18
discharged	test	Seasonal Naive (7d)	4.08	5.06	55.97	41.66	58.34	0.56	999	18
hhs_care	calibration	ETS	55.28	80.07	2.62	2.65	97.35	-12.72	1187	0
hhs_care	calibration	Ensemble	71.31	97.57	3.4	3.42	96.58	-16.92	1187	0
hhs_care	calibration	Gradient Boosting	90.83	125.76	4.32	4.35	95.65	-4.67	1187	0

target	period	model	MAE	RMSE	MAPE %	WAPE %	Accuracy %	Bias	n	zero y excluded
hhs_care	calibration	Moving Avg (7d)	111.56	140.88	5.29	5.34	94.66	48.67	1187	0
hhs_care	calibration	Naive	92.51	121.23	4.39	4.43	95.57	35.67	1187	0
hhs_care	calibration	Random Forest	105.86	137.71	5.07	5.07	94.93	-40.1	1187	0
hhs_care	calibration	SARIMA	55.1	79.04	2.61	2.64	97.36	-10.19	1187	0
hhs_care	calibration	Seasonal Naive (7d)	111.19	140.15	5.27	5.33	94.67	48.27	1187	0
hhs_care	test	ETS	31.33	45.3	1.34	1.35	98.65	-13.76	999	0
hhs_care	test	Ensemble	28.49	41.98	1.23	1.23	98.77	-17.08	999	0
hhs_care	test	Gradient Boosting	30.88	41.95	1.34	1.33	98.67	-13.93	999	0
hhs_care	test	Moving Avg (7d)	67.61	77.15	2.89	2.91	97.09	-67.18	999	0
hhs_care	test	Naive	55.69	66.87	2.38	2.39	97.61	-54.74	999	0
hhs_care	test	Random Forest	37.81	60.54	1.65	1.63	98.37	-24.74	999	0
hhs_care	test	SARIMA	32	46.03	1.37	1.38	98.62	-15.9	999	0
hhs_care	test	Seasonal Naive (7d)	67.11	76.52	2.87	2.89	97.11	-66.49	999	0
transferred	calibration	ETS	3.98	5.43	58.48	48.37	51.63	-1.99	1187	29
transferred	calibration	Ensemble	3.92	5.32	61.22	47.55	52.45	-1.67	1187	29
transferred	calibration	Gradient Boosting	4.12	5.59	61.07	50	50	-2.09	1187	29
transferred	calibration	Moving Avg (7d)	3.82	5.29	58.23	46.39	53.61	-1.27	1187	29
transferred	calibration	Naive	4.63	6.25	71.54	56.18	43.82	-1.16	1187	29
transferred	calibration	Random Forest	4.29	5.74	64.33	52.07	47.93	-2.3	1187	29
transferred	calibration	SARIMA	4.47	5.6	85.66	54.28	45.72	-0.28	1187	29
transferred	calibration	Seasonal Naive (7d)	4.65	6.36	72.24	56.45	43.55	-1.1	1187	29
transferred	test	ETS	3.98	5.09	52.55	38.7	61.3	0.29	999	0
transferred	test	Ensemble	4.07	5.05	53.64	39.59	60.41	0.1	999	0
transferred	test	Gradient Boosting	4.71	5.81	67.01	45.87	54.13	1.36	999	0
transferred	test	Moving Avg (7d)	4.23	5.37	60.02	41.19	58.81	1.06	999	0
transferred	test	Naive	5.12	6.73	67.13	49.84	50.16	0.66	999	0
transferred	test	Random Forest	4.44	5.47	60.92	43.2	56.8	0.79	999	0
transferred	test	SARIMA	4.04	5.39	42.64	39.34	60.66	-2.06	999	0
transferred	test	Seasonal Naive (7d)	5.09	6.61	65.78	49.52	50.48	0.76	999	0

F.2 Error by horizon

Table F2. Test-window MAE by model at selected horizons

Series	Model	h = 1	h = 7	h = 14	h = 21	h = 28
discharged	ETS	3.82	3.13	3.73	3.81	3.31
discharged	Ensemble	3.02	3.01	3.66	3.95	3.7
discharged	Gradient Boosting	3.01	3.19	4.23	4.27	4.25
discharged	Moving Avg (7d)	3.37	3.53	3.67	3.91	3.51
discharged	Naive	4.13	3.84	4.2	4.8	4.21
discharged	Random Forest	2.79	3.15	4.12	4.41	4.46
discharged	SARIMA	3.43	2.93	3.43	3.71	3.29
discharged	Seasonal Naive (7d)	3.51	3.84	4.2	4.8	4.21
hhs_care	ETS	5.76	19.1	32.23	44.1	58.25
hhs_care	Ensemble	9.07	17.23	26.78	40.31	56.93
hhs_care	Gradient Boosting	21.75	19.55	25.14	40.43	63.61
hhs_care	Moving Avg (7d)	17.47	40.38	67.7	95.95	123.22
hhs_care	Naive	5.72	29	55.48	83.33	111.24
hhs_care	Random Forest	11.78	20.14	30.31	59.69	75.69
hhs_care	SARIMA	5.8	19.19	32.5	43.99	58.64
hhs_care	Seasonal Naive (7d)	31.11	29	55.48	83.33	111.24
transferred	ETS	3.45	4.07	4.25	4.12	4.31
transferred	Ensemble	3.69	4.02	4.15	4.31	4.33
transferred	Gradient Boosting	4.37	4.56	4.58	4.85	5.01
transferred	Moving Avg (7d)	3.53	4.13	4.38	4.53	4.63
transferred	Naive	4.6	5.15	4.92	5.42	5.48
transferred	Random Forest	3.98	4.27	4.28	4.87	4.81
transferred	SARIMA	3.9	4.14	4.15	3.99	3.95
transferred	Seasonal Naive (7d)	4.84	5.15	4.92	5.42	5.48

The file metrics_by_horizon.csv in the artifacts folder has every horizon from 1 to 28.

Table F3. Multi-horizon scorecard for the champion models

Scorecard (test)	h = 1	h = 7	h = 14	h = 28
Care load, SARIMA: MAE / bias	5.8 / +1.6	19.2 / −3.0	32.5 / −13.8	58.6 / −44.0
Care load, Naive: MAE	5.7	29.0	55.5	111.2
Discharges, Ensemble: MAE / bias	3.02 / +0.31	3.01 / +0.27	3.66 / +1.12	3.70 / +1.78
Transfers, Moving Avg: MAE / bias	3.53 / +0.45	4.13 / +0.71	4.38 / +0.94	4.63 / +1.65
80% coverage (rolling 26-week rule): care / discharges / transfers	94 / 89 / 77%	91 / 85 / 71%	90 / 66 / 70%	88 / 71 / 52%

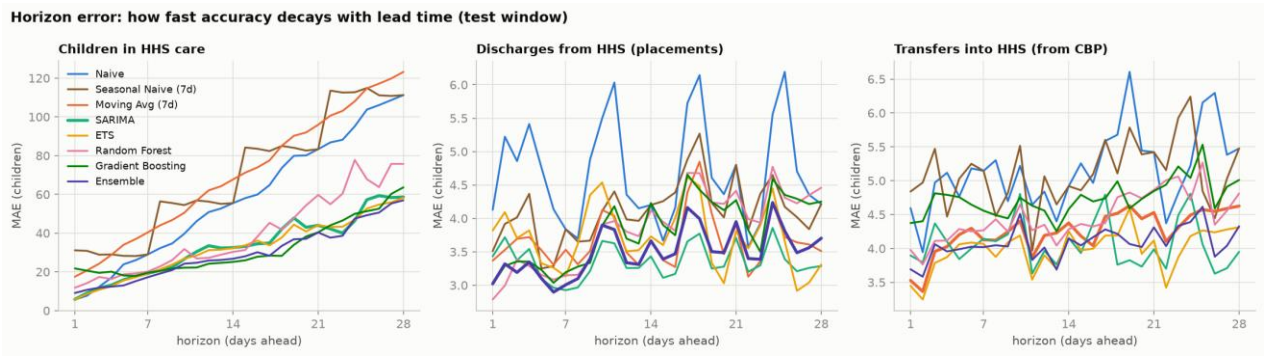


Figure F1. Error by horizon for all eight models (notebook version)

F.3 Statistical significance

Table F4. Diebold–Mariano p-values against naive

Series	h = 1	h = 7	h = 14	h = 28
Care load (SARIMA)	p = 0.89	p = 0.052	**p = 0.045**	**p < 0.001**
Discharges (Ensemble)	**p = 0.008**	**p = 0.003**	p = 0.072	p = 0.29
Transfers (Moving Avg)	**p = 0.035**	p = 0.12	p = 0.059	p = 0.58

For the care load, persistence is unbeatable at one day (tomorrow ≈ today), but the gains become significant at two and four weeks, the horizons that matter for capacity. For flows the gains are significant at short horizons.

Full test output (executed notebook output).

	target	champion	h	MAE champion	MAE naive	DM stat	p-value	significant (5%)
0	hhs_care	SARIMA	1	5.799	5.723	0.137	0.892	False
1	hhs_care	SARIMA	7	19.192	29.000	-1.990	0.052	False
2	hhs_care	SARIMA	14	32.498	55.480	-2.061	0.045	True
3	hhs_care	SARIMA	28	58.643	111.238	-20.918	0.000	True
4	discharged	Ensemble	1	3.021	4.128	-2.793	0.008	True
5	discharged	Ensemble	7	3.005	3.836	-3.056	0.003	True
6	discharged	Ensemble	14	3.664	4.200	-1.837	0.072	False
7	discharged	Ensemble	28	3.703	4.214	-1.066	0.292	False
8	transferred	Moving Avg (7d)	1	3.527	4.596	-2.176	0.035	True
9	transferred	Moving Avg (7d)	7	4.127	5.145	-1.588	0.118	False
10	transferred	Moving Avg (7d)	14	4.376	4.920	-1.930	0.059	False
11	transferred	Moving Avg (7d)	28	4.626	5.476	-0.563	0.577	False

F.4 Robustness

Table F5. Robustness by series and model

target	model	win-rate vs naive %	error growth wk4/wk1	coverage80 %	FSI	accuracy %
hhs_care	Naive	n/a	5.6	94.8	50.7	97.6
hhs_care	Seasonal Naive (7d)	0	3.8	94.6	63.5	97.1
hhs_care	Moving Avg (7d)	7.1	3.8	97.8	58.5	97.1
hhs_care	SARIMA	85.7	3.9	91.6	25.2	98.6
hhs_care	ETS	85.7	3.9	92.6	25.1	98.7
hhs_care	Random Forest	71.4	4	97.9	23.1	98.4
hhs_care	Gradient Boosting	92.9	2.7	100	33.1	98.7
hhs_care	Ensemble	100	3.7	97.6	30.2	98.8
discharged	Naive	n/a	1	95.7	49.9	53
discharged	Seasonal Naive (7d)	85.7	1.1	91.6	87.8	58.3
discharged	Moving Avg (7d)	85.7	1	94.5	86	62.9
discharged	SARIMA	85.7	1	86.2	87	65.9
discharged	ETS	78.6	1	89	73.7	62.3
discharged	Random Forest	85.7	1.4	74.2	87.4	61.2
discharged	Gradient Boosting	85.7	1.3	74.3	84.1	61.4
discharged	Ensemble	85.7	1.2	77.2	86.5	64.6
transferred	Naive	n/a	1.1	72.7	50.1	50.2
transferred	Seasonal Naive (7d)	50	1.1	71	86.6	50.5
transferred	Moving Avg (7d)	78.6	1.1	66	84.3	58.8
transferred	SARIMA	78.6	1	80	95.9	60.7
transferred	ETS	85.7	1.1	66.2	80.6	61.3
transferred	Random Forest	57.1	1.2	64	83.3	56.8
transferred	Gradient Boosting	50	1.1	59.5	82.2	54.1
transferred	Ensemble	64.3	1.1	68.4	87.6	60.4

F.5 Interval coverage

Table F6. Interval coverage and width on the test window

target	model	coverage80	coverage95	width80
discharged	ETS	0.89	0.982	15.223
discharged	Ensemble	0.772	0.963	11.829
discharged	Gradient Boosting	0.743	0.955	13.85
discharged	Moving Avg (7d)	0.945	0.99	16.883
discharged	Naive	0.957	0.996	21.246
discharged	Random Forest	0.742	0.949	14.925
discharged	SARIMA	0.862	0.988	12.703
discharged	Seasonal Naive (7d)	0.916	0.984	17.427
hhs_care	ETS	0.926	0.984	166.755
hhs_care	Ensemble	0.976	0.998	221.427
hhs_care	Gradient Boosting	1	1	301.447
hhs_care	Moving Avg (7d)	0.978	1	316.8
hhs_care	Naive	0.948	1	267.352
hhs_care	Random Forest	0.979	0.996	301.534
hhs_care	SARIMA	0.916	0.975	165.735
hhs_care	Seasonal Naive (7d)	0.946	1	318.754
transferred	ETS	0.662	0.881	11.29
transferred	Ensemble	0.684	0.907	11.049
transferred	Gradient Boosting	0.595	0.816	11.398
transferred	Moving Avg (7d)	0.66	0.843	11.187
transferred	Naive	0.727	0.908	13.669
transferred	Random Forest	0.64	0.855	11.67
transferred	SARIMA	0.8	0.958	12.435
transferred	Seasonal Naive (7d)	0.71	0.912	14.015

Table F7. Coverage by calibration rule

target	model	method	coverage80	coverage95	width80
discharged	ETS	fixed calibration (13 wk)	0.89	0.982	15.223
discharged	ETS	rolling origin (26 wk)	0.878	0.974	14.403
discharged	ETS	rolling origin (13 wk)	0.824	0.948	12.426
discharged	Ensemble	fixed calibration (13 wk)	0.772	0.963	11.829
discharged	Ensemble	rolling origin (26 wk)	0.787	0.961	11.313
discharged	Ensemble	rolling origin (13 wk)	0.779	0.929	10.162
discharged	Gradient Boosting	fixed calibration (13 wk)	0.743	0.955	13.85
discharged	Gradient Boosting	rolling origin (26 wk)	0.774	0.95	13.162
discharged	Gradient Boosting	rolling origin (13 wk)	0.751	0.927	11.063
discharged	Moving Avg (7d)	fixed calibration (13 wk)	0.945	0.99	16.883
discharged	Moving Avg (7d)	rolling origin (26 wk)	0.916	0.987	15.413
discharged	Moving Avg (7d)	rolling origin (13 wk)	0.863	0.952	13.379
discharged	Naive	fixed calibration (13 wk)	0.957	0.996	21.246
discharged	Naive	rolling origin (26 wk)	0.93	0.995	19.716
discharged	Naive	rolling origin (13 wk)	0.887	0.971	17.036
discharged	Random Forest	fixed calibration (13 wk)	0.742	0.949	14.925
discharged	Random Forest	rolling origin (26 wk)	0.776	0.945	14.259
discharged	Random Forest	rolling origin (13 wk)	0.751	0.917	11.091
discharged	SARIMA	fixed calibration (13 wk)	0.862	0.988	12.703
discharged	SARIMA	rolling origin (26 wk)	0.829	0.978	11.751
discharged	SARIMA	rolling origin (13 wk)	0.777	0.955	10.154
discharged	Seasonal Naive (7d)	fixed calibration (13 wk)	0.916	0.984	17.427
discharged	Seasonal Naive (7d)	rolling origin (26 wk)	0.887	0.977	15.985
discharged	Seasonal Naive (7d)	rolling origin (13 wk)	0.851	0.955	13.671
hhs_care	ETS	fixed calibration (13 wk)	0.926	0.984	166.755
hhs_care	ETS	rolling origin (26 wk)	0.91	0.976	152.369
hhs_care	ETS	rolling origin (13 wk)	0.803	0.927	110.29
hhs_care	Ensemble	fixed calibration (13 wk)	0.976	0.998	221.427
hhs_care	Ensemble	rolling origin (26 wk)	0.965	0.996	202.119
hhs_care	Ensemble	rolling origin (13 wk)	0.851	0.947	128.25
hhs_care	Gradient Boosting	fixed calibration (13 wk)	1	1	301.447
hhs_care	Gradient Boosting	rolling origin (26 wk)	0.991	1	273.465
hhs_care	Gradient Boosting	rolling origin (13 wk)	0.794	0.92	157.935
hhs_care	Moving Avg (7d)	fixed calibration (13 wk)	0.978	1	316.8
hhs_care	Moving Avg (7d)	rolling origin (26 wk)	0.931	0.996	301.117
hhs_care	Moving Avg (7d)	rolling origin (13 wk)	0.965	0.996	166.18
hhs_care	Naive	fixed calibration (13 wk)	0.948	1	267.352
hhs_care	Naive	rolling origin (26 wk)	0.937	0.998	253.096
hhs_care	Naive	rolling origin (13 wk)	0.974	1	137.748
hhs_care	Random Forest	fixed calibration (13 wk)	0.979	0.996	301.534
hhs_care	Random Forest	rolling origin (26 wk)	0.967	0.996	268.668
hhs_care	Random Forest	rolling origin (13 wk)	0.749	0.888	170.893
hhs_care	SARIMA	fixed calibration (13 wk)	0.916	0.975	165.735

target	model	method	coverage80	coverage95	width80
hhs_care	SARIMA	rolling origin (26 wk)	0.902	0.967	150.498
hhs_care	SARIMA	rolling origin (13 wk)	0.817	0.945	108.377
hhs_care	Seasonal Naive (7d)	fixed calibration (13 wk)	0.946	1	318.754
hhs_care	Seasonal Naive (7d)	rolling origin (26 wk)	0.911	0.99	304.149
hhs_care	Seasonal Naive (7d)	rolling origin (13 wk)	0.951	0.992	167.38
transferred	ETS	fixed calibration (13 wk)	0.662	0.881	11.29
transferred	ETS	rolling origin (26 wk)	0.7	0.891	11.747
transferred	ETS	rolling origin (13 wk)	0.707	0.894	12.11
transferred	Ensemble	fixed calibration (13 wk)	0.684	0.907	11.049
transferred	Ensemble	rolling origin (26 wk)	0.719	0.908	11.412
transferred	Ensemble	rolling origin (13 wk)	0.693	0.859	11.281
transferred	Gradient Boosting	fixed calibration (13 wk)	0.595	0.816	11.398
transferred	Gradient Boosting	rolling origin (26 wk)	0.675	0.848	12.038
transferred	Gradient Boosting	rolling origin (13 wk)	0.641	0.811	12.009
transferred	Moving Avg (7d)	fixed calibration (13 wk)	0.66	0.843	11.187
transferred	Moving Avg (7d)	rolling origin (26 wk)	0.684	0.861	11.58
transferred	Moving Avg (7d)	rolling origin (13 wk)	0.711	0.868	12.285
transferred	Naive	fixed calibration (13 wk)	0.727	0.908	13.669
transferred	Naive	rolling origin (26 wk)	0.736	0.904	14.144
transferred	Naive	rolling origin (13 wk)	0.76	0.943	15.501
transferred	Random Forest	fixed calibration (13 wk)	0.64	0.855	11.67
transferred	Random Forest	rolling origin (26 wk)	0.689	0.88	12.126
transferred	Random Forest	rolling origin (13 wk)	0.652	0.834	11.803
transferred	SARIMA	fixed calibration (13 wk)	0.8	0.958	12.435
transferred	SARIMA	rolling origin (26 wk)	0.815	0.962	12.932
transferred	SARIMA	rolling origin (13 wk)	0.737	0.945	12.104
transferred	Seasonal Naive (7d)	fixed calibration (13 wk)	0.71	0.912	14.015
transferred	Seasonal Naive (7d)	rolling origin (26 wk)	0.729	0.887	14.088
transferred	Seasonal Naive (7d)	rolling origin (13 wk)	0.756	0.92	15.531

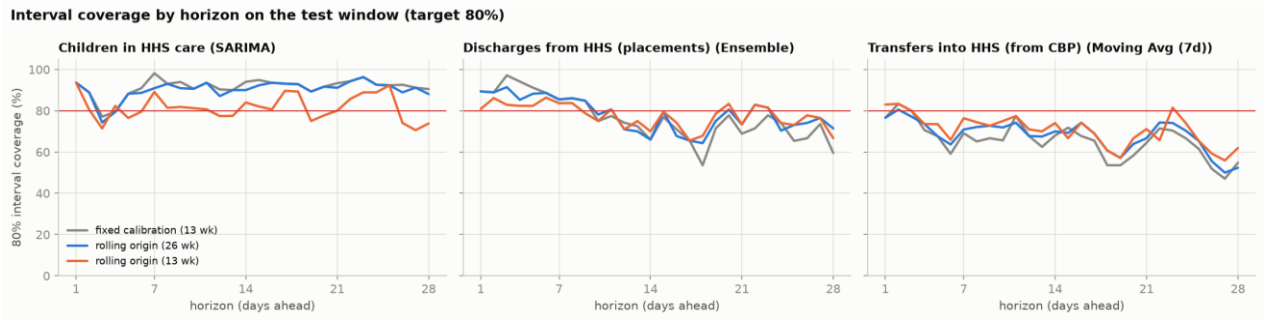


Figure F2. 80% coverage by horizon

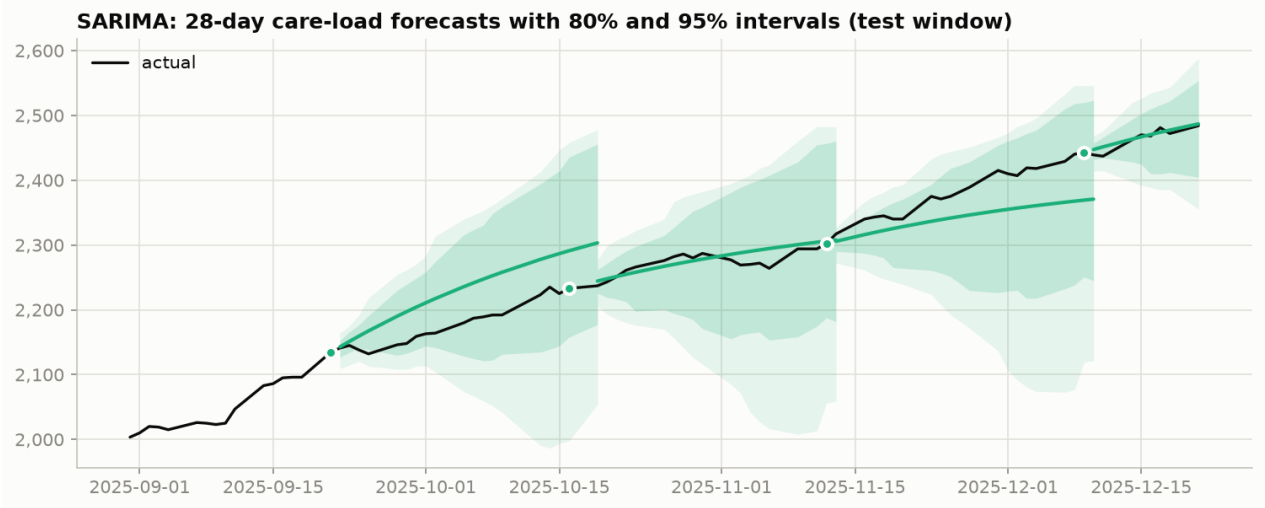


Figure F3. Care-load forecasts with intervals (notebook version)

F.6 Early warning

Table F8. Early-warning operating curve

alert at P(breach) ≥	events	detected %	median lead (days)	mean lead (days)	false alarms %	source
0.5	25	80	1	2.4	12.4	replay 2023–24
0.4	25	100	6	6.1	15.1	replay 2023–24
0.3	25	100	14	14.6	25.7	replay 2023–24
0.2	25	100	27	21.8	35.6	replay 2023–24
0.1	25	100	28	24.4	43.1	replay 2023–24
0.5	16	100	18.5	17.9	16	test 2025
0.4	16	100	23.5	22.4	20.5	test 2025
0.3	16	100	28	24.3	22.4	test 2025
0.2	16	100	28	25.4	23.9	test 2025
0.1	16	100	28	26.9	24.8	test 2025

Table F9. Lead time of every surge event at the 30% alert level

threshold	event_date	lead_days	detected	source
6,000	2023-06-29	9	True	replay 2023–24
6,000	2024-12-02	6	True	replay 2023–24
6,500	2023-07-12	16	True	replay 2023–24
6,500	2024-12-17	14	True	replay 2023–24
7,000	2023-07-19	13	True	replay 2023–24
7,000	2024-04-24	10	True	replay 2023–24
7,500	2023-07-25	8	True	replay 2023–24
7,500	2024-05-01	17	True	replay 2023–24
7,500	2024-05-30	28	True	replay 2023–24
8,000	2023-03-21	7	True	replay 2023–24
8,000	2023-04-25	1	True	replay 2023–24
8,000	2023-08-01	14	True	replay 2023–24
8,500	2023-05-02	6	True	replay 2023–24
8,500	2023-08-08	15	True	replay 2023–24
8,500	2024-02-14	10	True	replay 2023–24
9,000	2023-08-16	16	True	replay 2023–24
9,500	2023-08-16	15	True	replay 2023–24
9,500	2023-11-08	9	True	replay 2023–24
10,000	2023-08-23	22	True	replay 2023–24
10,000	2023-11-27	26	True	replay 2023–24
10,500	2023-09-12	22	True	replay 2023–24
10,500	2023-11-30	28	True	replay 2023–24
11,000	2023-09-19	14	True	replay 2023–24
11,000	2023-12-12	16	True	replay 2023–24
11,500	2023-12-20	23	True	replay 2023–24
2,100	2025-09-21	20	True	test 2025
2,125	2025-09-21	20	True	test 2025
2,150	2025-09-30	20	True	test 2025
2,175	2025-10-05	24	True	test 2025
2,200	2025-10-13	28	True	test 2025
2,225	2025-10-14	28	True	test 2025
2,250	2025-10-21	28	True	test 2025
2,275	2025-10-26	28	True	test 2025
2,300	2025-11-12	28	True	test 2025
2,325	2025-11-16	28	True	test 2025
2,350	2025-11-23	14	True	test 2025
2,375	2025-11-27	18	True	test 2025
2,400	2025-11-30	21	True	test 2025
2,425	2025-12-07	28	True	test 2025
2,450	2025-12-14	28	True	test 2025
2,475	2025-12-17	28	True	test 2025

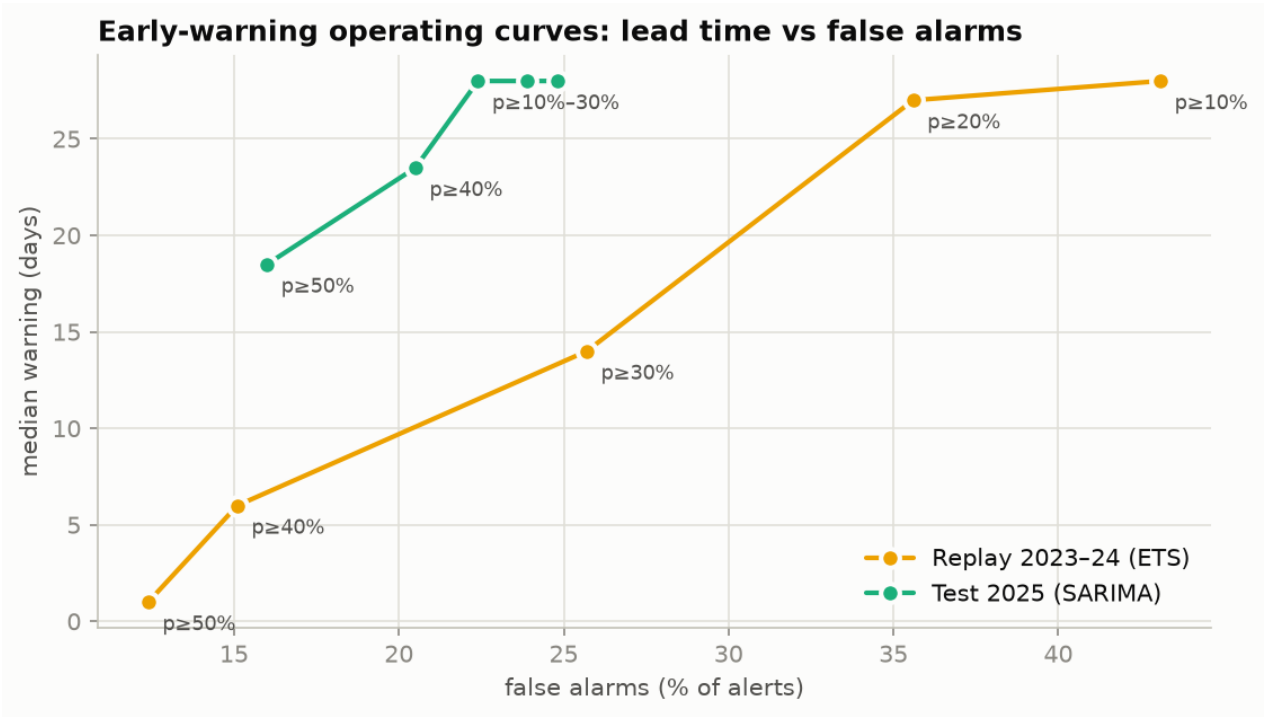


Figure F4. Early-warning operating curve

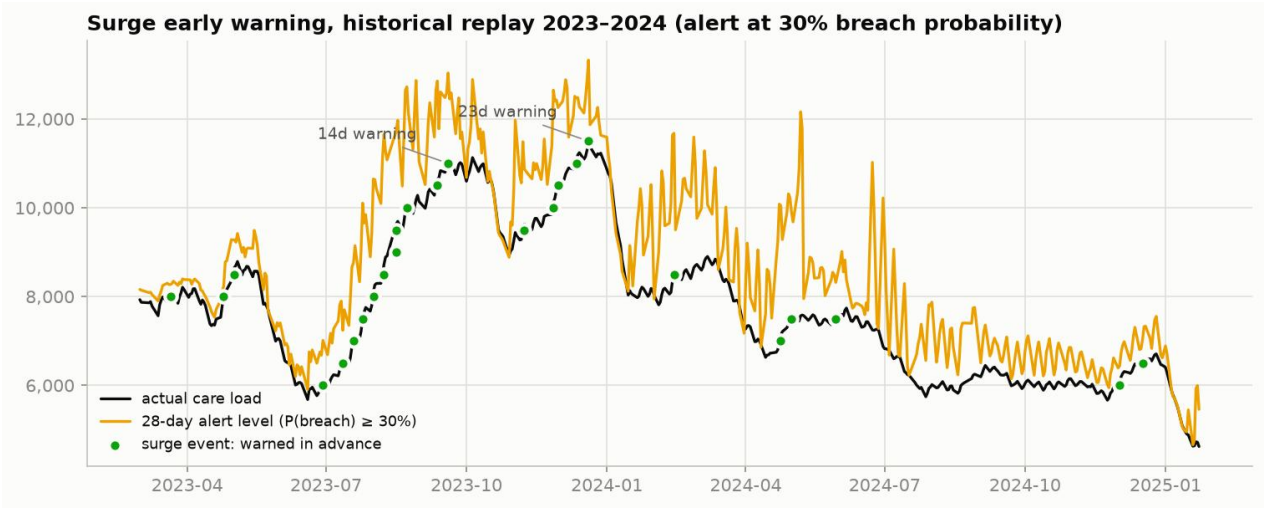


Figure F5. Surge replay (notebook version)

F.7 Production forecast and scenarios

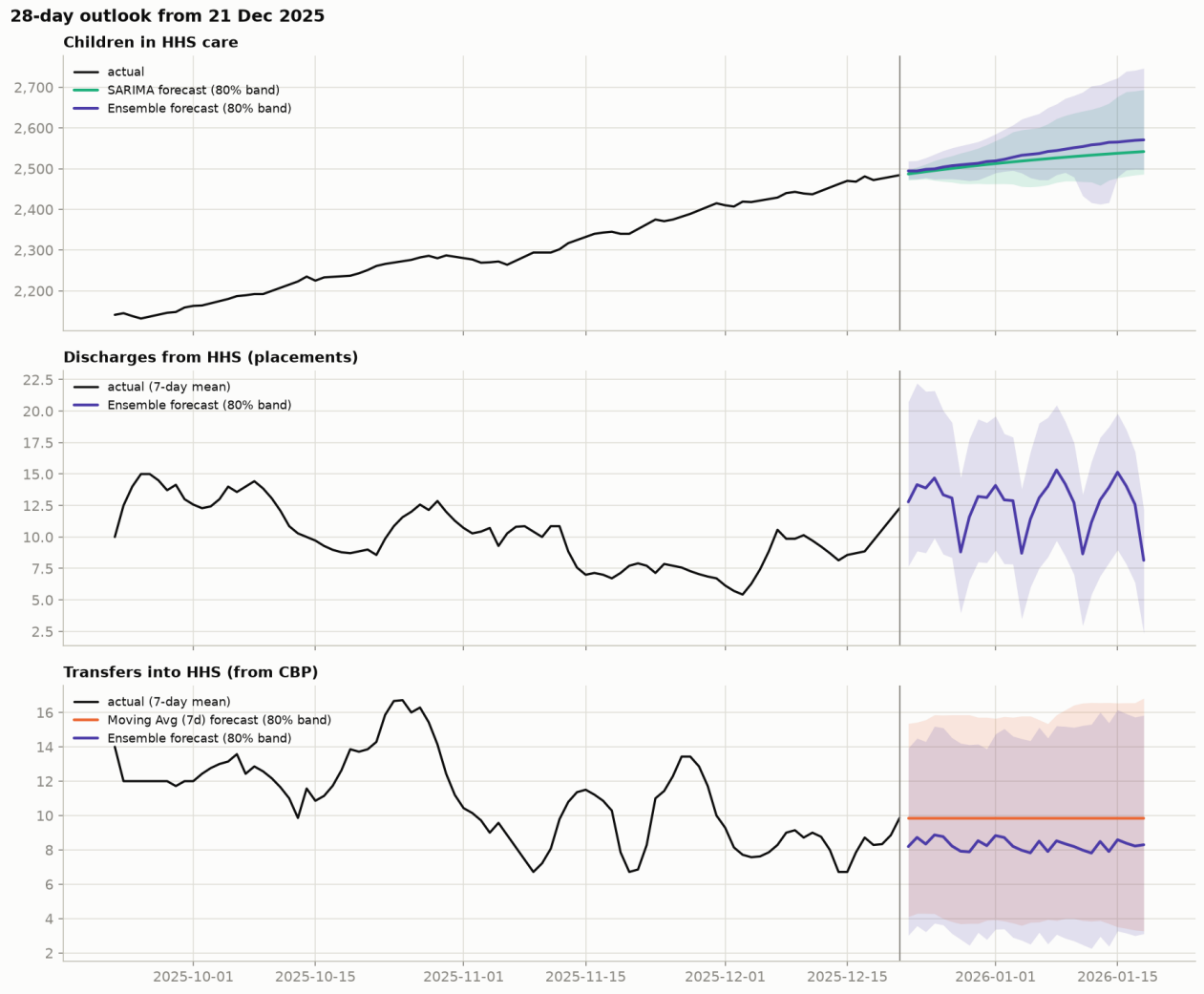


Figure F6. Production forecast from 21 December 2025

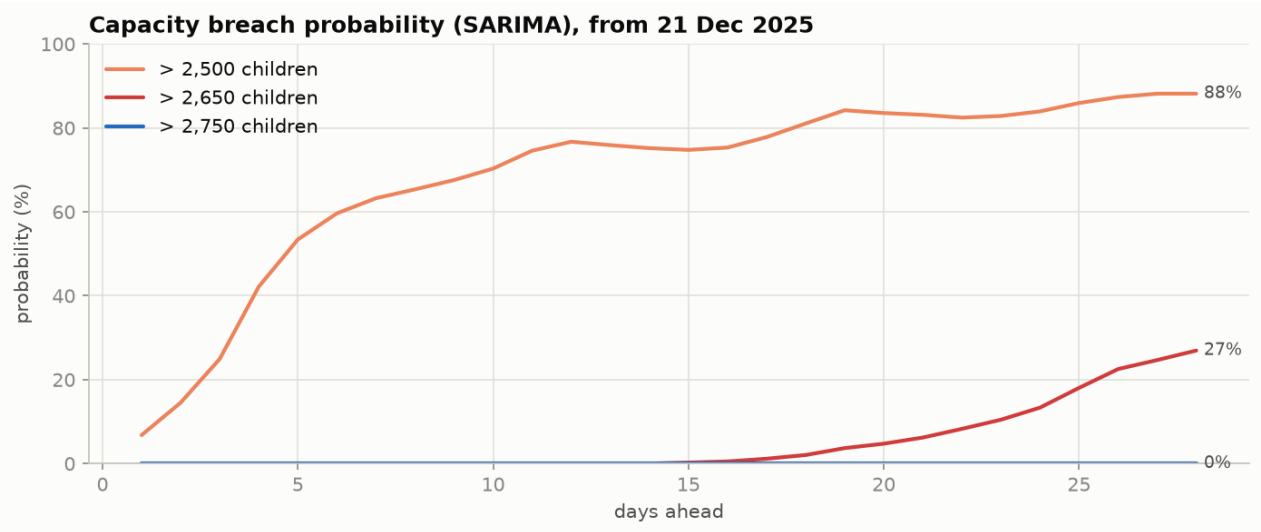


Figure F7. Capacity breach probability

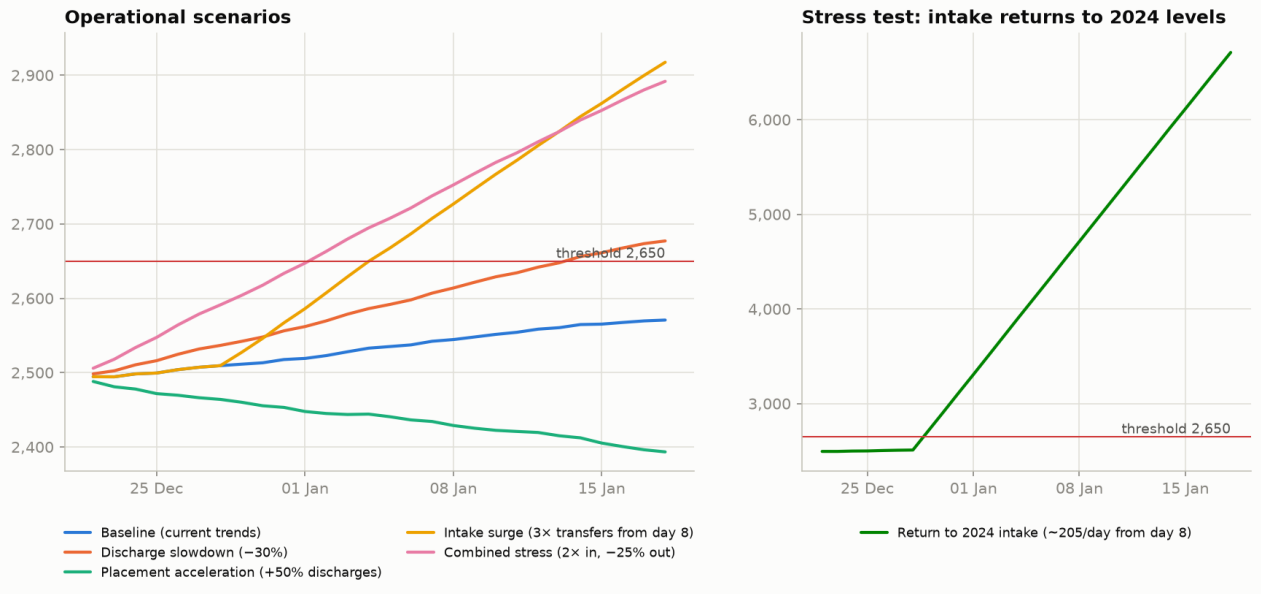


Figure F8. Scenario comparison (notebook version)

F.8 Pipeline analytics

Table F10. Indicators by period

Period	Indicator	Reports evaluated	% Watch or Alert	% Alert	Median value	Max value	Unit
Before the break	HHS census growth (backlog building)	245	21.63	6.53	-0.86	16.44	% vs 28 days earlier
Before the break	Discharge slowdown (placement bottleneck)	245	10.61	0.41	2.21	33.62	% below 8-week median rate
Before the break	CBP holding time (transfer bottleneck)	243	0	0	1.31	1.85	days (custody / transfers, 7-day)
Current regime	HHS census growth (backlog building)	185	42.16	0	4.46	8.68	% vs 28 days earlier
Current regime	Discharge slowdown (placement bottleneck)	183	42.62	22.4	10.01	87.41	% below 8-week median rate
Current regime	CBP holding time (transfer bottleneck)	183	95.63	43.72	2.88	6.57	days (custody / transfers, 7-day)
Last 90 days	HHS census growth (backlog building)	61	63.93	0	5.88	8.25	% vs 28 days earlier
Last 90 days	Discharge slowdown (placement bottleneck)	61	44.26	21.31	13.28	46.79	% below 8-week median rate
Last 90 days	CBP holding time (transfer bottleneck)	61	100	60.66	3.19	6.57	days (custody / transfers, 7-day)

Table F11. Share of reported days by overall status and year (%)

Year	Normal	Watch	Alert	Insufficient data	HHS census growth (backlog building) at Watch or Alert	Discharge slowdown (placement bottleneck) at Watch or Alert	CBP holding time (transfer bottleneck) at Watch or Alert
2023	45.7	14.3	33.9	6.1	50	5.3	n/a
2024	70.6	22.6	6.9	0	21.4	10.5	0
2025	16	30.4	53.6	0	32.9	41.7	79.6

Table F12. Every Watch and Alert episode, full history

Indicator	Start	End	Reported days	Calendar days	Highest level	Peak value	Unit
HHS census growth (backlog building)	2023-02-09	2023-02-22	9	14	Alert	20.44	% vs 28 days earlier
HHS census growth (backlog building)	2023-04-06	2023-04-06	1	1	Watch	5.03	% vs 28 days earlier
HHS census growth (backlog building)	2023-05-01	2023-05-18	14	18	Alert	16.71	% vs 28 days earlier
HHS census growth (backlog building)	2023-07-10	2023-09-28	55	81	Alert	37.61	% vs 28 days earlier
HHS census growth (backlog building)	2023-10-02	2023-10-05	4	4	Watch	7.37	% vs 28 days earlier
HHS census growth (backlog building)	2023-11-23	2023-12-28	25	36	Alert	17.46	% vs 28 days earlier
HHS census growth (backlog building)	2024-02-14	2024-02-15	2	2	Watch	5.45	% vs 28 days earlier

Indicator	Start	End	Reported days	Calendar days	Highest level	Peak value	Unit
HHS census growth (backlog building)	2024-02-22	2024-02-22	1	1	Watch	5.22	% vs 28 days earlier
HHS census growth (backlog building)	2024-02-27	2024-03-12	11	15	Alert	10.01	% vs 28 days earlier
HHS census growth (backlog building)	2024-05-05	2024-05-21	13	17	Alert	12.88	% vs 28 days earlier
HHS census growth (backlog building)	2024-09-04	2024-09-16	10	13	Watch	7.89	% vs 28 days earlier
HHS census growth (backlog building)	2024-12-09	2024-12-30	16	22	Alert	16.44	% vs 28 days earlier
HHS census growth (backlog building)	2025-04-07	2025-04-29	18	23	Watch	8.39	% vs 28 days earlier
HHS census growth (backlog building)	2025-05-15	2025-06-09	15	26	Watch	8.68	% vs 28 days earlier
HHS census growth (backlog building)	2025-09-15	2025-10-16	23	32	Watch	8.25	% vs 28 days earlier
HHS census growth (backlog building)	2025-10-22	2025-10-30	7	9	Watch	6.29	% vs 28 days earlier
HHS census growth (backlog building)	2025-11-30	2025-12-18	15	19	Watch	6.8	% vs 28 days earlier
Discharge slowdown (placement bottleneck)	2023-06-22	2023-06-22	1	1	Watch	21.68	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2023-07-09	2023-07-12	4	4	Watch	20.26	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2023-10-12	2023-10-12	1	1	Watch	16.84	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2023-11-27	2023-12-03	5	7	Watch	24.24	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-02-19	2024-02-19	1	1	Watch	17.08	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-02-22	2024-02-22	1	1	Watch	15.66	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-04-29	2024-05-02	4	4	Watch	18.27	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-05-27	2024-05-27	1	1	Watch	15.08	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-05-29	2024-05-30	2	2	Watch	25.32	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-07-11	2024-07-11	1	1	Watch	16.54	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-08-04	2024-08-06	3	3	Watch	20.39	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-08-27	2024-09-10	10	15	Alert	33.62	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2024-12-04	2024-12-08	3	5	Watch	18.98	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-02-17	2025-04-20	45	63	Alert	87.41	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-07-31	2025-08-07	6	8	Watch	20.97	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-08-17	2025-08-25	7	9	Alert	32.78	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-09-03	2025-09-09	5	7	Alert	36.41	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-09-11	2025-09-22	8	12	Watch	29.23	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-10-15	2025-10-23	7	9	Alert	31.24	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-11-05	2025-11-05	1	1	Watch	20.95	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-11-13	2025-12-04	15	22	Alert	46.79	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-12-08	2025-12-09	2	2	Watch	17.41	% below 8-week median rate
Discharge slowdown (placement bottleneck)	2025-12-14	2025-12-15	2	2	Watch	21.9	% below 8-week median rate
CBP holding time (transfer bottleneck)	2025-02-26	2025-02-26	1	1	Watch	2.02	days (custody / transfers, 7-day)
CBP holding time (transfer bottleneck)	2025-03-02	2025-03-24	17	23	Alert	3.8	days (custody / transfers, 7-day)
CBP holding time (transfer bottleneck)	2025-03-31	2025-05-08	30	39	Alert	3.11	days (custody / transfers, 7-day)
CBP holding time (transfer bottleneck)	2025-05-15	2025-06-08	14	25	Alert	3.3	days (custody / transfers, 7-day)

Indicator	Start	End	Reported days	Calendar days	Highest level	Peak value	Unit
CBP holding time (transfer bottleneck)	2025-06-10	2025-06-10	1	1	Watch	2.14	days (custody / transfers, 7-day)
CBP holding time (transfer bottleneck)	2025-06-22	2025-08-28	48	68	Alert	5.38	days (custody / transfers, 7-day)
CBP holding time (transfer bottleneck)	2025-09-02	2025-12-21	76	111	Alert	6.57	days (custody / transfers, 7-day)

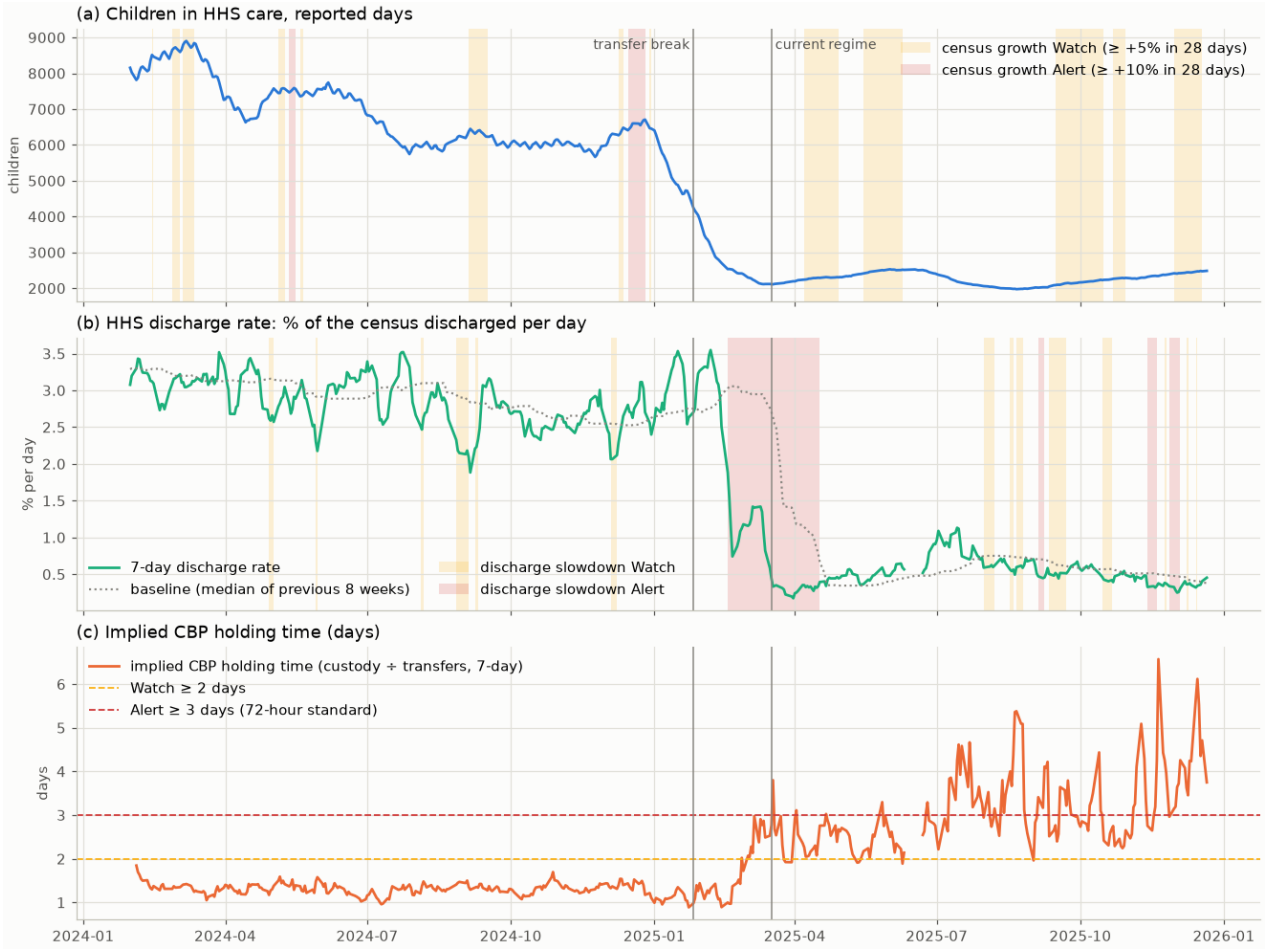


Figure F9. Pipeline monitoring (notebook version)

Appendix G. Software Validation and Reproducibility

This appendix explains how the work was tested and how to reproduce it.

G.1 Software validation

The notebook executes top to bottom in a clean kernel (205 cells, 0 errors, 8–26 minutes depending on machine load). An automated suite, organised into eight pytest categories (data validation, preprocessing and features, model units, forecast and metrics, integration, Streamlit application, end-to-end workflow, performance and reliability), executes the notebook's own code and the real dashboard script, including edge cases on synthetic files (empty, malformed, constant, short, weekly, out-of-range inputs; failing and invalid models; missing dependencies; corrupted artifacts). The Streamlit dashboard was also exercised in a real browser. Pass/fail counts, open defects and evidence are in `reports/Test_Report_v2.md`; this paper does not restate them, to keep a single source of truth.

A fast pytest suite in `tests/` (`python -m pytest`) covers the descriptive pipeline layer of Section 8 of the paper:

- data ingestion and validation, including a cell-by-cell reconciliation of the cleaned table with the raw export;
- KPI and aggregation definitions, checked against hand-computed examples and an independent recomputation from the raw file;

- the alert rules, at their threshold boundaries, with a no-look-ahead truncation test;
- the dashboard's controls, empty and invalid ranges, file handling and a real server start-up;
- the consistency of this paper, the executive summary and the README with the generated values.

Results are in `reports/Test_Report_v2.1.md`.

A separate end-to-end QA run is reported in reports/QA_Test_Execution_Report_v2.3.pdf.

G.2 Where everything is

Table G1. Reproducibility: where each result lives

Item	Location
Full executed analysis	`UAC_Care_Load_Forecasting.ipynb` (8–26 minutes in a clean kernel; deterministic seeds; editable source `notebook_source/UAC_Care_Load_Forecasting_v2.py`)
Interactive dashboard	`python -m streamlit run uac_forecast_dashboard.py`
Clean daily data with masks	`artifacts/daily_clean.csv`
Walk-forward forecasts (scored)	`artifacts/backtest.csv.gz`
Metrics, coverage, robustness	`artifacts/metrics_overall.csv`, `metrics_by_horizon.csv`, `interval_coverage.csv`, `interval_coverage_by_h.csv`, `interval_method_comparison.csv`, `robustness.csv`
Validation and audit	`artifacts/validation_report.csv`, `model_fit_log.csv`, `feature_leakage_audit.csv`
Production forecasts with intervals	`artifacts/forecast_future.csv`
Early-warning results	`artifacts/surge_lead_times.csv`, `ews_operating_curve.csv`, `surge_replay_paths.csv`
Run metadata	`artifacts/metadata.json`
Tests and test report	`QA_Testing/pytest_suite/`, `reports/Test_Report_v2.md`
Pipeline analytics (Section 8 of the paper)	`src/` (functions), `python tools/pipeline_analysis.py` (regenerates `reports/pipeline_*.csv`, `reports/pipeline_analysis.md` and the notebook-size pipeline chart)
Pipeline and consistency tests	`python -m pytest` (suite in `tests/`), results in `reports/Test_Report_v2.1.md`

Table G2. Rebuilding this paper

Item	Command
Main-body charts	`python tools/paper_figures.py`
This Appendix volume (Markdown)	`python tools/build_paper_appendix.py`
Word and PDF of both documents, then all quality checks	`python tools/build_paper.py`
Quality checks only	`python tools/check_paper.py`

Appendix H. Raw Source Code

This appendix reproduces all source code as it stood when this volume was built. Long lines wrap.

H.1 Analysis notebook source: notebook_source/UAC_Care_Load_Forecasting_v2.py

```
# %% [markdown]
# # Predictive Forecasting of Care Load & Placement Demand
# ### HHS Unaccompanied Alien Children (UAC) Program: from historical reporting to predictive intelligence
#
# **Unified Mentor Data Science Internship, October 2026** · notebook version 2.0 (validated and extended; the v1 baseline is preserved in `archive/v1_baseline_2026-10-05/`, every change is listed in `CHANGELOG.md`)
#
# | Planning question | What this notebook delivers |
# |---|---|
# | How many children will be in HHS care in the coming days or weeks? | 1–28 day care-load forecasts with calibrated 80% / 95% intervals |
# | Will discharges keep pace with incoming transfers? | Discharge and transfer forecasts, plus the net-pressure (imbalance) outlook |
# | When should shelters, medical staff and caseworkers be scaled up? | Capacity-breach probabilities, surge lead time, and what-if scenarios |
#
# **Roadmap**
# 1. Setup & configuration
# 2. Load the raw data, with **schema validation** (clear errors for malformed files)
# 3. Clean & type the data, with **plausibility validation** (impossible values quarantined, never zero-filled)
# 4. **Root-cause analysis (RCA) of data quality**: cadence, stale values, copied records, stock-flow identity, variable definitions
# 5. Time-series preparation: daily continuity, interpolation **and** masking
# 6. Exploratory analysis: regime break, length of stay, seasonality, interpolation check, monthly pattern, outliers, correlations, decomposition, stationarity
# 7. Feature engineering: lags, rolling mean/variance, net pressure, calendar and holiday effects, **per-feature leakage audit**
# 8. Train–test design: strict time order, walk-forward, multi-horizon, leakage guards, origin-time checks
# 9. Forecasting models: naive, seasonal naive, moving average, ARIMA/SARIMA, ETS, Random Forest, Gradient Boosting, Ensemble (model cards and fallback)
# 10. Walk-forward backtest
# 11. Evaluation: MAE, RMSE, MAPE, WAPE, bias, horizon error, statistical vs ML, ARIMA vs SARIMA, recursive vs direct
# 12. Uncertainty quantification: conformal intervals, coverage by horizon, rolling-origin recalibration
# 13. KPIs: accuracy, surge lead time, breach probability, stability, robustness
# 14. Production forecast to mid-January 2026
# 15. Scenario analysis (stock-flow what-ifs)
# 16. Export artifacts for the Streamlit dashboard
# 17. Insights & recommendations
#
# **How to read this notebook:** every chart, table and printed result is followed by an **Observation** cell that states the key finding in plain language, with the supporting numbers.


# %% [markdown]
# ### Brainstorm: what can go wrong, and how this notebook guards against it
#
# Before writing any model I listed the ways a forecast for this program could quietly fail. Every guard below is implemented and checked later in the notebook.
#
# | Risk | Why it matters | Guard used here |
# |---|---|---|
# | Malformed or implausible input on a refresh | A broken export or a typo (negative count, extra digit) silently drives forecasts and alerts | Schema validation with clear errors; impossible values quarantined to *missing* and listed |
# | Irregular reporting (no Friday/Saturday reports) | Breaks daily continuity; naive interpolation leaks *future* values into features | Daily grid + time interpolation **plus** an `is_reported` mask; forecast origins and scored targets are reported days only |
# | Copied or stale records | Silent corruption teaches models patterns that never happened | Duplicate-record audit + **stock-flow identity** test (care load must reconcile with in/out flows) |
# | Structural break (2025 policy shift) | A model fitted on 2023–24 volumes forecasts a world that no longer exists | Change-point detection, regime-aware training windows, scale-free ML targets |
# | Small counts and zero days | MAPE explodes when actuals are near 0 | WAPE-based accuracy KPI; MAPE computed on non-zero actuals |
```

```
# | Over-optimistic validation | Random splits leak the future into training | Strict time split, walk-forward origins, model selection on a calibration window, reporting on an untouched
      test window |
# | Point forecasts treated as certain | Capacity is planned on the tail, not the mean | Horizon-specific conformal intervals with measured coverage, breach probabilities |


# %% [markdown]
# ## 1. Setup & configuration


# %%
# Core libraries for data handling, statistics, machine learning and plotting
import json, os, re, time, warnings
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.ticker as mticker
from IPython.display import display
from pandas.tseries.holiday import USFederalHolidayCalendar
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.exponential_smoothing.ets import ETSModel
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from scipy import stats
warnings.filterwarnings("ignore")
pd.set_option("display.width", 160); pd.set_option("display.max_columns", 30)


# %%
# Project paths (portable): UAC_BASE_DIR if set, else the first folder (working directory or a parent) that holds the raw CSV
DATA_FILE = "HHS_Unaccompanied_Alien_Children_Program.csv"
_candidates = [Path(os.environ["UAC_BASE_DIR"])] if os.environ.get("UAC_BASE_DIR") else [Path.cwd(), *Path.cwd().parents]
BASE_DIR = next((p for p in _candidates if (p / DATA_FILE).exists()), Path.cwd())
DATA_PATH = BASE_DIR / DATA_FILE
ART_DIR, FIG_DIR = BASE_DIR / "artifacts", BASE_DIR / "figures"
ART_DIR.mkdir(exist_ok=True); FIG_DIR.mkdir(exist_ok=True)
print(f"Project folder: {BASE_DIR.name} | data file found: {DATA_PATH.exists()}")


# %% [markdown]
# **Observation:** The project folder was found from the working directory and the data file is present, so the notebook no longer depends on a personal absolute path; setting
      `UAC_BASE_DIR` points it at another folder that holds the same CSV.


# %%
# Global settings: forecast horizon (days), random seed, and the three series we forecast
H = 28                     # 4-week planning horizon = four full weekly cycles (30 days would add a partial week and is not validated)
SEED = 42
TARGETS = {"hhs_care": "stock", "discharged": "flow", "transferred": "flow"}   # stock = level, flow = daily count
LABELS = {"hhs_care": "Children in HHS care", "discharged": "Discharges from HHS (placements)",
          "transferred": "Transfers into HHS (from CBP)", "apprehended": "Apprehended (CBP intake)",
          "cbp_custody": "Children in CBP custody"}


# %%
# Plot style: quiet chrome, hairline grids, and one fixed colour per entity (colour follows the series/model, never its rank)
INK, INK2, MUTED, GRID, BASE, SURFACE = "#0b0b0b", "#52514e", "#898781", "#e1e0d9", "#c3c2b7", "#fcfcfb"
plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "axes.edgecolor": BASE,
```

```
                    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8, "axes.axisbelow": True,
                    "axes.spines.top": False, "axes.spines.right": False, "axes.labelcolor": INK2,
                    "xtick.color": MUTED, "ytick.color": MUTED, "text.color": INK, "axes.titlecolor": INK,
                    "axes.titleweight": "bold", "axes.titlesize": 12, "axes.titlelocation": "left",
                    "lines.linewidth": 2, "lines.solid_capstyle": "round", "font.size": 10,
                    "legend.frameon": False, "figure.dpi": 100, "savefig.dpi": 150, "savefig.bbox": "tight"})
SERIES_COLORS = {"hhs_care": "#2a78d6", "transferred": "#eb6834", "discharged": "#1baf7a",
                 "apprehended": "#eda100", "cbp_custody": "#e87ba4"}
MODEL_COLORS = {"Naive": "#2a78d6", "Seasonal Naive (7d)": "#8a5a2b", "Moving Avg (7d)": "#eb6834", "SARIMA": "#1baf7a", "ETS": "#eda100",
                "Random Forest": "#e87ba4", "Gradient Boosting": "#008300", "Ensemble": "#4a3aa7"}
STATUS = {"good": "#0ca30c", "warning": "#fab219", "serious": "#ec835a", "critical": "#d03b3b"}
COMMA = mticker.StrMethodFormatter("{x:,.0f}")



def savefig(fig, name):
    """Save a figure into figures/ so the research paper can reuse it."""
    fig.savefig(FIG_DIR / f"{name}.png")
    plt.show()

# %% [markdown]
# ## 2. Load the raw data

# %%
# Read the CSV; thousands="," turns quoted values such as "2,484" into numbers (a missing file stops with a clear message)
if not DATA_PATH.exists():
    raise FileNotFoundError(f"Input file not found: {DATA_PATH.name} in {DATA_PATH.parent}. Place {DATA_FILE} in the project folder or set UAC_BASE_DIR.")
raw = pd.read_csv(DATA_PATH, thousands=",")
print("Raw shape:", raw.shape)
raw.head()

# %% [markdown]
# **Observation:** The raw file loads as 1,170 rows × 6 columns, newest report first (21 Dec 2025: 2,484 children in HHS care, 14 discharges). The thousand separators parsed correctly,
     so all five measures arrive as numbers rather than text.

# %%
# Audit the raw file: data types, fully blank rows (export padding), and missing cells
print(raw.dtypes, "\n")
print("Fully blank rows:", int(raw.isna().all(axis=1).sum()))
print("Rows with any value:", int(raw.notna().any(axis=1).sum()))

# %% [markdown]
# **Observation:** Only 720 of the 1,170 rows (62%) contain data; the other 450 are empty export padding and must be dropped. Dates are stored as text such as "December 21, 2025", rows
     run newest-first, and one header carries a stray "*", so the file needs cleaning before any time-series work.

# %% [markdown]
# ### 2.1 Schema validation
#
# A refreshed export can arrive broken: a missing column, a header-only file, text in a count column, a different date format. The checks below stop the run **early with a message that
     names the column, file line and value**, instead of a raw pandas error several sections later. Policy:
#
# | Check | Rule | On failure |
# |---|---|---|
# | Columns | 6 columns in the documented order (matched by a key phrase per position); extra trailing columns are ignored and reported | stop |
# | Rows | at least one data row; at least 365 reports (training + two 13-week evaluation windows) | stop |
```

```
# | Dates | documented format ("December 21, 2025"); ISO `YYYY-MM-DD` accepted as a reported fallback; timestamps or other text rejected | stop |
# | Numbers | thousands separators and spaces allowed; blank / `N/A` = missing; any other text rejected | stop |
# | Columns with no values, constant targets, non-daily cadence (median gap > 3 days) | nothing meaningful to forecast | stop |


# %%
# Schema validation: stop early with a clear message naming the column, file line and value
class DataValidationError(ValueError):
    """The input file cannot be used safely; the message says what to fix."""



COLS = ["date", "apprehended", "cbp_custody", "transferred", "hhs_care", "discharged"]
CBP_COLS = ["apprehended", "cbp_custody", "transferred"]
EXPECTED_HEADER_KEYS = ["date", "apprehended", "in cbp custody", "transferred", "in hhs care", "discharged"]   # one key phrase per position
MISSING_MARKERS = {"", "NAN", "N/A", "NA", "NULL", "NONE"}
MIN_REPORTS = 365



def validate_schema(frame):
    """Check columns, rows, dates and numbers; return (typed frame with snake_case names, list of notes)."""
    notes = []
    if frame.shape[1] < len(COLS):
        raise DataValidationError(f"Expected {len(COLS)} columns, found {frame.shape[1]}: {list(frame.columns)}")
    heads = [str(c).replace("", "").replace("*", "").strip().lower() for c in frame.columns]
    for i, key in enumerate(EXPECTED_HEADER_KEYS):
        if key not in heads[i]:
            raise DataValidationError(f"Column {i + 1} is '{frame.columns[i]}'; expected a header containing '{key}'")
    if frame.shape[1] > len(COLS):
        notes.append(f"ignored extra column(s) {list(frame.columns[len(COLS):])}")
    data = frame.iloc[:, :len(COLS)].dropna(how="all").set_axis(COLS, axis=1).copy()
    if data.empty:
        raise DataValidationError("The file has a header but no data rows")
    for c in COLS:
        if data[c].isna().all():
            raise DataValidationError(f"Column '{c}' is entirely empty")
    txt = data["date"].astype(str).str.strip()
    primary = pd.to_datetime(txt, format="%B %d, %Y", errors="coerce")
    iso = pd.to_datetime(txt, format="%Y-%m-%d", errors="coerce")
    if (primary.isna() & iso.notna()).any():
        notes.append(f"{int((primary.isna() & iso.notna()).sum())} date(s) in ISO format accepted")
    data["date"] = primary.fillna(iso)
    if data["date"].isna().any():
        bad = [(int(i) + 2, txt[i]) for i in data.index[data["date"].isna()][:5]]          # file line = index + 2 (header is line 1)
        raise DataValidationError(f"{int(data['date'].isna().sum())} unparseable date(s), e.g. (file line, value): {bad}")
    for c in COLS[1:]:
        if data[c].dtype == object:
            s = data[c].astype(str).str.replace(",", "", regex=False).str.strip()
            num = pd.to_numeric(s, errors="coerce")
            text = num.isna() & data[c].notna() & ~s.str.upper().isin(MISSING_MARKERS)
            if text.any():
                bad = [(int(i) + 2, data.at[i, c]) for i in data.index[text][:5]]
                raise DataValidationError(f"Column '{c}' has {int(text.sum())} non-numeric value(s), e.g. (file line, value): {bad}")
            data[c] = num
    data[COLS[1:]] = data[COLS[1:]].astype(float)
    if len(data) < MIN_REPORTS:
        raise DataValidationError(f"Only {len(data)} reports; at least {MIN_REPORTS} are needed for training plus the calibration and test windows")
    median_gap = data["date"].sort_values().diff().dt.days.median()
```

```
    if median_gap > 3:
        raise DataValidationError(f"Median gap between reports is {median_gap:.0f} days: this is not a daily series")
    for c in TARGETS:
        if data[c].nunique() <= 1:
            raise DataValidationError(f"Target '{c}' is constant ({data[c].dropna().iloc[0]:g}): nothing to forecast; check the export")
    return data, notes


# %% [markdown]
# ## 3. Clean & type the data


# %%
# Validated, typed table with short snake_case names (the original meaning is kept in LABELS)
df, SCHEMA_NOTES = validate_schema(raw)
print(f"Schema valid: {len(df)} data rows × {df.shape[1]} columns | notes: {SCHEMA_NOTES or 'none'}")


# %% [markdown]
# **Observation:** The source file passes every schema check unchanged: 720 data rows × 6 columns, every date in the documented "Month DD, YYYY" format, every count numeric, and no notes
      (no extra columns, no ISO dates). The same checks stop a malformed refresh with a readable message; the negative cases (missing or renamed column, header-only file, text in a count,
      invalid dates, time-zone timestamps, empty or constant columns, short or weekly histories) are exercised on synthetic files in `QA_Testing/`.


# %%
# Sort oldest to newest (stable sort), resolve duplicate dates, and use the date as the index
dup_rows = df[df["date"].duplicated(keep=False)]
DUP_CONFLICTS = dup_rows.groupby("date").filter(lambda g: len(g.drop_duplicates()) > 1)      # same date, different values
if len(DUP_CONFLICTS):
    print("Conflicting records for the same date (the first record in file order is kept):"); display(DUP_CONFLICTS)
n_dupes = int(df["date"].duplicated().sum())
df = df.sort_values("date", kind="stable").drop_duplicates("date", keep="first").set_index("date").astype(float)
print(f"Duplicate dates removed: {n_dupes} | Rows: {len(df)} | Range: {df.index.min():%d %b %Y} → {df.index.max():%d %b %Y}")
df.describe().T.round(1)


# %% [markdown]
# **Observation:** The clean table has 720 daily reports from 12 Jan 2023 to 21 Dec 2025, with no duplicate dates. Care load ranged almost 6-fold, from 1,972 to 11,516 children, and its
      standard deviation (2,833) is almost half its mean (6,061). The data therefore span very different operating levels, which the regime analysis below confirms.


# %% [markdown]
# ### 3.1 Plausibility validation
#
# Schema-valid numbers can still be impossible. Values that cannot be true are **quarantined**: set to *missing* (so they are re-estimated by interpolation and never scored) and listed,
      never replaced by zero. Values that are only unusual are **flagged for review** and left unchanged.
#
# | Rule | Applies to | Action |
# |---|---|---|
# | Negative count | all five columns | quarantine |
# | Single-report stock spike: care load more than 1.5× (or less than 1/1.5) the median of the 3 reports before and after | `hhs_care` | quarantine |
# | Non-integer count | all five columns | flag |
# | Flow spike: more than 5× the local 15-report median and more than 50 above it | flows and CBP custody | flag |
#
# On the real data the largest care-load deviation from its neighbours is ×1.11 and the largest flow ratio is 3.2×, so these limits sit far outside normal variation.


# %%
# Plausibility validation: impossible values are quarantined (set to missing and listed); unusual values are flagged only
def validate_values(frame, stock_ratio=1.5, flow_ratio=5.0, flow_abs=50):
    out, issues = frame.copy(), []
    def log_issue(mask, col, rule, action):
```

```
        issues.extend({"date": d, "column": col, "value": frame.at[d, col], "rule": rule, "action": action} for d in out.index[mask])
    for c in out.columns:
        neg = out[c] < 0
        log_issue(neg, c, "negative count", "quarantined (set to missing)")
        out.loc[neg, c] = np.nan
        log_issue(out[c].notna() & (out[c] % 1 != 0), c, "non-integer count", "flagged for review")
    care = out["hhs_care"]
    ref = pd.concat([care.shift(k) for k in (1, 2, 3, -1, -2, -3)], axis=1).median(axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        spike = (np.abs(np.log(care / ref)) > np.log(stock_ratio)) & care.notna() & ref.notna()
    log_issue(spike, "hhs_care", f"stock spike (> {stock_ratio}x or < 1/{stock_ratio}x neighbouring reports)", "quarantined (set to missing)")
    out.loc[spike, "hhs_care"] = np.nan
    for c in ("apprehended", "cbp_custody", "transferred", "discharged"):
        local = out[c].rolling(15, center=True, min_periods=5).median()
        log_issue((out[c] > flow_ratio * local) & (out[c] - local > flow_abs), c, f"flow spike (> {flow_ratio:g}x local median)", "flagged for review")
    return out, pd.DataFrame(issues, columns=["date", "column", "value", "rule", "action"])



df, VALIDATION_ISSUES = validate_values(df)
print(f"Plausibility issues: {len(VALIDATION_ISSUES)} (quarantined: {VALIDATION_ISSUES['action'].str.startswith('quarantined').sum()})")
VALIDATION_ISSUES if len(VALIDATION_ISSUES) else print("No implausible values: the source data pass every rule unchanged.")


# %% [markdown]
# **Observation:** No value in the source data breaks a plausibility rule: no negative or fractional count, no single-report care-load spike and no flow spike, so nothing is quarantined
      and every later result is identical to the unvalidated pipeline. The rules protect future refreshes: in the synthetic tests a −5 discharge count and a 99,999 care load are set to
      missing and listed, while a genuine, lasting +40% level shift is left untouched.


# %% [markdown]
# ## 4. Root-cause analysis of data quality
#
# The brief calls this "high-quality daily data". Before trusting it I tested four things: reporting cadence, stale values, duplicated records, and whether the stock and flow columns
      reconcile.


# %% [markdown]
# ### 4.1 Reporting cadence: which days are actually reported?


# %%
# Count reports per weekday and the gaps (in days) between consecutive reports
dow_counts = df.index.day_name().value_counts().reindex(["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"], fill_value=0)
gaps = df.index.to_series().diff().dt.days
print("Reports per weekday:\n", dow_counts.to_string(), "\n")
print("Gap between reports (days):\n", gaps.value_counts().sort_index().to_string())


# %% [markdown]
# **Observation:** Monday–Thursday and Sunday each have 130–149 reports, while Friday has only 2 and Saturday none. 77% of gaps between reports are 1 day, but 17% are 3-day
      Thursday→Sunday jumps, so weekend values are missing by design, not at random.


# %%
# Visualise the cadence: weekday coverage and the gap distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 3.6))
axes[0].bar(dow_counts.index.str[:3], dow_counts.values, width=0.55, color=SERIES_COLORS["hhs_care"])
axes[0].set_title("Reports by weekday: Friday and Saturday are almost never reported")
gap_vc = gaps.value_counts().sort_index()
axes[1].bar(gap_vc.index.astype(int).astype(str), gap_vc.values, width=0.55, color=SERIES_COLORS["hhs_care"])
```

```
axes[1].set_title("Days between consecutive reports"); axes[1].set_xlabel("gap (days)")
for ax in axes: ax.set_ylabel("count")
savefig(fig, "01_reporting_cadence")


# %% [markdown]
# **Observation:** The chart confirms a fixed Sunday–Thursday reporting cycle: about a third of calendar days (355 of 1,075) have no report. Because the gaps are structural, interpolated
      weekend values are estimates only. They keep the series continuous for features, but they are masked from every forecast score.


# %% [markdown]
# ### 4.2 Stale (carried-forward) values


# %%
# A report whose care load AND discharges exactly repeat the previous report is a likely copy-forward
stale_mask = (df["hhs_care"].diff() == 0) & (df["discharged"].diff() == 0)
print("Stale HHS reports:", int(stale_mask.sum()))
df.loc[stale_mask | stale_mask.shift(-1, fill_value=False), ["hhs_care", "discharged", "transferred"]]


# %% [markdown]
# **Observation:** 5 reports repeat the previous day's care load and discharges exactly (e.g., 8,404 children and 329 discharges on both 25 and 26 Feb 2024), while their CBP transfers
      changed (366 vs 326). A double exact repeat alongside changing transfers points to a carry-forward rather than a fresh report, so these values are re-estimated by interpolation and
      never used as a scored target.


# %%
# Treat stale HHS values as missing so they are re-estimated by interpolation and never used as a scored target
df.loc[stale_mask, ["hhs_care", "discharged"]] = np.nan


# %% [markdown]
# ### 4.3 Duplicated CBP records: the key root-cause finding


# %%
# Each CBP row is a triple (apprehended, in custody, transferred); identical triples on different dates are extremely unlikely
cbp_key = df[CBP_COLS].astype("Int64").astype(str).agg("-".join, axis=1)
repeats = cbp_key.map(cbp_key.value_counts())
print(f"Rows whose CBP triple appears on 2+ different dates: {(repeats > 1).sum()} of {len(df)}")
cbp_key.value_counts().head(5)


# %% [markdown]
# **Observation:** 374 of 720 reports (52%) share an identical (apprehended, in custody, transferred) triple with another date, and five triples appear five times each. Exact three-
      number matches at these volumes are very unlikely by chance, so this is strong evidence of copied records.


# %%
# Flag the earlier copy of every repeated triple; the longest contiguous run marks the corrupted window
is_copy = cbp_key.duplicated(keep="last")
run_id = (is_copy != is_copy.shift()).cumsum()
runs = is_copy.groupby(run_id).agg(["first", "size"])
MIN_COPY_RUN = 14                                                   # isolated chance repeats at low counts are not a copied block
if runs["first"].any() and runs.loc[runs["first"], "size"].max() >= MIN_COPY_RUN:
    longest = runs[runs["first"]]["size"].idxmax()
    CBP_BAD_START, CBP_BAD_END = is_copy[run_id == longest].index.min(), is_copy[run_id == longest].index.max()
    N_COPIED = int(runs.loc[longest, "size"])
else:                                                               # clean refresh: no copied block, nothing to mask
    longest, N_COPIED = None, 0
    CBP_BAD_START = CBP_BAD_END = df.index.min() - pd.Timedelta(days=1)
```

```
print(f"Contiguous copied block: {CBP_BAD_START:%d %b %Y} → {CBP_BAD_END:%d %b %Y} ({N_COPIED} reports)")
print("Share of copied CBP rows by year:\n", is_copy.groupby(df.index.year).mean().round(2).to_string())


# %% [markdown]
# **Observation:** The copies form one unbroken block of 251 reports, from 12 Jan 2023 to 30 Jan 2024. 100% of 2023 CBP rows are copies, falling to 8% in 2024 (January only) and 0% in
      2025. The problem is confined to one period, so CBP data from February 2024 onward remain usable.


# %%
# Show one concrete example: the same triple reported on five different dates
example = cbp_key.value_counts().index[0]
df.loc[cbp_key == example, CBP_COLS + ["hhs_care", "discharged"]]


# %% [markdown]
# **Observation:** The same CBP triple (56, 82, 68) appears on five dates spread over 19 months, each time beside a completely different HHS care load (5,507 to 11,217). Identical intake
      numbers in such different situations confirm the CBP values were copied rather than reported.


# %% [markdown]
# ### 4.4 Proof via the stock-flow identity
#
# Care load is a **stock** and transfers/discharges are **flows**, so on consecutive days:
#
# $$\text{HHS}_t - \text{HHS}_{t-1} \;\approx\; \text{Transfers}_t - \text{Discharges}_t$$
#
# If a column is corrupted, this identity breaks. I test it separately for the copied block and the trusted period.


# %%
# Compute the identity residual on consecutive-day pairs only (multi-day gaps hide unreported flows)
sf = df.copy()
sf["gap"] = sf.index.to_series().diff().dt.days
sf["d_care"] = sf["hhs_care"].diff()
sf["net_flow"] = sf["transferred"] - sf["discharged"]
sf["residual"] = sf["d_care"] - sf["net_flow"]
sf["period"] = np.where(sf.index <= CBP_BAD_END, "Copied CBP block (2023–Jan 2024)", "Trusted (Feb 2024 →)")
pairs = sf[(sf["gap"] == 1) & sf["residual"].notna()]
identity = pairs.groupby("period").apply(lambda g: pd.Series({
    "pairs": len(g), "corr(ΔHHS, net flow)": g["d_care"].corr(g["net_flow"]),
    "mean residual / day": g["residual"].mean(), "median |residual|": g["residual"].abs().median()})).round(2)
identity


# %% [markdown]
# **Observation:** In the trusted period the daily change in care load tracks recorded transfers minus discharges closely (correlation 0.90, typical gap 6 children). In the copied block
      the match collapses (correlation 0.51) and about 190 children per day go unexplained. The HHS columns reconcile only where the CBP data are genuine, which pinpoints the CBP columns
      as the corrupted part.


# %%
# Scatter of daily care-load change vs recorded net flow: points should sit on the 45° line if the data reconcile
fig, axes = plt.subplots(1, 2, figsize=(12, 4.2), sharex=False)
for ax, (name, g) in zip(axes, pairs.groupby("period")):
    ax.scatter(g["net_flow"], g["d_care"], s=16, color=SERIES_COLORS["hhs_care"], alpha=0.6, edgecolor=SURFACE, linewidth=0.8)
    lim = np.nanpercentile(np.abs(np.r_[g["net_flow"], g["d_care"]]), 99)
    ax.plot([-lim, lim], [-lim, lim], color=MUTED, lw=1)
    ax.set_title(f"{name}\ncorr = {g['d_care'].corr(g['net_flow']):.2f}")
    ax.set_xlabel("recorded transfers − discharges"); ax.set_ylabel("actual change in HHS care")
savefig(fig, "02_stock_flow_identity")
```

```
# %% [markdown]
# **Observation:** Trusted-period points hug the 45° line, while copied-block points sit mostly above it: care load rose far more than the recorded flows allow. The visual gap matches
      the +190 children/day residual and shows that the copied transfers understate real admissions.


# %%
# Implied admissions = ΔHHS + discharges uses only HHS columns; compare with recorded CBP transfers by quarter
pairs = pairs.assign(implied_admissions=pairs["d_care"] + pairs["discharged"])
q = pairs.groupby(pairs.index.to_period("Q"))[["implied_admissions", "transferred"]].mean().round(0)
q["recorded / implied"] = (q["transferred"] / q["implied_admissions"]).round(2)
q


# %% [markdown]
# **Observation:** Admissions implied by the HHS columns alone ran at 281–401 a day in 2023, but recorded transfers captured only 34–57% of them. From Q2 2024 the ratio is 0.97–1.00. In
      2025 it is 0.67–0.92 at very low volumes, a gap of only about 2–4 children a day, consistent with small non-CBP referrals. **Conclusion:** the January 2023 – January 2024 CBP
      columns are corrupted and are masked. The HHS columns reconcile wherever they can be checked (February 2024 onward) and, apart from the 5 stale reports, are kept for the whole
      history; they cannot be cross-checked inside the copied block itself.


# %% [markdown]
# **RCA decision log**
#
# | Issue | Evidence | Fix |
# |---|---|---|
# | 450 blank padding rows | `isna().all()` | Dropped |
# | Newest-first, text dates, comma numbers | Raw preview | Parsed, sorted, typed |
# | Fri/Sat never reported | Weekday counts | Daily grid + interpolation, with an `is_reported` mask for scoring |
# | 5 stale carried-forward HHS reports | Exact repeats | Set to missing → interpolated, never scored |
# | CBP columns copied, Jan 2023–Jan 2024 | Duplicate triples + broken identity | CBP columns masked in that window; ML features use the trusted window only |
# | HHS columns | Reconcile with flows where CBP is trusted | Kept for the full history (used for EDA and the surge replay) |
# | Duplicate dates (none today) | Stable sort + conflict listing | Identical repeats dropped; for conflicting repeats the first record in file order is kept and printed (an assumption:
      the file carries no revision timestamp) |
# | Impossible values on a refresh (none today) | Plausibility rules (§3.1) | Quarantined to missing and listed; never zero-filled |
#
# **What each column measures (domain definitions supported by the file)**
#
# | Column | Type | Meaning | Role in the HHS population balance |
# |---|---|---|---|
# | `hhs_care` | stock (snapshot on the report day) | children in HHS care | closing population |
# | `transferred` | daily count | children transferred out of CBP custody **into HHS** | admissions from CBP (one movement, counted once) |
# | `discharged` | daily count | children discharged from HHS care (mostly to sponsors) | exits |
# | `apprehended` | daily count | children apprehended and placed in CBP custody | CBP intake, upstream; **not** part of the HHS balance |
# | `cbp_custody` | stock | children in CBP custody | upstream pipeline |
#
# The balance that can be tested is therefore *closing = opening + transfers in − discharges + (unrecorded other admissions − other exits)*. The file has no columns for non-CBP
      referrals, other exits or bed capacity, so the unexplained part is measured (§6 and §14) and reported as a reconciliation term rather than invented, and every capacity threshold in
      this notebook is a user input or an illustration.


# %% [markdown]
# ## 5. Time-series preparation: continuity, interpolation **and** masking


# %%
# Re-index onto a continuous daily calendar; is_reported marks the days with a genuine (non-stale) HHS report
daily = df.asfreq("D")
daily["is_reported"] = daily["hhs_care"].notna() & daily["discharged"].notna()
daily["cbp_trusted"] = daily.index > CBP_BAD_END
```

```
daily.loc[~daily["cbp_trusted"], CBP_COLS] = np.nan          # mask the copied CBP block
OBSERVED = daily[["apprehended", "cbp_custody", "transferred", "hhs_care", "discharged"]].notna()   # genuine values per column, before any imputation
print(f"Calendar days: {len(daily)} | reported: {daily['is_reported'].sum()} | to impute: {(~daily['is_reported']).sum()}")


# %% [markdown]
# **Observation:** The continuous calendar has 1,075 days: 715 (67%) hold a genuine, non-stale report and 360 (33%) must be imputed. One-third imputation is too much to treat
      interpolated values as real data, which is why every evaluation in this notebook scores reported days only.


# %%
# Time-weighted linear interpolation fills interior gaps only (no extrapolation at the ends)
value_cols = ["apprehended", "cbp_custody", "transferred", "hhs_care", "discharged"]
daily[value_cols] = daily[value_cols].interpolate(method="time", limit_area="inside")
daily["net_pressure"] = daily["transferred"] - daily["discharged"]           # flow-based imbalance signal
T_END = daily.index.max()
print("Remaining gaps after interpolation:\n", daily[value_cols].isna().sum().to_string())


# %% [markdown]
# **Observation:** After interpolation the HHS care and discharge series are complete (0 gaps). The three CBP series keep exactly 384 missing days, the length of the masked copied block.
      These gaps are intentional, so corrupted values cannot reach the models.


# %%
# Visual check on a recent month: interpolated weekend points sit on straight segments between real reports
win = daily.loc[T_END - pd.Timedelta(days=34):]
fig, ax = plt.subplots(figsize=(12, 3.6))
ax.plot(win.index, win["hhs_care"], color=SERIES_COLORS["hhs_care"], lw=2, label="daily series")
ax.scatter(win.index[win["is_reported"]], win.loc[win["is_reported"], "hhs_care"], s=36, color=SERIES_COLORS["hhs_care"], edgecolor=SURFACE, linewidth=2, zorder=3, label="reported")
ax.scatter(win.index[~win["is_reported"]], win.loc[~win["is_reported"], "hhs_care"], s=36, facecolor=SURFACE, edgecolor=MUTED, linewidth=1.5, zorder=3, label="interpolated (masked)")
ax.set_title("Continuity with masking: last five weeks of care load"); ax.yaxis.set_major_formatter(COMMA); ax.legend(loc="upper left")
savefig(fig, "03_interpolation_mask")


# %% [markdown]
# **Observation:** Over the last five weeks 24 days are real reports and 11 are interpolated: 10 Friday/Saturday days plus Wednesday 26 Nov, before Thanksgiving. They sit on straight
      lines between neighbouring reports. Care load climbed from 2,343 to 2,484 (+141) in this window, and the interpolation preserves that trend without inventing new peaks or dips.


# %% [markdown]
# **Leakage guard.** An interpolated Friday value is built from the following Sunday's report. If a forecast were issued "as of Friday", it would secretly know Sunday. So **every
      forecast origin is a reported day** and **every scored target is a reported day**. Interpolated days only fill the rolling-window features *between* reported days, which uses no
      information from after the origin.


# %% [markdown]
# ## 6. Exploratory analysis


# %%
# Three years at a glance: the care-load stock and the three daily flows (small multiples, one shared time axis)
fig, axes = plt.subplots(2, 1, figsize=(12, 7), sharex=True, gridspec_kw={"height_ratios": [1.1, 1]})
axes[0].plot(daily.index, daily["hhs_care"], color=SERIES_COLORS["hhs_care"])
axes[0].set_title("Children in HHS care (stock)"); axes[0].yaxis.set_major_formatter(COMMA)
for c in ["transferred", "discharged"]:
    axes[1].plot(daily.index, daily[c].rolling(7, min_periods=3).mean(), color=SERIES_COLORS[c], label=LABELS[c] + " (7-day mean)")
axes[1].axvspan(daily.index.min(), CBP_BAD_END, color=GRID, alpha=0.6, lw=0)
axes[1].text(CBP_BAD_START + pd.Timedelta(days=10), 20, "CBP columns masked (copied block)", color=INK2, fontsize=9)
axes[1].set_title("Daily flows into and out of HHS care"); axes[1].legend(loc="upper right")
savefig(fig, "04_history_overview")
```

```
# %% [markdown]
# **Observation:** Care load peaked at 11,516 children (20 Dec 2023) and swung between about 5,700 and 11,500 through 2023–24. It then collapsed from 6,405 on 1 Jan 2025 to 2,111 by mid-
      March 2025, as daily transfers and discharges fell from hundreds a day to about 10. The history therefore contains two very different operating regimes that should not be modelled
      as one.


# %%
# Change-point detection: the single split that best separates two mean levels of log(flow)
def single_break(series, min_seg=60):
    """Return the date that minimises within-segment squared error of log1p(series) (binary segmentation, 1 split).
    Returns None when the series is too short for two segments of min_seg points; the caller decides what to do."""
    s = series.dropna(); y = np.log1p(s.to_numpy())
    if len(y) <= 2 * min_seg:
        return None
    sse = [((y[:k] - y[:k].mean()) ** 2).sum() + ((y[k:] - y[k:].mean()) ** 2).sum() for k in range(min_seg, len(y) - min_seg)]
    return s.index[min_seg + int(np.argmin(sse))]


reported = df.dropna(subset=["hhs_care"])
brk_transfers = single_break(reported.loc[reported.index > CBP_BAD_END, "transferred"])
brk_discharges = single_break(reported["discharged"])
if brk_transfers is None or brk_discharges is None:
    raise DataValidationError("History too short for change-point detection: each series needs more than 120 trusted reports")
REGIME_START = max(brk_transfers, brk_discharges) + pd.Timedelta(days=28)   # 4-week buffer while the backlog drains
print(f"Break in transfers: {brk_transfers:%d %b %Y} | break in discharges: {brk_discharges:%d %b %Y} | current regime starts: {REGIME_START:%d %b %Y}")


# %% [markdown]
# **Observation:** The statistical break falls on 26 Jan 2025 for transfers and 17 Feb 2025 for discharges: intake dropped first, and placements followed about three weeks later. Models
      are trained on the current regime from 17 Mar 2025, after a four-week buffer for the backlog to drain.


# %%
# Quantify the regime shift: average daily volumes before vs after the break
pre = daily.loc[CBP_BAD_END:brk_transfers - pd.Timedelta(days=1)]
post = daily.loc[REGIME_START:]
shift = pd.DataFrame({"Feb 2024 – break": pre[["apprehended", "transferred", "discharged", "hhs_care"]].mean(),
                      "Current regime": post[["apprehended", "transferred", "discharged", "hhs_care"]].mean()})
shift["change %"] = (shift["Current regime"] / shift["Feb 2024 – break"] - 1) * 100
shift.round(1)


# %% [markdown]
# **Observation:** Daily intake, transfers and discharges all fell by about 95% (e.g., transfers 209.9 → 9.6 a day), but average care load fell only 66.5% (6,751 → 2,262). The census
      shrank far less than the flows, so children now stay much longer in care. That is the key change for planners.


# %%
# Little's law: average length of stay ≈ children in care ÷ daily discharges (28-day smoothing)
los = daily["hhs_care"].rolling(28).mean() / daily["discharged"].rolling(28).mean()
fig, ax = plt.subplots(figsize=(12, 3.6))
ax.plot(los.index, los, color=SERIES_COLORS["discharged"])
ax.axvline(brk_discharges, color=MUTED, lw=1); ax.text(brk_discharges + pd.Timedelta(days=6), los.max() * 0.9, "discharge break", color=INK2, fontsize=9)
ax.set_title("Implied average length of stay in HHS care (days)"); ax.set_ylabel("days")
savefig(fig, "05_length_of_stay")
print(f"Implied stay: {los.loc[:brk_discharges].tail(180).mean():.0f} days before the break vs {los.loc[REGIME_START:].mean():.0f} days in the current regime")


# %% [markdown]
```

```
# **Observation:** The implied average stay rose from about 36 days before the break to about 203 days in the current regime. It peaked at 369 days in April 2025 and stood at 258 days at
      the latest report. The bottleneck has moved from intake to placement: care load now depends mainly on how fast children are discharged to sponsors.


# %%
# Net pressure (transfers – discharges) is the leading indicator of care-load growth; positive values mean the system is filling
np28 = daily["net_pressure"].rolling(14, min_periods=7).mean()
fig, axes = plt.subplots(2, 1, figsize=(12, 6), sharex=True)
seg = daily.loc[REGIME_START:]
axes[0].plot(seg.index, seg["hhs_care"], color=SERIES_COLORS["hhs_care"]); axes[0].yaxis.set_major_formatter(COMMA)
axes[0].set_title("Current regime: children in HHS care")
vals = np28.loc[REGIME_START:]
axes[1].bar(vals.index, vals.clip(lower=0), width=1, color=STATUS["serious"], label="filling (net inflow)")
axes[1].bar(vals.index, vals.clip(upper=0), width=1, color=STATUS["good"], label="draining (net outflow)")
axes[1].axhline(0, color=BASE, lw=1); axes[1].legend(loc="lower right")
axes[1].set_title("Net pressure: transfers – discharges (14-day mean, children/day)")
savefig(fig, "06_net_pressure")


# %% [markdown]
# **Observation:** Net pressure explains the care-load path. A deep net outflow (down to −19 children/day in mid-July 2025) drove the fall from 2,528 (1 Jun) to 1,972 (21 Aug), and small
      net inflows since September (about +1/day on average) coincide with the climb to 2,484. Net pressure was positive on only 46% of days, so the system alternates between filling and
      draining. The recorded flows explain only part of the recent growth; the weekend analysis below accounts for the rest.


# %%
# Weekly seasonality in the current regime: mean flow per weekday on reported days only
rep_post = daily.loc[REGIME_START:].query("is_reported")
order = ["Sun", "Mon", "Tue", "Wed", "Thu"]                                  # Fri/Sat are (almost) never reported
dow_tbl = rep_post.groupby(rep_post.index.day_name().str[:3])[["transferred", "discharged"]].mean().reindex(order)
fig, ax = plt.subplots(figsize=(9, 3.6))
x = np.arange(len(order)); w = 0.36
for i, c in enumerate(["transferred", "discharged"]):
    ax.bar(x + (i - 0.5) * (w + 0.02), dow_tbl[c], width=w, color=SERIES_COLORS[c], label=LABELS[c])
ax.set_xticks(x, order); ax.set_title("Average daily flow by weekday (current regime, reported days)"); ax.legend(loc="upper left")
savefig(fig, "07_weekday_profile")
dow_tbl.round(1)


# %% [markdown]
# **Observation:** Discharges follow a clear weekly cycle: they peak on Thursday (14.9) and are lowest on Sunday (7.2), roughly half the Thursday level. Transfers are much flatter (8.7–
      10.8 a day). Placement activity is concentrated mid-week while arrivals are steady, which sets up the weekend pattern below.


# %%
# Where does care-load growth happen? Average change per day on weekday transitions vs across the unreported weekend
rep = daily.loc[daily["is_reported"], ["hhs_care"]].copy()
rep["gap"] = rep.index.to_series().diff().dt.days
rep["per_day"] = rep["hhs_care"].diff() / rep["gap"]
rep["transition"] = np.select([rep["gap"] == 1, (rep["gap"] == 3) & (rep.index.dayofweek == 6)],
                              ["weekday (consecutive reports)", "weekend (Thu → Sun)"], "other gap")
rep["period"] = np.select([rep.index >= REGIME_START, rep.index > CBP_BAD_END], ["3) current regime", "2) Feb 2024 – break"], "1) 2023 – Jan 2024")
WEEKEND = rep[rep["transition"] != "other gap"].pivot_table(index="period", columns="transition", values="per_day", aggfunc="mean").round(1)
WEEKEND


# %% [markdown]
# **Observation:** In the current regime care load falls slightly across weekday transitions (−0.3/day) but rises by +3.4/day across each Thursday→Sunday gap. In 2023–24 the pattern was
      the reverse (weekdays +19 to +67/day, weekends −65 to −77/day), so weekend dynamics changed with the regime.


# %%
```

```
# Chart the weekend ratchet in the current regime (children added per calendar day)
cur = WEEKEND.loc["3) current regime"]
fig, ax = plt.subplots(figsize=(7.5, 3.4))
ax.barh(cur.index, cur.values, height=0.5, color=[STATUS["good"] if v < 0 else STATUS["serious"] for v in cur.values])
for i, v in enumerate(cur.values):
    ax.annotate(f"{v:+.1f} / day", (v, i), xytext=(6 if v >= 0 else -6, 0), textcoords="offset points", va="center",
                ha="left" if v >= 0 else "right", fontsize=9.5, color=INK)
ax.axvline(0, color=BASE, lw=1); ax.grid(axis="y", visible=False)
ax.set_xlim(min(cur.min(), 0) - 1.4, cur.max() + 0.9)                          # room for value labels inside the plot
ax.set_title("Current regime: care load grows over weekends, not weekdays")
savefig(fig, "07b_weekend_ratchet")


# %% [markdown]
# **Observation:** Over the current regime care load rose by 373 children. Weekday transitions netted −40, while Thursday→Sunday gaps added +323 and other multi-day gaps (holidays, a
      June outage and removed stale reports) +90. Net growth happens across the weekend, when arrivals continue but placements slow (Sunday discharges are the week's lowest), so **weekend
      placement capacity is a direct lever on care load**. Friday/Saturday flows are unreported, so this is inferred from the stock; the forecast section adds a reconciliation term for
      it.


# %% [markdown]
# ### Is interpolating the Friday/Saturday *counts* appropriate?
#
# Interpolating a stock (care load) between two snapshots is natural. Interpolating daily **counts** needs evidence, because a missing count could mean "not reported", "zero activity",
     or "included in the next report". Three checks decide it:
# 1. **Are Sunday counts single-day counts or 3-day totals?** If Sunday's figure covered Friday–Sunday it would be about three times a weekday figure.
# 2. **Do the interpolated weekend flows reconcile with the stock?** The care-load change from Thursday to Sunday should match the three days of flows about as well as consecutive
     weekdays do.
# 3. **Does the choice change the model inputs?** Compare 14-day flow means from the interpolated grid with means computed from reported days only.
#
# Zero-filling is not considered: the stock keeps moving over weekends, so flows on those days are not zero.


# %%
# Interpolation check: Sunday counts vs weekday counts, and the unexplained stock change per day (consecutive weekdays vs Thu→Sun with interpolated Fri/Sat)
chk = []
for name, lo in [("Trusted (Feb 2024 →)", CBP_BAD_END), ("Current regime", REGIME_START)]:
    r = df.loc[lo + pd.Timedelta(days=1):]
    gap = r.index.to_series().diff().dt.days
    resid = r["hhs_care"].diff() - (r["transferred"] - r["discharged"])           # one-day identity residual
    wkends = [(e - pd.Timedelta(days=3), e) for e in r.index[(gap == 3) & (r.index.dayofweek == 6)]]
    w_unexp = [(daily.at[e, "hhs_care"] - daily.at[s, "hhs_care"] - daily.loc[s + pd.Timedelta(days=1):e, "net_pressure"].sum()) / 3 for s, e in wkends]
    chk.append({"period": name, "Sunday discharges": r.loc[r.index.dayofweek == 6, "discharged"].mean(),
                "Mon–Thu discharges": r.loc[r.index.dayofweek < 4, "discharged"].mean(),
                "unexplained/day: consecutive weekdays": resid[gap == 1].mean(),
                "unexplained/day: Thu→Sun (interpolated)": np.nanmean(w_unexp), "weekends": len(wkends)})
INTERP_CHECK = pd.DataFrame(chk).set_index("period").round(2)
sens = []
for c in ("transferred", "discharged"):
    m = daily["is_reported"] & (daily.index > CBP_BAD_END + pd.Timedelta(days=15))
    grid = daily[c].rolling(14).mean()[m]
    rep_only = daily[c].where(OBSERVED[c]).rolling(14, min_periods=7).mean()[m]
    sens.append({"flow": c, "mean |difference| %": ((grid - rep_only).abs() / rep_only).mean() * 100, "correlation": grid.corr(rep_only)})
display(INTERP_CHECK)
pd.DataFrame(sens).set_index("flow").round(3)


# %% [markdown]
# **Observation:** Sunday figures are single-day counts, not three-day totals: in the current regime Sunday discharges average 7.2 against 12.8 on Monday–Thursday, whereas a weekend
      total would be about three times a weekday. Interpolated Friday/Saturday flows reconcile with the stock almost as well as real reports: the unexplained care-load change is 3.4
```

```
      children/day across Thu→Sun gaps versus 3.0/day between consecutive weekday reports, a difference of about 0.45/day. In the 2024 high-volume period the gap was large (−18.8/day vs
      +2.6/day): weekend net outflows were then much bigger than straight-line interpolation implies, so 2024 weekend flow features are biased; they enter only the ML training rows, which
      a 180-day half-life down-weights. The choice barely moves the model inputs: 14-day flow means differ by 3–5% from means built on reported days only (correlation ≥ 0.999).
      Interpolation is therefore kept for features, and interpolated days are never scored.


# %%
# STL decomposition (weekly period, robust to outliers) of care load and discharges in the current regime
fig, axes = plt.subplots(4, 2, figsize=(13, 8.5), sharex=True)
for j, c in enumerate(["hhs_care", "discharged"]):
    res = STL(daily.loc[REGIME_START:, c], period=7, robust=True).fit()
    for i, (part, name) in enumerate(zip([res.observed, res.trend, res.seasonal, res.resid], ["observed", "trend", "seasonal (weekly)", "residual"])):
        axes[i, j].plot(part.index, part, color=SERIES_COLORS[c], lw=1.6 if i else 2)
        axes[i, j].set_title(f"{LABELS[c]}: {name}" if i == 0 else name, fontsize=10)
    strength = max(0, 1 - res.resid.var() / (res.seasonal + res.resid).var())
    print(f"{LABELS[c]}: weekly seasonal strength = {strength:.2f}")
savefig(fig, "08_stl_decomposition")


# %% [markdown]
# **Observation:** Weekly seasonality is strong for discharges (strength 0.42; on average Thursday runs about +3.5 and Sunday about −4.8 placements versus trend) but weak for care load
      (0.14). Care load is dominated by its trend, which fell from about 2,107 to 1,980 and then climbed to about 2,489. Weekly terms therefore matter for discharge forecasts and much
      less for the census.


# %%
# Monthly pattern (descriptive): month-end care load and mean daily discharges by calendar month and year; each month occurs at most three times
mon = daily.query("is_reported").resample("MS").agg({"hhs_care": "last", "discharged": "mean", "transferred": "mean"})
mon["year"], mon["month"] = mon.index.year, mon.index.month
years = sorted(mon["year"].unique())
YEAR_COLORS = dict(zip(years, plt.cm.Blues(np.linspace(0.45, 0.95, len(years)))))      # sequential: later years darker
fig, axes = plt.subplots(1, 2, figsize=(13, 3.8))
for yr, g in mon.groupby("year"):
    axes[0].plot(g["month"], g["hhs_care"], color=YEAR_COLORS[yr], marker="o", ms=4, label=str(yr))
    axes[1].plot(g["month"], g["discharged"], color=YEAR_COLORS[yr], marker="o", ms=4, label=str(yr))
axes[0].set_title("Month-end children in HHS care"); axes[0].yaxis.set_major_formatter(COMMA)
axes[1].set_title("Mean daily discharges (reported days)")
for ax in axes:
    ax.set_xticks(range(1, 13), ["J", "F", "M", "A", "M", "J", "J", "A", "S", "O", "N", "D"]); ax.legend(loc="upper right")
savefig(fig, "07c_monthly_pattern")
MONTHLY = mon.pivot_table(index="month", columns="year", values="hhs_care").round(0)
full_years = [y for y in years if mon.loc[mon["year"] == y, "month"].nunique() == 12][:2]        # first two complete calendar years
if len(full_years) == 2:
    a, b = (np.sign(MONTHLY[y].diff()) for y in full_years)
    print(f"Month-to-month direction of care load agrees between {full_years[0]} and {full_years[1]} in {int((a == b).sum())} of {int((a.notna() & b.notna()).sum())} months")


# %% [markdown]
# **Observation:** Month-end care load followed a different path in every year: it peaked in December 2023 (11,516 on 20 Dec), moved between about 5,700 and 9,000 through 2024, and
      collapsed in early 2025. The month-to-month direction agreed between 2023 and 2024 in only 5 of 11 months, no better than chance. With each calendar month seen at most three times,
      and once on each side of the 2025 break, there is no stable month-of-year pattern to estimate, which supports leaving month and week-of-year out of the features.


# %%
# Outliers and unusual periods: largest robust deviations (MAD z-scores vs a centred 29-day median, reported days) and the fastest 28-day rises in care load
out_rows = []
for c in ("transferred", "discharged"):
    x = daily[c].where(daily["is_reported"] & OBSERVED[c]).dropna()
    med = x.rolling(29, center=True, min_periods=10).median()
    mad = (x - med).abs().rolling(29, center=True, min_periods=10).median() * 1.4826
    z = ((x - med) / mad.replace(0, np.nan)).dropna()
```

```
    for d in z.abs().nlargest(3).index:
        out_rows.append({"series": c, "date": d.date(), "value": x[d], "local median": med[d], "robust z": z[d]})
OUTLIERS = pd.DataFrame(out_rows).round(1)
rise = (daily["hhs_care"] - daily["hhs_care"].shift(28)).dropna()
SURGES = []
for d in rise.sort_values(ascending=False).index:                       # three non-overlapping fastest 28-day rises
    if all(abs((d - s).days) > 56 for s in SURGES):
        SURGES.append(d)
    if len(SURGES) == 3:
        break
display(OUTLIERS)
pd.DataFrame({"28-day window ending": [d.date() for d in SURGES], "care load at end": [daily.at[d, "hhs_care"] for d in SURGES],
              "rise over 28 days": [rise[d] for d in SURGES], "rise %": [rise[d] / daily.at[d - pd.Timedelta(days=28), "hhs_care"] * 100 for d in SURGES]}).round(1)


# %% [markdown]
# **Observation:** The largest robust deviations occur at the low volumes of the current regime: 29 transfers on 11 Sep 2025 against a local median of 11 (robust z = 12.1) and 39
      discharges on 10 Jul 2025 against 17 (z = 4.9). They pass the plausibility rules and look like genuine operational spikes, so they are kept; Huber-loss boosting and median-based
      checks limit their influence. The fastest build-ups in care load were all in 2023: +2,806 children (+43%) in the 28 days to 13 Aug 2023, +1,342 (+20%) to 9 Feb 2023 and +1,695
      (+17%) to the 11,516 peak on 20 Dec 2023. These are the surges the early-warning replay is tested on.


# %%
# Correlations in the current regime (reported days): same-day links and CBP-to-HHS lags; these are associations, not causal effects
cr = daily.loc[REGIME_START:].query("is_reported").assign(care_change=lambda x: x["hhs_care"].diff())
CORR = cr[["apprehended", "cbp_custody", "transferred", "discharged", "care_change"]].corr().round(2)
lagged = {k: cr["transferred"].corr(daily["apprehended"].shift(k).reindex(cr.index)) for k in range(0, 4)}   # reported target days
print("corr(transfers today, apprehensions k days earlier):", {k: round(v, 2) for k, v in lagged.items()})
CORR


# %% [markdown]
# **Observation:** In the current regime the daily care-load change rises with transfers (+0.47) and falls with discharges (−0.61); the stronger discharge link is consistent with
      placement being the binding constraint. CBP intake leads HHS transfers by one to two days: transfers correlate 0.38 with same-day apprehensions but 0.59 and 0.58 with apprehensions
      one and two days earlier, which supports the lagged CBP features. These are associations only; they do not show that one series causes another.


# %%
# Stationarity: ADF (H0 = unit root) and KPSS (H0 = stationary) together; agreement gives a clear verdict
adf_rows = []
for c in ["hhs_care", "discharged", "transferred"]:
    s = daily.loc[REGIME_START:, c]
    for label, series in [("level", s), ("1st difference", s.diff().dropna())]:
        p_adf, p_kpss = adfuller(series, autolag="AIC")[1], kpss(series, regression="c", nlags="auto")[1]
        verdict = "stationary" if (p_adf < 0.05 and p_kpss > 0.05) else "non-stationary" if (p_adf >= 0.05 and p_kpss <= 0.05) else "inconclusive"
        adf_rows.append({"series": c, "form": label, "ADF p": round(p_adf, 3), "KPSS p": round(p_kpss, 3), "verdict": verdict})
pd.DataFrame(adf_rows)


# %% [markdown]
# **Observation:** Transfers are stationary in level, and discharges become stationary after one difference. For care load both tests are inconclusive (ADF p = 0.106 and 0.350; KPSS p =
      0.086 and 0.079) because the regime window is short. The differencing choice for care load therefore rests on structure: a stock accumulates flows, so it is modelled with one
      difference.


# %%
# ACF/PACF of differenced care load and log discharges guide the ARIMA orders (spikes at lag 7 = weekly pattern)
fig, axes = plt.subplots(2, 2, figsize=(12, 6))
care_d = daily.loc[REGIME_START:, "hhs_care"].diff().dropna()
dis_l = np.log1p(daily.loc[REGIME_START:, "discharged"])
plot_acf(care_d, lags=28, ax=axes[0, 0], color=SERIES_COLORS["hhs_care"], title="ACF: Δ care load")
```

```
plot_pacf(care_d, lags=28, ax=axes[0, 1], color=SERIES_COLORS["hhs_care"], title="PACF: Δ care load", method="ywm")
plot_acf(dis_l, lags=28, ax=axes[1, 0], color=SERIES_COLORS["discharged"], title="ACF: log discharges")
plot_pacf(dis_l, lags=28, ax=axes[1, 1], color=SERIES_COLORS["discharged"], title="PACF: log discharges", method="ywm")
fig.tight_layout(); savefig(fig, "09_acf_pacf")


# %% [markdown]
# **Observation:** Daily care-load changes are persistent. Autocorrelation is 0.60 at lag 1 and 0.51 at lag 7 on the daily series, and still 0.44 and 0.39 using only genuine consecutive
      reports (interpolated weekends repeat the same change, which inflates the raw figures). 18 of the 28 lags exceed the plotted significance band. The partial autocorrelation drops
      from 0.60 at lag 1 to 0.29 at lag 2, pointing to a short autoregressive term. Log discharges show a weekly pattern (partial autocorrelation 0.32 at lag 7 vs 0.03 at lag 2), which
      supports weekly seasonal terms. This momentum is what lets models beat "tomorrow = today" at longer horizons.


# %% [markdown]
# **Modelling decisions from the EDA**
# * **Care load:** one difference (d = 1) plus an autoregressive term to capture momentum.
# * **Flows:** log scale, no differencing (d = 0), with a constant, so forecasts revert to the regime's typical level instead of extrapolating a temporary push. The walk-forward test
      validates this choice.
# * **Seasonality:** period s = 7, kept only where AIC finds it useful.
# * **Training windows:** statistical models train on the current regime only; ML models use scale-free targets so the larger 2024 history stays informative without its volume level
      dominating.


# %% [markdown]
# ## 7. Feature engineering
#
# Every feature describes what is known **at the end of the origin day _t_**. A direct multi-horizon design then stacks one row per (origin, horizon) and adds the horizon `h` and the
      calendar of the *target* day.
#
# | Group | Features |
# |---|---|
# | Lags (t-1, t-7, t-14) | care-load log-changes; flow lags relative to their 14-day mean |
# | Rolling mean & variance (7 / 14 days) | care-load deviation from its rolling mean, rolling volatility; flow rolling means and log-variances |
# | Flow-based signals | net pressure (transfers − discharges) per 100 children: lag-1, 7-day and 14-day means; CBP pipeline per 100 children |
# | Calendar | target weekday, US federal-holiday window (±1 day), origin weekday, horizon |
# | Seasonal anchor | same weekday last week relative to today's level |
#
# **Why there is no month feature (an RCA finding from the first full run).** My first version included the target month. It became the *single most important* care-load feature (about
      30% of Random Forest importance), and the production forecast showed Random Forest dropping from 2,525 on 31 Dec to 2,282 on 1 Jan, a 10% cliff exactly at the month boundary. The
      trusted training history contains only **one** January, January 2025, which is the policy-break crash. So the model had learned "January ⇒ care load collapses", a regime change
      disguised as seasonality. With under two years of trusted data no month effect can be estimated reliably, so the feature was removed. The trade-off was measured, not assumed. On the
      Sep–Dec 2025 test window, removing it raised Random Forest's care-load MAE (25 → 38) but lowered Gradient Boosting's (40 → 31), and the Ensemble stayed the best model on test
      (28.5). A feature that can create a 10% forecast cliff is unacceptable in an early-warning system, whatever it does to one window's score. The holiday window was tested the same way
      and changes forecasts by at most about 4 children, so it stays.


# %%
# Origin-level features: everything known at the end of day t (rolling windows only look backwards)
HOLIDAYS = USFederalHolidayCalendar().holidays(daily.index.min(), T_END + pd.Timedelta(days=90))
HOLIDAY_WINDOW = HOLIDAYS.union(HOLIDAYS + pd.Timedelta(days=1)).union(HOLIDAYS - pd.Timedelta(days=1))



def origin_features(d):
    """Scale-free features at each origin day: lags, rolling mean/variance, net pressure and pipeline signals."""
    f = pd.DataFrame(index=d.index)
    care, net = d["hhs_care"], d["transferred"] - d["discharged"]
    for k in (1, 7, 14):
        f[f"care_logchg_{k}"] = np.log(care / care.shift(k))                # lagged momentum of the stock
    for w in (7, 14):
```

```
        f[f"care_dev_ma{w}"] = np.log(care / care.rolling(w).mean())          # level vs rolling mean
        f[f"care_vol{w}"] = care.diff().rolling(w).std() / care * 100         # rolling volatility (% of load)
        f[f"net_rate_ma{w}"] = net.rolling(w).mean() / care * 100            # net pressure per 100 children
    f["net_rate_lag1"] = net / care * 100
    f["cbp_pipeline_rate"] = d["cbp_custody"] / care * 100                   # children waiting at CBP per 100 in HHS
    for c in ("apprehended", "transferred", "discharged"):
        base = np.log1p(d[c].rolling(14).mean())
        for k in (0, 1, 7, 14):
            f[f"{c}_lag{k}"] = np.log1p(d[c].shift(k)) - base                # lags relative to the 14-day mean
        f[f"{c}_ma7"] = np.log1p(d[c].rolling(7).mean()) - base
        f[f"{c}_logvar7"] = np.log1p(d[c]).rolling(7).var()
        f[f"{c}_logvar14"] = np.log1p(d[c]).rolling(14).var()
        f[f"{c}_rate14"] = d[c].rolling(14).mean() / care * 100              # flow per 100 children in care
    f["origin_dow"] = d.index.dayofweek
    return f



FEATS = origin_features(daily)
print("Origin features:", FEATS.shape[1])
FEATS.loc[T_END].round(3).to_frame("value at last origin").T


# %% [markdown]
# **Observation:** At the last report (21 Dec 2025) care load is 2.2% above its level 14 days earlier and slightly above its 7- and 14-day averages, confirming the upward trend. Recorded
      net pressure is slightly negative (−0.08 to −0.12 per 100 children), and the CBP pipeline holds 0.7 children per 100 in care (18 children). Most of the 36 features are ratios or
      log-changes, so they stay comparable across the 2024 and 2025 regimes despite a twenty-fold difference in volumes.


# %%
# Scale-free targets let one model learn from both the high-volume (2024) and low-volume (2025) regimes
def to_target(future, d, col, kind):
    """stock: log(y[t+h] / y[t]); flow: log1p(y[t+h]) - log1p(14-day mean at t)."""
    if kind == "stock":
        return np.log(future / d[col])
    return np.log1p(future) - np.log1p(d[col].rolling(14).mean())



def from_target(z, d, t, col, kind):
    """Invert to_target at origin t to get forecasts in children."""
    if kind == "stock":
        return d.at[t, col] * np.exp(z)
    return np.clip(np.expm1(z + np.log1p(d.loc[:t, col].tail(14).mean())), 0, None)


# %%
# Direct multi-horizon stacking: one block per horizon h with target-day calendar features and a seasonal anchor
def horizon_block(d, col, kind, h, feats=None):
    """Rows = origins; adds h, target-day calendar, same-weekday-last-week anchor, target z and a validity mask."""
    X = (FEATS if feats is None else feats).copy()
    tdate = d.index + pd.Timedelta(days=h)
    X["h"] = h
    X["target_dow"] = tdate.dayofweek                                        # no month feature: see the note above
    X["target_holiday"] = tdate.isin(HOLIDAY_WINDOW).astype(int)
    lag = 7 * int(np.ceil(h / 7)) - h                                        # shift that lands on the same weekday, at or before t
    anchor = d[col].shift(lag)
    X["seasonal_anchor"] = np.log(anchor / d[col]) if kind == "stock" else np.log1p(anchor) - np.log1p(d[col].rolling(14).mean())
    X["z"] = to_target(d[col].shift(-h), d, col, kind)
    X["target_date"] = tdate
```

```
    target_ok = d["is_reported"] & OBSERVED[col].reindex(d.index, fill_value=False)          # reported day AND a genuine value for this series
    X["target_reported"] = target_ok.shift(-h, fill_value=False).to_numpy()
    return X



STACKS = {col: {h: horizon_block(daily, col, kind, h) for h in range(1, H + 1)} for col, kind in TARGETS.items()}
FEATURE_COLS = [c for c in STACKS["hhs_care"][1].columns if c not in ("z", "target_date", "target_reported")]
print(f"{len(FEATURE_COLS)} model features per row; example block for h=7:")
STACKS["hhs_care"][7].loc[T_END - pd.Timedelta(days=7), FEATURE_COLS[:6] + ["h", "target_dow", "seasonal_anchor", "z"]].to_frame().T.round(4)


# %% [markdown]
# **Observation:** Each training row holds 40 model features plus the target. In the example (origin 14 Dec 2025, horizon 7 days), the target z = 0.0089 means care load rose 0.9% over
      the following week (2,462 → 2,484): exactly the kind of small relative change the ML models learn to predict.


# %% [markdown]
# ### 7.1 Leakage audit
#
# Two complementary checks. **(1) Executed perturbation test:** every report after a cut-off date is scaled by 1.5, the whole preparation (daily grid, masks, interpolation) and every
      feature are recomputed, and any feature value that changes at a reported origin on or before the cut-off is a leak. A negative control (the target *z*, which looks ahead by design)
      must change, proving the test can detect look-ahead. **(2) A documented definition for every feature:** source columns, look-back window, availability at the forecast origin and
      suitability for each horizon.


# %%
# Leakage audit, part 1 (executed): perturb every report after the cut-off, recompute preparation + features, count changed feature values at origins up to the cut-off
def prep_daily(frame):
    """Replica of §5 (daily grid, masks, interpolation); checked below to reproduce `daily` exactly."""
    g = frame.asfreq("D")
    g["is_reported"] = g["hhs_care"].notna() & g["discharged"].notna()
    g["cbp_trusted"] = g.index > CBP_BAD_END
    g.loc[~g["cbp_trusted"], CBP_COLS] = np.nan
    g[value_cols] = g[value_cols].interpolate(method="time", limit_area="inside")
    g["net_pressure"] = g["transferred"] - g["discharged"]
    return g



replica_ok = prep_daily(df)[daily.columns].equals(daily)
LEAK_CUT = T_END - pd.Timedelta(days=60)
LEAK_CUT -= pd.Timedelta(days=(LEAK_CUT.dayofweek - 5) % 7)          # a Saturday: its interpolated value depends on the next (perturbed) Sunday report
pert = df.copy(); pert.loc[pert.index > LEAK_CUT, value_cols] *= 1.5
dp = prep_daily(pert); feats_p = origin_features(dp)
chk_origins = daily.index[(daily.index <= LEAK_CUT) & daily["is_reported"] & (daily.index > CBP_BAD_END + pd.Timedelta(days=15))]
LEAK_CHANGED = pd.Series(0, index=FEATURE_COLS)
z_changed = 0
for col, kind in TARGETS.items():
    for h in (1, 7, 14, 28):
        a = STACKS[col][h].loc[chk_origins]; b = horizon_block(dp, col, kind, h, feats=feats_p).loc[chk_origins]
        LEAK_CHANGED += (~np.isclose(a[FEATURE_COLS].to_numpy(float), b[FEATURE_COLS].to_numpy(float), equal_nan=True)).sum(axis=0)
        z_changed += int((~np.isclose(a["z"].to_numpy(float), b["z"].to_numpy(float), equal_nan=True)).sum())
unrep = daily.index[(daily.index <= LEAK_CUT) & (daily.index > LEAK_CUT - pd.Timedelta(days=7)) & ~daily["is_reported"]]
unrep_changed = int((~np.isclose(FEATS.loc[unrep].to_numpy(float), feats_p.loc[unrep].to_numpy(float), equal_nan=True)).any(axis=1).sum())
print(f"Replica of §5 reproduces `daily` exactly: {replica_ok} | cut-off {LEAK_CUT:%d %b %Y} | reported origins checked: {len(chk_origins)} × 3 series × 4 horizons")
print(f"Feature values changed at reported origins ≤ cut-off: {int(LEAK_CHANGED.sum())} (must be 0)")
print(f"Negative controls: target z changed in {z_changed} rows (must be > 0); interpolated days just before the cut-off changed: {unrep_changed} of {len(unrep)}")
```

```
# %% [markdown]
# **Observation:** The leakage test passes. The §5 replica reproduces the notebook's daily table exactly, and after every report from 19 Oct 2025 onward is scaled by 1.5, **0** feature values change at the 410 reported origins up to the 18 Oct 2025 cut-off (40 features, 3 series, horizons 1/7/14/28). The negative controls prove the test can see look-ahead: the target z changes in 99 rows (targets after the cut-off), and 2 of the 3 interpolated days just before the cut-off change, which is exactly why forecasts are issued only from reported days.


# %%
# Leakage audit, part 2: definition, sources, look-back window, availability at the origin and horizon suitability for every model feature
def describe_feature(f):
    m = lambda p: re.fullmatch(p, f)
    if (g := m(r"care_logchg_(\d+")):  return f"log(care[t] / care[t−{g[1]}])", "hhs_care", f"t−{g[1]} … t"
    if (g := m(r"care_dev_ma(\d+")):   return f"log(care[t] / {g[1]}-day mean of care)", "hhs_care", f"t−{int(g[1]) - 1} … t"
    if (g := m(r"care_vol(\d+")):      return f"std of daily care change over {g[1]} days ÷ care[t] × 100", "hhs_care", f"t−{g[1]} … t"
    if (g := m(r"net_rate_ma(\d+")):   return f"{g[1]}-day mean of (transfers − discharges) ÷ care[t] × 100", "transferred, discharged, hhs_care", f"t−{int(g[1]) - 1} … t"
    if f == "net_rate_lag1":           return "(transfers − discharges) on the origin day ÷ care[t] × 100", "transferred, discharged, hhs_care", "t"
    if f == "cbp_pipeline_rate":       return "children in CBP custody ÷ care[t] × 100", "cbp_custody, hhs_care", "t"
    if (g := m(r"(\w+)_lag(\d+)")):    return f"log1p({g[1]}[t−{g[2]}]) − log1p(14-day mean)", g[1], f"t−{max(int(g[2]), 13)} … t"
    if (g := m(r"(\w+)_ma7")):         return f"log1p(7-day mean) − log1p(14-day mean) of {g[1]}", g[1], "t−13 … t"
    if (g := m(r"(\w+)_logvar(\d+")):  return f"variance of log1p({g[1]}) over {g[2]} days", g[1], f"t−{int(g[2]) - 1} … t"
    if (g := m(r"(\w+)_rate14")):      return f"14-day mean of {g[1]} ÷ care[t] × 100", f"{g[1]}, hhs_care", "t−13 … t"
    if f == "origin_dow":              return "weekday of the origin day", "calendar", "t"
    if f == "h":                       return "forecast horizon in days", "design", "n/a"
    if f == "target_dow":              return "weekday of the target day t+h", "calendar", "known in advance"
    if f == "target_holiday":          return "target day within ±1 day of a US federal holiday", "calendar (USFederalHolidayCalendar)", "known in advance"
    if f == "seasonal_anchor":         return "same-weekday value at t+h−7⌈h/7⌉ (≤ t) relative to today's level (flows: 14-day mean)", "series being forecast", "t−13 … t"
    return "undocumented", "?", "?"



AUDIT = pd.DataFrame([dict(zip(["definition", "source columns", "look-back window"], describe_feature(f)), feature=f) for f in FEATURE_COLS]).set_index("feature")
AUDIT["available at origin t?"] = np.where(AUDIT["look-back window"].isin(["known in advance", "n/a"]), "yes (deterministic)", "yes (≤ t only)")
AUDIT["horizon suitability"] = np.where(AUDIT.index.isin(["target_dow", "target_holiday", "h", "seasonal_anchor"]), "built per horizon (direct strategy)", "all h; information ages with h")
AUDIT["changed under perturbation"] = LEAK_CHANGED
print(f"{len(AUDIT)} features documented; undocumented: {(AUDIT['definition'] == 'undocumented').sum()}; leaking (changed): {(AUDIT['changed under perturbation'] > 0).sum()}")
AUDIT


# %% [markdown]
# **Observation:** All 40 model features are documented and none leaks. 36 are origin features whose look-back windows end on the origin day (from t−14 … t to t alone); the other four are the horizon itself, the known calendar of the target day (weekday and federal-holiday window) and the same-weekday anchor taken at or before t. Every input is therefore available at prediction time for every horizon from 1 to 28 days, as the direct strategy requires.
#
# **Calendar features considered and excluded.** *Month* was removed after the incident described above (one trusted January, which was the policy-break crash). *Week of year* has the same flaw (each week occurs at most twice in the trusted history, once on each side of the break) and was never added. *Future flows* (tomorrow's transfers or discharges) are unavailable at the forecast origin, so multi-step forecasts use the **direct** strategy: one model sees the horizon `h` and only origin-time information, never predicted or actual future lags.


# %% [markdown]
# ## 8. Train–test design
#
# * **Strict time order.** No random sampling anywhere.
# * **Walk-forward.** A forecast is issued from **every reported day** in the evaluation period, for horizons 1–28. Models are **re-estimated every 14 days**; between re-estimations they absorb each new observation (state update), the way a production system would run.
# * **Two evaluation windows.** A *calibration* window (13 weeks) is used to choose the champion model and calibrate the intervals. The final *test* window (the last 13 weeks) stays untouched until reporting.
# * **Training windows from the RCA.** Statistical models train on the current regime. ML models train on the trusted-CBP period (Feb 2024 onward) with scale-free targets and recency weighting.
```

```
# %%
# Window boundaries: calibration (13 weeks) then test (13 weeks); ML history starts after the copied CBP block + 2-week warm-up
CAL_DAYS = TEST_DAYS = 91
TEST_START = T_END - pd.Timedelta(days=TEST_DAYS)
CAL_START = TEST_START - pd.Timedelta(days=CAL_DAYS)
ML_START = CBP_BAD_END + pd.Timedelta(days=15)
REFIT_EVERY = 14
ORIGINS = daily.index[(daily.index >= CAL_START) & (daily.index < T_END) & daily["is_reported"]]
print(f"ML training from {ML_START:%d %b %Y} | stats training from {REGIME_START:%d %b %Y}")
print(f"Calibration {CAL_START:%d %b %Y} → {TEST_START - pd.Timedelta(days=1):%d %b %Y} | Test {TEST_START:%d %b %Y} → {T_END:%d %b %Y} | {len(ORIGINS)} forecast origins")


# %% [markdown]
# **Observation:** The ML models learn from 22 months of trusted history (from 14 Feb 2024) and the statistical models from 9 months of the current regime (from 17 Mar 2025). Forecasts
      are issued from 124 reported days across two 13-week windows, giving a large, strictly out-of-sample evaluation set.


# %%
# Timeline of the design: which data each component sees
fig, ax = plt.subplots(figsize=(12, 2.6))
bars = [("HHS history (EDA, surge replay)", daily.index.min(), T_END, "#86b6ef"),
        ("ML training (trusted CBP)", ML_START, T_END, "#3987e5"),
        ("Statistical training (current regime)", REGIME_START, T_END, "#256abf"),
        ("Calibration window", CAL_START, TEST_START, STATUS["warning"]),
        ("Test window", TEST_START, T_END, STATUS["serious"])]
for i, (name, a, b, col) in enumerate(bars[::-1]):
    ax.barh(i, (b - a).days, left=mdates.date2num(a), height=0.55, color=col)
    if (b - a).days > 250:                                          # long bar: label inside
        ax.text(mdates.date2num(a) + 5, i, name, va="center", fontsize=9, color=INK if col != "#256abf" else "white")
    else:                                                           # short bar: label to its left
        ax.text(mdates.date2num(a) - 5, i, name, va="center", ha="right", fontsize=9, color=INK)
ax.xaxis_date(); ax.set_yticks([]); ax.grid(axis="y", visible=False)
ax.set_title("Train–test design: expanding training windows, walk-forward calibration and test")
savefig(fig, "10_train_test_design")


# %% [markdown]
# **Observation:** The design keeps strict time order. The calibration window (22 Jun – 20 Sep 2025) picks the models and sizes the intervals; the test window (21 Sep – 21 Dec 2025)
      comes after it and is used only for reporting. No model is fitted on data from after its forecast date. One caveat: the regime start date and the data-quality flags were set using
      the full history, a mild design-level look-ahead; the next cell tests whether it matters.


# %%
# Origin-time check of the design constants: would the copied-CBP block and the regime breaks have been found with data up to the FIRST forecast origin only?
def copy_block(key):
    """(start, end) of the longest contiguous run of earlier copies in a CBP key series, or None if there are no copies."""
    c = key.duplicated(keep="last"); rid = (c != c.shift()).cumsum(); r = c.groupby(rid).agg(["first", "size"])
    if not r["first"].any() or r.loc[r["first"], "size"].max() < MIN_COPY_RUN:
        return None
    blk = c[rid == r[r["first"]]["size"].idxmax()].index
    return blk.min(), blk.max()



fmt_rng = lambda b: "none" if b is None else f"{b[0]:%d %b %Y} → {b[1]:%d %b %Y}"
fmt_day = lambda d: "n/a (too short)" if d is None else f"{d:%d %b %Y}"
blk_then = copy_block(cbp_key.loc[:CAL_START])
rep_then = reported.loc[:CAL_START]
brk_t_then = single_break(rep_then.loc[rep_then.index > (blk_then[1] if blk_then else CBP_BAD_END), "transferred"])
```

```
brk_d_then = single_break(rep_then["discharged"])
ORIGIN_TIME_CHECK = pd.DataFrame({
    "full history (used)": [fmt_rng(copy_block(cbp_key)), fmt_day(brk_transfers), fmt_day(brk_discharges)],
    f"data up to {CAL_START:%d %b %Y} only": [fmt_rng(blk_then), fmt_day(brk_t_then), fmt_day(brk_d_then)]},
    index=["copied CBP block", "break in transfers", "break in discharges"])
ORIGIN_TIME_CHECK["identical"] = ORIGIN_TIME_CHECK.iloc[:, 0] == ORIGIN_TIME_CHECK.iloc[:, 1]
originals = [cbp_key.index[(cbp_key == cbp_key[d]) & (cbp_key.index > d)].max() for d in is_copy.index[is_copy & (is_copy.index <= CBP_BAD_END)]]
if originals:
    print(f"Originals of the copied rows are dated {min(originals):%d %b %Y} – {max(originals):%d %b %Y}; first forecast origin {CAL_START:%d %b %Y}")
ORIGIN_TIME_CHECK


# %% [markdown]
# **Observation:** The design-level look-ahead has no effect. With data only up to the first forecast origin (22 Jun 2025) the copy detection finds the identical block (12 Jan 2023 → 30 Jan 2024) and the change-point search finds the identical breaks (26 Jan and 17 Feb 2025), because every original of a copied row is dated 26 Jun 2024 – 12 Feb 2025. All masks and training windows used in the backtest would have been available in real time.


# %% [markdown]
# ## 9. Forecasting models
#
# Every model has the same two-method interface: `fit(t, col, kind)` estimates parameters with data up to origin *t*, and `predict(t, col, kind)` returns a 28-day path issued at *t*.
#
# | Family | Model | Idea |
# |---|---|---|
# | Baseline | Naive persistence | tomorrow = today (hard to beat for a slow-moving stock) |
# | Baseline | Seasonal naive (7d) | each target day repeats the same weekday of the last week (captures the weekly cycle, nothing else) |
# | Baseline | Moving average (7d) | flat forecast at last week's mean |
# | Statistical | ARIMA / SARIMA | ARIMA with optional weekly seasonal terms; orders chosen by AIC (the table below compares the best ARIMA and the best SARIMA) |
# | Statistical | ETS (Holt-Winters) | exponential smoothing; damped trend for care load, weekly season |
# | ML | Random Forest | direct multi-horizon, scale-free target, engineered features |
# | ML | Gradient Boosting | same design, Huber loss (robust to outlier days) |
# | Hybrid | Ensemble | equal-weight mean of SARIMA, ETS, RF and GB |
#
# **Model cards** (what each model needs, how it can fail, and what happens then)
#
# | Model | Training data & features | Hyper-parameters | Horizons | Failure conditions → fallback | Cost (whole backtest) | Uncertainty |
# |---|---|---|---|---|---|---|
# | Naive | last reported value | none | any | none | negligible | empirical (walk-forward errors) |
# | Seasonal naive | value 7·⌈h/7⌉ − h days before the target day's weekday (≤ origin) | none | any | none | negligible | empirical |
# | Moving avg (7d) | last 7 days | window 7 | any | none | negligible | empirical |
# | SARIMA | current regime only; target series (log1p for flows) | (p,d,q)(P,0,Q)₇ by AIC on pre-calibration data; d = 1 for the stock | any (iterated state-space forecast) | non-convergence or < 56 days of history → previous parameters, else naive | seconds | empirical (calibrated); analytic available |
# | ETS | current regime only; target series | additive error and weekly season; damped trend for the stock | any | < 28 days of history (4 weekly cycles) or fit error → previous parameters, else naive | seconds | empirical; analytic used in the replay |
# | Random Forest | trusted window (Feb 2024 →), 40 scale-free features, 180-day recency weights | 200 trees, leaf ≥ 5, 50% features, 70% bootstrap | 1–28 (direct, `h` is a feature) | < 200 training rows or fit error → previous model, else naive | 25–80 s per series | empirical |
# | Gradient Boosting | as RF | 150 trees, depth 3, learning rate 0.07, subsample 0.8, Huber loss | 1–28 (direct) | as RF | 40–120 s per series | empirical |
# | Ensemble | the four model forecasts | equal weights (no fitted weights, so nothing to over-fit) | 1–28 | a component that failed contributes its (flagged) naive fallback | none | empirical |
#
# Every failed fit or prediction is logged in `FIT_LOG` and the walk-forward continues; a forecast that is not finite or has the wrong length is replaced by naive persistence and marked `fallback = True`.


# %%
# Baseline models: naive persistence and a 7-day moving average
class NaiveModel:
    name = "Naive"
    def fit(self, t, col, kind): pass
```

```
    def predict(self, t, col, kind): return np.repeat(daily.at[t, col], H)



class SeasonalNaiveModel:
    name = "Seasonal Naive (7d)"
    def fit(self, t, col, kind): pass
    def predict(self, t, col, kind):
        h = np.arange(1, H + 1)
        return daily[col].reindex(t + pd.to_timedelta(h - 7 * np.ceil(h / 7).astype(int), "D")).to_numpy()   # same weekday, at or before t



class MovingAverageModel:
    name = "Moving Avg (7d)"
    def fit(self, t, col, kind): pass
    def predict(self, t, col, kind): return np.repeat(daily.loc[:t, col].tail(7).mean(), H)

# %%
# Statistical models work on the current regime; flows are modelled on the log scale (log1p) so forecasts stay positive
def stat_series(t, col, kind):
    y = daily.loc[REGIME_START:t, col].asfreq("D")
    return y if kind == "stock" else np.log1p(y)



def stat_back(f, kind):
    return np.asarray(f) if kind == "stock" else np.clip(np.expm1(np.asarray(f)), 0, None)



class SarimaModel:
    name = "SARIMA"
    def __init__(self, order, seasonal_order, trend): self.spec = dict(order=order, seasonal_order=seasonal_order, trend=trend)
    def fit(self, t, col, kind):
        y = stat_series(t, col, kind)
        if len(y) < 56:
            raise ValueError(f"SARIMA needs at least 56 days of current-regime history, got {len(y)}")
        self.res = SARIMAX(y, **self.spec).fit(disp=False)
    def predict(self, t, col, kind):
        return stat_back(self.res.apply(stat_series(t, col, kind)).forecast(H), kind)   # same parameters, updated state



class EtsModel:
    name = "ETS"
    def _model(self, y, kind):
        if kind == "stock":
            return ETSModel(y, error="add", trend="add", damped_trend=True, seasonal="add", seasonal_periods=7)
        return ETSModel(y, error="add", trend=None, seasonal="add", seasonal_periods=7)
    def fit(self, t, col, kind):
        y = stat_series(t, col, kind)
        if len(y) < 28:
            raise ValueError(f"ETS with a weekly season needs at least 28 days (4 cycles), got {len(y)}")
        self.params = self._model(y, kind).fit(disp=False).params
    def predict(self, t, col, kind):
        return stat_back(self._model(stat_series(t, col, kind), kind).smooth(self.params).forecast(H), kind)
```

```
# %%
# SARIMA order selection by AIC on data available before the calibration window (no peeking at evaluation data)
def select_sarima(col, kind):
    y = stat_series(CAL_START - pd.Timedelta(days=1), col, kind)
    d, trend = (1, "n") if kind == "stock" else (0, "c")
    rows = []
    for p in (0, 1, 2):
        for q in (0, 1, 2):
            for P, Q in ((0, 0), (1, 1)):
                try:
                    aic = SARIMAX(y, order=(p, d, q), seasonal_order=(P, 0, Q, 7), trend=trend).fit(disp=False).aic
                    rows.append({"order": (p, d, q), "seasonal": (P, 0, Q, 7), "trend": trend, "AIC": aic})
                except Exception:
                    pass
    return pd.DataFrame(rows).sort_values("AIC").reset_index(drop=True)



SARIMA_SPECS, GRIDS = {}, {}
for col, kind in TARGETS.items():
    grid = GRIDS[col] = select_sarima(col, kind)
    best = grid.iloc[0]
    SARIMA_SPECS[col] = (best["order"], best["seasonal"], best["trend"])
    print(f"{col:12s} best SARIMA{best['order']}x{best['seasonal']} trend={best['trend']}  AIC={best['AIC']:.1f}")


# %% [markdown]
# **Observation:** AIC keeps weekly seasonal terms only for discharges (SARIMA(1,0,1)(1,0,1)₇, AIC 74.9), matching their strong weekly cycle. For care load it selects a simple
      ARIMA(1,1,1) (AIC 617.9) and for transfers an AR(1) (AIC 84.6), because weekly terms did not improve the fit enough. The name "SARIMA" is kept because the seasonal search is part of
      the method.


# %%
# ARIMA vs SARIMA: best non-seasonal vs best weekly-seasonal specification per series (AIC on pre-calibration data; ΔAIC < −2 favours SARIMA)
cmp = []
for col in TARGETS:
    g = GRIDS[col]
    seasonal = g["seasonal"].apply(lambda s: s[0] > 0 or s[2] > 0)
    a, s = g[~seasonal].iloc[0], g[seasonal].iloc[0]
    cmp.append({"series": col, "best ARIMA": str(a["order"]), "AIC ARIMA": a["AIC"], "best SARIMA": f"{s['order']}x{s['seasonal']}",
                "AIC SARIMA": s["AIC"], "ΔAIC (SARIMA − ARIMA)": s["AIC"] - a["AIC"], "specs fitted": len(g)})
ARIMA_VS_SARIMA = pd.DataFrame(cmp).set_index("series").round(1)
ARIMA_VS_SARIMA["selected"] = np.where(ARIMA_VS_SARIMA["ΔAIC (SARIMA − ARIMA)"] < 0, "SARIMA", "ARIMA")
ARIMA_VS_SARIMA


# %% [markdown]
# **Observation:** Weekly seasonal terms earn their place only for discharges, where the best SARIMA beats the best ARIMA by 11.5 AIC points (74.9 vs 86.4). For care load and transfers
      the seasonal versions are worse by 3.4 and 3.7 points, so plain ARIMA(1,1,1) and AR(1) are kept. Each choice comes from 18 fitted specifications on pre-calibration data only.


# %%
# ML models: direct multi-horizon learners on the stacked (origin, horizon) table, with recency weights
ML_HALF_LIFE = 180   # days: a sample from 6 months ago counts half as much as today's



def make_rf():
```

```
    return RandomForestRegressor(n_estimators=200, min_samples_leaf=5, max_features=0.5, max_samples=0.7, n_jobs=-1, random_state=SEED)


def make_gb():
    return GradientBoostingRegressor(n_estimators=150, learning_rate=0.07, max_depth=3, subsample=0.8, max_features=0.5, loss="huber", random_state=SEED)


# Pre-built (day, horizon, feature) tensor so a 28-day forecast is a single fast array lookup
XARR = {col: np.stack([STACKS[col][h][FEATURE_COLS].to_numpy(dtype=float) for h in range(1, H + 1)], axis=1) for col in TARGETS}
POS = pd.Series(np.arange(len(daily)), index=daily.index)


class MLDirectModel:
    def __init__(self, name, factory): self.name, self.factory = name, factory
    def training_table(self, t, col):
        """Rows with origin and target both reported and target date <= t (strictly known at time t)."""
        parts = []
        for h, X in STACKS[col].items():
            ok = (X.index >= ML_START) & (X.index <= t) & daily["is_reported"].to_numpy() & X["target_reported"] & (X["target_date"] <= t)
            parts.append(X[ok])
        tr = pd.concat(parts)
        return tr[np.isfinite(tr[FEATURE_COLS + ["z"]].to_numpy(dtype=float)).all(axis=1)]     # drops NaN and ±inf rows (e.g. a zero care load)
    def fit(self, t, col, kind):
        tr = self.training_table(t, col)
        if len(tr) < 200:
            raise ValueError(f"{self.name} needs at least 200 training rows, got {len(tr)}")
        weights = 0.5 ** ((t - tr.index).days / ML_HALF_LIFE)
        self.model = self.factory().fit(tr[FEATURE_COLS].to_numpy(dtype=float), tr["z"].to_numpy(), sample_weight=weights)
        if hasattr(self.model, "n_jobs"): self.model.set_params(n_jobs=1)    # single-thread predict is faster for 28 rows
        self.n_train = len(tr)
    def predict(self, t, col, kind):
        return from_target(self.model.predict(XARR[col][POS[t]]), daily, t, col, kind)


# %% [markdown]
# **Design experiment (run during development, not repeated here to save runtime).** Which history should the ML models learn from? I ran the same walk-forward protocol with three
      training designs (MAE, children; calibration → test):
#
# | ML training design | Care load RF | Care load GB | Discharges GB |
# |---|---|---|---|
# | Trusted window (Feb 2024 →) + 180-day recency weights **(chosen)** | 89 → 25 | 97 → 39 | 4.2 → 3.7 |
# | Trusted window, no weights | – | 99 → 38 | 4.4 → 3.7 |
# | Current regime only | 125 → 24 | 126 → 26 | 5.8 → 3.8 |
#
# Regime-only training is slightly better late in 2025 but **much worse early**, when the new regime had only about three months of history. The chosen design is the most *robust* across
      both windows, which matters more for an early-warning system than a marginal best case.
#
# **Which evidence made the decision (leakage audit of the design).** The choice follows from the **calibration** column alone (89 vs 125 for Random Forest, 97 vs 126 for Gradient
      Boosting, 4.2 vs 5.8 for discharges), so it would be the same without the test window; test values are shown for transparency only. Likewise, the month feature was removed because
      of the 10% production-forecast cliff, not because of a test score.


# %%
# Assemble the seven base models for a target (Ensemble is added after the backtest)
def build_models(col):
    return [NaiveModel(), SeasonalNaiveModel(), MovingAverageModel(), SarimaModel(*SARIMA_SPECS[col]), EtsModel(),
```

```
            MLDirectModel("Random Forest", make_rf), MLDirectModel("Gradient Boosting", make_gb)]

# %% [markdown]
# ## 10. Walk-forward backtest
#
# Expect several minutes of runtime: 3 targets × 7 models × 124 origins × 28 horizons, about 73,000 forecasts.


# %%
# Walk-forward engine: re-estimate every REFIT_EVERY days, forecast from every reported origin; a failing model never stops the run
FIT_LOG = []



def safe_forecast(m, t, col, kind, fitted):
    """Model forecast, or naive persistence (logged, fallback=True) if the model has no valid fit or returns a bad path."""
    try:
        if not fitted:
            raise RuntimeError("no successful fit yet")
        yhat = np.asarray(m.predict(t, col, kind), dtype=float)
        if yhat.shape != (H,) or not np.isfinite(yhat).all():
            raise ValueError("forecast is not a finite path of length H")
        return yhat, False
    except Exception as e:
        FIT_LOG.append({"target": col, "model": m.name, "origin": t, "stage": "predict", "error": f"{type(e).__name__}: {e}"[:200]})
        return np.repeat(daily.at[t, col], H).astype(float), True



def walk_forward(col, kind, origins=ORIGINS, models=None):
    models = build_models(col) if models is None else models
    rows, last_fit, timing, fitted = [], None, {}, {}
    for t in origins:
        if last_fit is None or (t - last_fit).days >= REFIT_EVERY:
            for m in models:
                t0 = time.time()
                try:
                    m.fit(t, col, kind); fitted[m.name] = True
                except Exception as e:                                          # keep the previous fit (if any) and log the failure
                    FIT_LOG.append({"target": col, "model": m.name, "origin": t, "stage": "fit", "error": f"{type(e).__name__}: {e}"[:200]})
                timing[m.name] = timing.get(m.name, 0) + time.time() - t0
            last_fit = t
        for m in models:
            t0 = time.time(); yhat, fb = safe_forecast(m, t, col, kind, fitted.get(m.name, False)); timing[m.name] = timing.get(m.name, 0) + time.time() - t0
            rows.append(pd.DataFrame({"target": col, "model": m.name, "origin": t, "h": np.arange(1, H + 1),
                                      "date": t + pd.to_timedelta(np.arange(1, H + 1), "D"), "yhat": yhat, "fallback": fb}))
    return pd.concat(rows, ignore_index=True), timing



t_start, parts, timings = time.time(), [], {}
for col, kind in TARGETS.items():
    out, timings[col] = walk_forward(col, kind)
    parts.append(out)
    print(f"{col:12s} done in {time.time() - t_start:5.0f}s  (cumulative)")
BT = pd.concat(parts, ignore_index=True)
print(f"Failed fits/predictions logged: {len(FIT_LOG)} | fallback forecasts used: {int(BT['fallback'].sum())}")
pd.DataFrame(timings).round(1).rename_axis("seconds")
```

```
# %% [markdown]
# **Observation:** In this run the walk-forward backtest took about 8.6 minutes for 3 series × 7 models, with no failed fit and no fallback forecast (wall-clock times vary with machine
      load: identical runs took between 4.9 and 8.6 minutes). Gradient Boosting is the most expensive model (68–119 s per series), followed by Random Forest (57–80 s). The statistical
      models need 3–17 s and the three baselines at most 0.2 s, so all of them are cheap to re-estimate.


# %%
# Add the equal-weight Ensemble, attach actuals, and keep only scoreable rows (reported target dates)
comp = ["SARIMA", "ETS", "Random Forest", "Gradient Boosting"]
ens = BT[BT["model"].isin(comp)].groupby(["target", "origin", "h", "date"], as_index=False)["yhat"].mean().assign(model="Ensemble")
BT = pd.concat([BT, ens], ignore_index=True)
BT["fallback"] = BT["fallback"].fillna(False).astype(bool)
BT["y"], BT["observed"] = np.nan, False
for col in TARGETS:
    m = BT["target"] == col
    BT.loc[m, "y"] = daily[col].reindex(BT.loc[m, "date"]).to_numpy()
    BT.loc[m, "observed"] = (daily["is_reported"] & OBSERVED[col]).reindex(BT.loc[m, "date"], fill_value=False).to_numpy()
BT["scored"] = BT["observed"].astype(bool) & BT["y"].notna()          # reported day with a genuine (non-imputed) value for this series
BT["period"] = np.where(BT["origin"] < TEST_START, "calibration", "test")
BT["error"] = BT["y"] - BT["yhat"]
MODEL_ORDER = list(MODEL_COLORS)
print(f"Backtest rows: {len(BT):,} | scored: {BT['scored'].sum():,}")


# %% [markdown]
# **Observation:** The backtest produced 83,328 forecasts (7 models plus the Ensemble). 52,464 (63%) fall on reported days with a genuine value and can be scored; the rest target
      unreported weekends or dates after the data end. Scoring only real reports keeps every accuracy figure honest.


# %% [markdown]
# ## 11. Evaluation
#
# | Metric | Purpose |
# |---|---|
# | MAE | average absolute miss, in children |
# | RMSE | penalises large misses (capacity planning cares about those) |
# | MAPE | mean of \|error\| ÷ actual over **non-zero actuals only**; the number of excluded zero actuals is reported (`zero y excluded`) |
# | WAPE | Σ\|error\| ÷ Σ\|actual\|: total miss as a share of total volume, stable for low counts |
# | **Forecast Accuracy %** | **100 − WAPE** = 100 × (1 − Σ\|y − ŷ\| ÷ Σ\|y\|). A volume-weighted complement of the relative error, *not* a transformation of MAE or MAPE. It is 100 for a
      perfect forecast, can fall below 0 when errors exceed volumes, and is undefined (NaN) when every actual is zero |
# | Bias | mean(ŷ − y): positive = over-forecast, negative = under-forecast |
# | Horizon error | each metric by horizon h = 1 … 28 (scorecard at h = 1, 7, 14, 28 below) |
# | Interval coverage | share of actuals inside the 80% / 95% interval (§12) |
#
# Every metric is computed on reported target days with a genuine value; an undefined ratio is returned as NaN, never as a number.


# %%
# Metric helper used for every table in this section; undefined ratios (empty input, all-zero actuals) are NaN, never invented
def score(g):
    e, y = g["error"], g["y"]
    if len(g) == 0:
        return pd.Series({"MAE": np.nan, "RMSE": np.nan, "MAPE %": np.nan, "WAPE %": np.nan, "Accuracy %": np.nan, "Bias": np.nan, "n": 0, "zero y excluded": 0})
    nz, vol = y > 0, y.abs().sum()
    wape = e.abs().sum() / vol * 100 if vol > 0 else np.nan
    return pd.Series({"MAE": e.abs().mean(), "RMSE": np.sqrt((e ** 2).mean()),
                      "MAPE %": (e[nz].abs() / y[nz]).mean() * 100 if nz.any() else np.nan, "WAPE %": wape,
                      "Accuracy %": 100 - wape, "Bias": -e.mean(), "n": len(g), "zero y excluded": int((~nz).sum())})
```

```
S = BT[BT["scored"]]
leader = S.groupby(["target", "period", "model"]).apply(score).reset_index().astype({"n": int, "zero y excluded": int})
CHAMPION = (leader[leader["period"] == "calibration"].sort_values("MAE").groupby("target").first()["model"].to_dict())
print("Champion per target (chosen on the CALIBRATION window):", CHAMPION)


# %% [markdown]
# **Observation:** Chosen on the calibration window alone, the champions are SARIMA for care load, the Ensemble for discharges and the 7-day moving average for transfers. The test window
      below shows whether those choices held up on unseen data.


# %%
# Test-window leaderboard per target (sorted by MAE); the champion was fixed before looking at these numbers
test_board = leader[leader["period"] == "test"].drop(columns="period")
for col in TARGETS:
    print(f"\n=== {LABELS[col]} | test window ===")
    display(test_board[test_board["target"] == col].drop(columns="target").sort_values("MAE").set_index("model").round(2))


# %% [markdown]
# **Observation:**
# * **Care load:** every fitted model beats naive persistence, while the flat and weekly baselines do not (moving average 67.6, seasonal naive 67.1), because a rising stock has no weekly
      repeat. The Ensemble is the most accurate (MAE 28.5, 49% below naive's 55.7). The champion SARIMA reaches 32.0 (−43%) at 98.6% accuracy, so the calibration choice held up but was
      not the single best model on new data.
# * All fitted care-load models **under-forecast** (bias −14 to −25 children; naive −55) because care load rose steadily in the test window. Capacity planning should lean on the upper
      interval.
# * **Discharges:** SARIMA is best (MAE 3.33) and the champion Ensemble is close (3.46); both cut naive error by about 25–28%. Seasonal naive captures part of the weekly cycle (4.08,
      −11% vs naive).
# * **Transfers:** ETS is best (3.98), and the champion moving average reaches 4.23 (17% below naive); seasonal naive (5.09) is no better than naive (5.12) because intake has little
      weekly pattern. Intake is the hardest series (accuracy 50–61%).


# %%
# Horizon error curves: MAE by forecast horizon in the test window (small multiples, one per target)
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
by_h = S[S["period"] == "test"].groupby(["target", "model", "h"])["error"].apply(lambda e: e.abs().mean()).reset_index(name="MAE")
for ax, col in zip(axes, TARGETS):
    for m in MODEL_ORDER:
        g = by_h[(by_h["target"] == col) & (by_h["model"] == m)]
        ax.plot(g["h"], g["MAE"], color=MODEL_COLORS[m], lw=2.6 if m == CHAMPION[col] else 1.6, label=m)
    ax.set_title(LABELS[col], fontsize=10.5); ax.set_xlabel("horizon (days ahead)"); ax.set_ylabel("MAE (children)")
    ax.set_xticks([1, 7, 14, 21, 28])
axes[0].legend(loc="upper left", fontsize=8.5)
fig.suptitle("Horizon error: how fast accuracy decays with lead time (test window)", x=0.01, ha="left", fontweight="bold")
fig.tight_layout(); savefig(fig, "11_horizon_error")


# %% [markdown]
# **Observation:** For care load, naive error grows almost linearly from 5.7 children at 1 day to 111 at 28 days. At 1 day nothing beats "tomorrow = today". By day 14 the Ensemble and
      Gradient Boosting halve naive's error (26.8 and 25.1 vs 55.5) and SARIMA/ETS cut it by about 42%; by day 28 SARIMA, ETS and the Ensemble are all about half of naive (57–59). The
      Ensemble is the best care-load model at 15 of the 28 horizons. Flow errors stay almost flat (about 3–5 a day at every horizon), and SARIMA is the best discharge model at 19 of 28
      horizons.


# %%
# Short- vs medium-term reliability: MAE by horizon bucket (week 1 … week 4)
S = S.assign(bucket=pd.cut(S["h"], [0, 7, 14, 21, 28], labels=["1–7d", "8–14d", "15–21d", "22–28d"]))
bucket_tbl = (S[S["period"] == "test"].groupby(["target", "model", "bucket"], observed=True)["error"]
```

```
            .apply(lambda e: e.abs().mean()).unstack("bucket").round(1))
bucket_tbl


# %% [markdown]
# **Observation:** Week-1 care-load forecasts are already precise (about 13 children for SARIMA, ETS and the Ensemble). By week 4 those models plus Gradient Boosting err by 48–53
      children versus 100.5 for naive, about half. Gradient Boosting is the best model in weeks 2–3 (23.0 and 32.3), while SARIMA keeps discharge error flat at 3.3–3.4 in every week.


# %%
# Statistical vs machine-learning: best model of each family by horizon bucket (test MAE)
FAMILY = {"Naive": "Baseline", "Seasonal Naive (7d)": "Baseline", "Moving Avg (7d)": "Baseline", "SARIMA": "Statistical", "ETS": "Statistical",
          "Random Forest": "Machine learning", "Gradient Boosting": "Machine learning", "Ensemble": "Ensemble"}
fam = bucket_tbl.reset_index().assign(family=lambda d: d["model"].map(FAMILY))
fam_best = fam.groupby(["target", "family"])[["1–7d", "8–14d", "15–21d", "22–28d"]].min()
fam_best


# %% [markdown]
# **Observation:** Statistical models are the best family for discharges from week 2 onwards (3.3–3.4 vs 3.7–4.2 for ML) and beat ML for transfers in every week. For the care-load stock,
      ML is best in weeks 2–3 and the Ensemble in weeks 1 and 4. No single family wins everywhere: statistical models suit the flows, while ML and the Ensemble add value for the stock.
      The Ensemble is never the worst family for any series or week, which makes it a safe default when the best model for a series is uncertain.


# %%
# Diebold–Mariano test: is the champion significantly better than naive persistence? (HAC variance, Harvey correction)
def diebold_mariano(e1, e2, h):
    e1, e2 = np.asarray(e1, float), np.asarray(e2, float)
    if len(e1) != len(e2) or len(e1) < 2:
        return np.nan, np.nan                       # undefined: mismatched inputs or too few pairs
    d = np.abs(e1) - np.abs(e2)                     # loss differential (absolute error)
    n, dbar = len(d), d.mean()
    gamma = [np.mean((d[k:] - dbar) * (d[:n - k] - dbar)) for k in range(min(h, n))]
    var = (gamma[0] + 2 * sum((1 - k / h) * gamma[k] for k in range(1, len(gamma)))) / n   # Bartlett weights keep the variance positive
    corr = (n + 1 - 2 * h + h * (h - 1) / n) / n    # Harvey, Leybourne & Newbold small-sample correction
    if var <= 0 or corr <= 0:
        return (0.0, 1.0) if np.allclose(d, 0) else (np.nan, np.nan)    # identical forecasts: no difference; otherwise undefined
    stat = dbar / np.sqrt(var) * np.sqrt(corr)
    return stat, 2 * stats.t.sf(abs(stat), df=n - 1)



dm_rows = []
for col in TARGETS:
    T = S[(S["period"] == "test") & (S["target"] == col)]
    for h in (1, 7, 14, 28):
        a = T[(T["model"] == CHAMPION[col]) & (T["h"] == h)].set_index("origin")["error"]
        b = T[(T["model"] == "Naive") & (T["h"] == h)].set_index("origin")["error"]
        a, b = a.align(b, join="inner")
        if CHAMPION[col] == "Naive" or len(a) < 10:
            continue
        stat, p = diebold_mariano(a.to_numpy(), b.to_numpy(), h)
        dm_rows.append({"target": col, "champion": CHAMPION[col], "h": h, "MAE champion": a.abs().mean(),
                        "MAE naive": b.abs().mean(), "DM stat": stat, "p-value": p, "significant (5%)": p < 0.05})
pd.DataFrame(dm_rows).round(3)


# %% [markdown]
# **Observation:** For care load the champion's gain over naive is not significant at 1 day (p = 0.89) but becomes significant at 14 days (p = 0.045) and 28 days (p < 0.001), the
      horizons that matter for capacity planning. For discharges the Ensemble's gain is significant at 1 and 7 days (p ≤ 0.008), and for transfers only at 1 day (p = 0.035). At longer
      horizons the flow forecasts still have 11–20% lower error than naive, but that advantage is not statistically significant.
```

```
# %%
# ARIMA vs SARIMA out of sample: walk-forward the best specification of the OTHER family and compare MAE (calibration = decision window, test = report)
def fmt_spec(order, seas):
    return f"ARIMA{tuple(order)}" if seas[0] == 0 and seas[2] == 0 else f"SARIMA{tuple(order)}x{tuple(seas)}"



alt_rows, t_alt = [], time.time()
lead_idx = leader.set_index(["target", "period", "model"])
for col, kind in TARGETS.items():
    g = GRIDS[col]; seasonal = g["seasonal"].apply(lambda s: s[0] > 0 or s[2] > 0)
    alt = g[~seasonal].iloc[0] if seasonal.iloc[0] else g[seasonal].iloc[0]
    m_alt = SarimaModel(alt["order"], alt["seasonal"], alt["trend"]); m_alt.name = "alternative"
    out, _ = walk_forward(col, kind, models=[m_alt])
    out["y"] = daily[col].reindex(out["date"]).to_numpy()
    keep = (daily["is_reported"] & OBSERVED[col]).reindex(out["date"], fill_value=False).to_numpy() & out["y"].notna().to_numpy()
    out = out[keep].assign(error=lambda x: x["y"] - x["yhat"], period=lambda x: np.where(x["origin"] < TEST_START, "calibration", "test"))
    for per, gg in out.groupby("period"):
        alt_rows.append({"series": col, "period": per, "selected (lowest AIC)": fmt_spec(*SARIMA_SPECS[col][:2]),
                         "MAE selected": lead_idx.at[(col, per, "SARIMA"), "MAE"], "other family": fmt_spec(alt["order"], alt["seasonal"]),
                         "MAE other": gg["error"].abs().mean()})
ARIMA_OOS = pd.DataFrame(alt_rows).set_index(["series", "period"]).round(2)
print(f"Alternative specifications walked forward in {time.time() - t_alt:.0f}s")
ARIMA_OOS


# %% [markdown]
# **Observation:** Out-of-sample errors on the calibration window, the decision window, confirm the AIC choices for all three series: care load 55.1 vs 55.3, discharges 4.09 vs 4.76 and
      transfers 4.47 vs 4.60 (selected vs other family). On the test window the seasonal care-load model happens to be slightly better (31.3 vs 32.0 children, 2%). Switching after seeing
      that number would be test-set selection, so the specification stands.


# %% [markdown]
# ### Recursive vs direct multi-horizon strategy
#
# The main ML models use the **direct** strategy (one model that is told the horizon). The alternative is **recursive**: a one-step model fed its own predictions for 28 steps. To isolate
      the strategy, both versions below use the same learner (Gradient Boosting, 100 trees, depth 3, Huber loss), the same recency weights, the same target-only inputs (the last 14
      transformed values, their 7- and 14-day means, and the target weekday) and the same walk-forward origins, refitted every 28 days. The recursive model must step through every
      calendar day, so it is trained on interpolated Friday/Saturday values too; the direct model trains on reported targets only. Both are scored on reported days only.


# %%
# Recursive vs direct, same learner and inputs: walk-forward on every origin, refit every 28 days (care load = log-change, discharges = log1p)
RD_LAGS = 14
DOW = daily.index.dayofweek.to_numpy()



def rd_transform(col, kind):
    y = daily[col]
    return (np.log(y / y.shift(1)) if kind == "stock" else np.log1p(y)).to_numpy()



def rd_rows(x, pos):
    """Target-only inputs at positions pos: the last 14 transformed values and their 7- and 14-day means."""
    L = np.stack([x[pos - k] for k in range(RD_LAGS)], axis=1)
    return np.column_stack([L, L[:, :7].mean(1), L.mean(1)])
```

```
def make_rd_gb():
    return GradientBoostingRegressor(n_estimators=100, learning_rate=0.08, max_depth=3, subsample=0.8, loss="huber", random_state=SEED)



rd_out, t_rd = [], time.time()
rep_pos = np.flatnonzero(daily["is_reported"].to_numpy())
for col, kind in (("hhs_care", "stock"), ("discharged", "flow")):
    x, y = rd_transform(col, kind), daily[col].to_numpy()
    ok_target = (daily["is_reported"] & OBSERVED[col]).to_numpy()
    start, last_fit, hs = POS[ML_START], None, np.arange(1, H + 1)
    for t in ORIGINS:
        p = POS[t]
        if last_fit is None or (t - last_fit).days >= 28:
            wts = lambda pos: 0.5 ** ((p - pos) / ML_HALF_LIFE)
            s1 = np.arange(start, p)                                          # one-step pairs (s → s+1), all known at t
            rec = make_rd_gb().fit(np.column_stack([rd_rows(x, s1), DOW[s1 + 1]]), x[s1 + 1], sample_weight=wts(s1))
            Xs, zs, ws = [], [], []
            for h in hs:
                s = rep_pos[(rep_pos >= start) & (rep_pos + h <= p)]
                s = s[ok_target[s + h]]
                Xs.append(np.column_stack([rd_rows(x, s), DOW[s + h], np.full(len(s), h)]))
                zs.append(np.log(y[s + h] / y[s]) if kind == "stock" else x[s + h]); ws.append(wts(s))
            direct = make_rd_gb().fit(np.vstack(Xs), np.concatenate(zs), sample_weight=np.concatenate(ws))
            last_fit = t
        base = rd_rows(x, np.array([p]))[0]
        zd = direct.predict(np.column_stack([np.tile(base, (H, 1)), (DOW[p] + hs) % 7, hs]))
        f_direct = y[p] * np.exp(zd) if kind == "stock" else np.clip(np.expm1(zd), 0, None)
        hist, level, f_rec = list(x[p - RD_LAGS + 1:p + 1]), y[p], []
        for h in hs:
            L = np.array(hist[::-1][:RD_LAGS])
            xh = rec.predict(np.r_[L, L[:7].mean(), L.mean(), (DOW[p] + h) % 7][None])[0]
            hist.append(xh)
            level = level * np.exp(xh) if kind == "stock" else max(np.expm1(xh), 0)
            f_rec.append(level)
        for strat, f in (("Direct", f_direct), ("Recursive", np.array(f_rec))):
            rd_out.append(pd.DataFrame({"target": col, "strategy": strat, "origin": t, "h": hs, "date": t + pd.to_timedelta(hs, "D"), "yhat": f}))
RD = pd.concat(rd_out, ignore_index=True)
RD["y"] = [daily.at[d, c] if d <= T_END else np.nan for d, c in zip(RD["date"], RD["target"])]
RD["ok"] = [bool(daily.at[d, "is_reported"] and OBSERVED.at[d, c]) if d <= T_END else False for d, c in zip(RD["date"], RD["target"])]
RD = RD[RD["ok"]].assign(error=lambda x: x["y"] - x["yhat"], period=lambda x: np.where(x["origin"] < TEST_START, "calibration", "test"))
RD_TABLE = (RD.assign(bucket=pd.cut(RD["h"], [0, 7, 14, 21, 28], labels=["1–7d", "8–14d", "15–21d", "22–28d"]))
            .groupby(["target", "period", "strategy", "bucket"], observed=True)["error"].apply(lambda e: e.abs().mean()).unstack("bucket").round(2))
print(f"Recursive vs direct: {RD['origin'].nunique()} origins × 2 series in {time.time() - t_rd:.0f}s")
RD_TABLE


# %% [markdown]
# **Observation:** With the same learner and inputs, the **recursive** strategy is more accurate for care load in every week of both windows (test: 13.0 vs 24.3 children in week 1, 40.6
      vs 64.7 in week 4), because a one-step model of the daily log-change compounds smoothly. For discharges the result is mixed: recursive is better in week 1 (3.16 vs 3.47 on test) but
      worse from week 3 (4.13–4.17 vs 3.61–3.68), as its own errors feed back into its inputs. On the calibration window neither variant comes close to the care-load champion (week
      averages of about 81 children for recursive vs SARIMA's 55), although recursive would have beaten it on the test window (about 28 vs 32). The production design is therefore
      unchanged, and a recursive care-load learner is recorded as a candidate for the next model review.


# %%
```

```
# Horizon error of the two strategies on the test window, with the full 40-feature direct Gradient Boosting model for reference
fig, axes = plt.subplots(1, 2, figsize=(13, 3.8))
STRAT_COLORS = {"Direct": "#2a78d6", "Recursive": "#eb6834"}
for ax, col in zip(axes, ("hhs_care", "discharged")):
    for strat, colr in STRAT_COLORS.items():
        g = RD[(RD["target"] == col) & (RD["strategy"] == strat) & (RD["period"] == "test")].groupby("h")["error"].apply(lambda e: e.abs().mean())
        ax.plot(g.index, g.values, color=colr, lw=2, label=f"{strat} (target-only inputs)")
    ref = by_h[(by_h["target"] == col) & (by_h["model"] == "Gradient Boosting")]
    ax.plot(ref["h"], ref["MAE"], color=MODEL_COLORS["Gradient Boosting"], lw=1.4, ls="--", label="Gradient Boosting (40 features, direct)")
    ax.set_title(LABELS[col], fontsize=10.5); ax.set_xlabel("horizon (days ahead)"); ax.set_ylabel("MAE"); ax.set_xticks([1, 7, 14, 21, 28])
axes[0].legend(loc="upper left", fontsize=8.5)
fig.suptitle("Recursive vs direct multi-horizon forecasting (test window)", x=0.01, ha="left", fontweight="bold")
fig.tight_layout(); savefig(fig, "11b_recursive_vs_direct")


# %% [markdown]
# **Observation:** On the test window, recursive care-load errors stay below the direct target-only model at every horizon. Against the full 40-feature Gradient Boosting model they are
      lower in week 1 (13.0 vs 19.7) and week 4 (40.6 vs 53.3) and level in weeks 2–3 (23.9 vs 23.0, 33.1 vs 32.3), so the strategy matters most at the shortest and longest lead times.
      For discharges the two strategies cross after about a week, after which the direct one is steadier. This supports direct models for the noisy flows and a recursive trial for the
      smooth stock.


# %% [markdown]
# ## 12. Uncertainty quantification: conformal intervals
#
# A point forecast cannot answer "how bad could it get?". For every model, target and horizon I take the **empirical distribution of calibration-window errors** (pooled over neighbouring
      horizons h ± 2 for stability) and add its quantiles to the point forecast. This is split-conformal prediction: distribution-free and honest, because the test window never touched
      the calibration.


# %%
# Residual quantiles per (target, model, horizon) from a chosen evaluation period
QUANTS = {"lo95": 0.025, "lo80": 0.10, "hi80": 0.90, "hi95": 0.975}



def residual_quantiles(frame):
    out = []
    for (col, m), g in frame.groupby(["target", "model"]):
        for h in range(1, H + 1):
            e = g.loc[(g["h"] - h).abs() <= 2, "error"]
            out.append({"target": col, "model": m, "h": h, **{k: e.quantile(q) for k, q in QUANTS.items()}})
    return pd.DataFrame(out)



Q_CAL = residual_quantiles(S[S["period"] == "calibration"])
TEST = S[S["period"] == "test"].merge(Q_CAL, on=["target", "model", "h"])
for k in QUANTS:
    TEST[k] = TEST["yhat"] + TEST[k]
TEST["in80"] = TEST["y"].between(TEST["lo80"], TEST["hi80"])
TEST["in95"] = TEST["y"].between(TEST["lo95"], TEST["hi95"])
coverage = TEST.groupby(["target", "model"]).agg(coverage80=("in80", "mean"), coverage95=("in95", "mean"),
                                                 width80=("hi80", lambda s: (s - TEST.loc[s.index, "lo80"]).mean())).round(3)
coverage


# %% [markdown]
# **Observation:**
```

```
# * **Care load:** intervals are conservative. Nominal 80% bands contain the actual value 92–100% of the time because the calibration window included the larger summer-2025 swings;
      erring wide is the safe side for capacity planning.
# * **Discharges:** the statistical models are close to target (86–89%); Random Forest and Gradient Boosting are slightly narrow (74%), the Ensemble 77%.
# * **Transfers:** most bands are too narrow (60–73%), with only SARIMA on target (80%). The cause is not higher volatility, since transfers were actually steadier in the test window (SD
      4.7 vs 5.0 a day). Instead, most models under-forecast by 1–2 transfers a day during calibration but not during the test, so the calibrated bands ended up off-centre. Use the 95%
      band for intake planning; the production forecast recalibrates on the latest 26 weeks.


# %% [markdown]
# ### 12.1 Rolling-origin recalibration (open defect D-MOD-01)
#
# The fixed calibration window cannot react when a model's bias changes. The production forecast already uses a different rule: at its origin it takes the error quantiles of **all walk-
      forward forecasts whose target date has already been observed**, from the last 26 weeks of origins. Here that exact rule is replayed at every test origin *t*: only errors with
      target date ≤ *t* (observable at the time) from origins in the previous 182 days (and, as a sensitivity check, 91 days) are used, so there is no look-ahead. The method was fixed
      before looking at its test results; the test window only reports how it would have performed in real time.


# %%
# Rolling-origin conformal: at each test origin use only errors already observable (target date <= origin) from the previous L days of origins
def rolling_quantiles(frame, lookback, min_errors=30):
    out = []
    test_origins = ORIGINS[ORIGINS >= TEST_START]
    for (col, m), g in frame.groupby(["target", "model"]):
        o, d, hh, e = (g[c].to_numpy() for c in ("origin", "date", "h", "error"))
        for t in test_origins:
            known = (d <= t.to_datetime64()) & (o >= (t - pd.Timedelta(days=lookback)).to_datetime64())
            for h in range(1, H + 1):
                sel = e[known & (np.abs(hh - h) <= 2)]
                q = np.quantile(sel, list(QUANTS.values())) if len(sel) >= min_errors else [np.nan] * len(QUANTS)
                out.append((col, m, t, h, *q))
    return pd.DataFrame(out, columns=["target", "model", "origin", "h", *QUANTS])



def coverage_of(qtab, keys):
    T = S[S["period"] == "test"].merge(qtab, on=keys)
    for k in QUANTS:
        T[k] = T["yhat"] + T[k]
    T = T.dropna(subset=list(QUANTS))
    return T.assign(in80=T["y"].between(T["lo80"], T["hi80"]), in95=T["y"].between(T["lo95"], T["hi95"]), width80=T["hi80"] - T["lo80"])



t_rq = time.time()
COV_METHODS = {"fixed calibration (13 wk)": coverage_of(Q_CAL, ["target", "model", "h"])}
for L in (182, 91):
    COV_METHODS[f"rolling origin ({L // 7} wk)"] = coverage_of(rolling_quantiles(S, L), ["target", "model", "origin", "h"])
COV_COMPARE = pd.concat({name: T.groupby(["target", "model"]).agg(coverage80=("in80", "mean"), coverage95=("in95", "mean"), width80=("width80", "mean"))
                         for name, T in COV_METHODS.items()}, axis=1).round(3)
print(f"Rolling quantiles computed in {time.time() - t_rq:.0f}s")
COV_COMPARE.loc[list(dict.fromkeys([(c, CHAMPION[c]) for c in TARGETS] + [(c, "Ensemble") for c in TARGETS]))].sort_index()


# %% [markdown]
# **Observation:** Rolling recalibration helps only modestly and does **not** fix D-MOD-01. For the transfer champion (7-day moving average) 80% coverage moves from 66.0% with the fixed
      calibration window to 68.4% with the production 26-week rule and 71.1% with a 13-week window, still well below 80%; the transfer Ensemble improves from 68.4% to 71.9%. For care load
      the rolling rule narrows the over-wide bands (SARIMA width 166 → 150 children at 90% coverage), and a 13-week window is almost exactly calibrated (81.7%) with bands about 35%
      narrower (108). The discharge Ensemble moves closer to target (77.2% → 78.7%). Shorter windows react faster but lose 95% coverage (transfers 86–87%, care 94.5%). The remaining
      transfer gap reflects the early-test bias shift, which an empirical window absorbs only slowly; adaptive conformal methods are the documented next step, and intake planning should
      keep using the 95% band.
```

```
# %%
# Coverage by horizon (champion models): does calibration hold from day 1 to day 28 under both methods?
cov_h = pd.concat({name: T.groupby(["target", "model", "h"])["in80"].mean() for name, T in COV_METHODS.items()}, axis=1)
COV_BY_H = cov_h.reset_index()
fig, axes = plt.subplots(1, 3, figsize=(15, 3.8), sharey=True)
METHOD_COLORS = dict(zip(COV_METHODS, ["#898781", "#2a78d6", "#eb6834"]))
for ax, col in zip(axes, TARGETS):
    g = COV_BY_H[(COV_BY_H["target"] == col) & (COV_BY_H["model"] == CHAMPION[col])]
    for name, colr in METHOD_COLORS.items():
        ax.plot(g["h"], g[name] * 100, color=colr, lw=2, label=name)
    ax.axhline(80, color=STATUS["critical"], lw=1); ax.set_ylim(0, 105)
    ax.set_title(f"{LABELS[col]} ({CHAMPION[col]})", fontsize=10.5); ax.set_xlabel("horizon (days ahead)"); ax.set_xticks([1, 7, 14, 21, 28])
axes[0].set_ylabel("80% interval coverage (%)"); axes[0].legend(loc="lower left", fontsize=8.5)
fig.suptitle("Interval coverage by horizon on the test window (target 80%)", x=0.01, ha="left", fontweight="bold")
fig.tight_layout(); savefig(fig, "12b_coverage_by_horizon")


# %% [markdown]
# **Observation:** Under-coverage is concentrated in multi-week flow forecasts. The discharge Ensemble covers 84–91% of actuals in week 1 but 70–76% in week 4 (depending on the method),
      and the transfer moving average 72–77% in week 1 versus 61–66% in week 4, with single horizons as low as 47–56%. Care-load coverage averages 82–93% per week under every method,
      although single horizons dip to 71–77%. Each point rests on only 42–55 test forecasts, so the curves are noisy and week-level averages are the safer reading.


# %%
# Multi-horizon scorecard at operational lead times (1, 7, 14, 28 days; test window): accuracy, bias and coverage of the rolling 26-week intervals
roll = COV_METHODS["rolling origin (26 wk)"].set_index(["target", "model", "origin", "h"])[["in80", "in95"]]
Tm = S[(S["period"] == "test") & S["h"].isin([1, 7, 14, 28])].join(roll, on=["target", "model", "origin", "h"])
Tm[["in80", "in95"]] = Tm[["in80", "in95"]].astype(float)          # NaN where too few observable errors existed
SCORECARD = Tm.groupby(["target", "model", "h"]).apply(lambda g: pd.concat([score(g), pd.Series({"coverage80": g["in80"].mean(), "coverage95": g["in95"].mean()})]))
SCORECARD = SCORECARD.drop(columns=["zero y excluded"]).astype({"n": int})
show = [(c, m) for c in TARGETS for m in dict.fromkeys(["Naive", "Seasonal Naive (7d)", CHAMPION[c], "Ensemble"])]
SCORECARD.reset_index().set_index(["target", "model"]).loc[show].reset_index().set_index(["target", "model", "h"]).round(2)


# %% [markdown]
# **Observation:** The scorecard gives each planning lead time its own numbers. SARIMA's care-load error grows from 5.8 children at 1 day to 19.2 at 7, 32.5 at 14 and 58.6 at 28 days
      (accuracy 99.8% → 97.5%), 34% below naive at 7 days and 41–47% below at 14–28 days, with a growing under-forecast during the steady test-window climb (bias −3 at 7 days, −44 at 28).
      Seasonal naive equals naive at 7, 14 and 28 days by construction (the anchor is the origin day itself) and is worse at 1 day (31.1). Discharge Ensemble error stays at 3.0–3.7 a day
      with a small over-forecast (+0.3 to +1.8), and the transfer moving average at 3.5–4.6. Coverage of the rolling 26-week 80% intervals is 88–94% for care load but only 66–71% for
      discharges at 14–28 days and 52–77% for transfers, which again marks the flow intervals as the weak spot.


# %%
# Fan chart: champion care-load forecasts issued at four test origins, with calibrated 80% / 95% bands
col = "hhs_care"; champ = CHAMPION[col]
fan_origins = ORIGINS[ORIGINS >= TEST_START][::18][:4]
fig, ax = plt.subplots(figsize=(12, 4.4))
hist = daily.loc[TEST_START - pd.Timedelta(days=21):, col]
ax.plot(hist.index, hist, color=INK, lw=1.6, label="actual")
for o in fan_origins:
    g = TEST[(TEST["target"] == col) & (TEST["model"] == champ) & (TEST["origin"] == o)].sort_values("date")
    ax.fill_between(g["date"], g["lo95"], g["hi95"], color=MODEL_COLORS[champ], alpha=0.10, lw=0)
    ax.fill_between(g["date"], g["lo80"], g["hi80"], color=MODEL_COLORS[champ], alpha=0.18, lw=0)
    ax.plot(g["date"], g["yhat"], color=MODEL_COLORS[champ], lw=2)
    ax.scatter([o], [daily.at[o, col]], s=40, color=MODEL_COLORS[champ], edgecolor=SURFACE, linewidth=2, zorder=4)
ax.yaxis.set_major_formatter(COMMA)
ax.set_title(f"{champ}: 28-day care-load forecasts with 80% and 95% intervals (test window)")
ax.legend(["actual"], loc="upper left")
savefig(fig, "12_fan_chart")
```

```
# %% [markdown]
# **Observation:** For the four plotted forecast dates, 89–100% of actual values fall inside the 80% band, consistent with the conservative coverage above. The 16 Oct forecast was within
      9 children at day 28, while the 12 Nov forecast under-shot by 68 (2,371 vs 2,439) because the climb accelerated. The bands widen with lead time, which is the uncertainty planners
      need to budget for.


# %% [markdown]
# ## 13. Key performance indicators
#
# | KPI | Definition used here |
# |---|---|
# | **Forecast Accuracy (%)** | 100 − WAPE on the test window (robust for small counts) |
# | **Surge Lead Time** | days between the first sustained forecast alert and the actual threshold crossing |
# | **Capacity Breach Probability** | P(care load > threshold) from the calibrated forecast distribution |
# | **Forecast Stability Index (FSI)** | 100 / (1 + mean revision ÷ mean actual daily change); higher = steadier forecasts |
# | **Model robustness** | win-rate vs naive across test weeks, error growth from week 1 to week 4, interval coverage |


# %%
# KPI 1: Forecast Accuracy (%) for every model and target on the untouched test window
ACCURACY = test_board.pivot(index="model", columns="target", values="Accuracy %").reindex(MODEL_ORDER)[list(TARGETS)].round(1)
ACCURACY


# %% [markdown]
# **Observation:** Care load is forecast with 97–99% accuracy by every model, so the differences that matter are at longer horizons and in the tails, not in this headline number. Flows
      are much harder: the best discharge model reaches 65.9% and the best transfer model 61.3%, versus 50–53% for naive. At about 10 events a day much of the daily variation is random,
      but it averages out over a week: summing the reported (Sunday–Thursday) days within each week cuts SARIMA's relative discharge error from 34% to 16% (naive: 37%). Weekly totals are
      therefore the better planning unit for placements.


# %%
# KPI 4: Forecast Stability Index: revisions to the forecast for the same date between consecutive origins
def stability_index(col, model):
    f = BT[(BT["target"] == col) & (BT["model"] == model) & (BT["period"] == "test")].sort_values(["date", "origin"])
    revision = f.groupby("date")["yhat"].diff().abs().mean()
    rep = daily.loc[TEST_START:T_END].query("is_reported")[col]
    natural = rep.diff().abs().mean()
    return 100 / (1 + revision / natural)



FSI = pd.DataFrame({col: {m: stability_index(col, m) for m in MODEL_ORDER} for col in TARGETS}).round(1)
FSI


# %% [markdown]
# **Observation:** Flow forecasts are much steadier than naive (FSI 74–96 vs 50), so they do not chase daily noise. The trend-following care-load models are less stable than naive (FSI
      23–33 vs 51; the flat baselines are steadiest: seasonal naive 63.5 and moving average 58.5): when a new report changes the recent slope, the whole 28-day path tilts. Planners should
      read each run together with its interval and judge trends over several days rather than reacting to a single forecast.


# %%
# KPI 5: robustness scorecard (weekly win-rate vs naive, week-4 / week-1 error growth, 80% interval coverage)
def robustness(col, model):
    T = S[(S["period"] == "test") & (S["target"] == col)]
    wk = T.assign(week=T["origin"].dt.to_period("W")).groupby(["week", "model"])["error"].apply(lambda e: e.abs().mean()).unstack()
    win = (wk[model] < wk["Naive"]).mean() * 100 if model != "Naive" else np.nan
    mt = T[T["model"] == model]
```

```
    growth = mt.loc[mt["h"] > 21, "error"].abs().mean() / mt.loc[mt["h"] <= 7, "error"].abs().mean()
    return pd.Series({"win-rate vs naive %": win, "error growth wk4/wk1": growth,
                      "coverage80 %": coverage.loc[(col, model), "coverage80"] * 100, "FSI": FSI.loc[model, col],
                      "accuracy %": ACCURACY.loc[model, col]})



ROBUST = pd.concat({col: pd.DataFrame({m: robustness(col, m) for m in MODEL_ORDER}).T for col in TARGETS}).round(1)
ROBUST


# %% [markdown]
# **Observation:** The Ensemble is the most robust care-load model: it beat naive in 100% of test weeks at 98.8% accuracy. Gradient Boosting has the slowest error growth (2.7× from week
      1 to week 4, vs 5.6× for naive), partly because its week-1 error is already high (19.7 vs naive's 17.9). Random Forest is the weakest modelled option for care load (71% win-rate,
      4.0× growth). For flows, error barely grows with horizon (1.0–1.4×), so 4-week flow forecasts are about as reliable as 1-week ones.


# %% [markdown]
# ### KPI 2: Surge lead time
#
# A surge **event** is the first reported day the care load crosses above a threshold *C* after at least 14 days below it. An **alert** is raised at origin *t* when the **probability of
      exceeding *C* within 28 days reaches p\***. **Lead time** is the number of days between the start of the unbroken alert streak and the event. A threshold sweep turns one series into
      many events, which makes the KPI far more robust than a single hand-picked capacity number.
#
# Alerting on the point forecast alone (p\* = 50%) proved too late during development: trend models smooth over the start of a surge. So I evaluate the whole **operating curve** (p\*
      from 50% down to 10%) and report detection rate, median lead and false-alarm rate, letting planners pick the sensitivity.
#
# I measure it twice:
# 1. **Test window (2025)** with the champion care-load model and its calibrated (conformal) error distribution.
# 2. **Historical replay (2023–2024)**, which contains the program's large real surges (care load rose from about 5,700 to over 11,000). Because only the HHS columns are trustworthy in
      2023, the replay uses a univariate damped-trend ETS model with analytic prediction intervals, re-estimated on a rolling 120-day window. This simulates what a planner would have seen
      in real time.


# %%
# Event detection, lead-time and false-alarm scoring shared by both evaluations
P_LEVELS = [0.5, 0.4, 0.3, 0.2, 0.1]   # alert when P(breach within 28 days) >= p*
P_STAR = 0.3                           # recommended operating point, chosen on the 2023–24 replay curve (the test window plays no part in the choice)



def surge_events(actual, thresholds, min_below=14):
    """First reported day above threshold C after at least `min_below` days below it."""
    events = []
    for C in thresholds:
        last_above, prev = None, None
        for d, v in actual.items():
            if v > C and prev is not None and prev <= C and (last_above is None or (d - last_above).days >= min_below):
                events.append((C, d))
            if v > C:
                last_above = d
            prev = v
    return pd.DataFrame(events, columns=["threshold", "event_date"])



def lead_times(events, peak, origins):
    """peak: Series by origin = highest forecast quantile over the next 28 days. Lead = length of the unbroken alert streak."""
    out = []
```

```
    for C, E in events.itertuples(index=False):
        prior = [o for o in origins if E - pd.Timedelta(days=H) <= o < E]
        streak = None
        for o in reversed(prior):
            if peak[o] >= C:
                streak = o
            else:
                break
        out.append({"threshold": C, "event_date": E, "lead_days": (E - streak).days if streak is not None else 0, "detected": streak is not None})
    return pd.DataFrame(out)



def false_alarm_rate(peak, actual, thresholds):
    """Share of alerts after which the actual load did NOT exceed the threshold within 28 days."""
    rates = []
    for C in thresholds:
        alerts = [o for o in peak.index[peak >= C] if o + pd.Timedelta(days=H) <= actual.index.max()]
        if alerts:
            rates.append(np.mean([actual.loc[o + pd.Timedelta(days=1):o + pd.Timedelta(days=H)].max() <= C for o in alerts]))
    return np.mean(rates) if rates else np.nan



def operating_curve(peaks, actual, events, thresholds, fa_origins=None):
    rows = []
    for p in P_LEVELS:
        lt = lead_times(events, peaks[p], list(peaks.index))
        fa = peaks[p] if fa_origins is None else peaks[p].loc[fa_origins]
        rows.append({"alert at P(breach) ≥": p, "events": len(lt), "detected %": lt["detected"].mean() * 100,
                     "median lead (days)": lt["lead_days"].median(), "mean lead (days)": lt["lead_days"].mean(),
                     "false alarms %": false_alarm_rate(fa, actual, thresholds) * 100})
    return pd.DataFrame(rows).round(1)

# %%
# (1) Test window: champion forecast + calibrated error quantiles give each origin's (1 − p) forecast quantile path
col, champ = "hhs_care", CHAMPION["hhs_care"]
cal_err = S[(S["period"] == "calibration") & (S["target"] == col) & (S["model"] == champ)]
qtab = pd.DataFrame({p: [cal_err.loc[(cal_err["h"] - h).abs() <= 2, "error"].quantile(1 - p) for h in range(1, H + 1)] for p in P_LEVELS},
                    index=range(1, H + 1))
paths = BT[(BT["target"] == col) & (BT["model"] == champ)]                      # walk-forward paths, no look-ahead
TEST_PEAKS = pd.DataFrame({p: (paths["yhat"] + paths["h"].map(qtab[p])).groupby(paths["origin"]).max() for p in P_LEVELS})
test_actual = daily.loc[TEST_START - pd.Timedelta(days=28):T_END].query("is_reported")[col]
lo, hi = np.floor(test_actual.min() / 25) * 25, np.ceil(test_actual.max() / 25) * 25
TEST_THRESHOLDS = np.arange(lo + 25, hi, 25)
ev_test = surge_events(test_actual, TEST_THRESHOLDS)
ev_test = ev_test[ev_test["event_date"] >= TEST_START]                           # score crossings inside the test window only
OC_TEST = operating_curve(TEST_PEAKS, test_actual, ev_test, TEST_THRESHOLDS, fa_origins=TEST_PEAKS.index >= TEST_START)
OC_TEST


# %% [markdown]
# **Observation:** In the 2025 test window all 16 threshold crossings were flagged in advance at every alert level. Even the strictest rule (50% probability) gave a median 18.5 days'
      warning with 16% false alarms, and a 30% trigger extends the median warning to the full 28 days with 22% false alarms. The steady 2025 climb makes this an easy case.


# %%
# (2) Historical replay 2023–2024: rolling 120-day damped ETS with analytic intervals, refit every 14 days, issued from every reported day
REPLAY_START, REPLAY_END, ROLL = pd.Timestamp("2023-03-01"), brk_transfers - pd.Timedelta(days=1), 120
```

```
def ets_stock(y):
    return ETSModel(y, error="add", trend="add", damped_trend=True, seasonal="add", seasonal_periods=7)



replay_origins = daily.index[(daily.index >= REPLAY_START) & (daily.index <= REPLAY_END) & daily["is_reported"]]
peaks, params, last_fit, t0 = {}, None, None, time.time()
REPLAY_PI = {}                                                          # analytic 80% intervals, kept for the high-demand coverage check
z90 = stats.norm.ppf(0.9)
for o in replay_origins:
    y = daily.loc[o - pd.Timedelta(days=ROLL - 1):o, "hhs_care"].asfreq("D")
    if last_fit is None or (o - last_fit).days >= REFIT_EVERY:
        params, last_fit = ets_stock(y).fit(disp=False).params, o
    fr = ets_stock(y).smooth(params).get_prediction(start=len(y), end=len(y) + H - 1).summary_frame(alpha=0.2)
    sd = (fr["pi_upper"] - fr["mean"]) / z90                            # forecast standard deviation per horizon
    REPLAY_PI[o] = fr[["mean", "pi_lower", "pi_upper"]].to_numpy()
    peaks[o] = {p: (fr["mean"] + stats.norm.ppf(1 - p) * sd).max() for p in P_LEVELS}
REPLAY_PEAKS = pd.DataFrame(peaks).T
replay_actual = daily.loc[REPLAY_START:REPLAY_END].query("is_reported")["hhs_care"]
REPLAY_THRESHOLDS = np.arange(6000, 11501, 500)
ev_hist = surge_events(replay_actual, REPLAY_THRESHOLDS)
OC_HIST = operating_curve(REPLAY_PEAKS, replay_actual, ev_hist, REPLAY_THRESHOLDS)
print(f"Replay: {len(replay_origins)} origins in {time.time() - t0:.0f}s, {len(ev_hist)} surge events")
OC_HIST


# %% [markdown]
# **Observation:** The 2023–24 replay is the tougher test, with 25 surge events across thresholds from 6,000 to 11,500. Alerting on the point forecast (50%) warns only about 1 day ahead
      and misses 20% of surges. A 30% trigger catches all 25 with a median 14-day warning and about 1 false alarm in 4 (26%). A 20% trigger extends the warning to 27 days, but 36% of
      alerts are false. The trigger is a policy trade-off; 30% keeps false alarms near one in four while giving two weeks to act.


# %%
# Interval behaviour in high-demand periods: coverage of the replay's analytic 80% ETS intervals when care load was building up fast vs other times
rp_rows = []
for o, arr in REPLAY_PI.items():
    dates = o + pd.to_timedelta(np.arange(1, H + 1), "D")
    act = daily["hhs_care"].reindex(dates).to_numpy()
    ok = (daily["is_reported"] & OBSERVED["hhs_care"]).reindex(dates, fill_value=False).to_numpy()
    rp_rows.append(pd.DataFrame({"origin": o, "h": np.arange(1, H + 1), "inside": (act >= arr[:, 1]) & (act <= arr[:, 2]),
                                 "below": act < arr[:, 1], "above": act > arr[:, 2], "width80": arr[:, 2] - arr[:, 1], "ok": ok}))
RP = pd.concat(rp_rows, ignore_index=True).query("ok")
growth28 = daily["hhs_care"] / daily["hhs_care"].shift(28) - 1
RP["phase"] = np.where(growth28.reindex(RP["origin"]).to_numpy() > 0.05, "build-up (care +5% or more in the past 28 days)", "other periods")
REPLAY_COV = RP.groupby("phase").agg(origins=("origin", "nunique"), coverage80=("inside", "mean"), actual_above_band=("above", "mean"),
                                     actual_below_band=("below", "mean"), mean_width80=("width80", "mean")).round(3)
REPLAY_COV


# %% [markdown]
# **Observation:** In the 2023–24 replay the analytic 80% ETS intervals stay calibrated during high demand: 80.4% coverage at the 153 origins where care load had risen at least 5% in the
      previous 28 days, against 84.3% at other times. During build-ups the misses are lopsided: the actual falls *below* the band 14.7% of the time and above it only 4.8%, because a
      damped trend fitted during a surge tends to overshoot as the surge slows. Bands are also wider then (about 2,980 vs 2,600 children). For capacity planning this is the safe
      direction: the upper bound was rarely too low while demand was rising.


# %%
```

```
# Operating curves: earlier warning costs more false alarms; p* = 30% balances notice against false alarms
fig, ax = plt.subplots(figsize=(8.5, 4.4))
for oc, name, colr in [(OC_HIST, "Replay 2023–24 (ETS)", MODEL_COLORS["ETS"]), (OC_TEST, f"Test 2025 ({champ})", MODEL_COLORS[champ])]:
    ax.plot(oc["false alarms %"], oc["median lead (days)"], color=colr, lw=2, marker="o", ms=8, mec=SURFACE, mew=2, label=name)
    pts = oc.groupby("median lead (days)").agg(fa=("false alarms %", "min"), lo=("alert at P(breach) ≥", "min"), hi=("alert at P(breach) ≥", "max"))
    for lead, r in pts.iterrows():                                          # one label per distinct point (merge saturated levels)
        txt = f"p≥{r['hi']:.0%}" if r["lo"] == r["hi"] else f"p≥{r['lo']:.0%}–{r['hi']:.0%}"
        ax.annotate(txt, (r["fa"], lead), xytext=(6, -12), textcoords="offset points", fontsize=8.5, color=INK2)
ax.set_xlabel("false alarms (% of alerts)"); ax.set_ylabel("median warning (days)"); ax.legend(loc="lower right")
ax.set_title("Early-warning operating curves: lead time vs false alarms")
savefig(fig, "13a_operating_curve")


# %% [markdown]
# **Observation:** Both curves show that earlier warning costs more false alarms. In the test window the curve flattens at 30% (28 days' warning, 22% false alarms). In the harder replay,
      each step down in the trigger buys more notice but adds 3–11 percentage points of false alarms. At similar false-alarm rates the replay gives far less notice (6 days at 15% vs 18.5
      days at 16% in the test), so the replay figures are the more realistic guide for a future surge.


# %%
# Per-event lead times at the recommended operating point p* (historical replay)
LT_HIST = lead_times(ev_hist, REPLAY_PEAKS[P_STAR], list(REPLAY_PEAKS.index))
LT_TEST = lead_times(ev_test, TEST_PEAKS[P_STAR], list(TEST_PEAKS.index))
LT_HIST


# %% [markdown]
# **Observation:** At the 30% trigger all 25 historical surges were flagged, with warnings from 1 to 28 days. The large 2023 climb gave the longest notice: crossings of 10,000–11,500
      children were flagged 14–28 days ahead. The shortest warnings came with abrupt jumps (8,000 on 25 Apr 2023 after a +661 rise in six days: 1 day) or at levels the census was hovering
      around (6,000 in Dec 2024: 6 days).


# %%
# Visualise the replay: care load, the real-time 28-day alert level at p*, and each surge event with its warning
fig, ax = plt.subplots(figsize=(12, 4.4))
ax.plot(replay_actual.index, replay_actual, color=INK, lw=1.6, label="actual care load")
ax.plot(REPLAY_PEAKS.index, REPLAY_PEAKS[P_STAR], color=MODEL_COLORS["ETS"], lw=1.6, label=f"28-day alert level (P(breach) ≥ {P_STAR:.0%})")
det, miss = LT_HIST[LT_HIST["detected"]], LT_HIST[~LT_HIST["detected"]]
ax.scatter(det["event_date"], det["threshold"], s=48, color=STATUS["good"], edgecolor=SURFACE, linewidth=2, zorder=4, label="surge event: warned in advance")
if len(miss):
    ax.scatter(miss["event_date"], miss["threshold"], s=48, color=STATUS["critical"], edgecolor=SURFACE, linewidth=2, zorder=4, label="surge event: missed")
for r in det[det["event_date"].dt.year == 2023].nlargest(2, "threshold").itertuples():
    ax.annotate(f"{r.lead_days}d warning", (r.event_date, r.threshold), xytext=(-75, 14), textcoords="offset points",
                fontsize=9, color=INK2, arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8))
ax.yaxis.set_major_formatter(COMMA); ax.legend(loc="lower left", fontsize=8.5)
ax.set_title(f"Surge early warning, historical replay 2023–2024 (alert at {P_STAR:.0%} breach probability)")
savefig(fig, "13_surge_replay")


# %% [markdown]
# **Observation:** In real time the alert level rises ahead of the census during each 2023 build-up, so every crossing from 6,000 to 11,500 was flagged before it happened. The alert line
      is jumpy in volatile periods (for example early 2024), so a single alert should be read together with the recent trend before acting.


# %%
# KPI 2 summary: Surge Lead Time at the recommended operating point, for both evaluations
LT_ALL = pd.concat([LT_HIST.assign(source="replay 2023–24"), LT_TEST.assign(source="test 2025")])
SURGE_LEAD = LT_ALL.groupby("source").agg(events=("lead_days", "size"), detection_rate=("detected", "mean"),
                                          median_lead_days=("lead_days", "median"), mean_lead_days=("lead_days", "mean")).round(2)
fa_at_star = {"replay 2023–24": OC_HIST, "test 2025": OC_TEST}
SURGE_LEAD["false_alarm_rate"] = SURGE_LEAD.index.map(lambda s: fa_at_star[s].set_index("alert at P(breach) ≥").at[P_STAR, "false alarms %"] / 100)
```

```
SURGE_LEAD
```

```
# %% [markdown]
# **Observation:** At the recommended 30% trigger the system detected 100% of surge events in both evaluations: 25 historical events with a median 14-day warning (mean 14.6, 26% false
      alarms) and 16 test-window events with 28 days' warning (22% false alarms). The historical figure, about two weeks, is the realistic planning lead time.
```

```
# %% [markdown]
# ## 14. Production forecast: 28 days beyond the last report
#
# Every model is re-fitted on all data up to the last report. Intervals use the residual quantiles from **both** evaluation windows (the most recent 26 weeks of errors).
```

```
# %%
# Fit every model at the final origin and forecast 28 days ahead, then add the Ensemble
FUTURE_DATES = T_END + pd.to_timedelta(np.arange(1, H + 1), "D")
rows, PROD_MODELS = [], {}
for col, kind in TARGETS.items():
    for m in build_models(col):
        try:
            m.fit(T_END, col, kind); fit_ok = True
        except Exception as e:                                          # logged; safe_forecast falls back to naive persistence
            FIT_LOG.append({"target": col, "model": m.name, "origin": T_END, "stage": "production fit", "error": f"{type(e).__name__}: {e}"[:200]}); fit_ok = False
        PROD_MODELS[(col, m.name)] = m
        yhat, fb = safe_forecast(m, T_END, col, kind, fit_ok)
        rows.append(pd.DataFrame({"target": col, "model": m.name, "h": np.arange(1, H + 1), "date": FUTURE_DATES, "yhat": yhat, "fallback": fb}))
FC = pd.concat(rows, ignore_index=True)
FC = pd.concat([FC, FC[FC["model"].isin(comp)].groupby(["target", "h", "date"], as_index=False)["yhat"].mean().assign(model="Ensemble", fallback=False)], ignore_index=True)
print(f"Production forecasts: {FC['model'].nunique()} models × {FC['target'].nunique()} series × {H} days | fallbacks: {int(FC['fallback'].sum())}")
```

```
# %% [markdown]
# **Observation:** All 8 models (7 fitted at the last report plus the Ensemble) produced a 28-day path for each of the 3 series with no fallback, so the production forecast comes from
      exactly the implementations validated in the backtest.
```

```
# %%
# What drives the ML forecasts? Top Random Forest feature importances for care load and discharges
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
for ax, col in zip(axes, ["hhs_care", "discharged"]):
    imp = pd.Series(PROD_MODELS[(col, "Random Forest")].model.feature_importances_, index=FEATURE_COLS).nlargest(12)[::-1]
    ax.barh(imp.index, imp.values, height=0.6, color=SERIES_COLORS[col])
    ax.set_title(f"{LABELS[col]}: top drivers (RF importance)", fontsize=10.5); ax.grid(axis="y", visible=False)
fig.tight_layout(); savefig(fig, "17_feature_importance")
```

```
# %% [markdown]
# **Observation:** For care load the 14-day net pressure (transfers − discharges per 100 children) is the top driver (22% of importance), followed by the horizon (19%) and the 14-day
      care-load trend (13%). Calendar features carry under 1%, so the forecast is driven by flows and momentum, not dates. For discharges the same net-pressure signal leads (15%),
      followed by the care-load trend (12%), weekday (10%), horizon (9%) and the same-weekday-last-week anchor (8%), matching their weekly cycle.
```

```
# %%
# Calibrated intervals from the last 26 weeks of walk-forward errors; flows are floored at zero
Q_ALL = residual_quantiles(S)
FC = FC.merge(Q_ALL, on=["target", "model", "h"])
for k in QUANTS:
    FC[k] = FC["yhat"] + FC[k]
    FC.loc[FC["target"] != "hhs_care", k] = FC.loc[FC["target"] != "hhs_care", k].clip(lower=0)
FC[(FC["model"] == CHAMPION["hhs_care"]) & (FC["target"] == "hhs_care") & FC["h"].isin([1, 7, 14, 21, 28])].round(0)
```

```
# %% [markdown]
# **Observation:** SARIMA expects care load to edge up from 2,487 tomorrow to 2,542 by 18 Jan 2026 (+58 from today's 2,484, about +2 a day). The 80% band widens from 25 children on day 1
      to about 207 on day 28 (2,486–2,694), so near-term numbers are firm while the 4-week figure should be planned as a range.


# %%
# Interval sanity checks on the production forecast (bound ordering, finiteness, width growth) and the zero floor on flows (a truncation, not a count model)
fcv = FC.assign(width80=FC["hi80"] - FC["lo80"])
checks = fcv.groupby("target").agg(
    rows=("yhat", "size"), finite=("yhat", lambda s: bool(np.isfinite(fcv.loc[s.index, ["yhat", *QUANTS]].to_numpy()).all())),
    ordered=("yhat", lambda s: bool(((fcv.loc[s.index, "lo95"] <= fcv.loc[s.index, "lo80"]) & (fcv.loc[s.index, "lo80"] <= fcv.loc[s.index, "hi80"])
                                     & (fcv.loc[s.index, "hi80"] <= fcv.loc[s.index, "hi95"])).all())),
    negative_point=("yhat", lambda s: int((s < 0).sum())),
    lo80_at_zero=("lo80", lambda s: int((s == 0).sum())), lo95_at_zero=("lo95", lambda s: int((s == 0).sum())))
w = fcv[fcv["h"].isin([1, H])].pivot_table(index="target", columns="h", values="width80", aggfunc="mean")
checks["mean width80 day 1"], checks[f"mean width80 day {H}"] = w[1].round(1), w[H].round(1)
zero_bt = BT[(BT["target"] != "hhs_care") & (BT["yhat"] == 0)].groupby(["target", "model"]).size()
print("Backtest flow point forecasts exactly at zero (by model):", zero_bt.to_dict() or "none")
checks


# %% [markdown]
# **Observation:** All 672 production forecasts and their bounds are finite and correctly ordered (lo95 ≤ lo80 ≤ hi80 ≤ hi95), and no point forecast is negative. Care-load 80% bands
      widen from 60 to 320 children between day 1 and day 28 (average over the 8 models). For the flows the zero floor binds often at the 95% level (73 of 224 discharge and 116 of 224
      transfer lower bounds) but rarely at 80% (8 and 12), where the empirical errors would otherwise imply negative counts. Flooring cannot change coverage for non-negative actuals, but
      it is a truncation rather than a count model, so a lower flow bound of 0 means "zero or close to it". In the backtest only the two naive baselines ever forecast exactly zero, each
      time after a zero-count report.


# %%
# Forecast dashboard view: care load (champion + ensemble) and the two flows, each with the 80% band
fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)
for ax, col in zip(axes, TARGETS):
    hist = daily.loc[T_END - pd.Timedelta(days=90):, col]
    ax.plot(hist.index, hist if col == "hhs_care" else hist.rolling(7, min_periods=1).mean(), color=INK, lw=1.6,
            label="actual" if col == "hhs_care" else "actual (7-day mean)")
    for m in dict.fromkeys([CHAMPION[col], "Ensemble"]):
        g = FC[(FC["target"] == col) & (FC["model"] == m)]
        ax.fill_between(g["date"], g["lo80"], g["hi80"], color=MODEL_COLORS[m], alpha=0.15, lw=0)
        ax.plot(g["date"], g["yhat"], color=MODEL_COLORS[m], lw=2, label=f"{m} forecast (80% band)")
    ax.axvline(T_END, color=MUTED, lw=1)
    ax.set_title(LABELS[col], fontsize=10.5); ax.legend(loc="upper left", fontsize=8.5)
    if col == "hhs_care":
        ax.yaxis.set_major_formatter(COMMA)
fig.suptitle(f"28-day outlook from {T_END:%d %b %Y}", x=0.01, ha="left", fontweight="bold")
fig.tight_layout(); savefig(fig, "14_production_forecast")


# %% [markdown]
# **Observation:** Both care-load forecasts continue the climb but more slowly than the last month (+4.6/day): SARIMA reaches 2,542 and the Ensemble 2,571 by 18 Jan 2026, and their 80%
      bands overlap for most of their width (about 65–75% after the first week). The Ensemble expects about 12.7 discharges a day, above the last 28 days' average of about 9.4, because it
      follows the late-December uptick (16 and 14 on the last two reports). If placements fall back to their recent average, care load would grow faster than forecast. Transfers are
      expected to stay near 8 a day, in line with the recent 8.7.


# %%
# Future imbalance: forecast transfers vs discharges (ensemble), net pressure and the flow-implied care load (coherence check)
fin = FC[(FC["target"] == "transferred") & (FC["model"] == "Ensemble")].set_index("date")["yhat"]
fout = FC[(FC["target"] == "discharged") & (FC["model"] == "Ensemble")].set_index("date")["yhat"]
```

```
w60 = daily.loc[T_END - pd.Timedelta(days=59):]
# Reconciliation: daily care-load change NOT explained by the recorded flows (unreported weekend flows + non-CBP referrals)
RECON_PER_DAY = float((w60["hhs_care"].iloc[-1] - w60["hhs_care"].iloc[0] - w60["net_pressure"].iloc[1:].sum()) / (len(w60) - 1))
fcare_direct = FC[(FC["target"] == "hhs_care") & (FC["model"] == "Ensemble")].set_index("date")["yhat"]
fcare_flow = daily.at[T_END, "hhs_care"] + (fin - fout + RECON_PER_DAY).cumsum()
IMBALANCE = pd.DataFrame({"transfers in": fin, "discharges out": fout, "net pressure (recorded flows)": fin - fout,
                          "care (direct forecast)": fcare_direct, "care (flow-implied)": fcare_flow})
print(f"Recorded-flow net pressure: {(fin - fout).mean():+.1f}/day | reconciliation (weekend + non-CBP): {RECON_PER_DAY:+.1f}/day "
      f"| effective imbalance: {(fin - fout).mean() + RECON_PER_DAY:+.1f}/day → {((fin - fout).mean() + RECON_PER_DAY) * H:+.0f} children over {H} days")
print(f"Coherence: direct vs flow-implied care load at day 28 differ by {abs(fcare_direct.iloc[-1] - fcare_flow.iloc[-1]):.0f} children")
IMBALANCE.iloc[[0, 6, 13, 20, 27]].round(1)


# %% [markdown]
# **Observation:** The forecast flows alone imply a slight decline: recorded net pressure is −4.4/day, or −1.0/day after the +3.3/day weekend and non-CBP reconciliation, giving 2,455 on
      day 28 against 2,571 from the direct Ensemble forecast. Most of this 115-child gap (about 92 children, or 80%) comes from the discharge forecast: the Ensemble expects 12.7
      discharges a day against a recent average of 9.4. The slightly lower transfer forecast (8.3 vs 8.7 a day) adds about 11 more. Placements are therefore the swing factor in the
      outlook, which is why capacity decisions should rest on intervals rather than a single line.


# %%
# KPI 3: Capacity Breach Probability from the empirical forecast distribution (point forecast + calibrated errors)
def breach_probability(col, model, threshold, base_path):
    errs = S[(S["target"] == col) & (S["model"] == model)]
    probs = []
    for h in range(1, H + 1):
        e = errs.loc[(errs["h"] - h).abs() <= 2, "error"].to_numpy()
        probs.append(float(np.mean(base_path[h - 1] + e > threshold)))
    return np.array(probs)



champ = CHAMPION["hhs_care"]
champ_path = FC[(FC["target"] == "hhs_care") & (FC["model"] == champ)].sort_values("h")["yhat"].to_numpy()
CURRENT = daily.at[T_END, "hhs_care"]
CAPACITY = int(np.ceil(CURRENT * 1.05 / 50) * 50)          # illustrative planning threshold: about 5% above today's load
risk_thresholds = np.arange(np.floor(CURRENT / 50) * 50, CURRENT * 1.12, 50)
probs_by_c = {c: breach_probability("hhs_care", champ, c, champ_path) for c in risk_thresholds}          # one 28-day curve per threshold
RISK = pd.DataFrame({f"day {h}": [probs_by_c[c][h - 1] for c in risk_thresholds] for h in (7, 14, 21, 28)},
                    index=pd.Index(risk_thresholds.astype(int), name="threshold")).round(3)
print(f"Current care load {CURRENT:,.0f}; illustrative capacity threshold {CAPACITY:,} (≈ +5%)")
RISK


# %% [markdown]
# **Observation:** With today's load at 2,484, exceeding 2,500 is likely (88% by day 28), but higher thresholds are much less likely by day 28: 44% for 2,600, 27% for the illustrative
      2,650 capacity, and 7% for 2,700. No threshold of 2,600 or more has any probability within the first week, so there is time to act if the trend continues.


# %%
# Breach-probability curve over the horizon for a few thresholds (status colours carry risk, labels carry the value)
fig, ax = plt.subplots(figsize=(11, 4))
th_show = [int(CURRENT // 50 * 50 + 50), CAPACITY, CAPACITY + 100]
for c, colr in zip(th_show, [STATUS["serious"], STATUS["critical"], "#256abf"]):
    p = breach_probability("hhs_care", champ, c, champ_path)
    ax.plot(np.arange(1, H + 1), p * 100, color=colr, lw=2, label=f"> {c:,} children")
    ax.annotate(f"{p[-1]:.0%}", (H, p[-1] * 100), xytext=(4, 0), textcoords="offset points", va="center", fontsize=9, color=INK2)
ax.set_ylim(0, 100); ax.set_xlabel("days ahead"); ax.set_ylabel("probability (%)"); ax.legend(loc="upper left")
ax.set_title(f"Capacity breach probability ({champ}), from {T_END:%d %b %Y}")
savefig(fig, "15_breach_probability")
```

```
# %% [markdown]
# **Observation:** The probability of exceeding 2,650 stays near zero for about two weeks and reaches 27% by day 28, just below the recommended 30% alert trigger. Exceeding 2,500 passes
      50% within about 5 days. A program with real capacity near 2,650 therefore has roughly two weeks of low risk before it needs to prepare, consistent with the early-warning lead times
      above.


# %% [markdown]
# ## 15. Scenario analysis: stock-flow what-ifs
#
# Scenarios change the **flows** and push the change through the identity *care(t+h) = care(t+h)_baseline + Σ(Δtransfers − Δdischarges)*. Each scenario is therefore anchored to the best
      direct forecast and stays physically consistent. The same engine powers the dashboard's scenario lab.
#
# **Status of these numbers.** The baseline path is a model forecast; every other path is a **hypothetical assumption** about future flows (for example "discharges fall 30%"), not a
      prediction that it will happen. The capacity line is an **illustrative threshold** (≈ 5% above today's load), because the file contains no bed-capacity data. Scenario paths carry no
      probability of their own; the breach probabilities above apply to the model forecast only.


# %%
# Scenario engine (identical logic is used in the Streamlit app)
def run_scenario(base_care, base_in, base_out, in_mult=1.0, out_mult=1.0, start_day=1, extra_in=0.0):
    days = np.arange(1, len(base_care) + 1)
    active = (days >= start_day).astype(float)
    d_in = (np.asarray(base_in) * (in_mult - 1) + extra_in) * active
    d_out = np.asarray(base_out) * (out_mult - 1) * active
    return np.asarray(base_care) + np.cumsum(d_in - d_out)



base_care = FC[(FC["target"] == "hhs_care") & (FC["model"] == "Ensemble")].sort_values("h")["yhat"].to_numpy()
base_in, base_out = fin.to_numpy(), fout.to_numpy()
pre_break_in = daily.loc[ML_START:brk_transfers, "transferred"].mean()
SCENARIOS = {
    "Baseline (current trends)": dict(),
    "Discharge slowdown (−30%)": dict(out_mult=0.7),
    "Placement acceleration (+50% discharges)": dict(out_mult=1.5),
    "Intake surge (3× transfers from day 8)": dict(in_mult=3.0, start_day=8),
    "Combined stress (2× in, −25% out)": dict(in_mult=2.0, out_mult=0.75),
    f"Return to 2024 intake (~{pre_break_in:.0f}/day from day 8)": dict(extra_in=pre_break_in - base_in.mean(), start_day=8),
}
SCEN = pd.DataFrame({name: run_scenario(base_care, base_in, base_out, **kw) for name, kw in SCENARIOS.items()}, index=FUTURE_DATES)
summary = pd.DataFrame({"care load day 14": SCEN.iloc[13], "care load day 28": SCEN.iloc[-1],
                        "change vs today": SCEN.iloc[-1] - CURRENT,
                        "first day above capacity": [next((i + 1 for i, v in enumerate(SCEN[c]) if v > CAPACITY), None) for c in SCEN]}).round(0)
summary


# %% [markdown]
# **Observation:** Under current trends care load reaches 2,571 by day 28 (+87). A 30% slowdown in placements adds about 100 more children (2,677) and crosses 2,650 on day 24. Tripled
      transfers cross it on day 15, and a combined shock on day 12. A return to 2024 intake (~205/day) would add about 4,200 children within three weeks. A 50% increase in placements is
      the only scenario here that reverses growth (−91), so expanding placements is the clearest way to bring the census down.


# %%
# Scenario comparison chart (log-free, one axis): the surge scenario dwarfs the others, so it gets its own panel
fig, axes = plt.subplots(1, 2, figsize=(14, 5.2), gridspec_kw={"width_ratios": [1.4, 1]})
pal = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300"]
for (name, colr) in zip(SCEN.columns, pal):
    ax = axes[1] if name.startswith("Return") else axes[0]
    ax.plot(SCEN.index, SCEN[name], color=colr, lw=2, label=name)
```

```
for ax in axes:
    ax.axhline(CAPACITY, color=STATUS["critical"], lw=1); ax.yaxis.set_major_formatter(COMMA)
    ax.text(FUTURE_DATES[-1], CAPACITY, f"threshold {CAPACITY:,}", color=INK2, fontsize=9, ha="right", va="bottom")
    ax.xaxis.set_major_locator(mdates.DayLocator(interval=7)); ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.1), ncol=2, fontsize=8.5)        # legend below the plot, off the lines
axes[0].set_ylim(top=max(CAPACITY, SCEN.drop(columns=[c for c in SCEN if c.startswith("Return")]).max().max()) + 40)
axes[0].set_title("Operational scenarios"); axes[1].set_title("Stress test: intake returns to 2024 levels")
savefig(fig, "16_scenarios")


# %% [markdown]
# **Observation:** The operational scenarios diverge steadily: by 18 Jan the gap between the best case (faster placements, 2,393) and the worst (tripled intake, 2,917) is about 520
      children, roughly a fifth of today's census. The stress test needs its own scale: a return to 2024 intake crosses the illustrative threshold as soon as it starts and reaches 6,710
      by day 28, 2.7 times today's census.


# %% [markdown]
# ## 16. Export artifacts for the dashboard and the paper


# %%
# Persist clean data, backtest, metrics, forecasts, residuals and KPI metadata into artifacts/
daily.assign(regime=np.where(daily.index >= REGIME_START, "current", "pre-break")).to_csv(ART_DIR / "daily_clean.csv", index_label="date")
BT[BT["scored"]][["target", "model", "origin", "h", "date", "y", "yhat", "period"]].to_csv(ART_DIR / "backtest.csv.gz", index=False)
leader.to_csv(ART_DIR / "metrics_overall.csv", index=False)
by_h_all = S.groupby(["target", "period", "model", "h"]).apply(score).reset_index().drop(columns=["zero y excluded"]).astype({"n": int})
by_h_all.to_csv(ART_DIR / "metrics_by_horizon.csv", index=False)
coverage.reset_index().to_csv(ART_DIR / "interval_coverage.csv", index=False)
COV_BY_H.rename(columns=lambda c: c.replace(" ", "_").replace("(", "").replace(")", "")).to_csv(ART_DIR / "interval_coverage_by_h.csv", index=False)
COV_COMPARE.stack(level=0, future_stack=True).rename_axis(["target", "model", "method"]).reset_index().to_csv(ART_DIR / "interval_method_comparison.csv", index=False)
VALIDATION_ISSUES.to_csv(ART_DIR / "validation_report.csv", index=False)
pd.DataFrame(FIT_LOG, columns=["target", "model", "origin", "stage", "error"]).to_csv(ART_DIR / "model_fit_log.csv", index=False)
AUDIT.reset_index().to_csv(ART_DIR / "feature_leakage_audit.csv", index=False)
ROBUST.reset_index(names=["target", "model"]).to_csv(ART_DIR / "robustness.csv", index=False)
FC.to_csv(ART_DIR / "forecast_future.csv", index=False)
S[["target", "model", "h", "error", "period"]].to_csv(ART_DIR / "residuals.csv.gz", index=False)
LT_ALL.to_csv(ART_DIR / "surge_lead_times.csv", index=False)
pd.concat([OC_HIST.assign(source="replay 2023–24"), OC_TEST.assign(source="test 2025")]).to_csv(ART_DIR / "ews_operating_curve.csv", index=False)
(REPLAY_PEAKS[[P_STAR, 0.5]].set_axis(["peak", "peak_p50"], axis=1).rename_axis("origin")
 .join(replay_actual.rename("actual"), how="left").to_csv(ART_DIR / "surge_replay_paths.csv"))
META = {"data_end": f"{T_END:%Y-%m-%d}", "regime_start": f"{REGIME_START:%Y-%m-%d}", "break_transfers": f"{brk_transfers:%Y-%m-%d}",
        "break_discharges": f"{brk_discharges:%Y-%m-%d}", "cbp_bad_start": f"{CBP_BAD_START:%Y-%m-%d}", "cbp_bad_end": f"{CBP_BAD_END:%Y-%m-%d}",
        "cal_start": f"{CAL_START:%Y-%m-%d}", "test_start": f"{TEST_START:%Y-%m-%d}", "horizon": H, "champion": CHAMPION,
        "sarima_specs": {k: [list(v[0]), list(v[1]), v[2]] for k, v in SARIMA_SPECS.items()}, "recon_per_day": RECON_PER_DAY,
        "alert_probability": P_STAR,
        "pre_break_transfers_per_day": float(pre_break_in), "current_care_load": float(CURRENT), "capacity_default": CAPACITY,
        "stale_reports": int(stale_mask.sum()), "blank_rows": int(raw.isna().all(axis=1).sum()), "copied_cbp_reports": N_COPIED,
        "surge_lead": SURGE_LEAD.reset_index().to_dict(orient="records"),
        "accuracy_champion": {c: float(ACCURACY.loc[CHAMPION[c], c]) for c in TARGETS},
        "notebook_version": "2.0", "models": MODEL_ORDER, "model_colors": MODEL_COLORS,
        "interval_method": "empirical walk-forward error quantiles (h ± 2) from the last 26 weeks of origins, target dates observed at the forecast origin",
        "accuracy_definition": "Accuracy % = 100 - WAPE = 100 x (1 - sum|y - yhat| / sum|y|), reported days of the test window",
        "capacity_note": "No bed-capacity data in the source file: capacity_default is an illustrative threshold (about 5% above the latest care load)",
        "validation_issues": int(len(VALIDATION_ISSUES)), "fit_failures": int(len(FIT_LOG)),
        "coverage80_rolling_26wk": {c: float(COV_COMPARE.loc[(c, CHAMPION[c]), ("rolling origin (26 wk)", "coverage80")]) for c in TARGETS}}
(ART_DIR / "metadata.json").write_text(json.dumps(META, indent=2, default=str))
print("Artifacts written:", sorted(p.name for p in ART_DIR.iterdir()))
print("Figures written:", len(list(FIG_DIR.glob("*.png"))))
```

```
# %% [markdown]
# **Observation:** All 17 artifact files and 22 figures were written (v2 adds the validation report, model-fit log, feature leakage audit, coverage by horizon and the interval-method
      comparison). The Streamlit dashboard reads these files directly, so it always shows exactly this notebook's results, and rerunning the notebook refreshes the dashboard.


# %%
# One-screen KPI summary for stakeholders
kpi = pd.DataFrame({
    "Champion model": CHAMPION,
    "Forecast accuracy % (test)": {c: ACCURACY.loc[CHAMPION[c], c] for c in TARGETS},
    "MAE (test)": {c: test_board.set_index(["target", "model"]).loc[(c, CHAMPION[c]), "MAE"] for c in TARGETS},
    "80% interval coverage %": {c: coverage.loc[(c, CHAMPION[c]), "coverage80"] * 100 for c in TARGETS},
    "Stability index": {c: FSI.loc[CHAMPION[c], c] for c in TARGETS},
    "Day-28 forecast": {c: FC[(FC["target"] == c) & (FC["model"] == CHAMPION[c]) & (FC["h"] == H)]["yhat"].iloc[0] for c in TARGETS},
}).round(1)
kpi


# %% [markdown]
# **Observation:** Care load is forecast at 98.6% accuracy (MAE 32 children) with conservative 80% intervals (91.6% coverage). Discharges reach 64.6% (MAE 3.5/day) and transfers 58.8%
      (MAE 4.2/day), with transfer intervals too narrow (66% coverage). The day-28 values are 2,542 children, 8.1 discharges (18 Jan is a Sunday, the weekly low; the 28-day average is
      about 12.7) and 9.9 transfers (the moving-average champion). Stability is the care-load champion's main weakness (FSI 25), so decisions should rest on intervals and breach
      probabilities rather than a single run.


# %% [markdown]
# ## 17. Insights & recommendations
#
# ### What the data say
# 1. **The raw file needs repair before it can be trusted.** 450 blank padding rows; 5 stale carried-forward HHS reports; and **251 CBP reports (12 Jan 2023 – 30 Jan 2024) copied from
      later dates**, proven by the broken stock-flow identity (correlation 0.51 vs 0.90 in the trusted period). The HHS columns reconcile wherever they can be checked (February 2024
      onward) and are kept for the whole history.
# 2. **The program changed regime in early 2025.** Transfers broke on 26 Jan 2025 and discharges on 17 Feb 2025. Intake fell about 95%, average care load fell 66% (6,751 → 2,262), and
      the **implied length of stay rose from about 36 to about 203 days**. The binding constraint is no longer intake; it is **placement (discharge) capacity**.
# 3. **The weekend ratchet.** Since the current regime began (17 Mar 2025) care load rose by 373. Weekday transitions netted **–40**, while Thursday→Sunday gaps added **+323** and other
      multi-day reporting gaps (holidays, an outage, removed stale reports) **+90**. Even during the recent climb (1,972 on 21 Aug → 2,484), weekend and other reporting gaps account for
      about 70% of the growth.
# 4. **The recorded flows are not a complete balance.** Even between consecutive weekday reports care load rises about 3 children a day more than transfers – discharges (non-CBP
      admissions or definitional differences; the file has no "other admissions" or "other exits" columns). Sunday counts are single-day counts, and interpolated Friday/Saturday flows
      reconcile with the stock to within about 0.45 children a day in the current regime, so interpolation is acceptable for features but never scored.
# 5. **Validation and leakage checks pass on the real data.** No implausible value was found (the rules quarantine negative counts and stock spikes on future refreshes), the executed
      perturbation test found 0 leaking feature values among 40 features, and the copied-CBP block and regime breaks are identical when detected with data up to the first forecast origin
      only.
#
# ### How well the future can be predicted (untouched 13-week test window)
# | Series | Champion (chosen on calibration) | Test MAE | vs naive | Accuracy % | Best on test |
# |---|---|---|---|---|---|
# | Children in HHS care | SARIMA | 32.0 children | –43% | 98.6 | Ensemble (28.5, –49%) |
# | Discharges / day | Ensemble | 3.5 | –25% | 64.6 | SARIMA (3.3) |
# | Transfers / day | Moving Avg (7d) | 4.2 | –17% | 58.8 | ETS (4.0) |
#
# * **Care load:** at 4 weeks ahead the modelled error is about half of naive persistence (48–52 vs 100 children), and the gain is statistically significant at 14 and 28 days (Diebold–
      Mariano p < 0.05). The Ensemble beat naive in **every** test week.
# * **Discharges** are close to their noise floor. With about 10 placements a day, pure Poisson randomness alone implies a mean absolute error of about 2.5, against 3.3 achieved. Plan
      discharge capacity on **weekly totals**: summing the reported days of each week roughly halves the relative error (16% vs 34% for SARIMA).
# * **Statistical vs ML:** statistical models win on the *flows* at almost every horizon (SARIMA's discharge error is flat at about 3.3 from day 1 to day 28). ML and the Ensemble win on
      the *stock* at medium horizons (8–21 days). The **Ensemble is the most robust all-rounder**, which is why the dashboard defaults to it alongside each champion.
# * **Early warning:** alerting when the breach probability reaches **30%** detected **all 25 historical surges with a median 14-day warning** (about 1 false alarm in 4). Alerting on the
      point forecast warned only about 1 day ahead.
```

```
# * **Baselines and model families (v2 additions):** seasonal naive helps only for discharges (−11% vs naive) and adds nothing for the stock; weekly seasonal ARIMA terms help only for
      discharges (ΔAIC −11.5), and out-of-sample calibration errors confirm each AIC choice. With identical inputs, a **recursive** Gradient Boosting model beat the direct one for care
      load at every horizon, but not the SARIMA champion on the calibration window, so it is a candidate for the next review rather than a change now.
# * **Uncertainty (v2):** the production rule (error quantiles of the last 26 weeks of forecasts already observed) was replayed in real time on the test window: care-load and discharge
      coverage stay near or above target, but transfer 80% coverage rises only from 66% to 68% (71% with a 13-week window). The intake interval defect **D-MOD-01 remains open**; plan
      intake on the 95% band. In the 2023–24 surge replay the analytic intervals kept 80% coverage during build-ups, with misses mostly on the low side.
#
# ### Outlook from 21 Dec 2025 (28 days, to 18 Jan 2026)
# * Care load keeps edging up: **SARIMA 2,542 by 18 Jan (80% interval 2,486–2,694)**, Ensemble 2,571.
# * Probability of exceeding **2,600** by day 28 is **44%**, above the recommended 30% alert trigger; for **2,650** it is **27%**, just below it.
# * Expected flows: about 8 transfers and 8–15 discharges per day (average about 12.7). The *flow-implied* route (forecast flows plus the +3.3/day weekend/non-CBP reconciliation) points
      to a slight decline (2,455 on day 28, −29), while the *direct* care-load models point to +60 to +90. Most of that gap of about 115 children (about 80%) comes from the discharge
      forecast (12.7/day expected vs 9.4 recently), so placements are the swing factor. The flow-implied value sits inside the champion's 95% interval, which is why capacity decisions
      should use intervals and breach probabilities, not a single line.
# * Stress tests: a 30% discharge slowdown crosses 2,650 on **day 24**; tripled intake on **day 15**; a combined shock on **day 12**. A **return to 2024 intake (~205/day) would add about
      4,200 children in three weeks**. Conversely, **+50% placements reverses the growth** (−91 children in 4 weeks instead of +87).
#
# ### Recommendations for HHS planners
# 1. **Treat discharge capacity as the primary lever.** Expand sponsor vetting and case-management throughput; +50% placements turns growth into decline within a month.
# 2. **Stop the weekend ratchet.** Add weekend placement and discharge processing; weekend gaps are where most of the regime's net growth has accumulated.
# 3. **Adopt a probability-based trigger.** Start scaling beds and staff when the 28-day breach probability for your real capacity reaches **30%** (dashboard status "Elevated"). In the
      2023–24 replay this gave a median two weeks' warning for gradual build-ups.
# 4. **Keep surge contingency warm.** A return to 2024-level intake would add about 4,200 children in three weeks, more than doubling today's census. Warnings averaged two weeks for
      gradual build-ups but were as short as 1 day for an abrupt jump (25 Apr 2023), so standby capacity should be activatable within days and the early-warning system should run on every
      new report.
# 5. **Fix the data at the source.** Publish Friday/Saturday flows (or weekend totals), publish non-CBP admissions and other exits so the population balance closes, keep the ingestion
      validation built here (schema, plausibility, duplicate-record and stock-flow checks) in front of every refresh, and correct the 2023 CBP records.
# 6. **Govern the models.** Re-estimate every 2 weeks, monitor interval coverage by horizon (intake bands are currently too narrow, so use 95%), review every calendar feature for regime
      leakage, and check the model-fit log after every run (a failing model silently falls back to naive persistence and is flagged).
# 7. **Next model review.** Evaluate a recursive care-load learner and an adaptive conformal method for intake intervals on a *new* evaluation window, with the selection rule fixed in
      advance.
#
# ### Limitations
# * The current regime is short (~9 months). Every model will need re-validation after the next policy change.
# * There is no official capacity data in the file, so thresholds here are illustrative. Planners should enter real bed capacity in the dashboard.
# * The test window (Sep–Dec 2025) was a steady rise. The 2023–24 replay supplies the surge stress test but uses a univariate model, because CBP data are corrupted in that period.
# * Tree-based ML models cannot extrapolate beyond the ranges they were trained on. Large shocks are therefore handled by the statistical models and the stock-flow scenario engine.
# * External drivers (border events, policy announcements, sponsor-vetting rules) are not in the data. Forecasts describe the system *as it currently behaves*.
# * The horizon is 28 days (four weekly cycles); 30-day forecasts were not validated. Per-horizon metrics rest on 42–55 test forecasts each, so single-horizon differences are noisy.
# * Transfer (intake) 80% intervals under-cover (66–71% depending on the calibration window; D-MOD-01 open). Flow lower bounds are floored at zero, a truncation rather than a count
      model.
# * The recorded flows leave about 3 children a day unexplained; the reconciliation term is an empirical average, not a measured flow. In 2024 interpolated weekend flows were biased,
      which affects only down-weighted ML training rows.
# * Scenario paths are hypothetical assumptions applied to the model forecast, not predictions, and carry no probability of their own.
#
# **Deliverables produced by this notebook:** `artifacts/` (data, backtest, metrics, forecasts, residuals, KPIs, validation report, model-fit log, feature leakage audit, coverage by
      horizon, interval-method comparison) feeding `uac_forecast_dashboard.py`; `figures/` used in the research paper and executive summary.
```

H.2 Streamlit dashboard: uac_forecast_dashboard.py

```
"""
UAC Care Load & Placement Demand: Streamlit forecasting dashboard (v2.1).


Run from this folder (after executing UAC_Care_Load_Forecasting.ipynb, which writes ./artifacts):
    python -m streamlit run uac_forecast_dashboard.py


v2.1 adds the "Pipeline analytics" tab (descriptive KPIs, bottleneck alerts, data explorer, CSV downloads); its logic
```

```
lives in ./src as pure functions with their own tests in ./tests.
"""
import json
import sys
from pathlib import Path


import numpy as np
import pandas as pd
import streamlit as st


st.set_page_config(page_title="UAC Care Load Forecast", page_icon="📈", layout="wide")


try:  # a missing plotting dependency becomes an actionable message instead of a traceback
    import plotly.graph_objects as go
except ImportError as e:
    st.error(f"A required package is missing ({e.name}). Install the pinned dependencies with `pip install -r requirements.txt`, then restart the app.")
    st.stop()


ART = Path(__file__).resolve().parent / "artifacts"
APP_DIR = Path(__file__).resolve().parent
if str(APP_DIR) not in sys.path:            # ./src must be importable whatever the working directory
    sys.path.insert(0, str(APP_DIR))
try:  # descriptive pipeline analytics: a missing ./src disables only that tab, never the forecasts
    from src import visualizations as pviz
    from src.bottleneck_detection import INDICATORS as PIPE_INDICATORS, LEVELS as PIPE_LEVELS, Thresholds, alert_episodes, detect_bottlenecks
    from src.data_loader import DataValidationError, reported_days
    from src.kpi_calculations import KPI_DEFINITIONS, add_daily_metrics, aggregate, filter_period, period_kpis
    PIPELINE_ERROR = None
except ImportError as e:
    PIPELINE_ERROR = (f"The pipeline-analytics module could not be loaded ({e}). Keep the `src/` folder next to `{Path(__file__).name}` "
                      "and install the pinned dependencies with `pip install -r requirements.txt`. The forecast tabs are unaffected.")


# Fixed colour per entity (colour follows the model, never its rank) and reserved status colours
DEFAULT_MODEL_COLORS = {"Naive": "#2a78d6", "Seasonal Naive (7d)": "#8a5a2b", "Moving Avg (7d)": "#eb6834", "SARIMA": "#1baf7a",
                        "ETS": "#eda100", "Random Forest": "#e87ba4", "Gradient Boosting": "#008300", "Ensemble": "#4a3aa7"}
SPARE_COLORS = ["#0f8b8d", "#b8336a", "#5c6bc0", "#7a7a7a"]
SCENARIO_COLORS = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#4a3aa7", "#e34948"]
STATUS = {"good": ("#0ca30c", "☑", "Low risk"), "warning": ("#fab219", "⚠", "Watch"),
          "serious": ("#ec835a", "🟠", "Elevated"), "critical": ("#d03b3b", "🔴", "Critical")}
UNAVAILABLE = ("#898781", "❓", "Unavailable (no error history for this model)")
ACTUAL = "#898781"
LABELS = {"hhs_care": "Children in HHS care", "discharged": "Discharges (sponsor placements)",
          "transferred": "Transfers into HHS (from CBP)"}



# ------------------------------------------------------------------------------ data
REQUIRED = ["daily_clean.csv", "forecast_future.csv", "metrics_overall.csv", "metrics_by_horizon.csv", "interval_coverage.csv",
            "robustness.csv", "residuals.csv.gz", "surge_lead_times.csv", "surge_replay_paths.csv", "ews_operating_curve.csv", "metadata.json"]
OPTIONAL = {"cov_by_h": "interval_coverage_by_h.csv", "cov_methods": "interval_method_comparison.csv", "validation": "validation_report.csv"}
META_KEYS = ["data_end", "horizon", "champion", "recon_per_day", "alert_probability", "capacity_default", "pre_break_transfers_per_day",
             "accuracy_champion", "surge_lead", "regime_start", "break_transfers", "break_discharges", "cbp_bad_start", "cbp_bad_end",
             "blank_rows", "stale_reports", "copied_cbp_reports"]
FC_COLS = ["target", "model", "h", "date", "yhat", "lo80", "hi80", "lo95", "hi95"]
DAILY_COLS = ["hhs_care", "discharged", "transferred", "is_reported"]      # read by the forecast tabs on every run
```

```
def artifact_signature():
    """Changes whenever an artifact is rewritten, so the cache reloads after the notebook is re-run (QA D-UI-04)."""
    if not ART.is_dir():
        return ()
    files = tuple(sorted((p.name, p.stat().st_mtime_ns, p.stat().st_size) for p in ART.iterdir() if p.is_file()))
    return (str(ART),) + files if files else ()     # folder path in the key: two artifact folders never share a cache entry



def check_contract(d):
    """Return a readable problem description if the artifacts are internally inconsistent, else None."""
    meta, fc = d["meta"], d["fc"]
    missing_keys = [k for k in META_KEYS if k not in meta]
    if missing_keys:
        return f"metadata.json lacks required key(s): {', '.join(missing_keys)}"
    if not isinstance(meta["horizon"], int) or not 1 <= meta["horizon"] <= 90:
        return f"metadata.json has an unsupported horizon ({meta['horizon']!r}); expected an integer between 1 and 90"
    missing_cols = [c for c in FC_COLS if c not in fc.columns]
    if missing_cols:
        return f"forecast_future.csv lacks column(s): {', '.join(missing_cols)}"
    for tg, champ in meta["champion"].items():
        n = int(((fc["target"] == tg) & (fc["model"] == champ)).sum())
        if n < meta["horizon"]:
            return f"forecast_future.csv has {n} of {meta['horizon']} forecast days for the champion '{champ}' ({tg})"
    if not np.isfinite(fc[["yhat", "lo80", "hi80", "lo95", "hi95"]].to_numpy(float)).all():
        return "forecast_future.csv contains non-finite forecasts or interval bounds"
    return check_daily(d["daily"])



def check_daily(daily):
    """Problems with the cleaned daily table that every tab reads, else None (a damaged table used to crash the app: QA E2E-03)."""
    if len(daily) < 8:
        return f"daily_clean.csv has {len(daily)} row(s); at least 8 days are needed"
    if not isinstance(daily.index, pd.DatetimeIndex):
        return "daily_clean.csv has dates that cannot be parsed"
    missing = [c for c in DAILY_COLS if c not in daily.columns]
    if missing:
        return f"daily_clean.csv lacks column(s): {', '.join(missing)}"
    if daily.index.has_duplicates:
        return f"daily_clean.csv has {int(daily.index.duplicated().sum())} duplicate date(s)"
    if not daily.index.is_monotonic_increasing:
        return "daily_clean.csv is not sorted by date"
    not_numeric = [c for c in DAILY_COLS[:3] if not pd.api.types.is_numeric_dtype(daily[c])]
    if not_numeric or not pd.api.types.is_bool_dtype(daily["is_reported"]):
        return f"daily_clean.csv has values of the wrong type in: {', '.join(not_numeric or ['is_reported'])}"
    gaps = [c for c in ("hhs_care", "discharged") if daily[c].isna().any()]   # the notebook fills every calendar day
    if gaps:
        return f"daily_clean.csv has missing values in: {', '.join(gaps)}"
    return None



@st.cache_data(show_spinner=False)   # spinner shown by the caller on every run, see below
def load_artifacts(signature):
```

```
    """Return (artifacts, error message). Failures become a readable message instead of a traceback (QA D-UI-03)."""
    if not signature:
        return None, "No artifacts found. Run `UAC_Care_Load_Forecasting.ipynb` first. It writes the `artifacts/` folder this app reads."
    missing = [f for f in REQUIRED if not (ART / f).exists()]
    if missing:
        return None, f"Missing artifact file(s): {', '.join(missing)}. Re-run `UAC_Care_Load_Forecasting.ipynb` to regenerate `artifacts/`."
    try:
        d = {
            "daily": pd.read_csv(ART / "daily_clean.csv", parse_dates=["date"]).set_index("date"),
            "fc": pd.read_csv(ART / "forecast_future.csv", parse_dates=["date"]),
            "overall": pd.read_csv(ART / "metrics_overall.csv"),
            "by_h": pd.read_csv(ART / "metrics_by_horizon.csv"),
            "coverage": pd.read_csv(ART / "interval_coverage.csv"),
            "robust": pd.read_csv(ART / "robustness.csv"),
            "lead": pd.read_csv(ART / "surge_lead_times.csv", parse_dates=["event_date"]),
            "replay": pd.read_csv(ART / "surge_replay_paths.csv", parse_dates=["origin"]),
            "oc": pd.read_csv(ART / "ews_operating_curve.csv"),
            "meta": json.loads((ART / "metadata.json").read_text()),
        }
        for key, name in OPTIONAL.items():           # newer artifacts: used when present, never required
            d[key] = pd.read_csv(ART / name) if (ART / name).exists() else None
        problem = check_contract(d)
        if problem:
            return None, f"The artifacts are inconsistent: {problem}. Re-run `UAC_Care_Load_Forecasting.ipynb` to regenerate them."
        resid = pd.read_csv(ART / "residuals.csv.gz")
        horizon_max = int(d["meta"]["horizon"])
        # Pre-pool walk-forward errors once per (series, model, horizon h±2): breach probabilities become O(1) lookups (QA D-PERF-01)
        d["err"] = {(tg, m, h): g.loc[(g["h"] - h).abs() <= 2, "error"].to_numpy()
                    for (tg, m), g in resid.groupby(["target", "model"]) for h in range(1, horizon_max + 1)}
    except Exception as e:  # corrupted/partial artifacts
        return None, f"The artifacts could not be read ({type(e).__name__}: {e}). Re-run `UAC_Care_Load_Forecasting.ipynb` to regenerate them."
    return d, None



SIGNATURE = artifact_signature()
# A cache spinner adds a page element only when the cache misses, which moved the tab bar one slot down on the first run
# after a start or data refresh; on the next run the tabs were remounted and the user's tab reset to Overview (QA E2E-01).
# st.spinner around the call adds its element on every run, so every element keeps its position.
with st.spinner("Loading forecast artifacts…"):
    A, load_error = load_artifacts(SIGNATURE)
if A is None:
    st.error(load_error)
    st.stop()


daily, fc, meta, ERR = A["daily"], A["fc"], A["meta"], A["err"]
H_MAX = int(meta["horizon"])
T_END = pd.Timestamp(meta["data_end"])
CHAMP = meta["champion"]
present = set(fc["model"])
MODELS = [m for m in dict.fromkeys(meta.get("models", list(DEFAULT_MODEL_COLORS)) + sorted(present)) if m in present]   # models come from the artifacts
MODEL_COLORS = {**DEFAULT_MODEL_COLORS, **meta.get("model_colors", {})}
for i, m in enumerate(m for m in MODELS if m not in MODEL_COLORS):
    MODEL_COLORS[m] = SPARE_COLORS[i % len(SPARE_COLORS)]



# ------------------------------------------------------------------------------ helpers
```

```
def rgba(hex_color, alpha):
    h = hex_color.lstrip("#")
    return f"rgba({int(h[0:2], 16)},{int(h[2:4], 16)},{int(h[4:6], 16)},{alpha})"



def style(fig, height=420, yfmt=",.0f", ytitle=None):
    # legend below the plot so long legends never collide with the chart toolbar (QA D-UI-08)
    fig.update_layout(height=height, margin=dict(l=8, r=8, t=30, b=8), hovermode="x unified",
                      legend=dict(orientation="h", yanchor="top", y=-0.14, x=0, title=None))
    fig.update_xaxes(showgrid=False)
    fig.update_yaxes(tickformat=yfmt, gridwidth=1, title=ytitle, zeroline=False)
    return fig



def add_band(fig, x, lo, hi, color, name, alpha=0.15):
    fig.add_trace(go.Scatter(x=x, y=hi, mode="lines", line=dict(width=0), showlegend=False, hoverinfo="skip"))
    fig.add_trace(go.Scatter(x=x, y=lo, mode="lines", line=dict(width=0), fill="tonexty", fillcolor=rgba(color, alpha),
                             name=name, hoverinfo="skip"))



def mark_forecast_start(fig):
    """Vertical line at the last report: observed data to the left, model forecast to the right."""
    fig.add_vline(x=T_END, line=dict(color=ACTUAL, width=1, dash="dot"))
    fig.add_annotation(x=T_END, y=1, yref="paper", text="last report → forecast", showarrow=False, xanchor="left", yanchor="bottom",
                       font=dict(size=11, color=ACTUAL))



def errors_for(target, model, h):
    return ERR.get((target, model, h), np.array([]))



def breach_prob(path, target, model, threshold):
    """P(care load > threshold) per day = share of walk-forward errors that would push the path above it; NaN if no errors exist."""
    out = []
    for h in range(1, len(path) + 1):
        e = errors_for(target, model, h)
        out.append(float(np.mean(path[h - 1] + e > threshold)) if len(e) else np.nan)
    return np.array(out)



def risk_status(p):
    if not np.isfinite(p):
        return UNAVAILABLE
    key = "good" if p < 0.10 else "warning" if p < 0.30 else "serious" if p < 0.60 else "critical"
    return STATUS[key]



def model_path(target, model, horizon):
    return fc[(fc["target"] == target) & (fc["model"] == model) & (fc["h"] <= horizon)].sort_values("h")
```

```
def run_scenario(base_care, base_in, base_out, in_mult=1.0, out_mult=1.0, start_day=1, extra_in=0.0):
    """Shift the baseline care path by the cumulative change in flows (stock-flow identity)."""
    days = np.arange(1, len(base_care) + 1)
    active = (days >= start_day).astype(float)
    d_in = (np.asarray(base_in) * (in_mult - 1) + extra_in) * active
    d_out = np.asarray(base_out) * (out_mult - 1) * active
    return np.asarray(base_care) + np.cumsum(d_in - d_out)



def pct(p):
    return f"{p:.0%}" if np.isfinite(p) else "n/a"



# ---------------------------------------------------------------------------- sidebar
with st.sidebar:
    st.header("Controls")
    horizon = st.slider("Forecast horizon (days)", 1, H_MAX, H_MAX, step=1,
                        help=f"Any horizon from 1 to {H_MAX} days was validated in the walk-forward backtest; longer horizons are not supported.")
    default_models = [m for m in dict.fromkeys([CHAMP["hhs_care"], "Ensemble"]) if m in MODELS]
    models_sel = st.multiselect("Models to display", MODELS, default=default_models)
    interval = st.radio("Confidence interval", ["80%", "95%", "None"], horizontal=True)
    cap_default = int(np.clip(meta["capacity_default"], 100, 20000))
    capacity = st.number_input("Capacity / stress threshold (children)", min_value=100, max_value=20000, value=cap_default, step=50,
                               help="Enter the program's funded bed capacity or an internal stress trigger. The source data contain no capacity figures, "
                                    "so the default is only an illustrative threshold about 5% above today's load.")
    history_days = st.select_slider("History shown", options=[30, 60, 90, 180, 365], value=90)
    st.caption(f"Data through **{T_END:%d %b %Y}**. Champion models are selected on the calibration window and scored on the untouched test window.")


lo_col, hi_col = {"80%": ("lo80", "hi80"), "95%": ("lo95", "hi95"), "None": (None, None)}[interval]
if not models_sel:
    st.warning("Select at least one model in the sidebar.")
    st.stop()


# ---------------------------------------------------------------------------- header
st.title("UAC care load & placement demand forecast")
st.caption("HHS Unaccompanied Alien Children Program · forward-looking capacity intelligence · walk-forward validated forecasts with calibrated uncertainty")


tabs = st.tabs(["Overview", "Care load forecast", "Discharge demand", "Model comparison", "Scenario lab", "Early warning & data quality",
                "Pipeline analytics & data explorer"])


# ============================================================================ Overview
with tabs[0]:
    champ = CHAMP["hhs_care"]
    current = float(daily["hhs_care"].iloc[-1])
    path = model_path("hhs_care", champ, horizon)
    end_val = float(path["yhat"].iloc[-1])
    p_breach = breach_prob(path["yhat"].to_numpy(), "hhs_care", champ, capacity)
    p_max = np.nanmax(p_breach) if np.isfinite(p_breach).any() else np.nan
    colr, icon, label = risk_status(p_max)
    fin = model_path("transferred", "Ensemble", horizon)["yhat"].to_numpy()
    fout = model_path("discharged", "Ensemble", horizon)["yhat"].to_numpy()
    acc = meta["accuracy_champion"]
```

```
    lead = pd.DataFrame(meta["surge_lead"])
    acc_help = meta.get("accuracy_definition", "Accuracy % = 100 − WAPE on the test window")


    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Children in HHS care (latest report)", f"{current:,.0f}",
              delta=f"{current - daily['hhs_care'].iloc[-8]:+,.0f} vs 7 days earlier", delta_color="off", border=True,
              chart_data=daily["hhs_care"].iloc[-60:].round().tolist())
    c2.metric(f"Forecast in {horizon} days ({champ})", f"{end_val:,.0f}", delta=f"{end_val - current:+,.0f}", delta_color="inverse", border=True)
    flow_view = (fin - fout).mean() + meta["recon_per_day"]
    c3.metric(f"Change per day ({champ})", f"{(end_val - current) / horizon:+.1f}/day",
              help=f"Direct care-load forecast. Flow-based cross-check (Ensemble transfers − discharges + {meta['recon_per_day']:.1f}/day "
                   f"weekend/non-CBP reconciliation): {flow_view:+.1f}/day. Small daily flow biases compound, so the two routes can differ; see the Discharge demand tab.",
              border=True)
    c4.metric(f"Breach risk > {capacity:,} ({horizon}d)", pct(p_max),
              help="Highest single-day probability, over the horizon, that care load exceeds the threshold (champion model + calibrated walk-forward errors). "
                   "The threshold is your input, not program data.", border=True)


    st.markdown(f"#### {icon} Capacity status: **{label}**")
    st.caption("Status bands: Low risk < 10% · Watch 10–30% · Elevated 30–60% · Critical ≥ 60% probability of exceeding the threshold within the horizon. "
               f"'Elevated' (≥ {meta['alert_probability']:.0%}) is the recommended trigger to start scaling capacity (see Early warning tab).")


    c5, c6, c7 = st.columns(3)
    c5.metric("Forecast accuracy: care load (test)", f"{acc['hhs_care']:.1f}%", help=acc_help, border=True)
    c6.metric("Forecast accuracy: discharges (test)", f"{acc['discharged']:.1f}%", help=acc_help, border=True)
    if not lead.empty:
        rep = lead[lead["source"].str.startswith("replay")]
        if not rep.empty:
            c7.metric("Surge lead time (2023–24 replay, median)", f"{rep['median_lead_days'].iloc[0]:.0f} days",
                      help=f"Detection rate {rep['detection_rate'].iloc[0]:.0%} across {int(rep['events'].iloc[0])} threshold-crossing events.", border=True)


    fig = go.Figure()
    hist = daily["hhs_care"].iloc[-history_days:]
    fig.add_trace(go.Scatter(x=hist.index, y=hist, name="Observed (Fri/Sat interpolated)", line=dict(color=ACTUAL, width=2)))
    if lo_col:
        add_band(fig, path["date"], path[lo_col], path[hi_col], MODEL_COLORS[champ], f"{champ} {interval} interval")
    fig.add_trace(go.Scatter(x=path["date"], y=path["yhat"], name=f"{champ} forecast", line=dict(color=MODEL_COLORS[champ], width=2)))
    fig.add_hline(y=capacity, line=dict(color=STATUS["critical"][0], width=1), annotation_text=f"threshold {capacity:,} (user input)", annotation_position="top left")
    mark_forecast_start(fig)
    st.plotly_chart(style(fig, 380, ytitle="children"), key="overview_chart")


# ============================================================================== Care load forecast
with tabs[1]:
    st.subheader("Future care load forecast")
    fig = go.Figure()
    hist = daily["hhs_care"].iloc[-history_days:]
    fig.add_trace(go.Scatter(x=hist.index, y=hist, name="Actual", line=dict(color=ACTUAL, width=2)))
    for m in models_sel:
        p = model_path("hhs_care", m, horizon)
        if lo_col:
            add_band(fig, p["date"], p[lo_col], p[hi_col], MODEL_COLORS[m], f"{m} {interval}", alpha=0.12)
        fig.add_trace(go.Scatter(x=p["date"], y=p["yhat"], name=m, line=dict(color=MODEL_COLORS[m], width=2)))
    fig.add_hline(y=capacity, line=dict(color=STATUS["critical"][0], width=1))
    mark_forecast_start(fig)
    st.plotly_chart(style(fig, 460, ytitle="children in care"), key="care_chart")
```

```
    tbl = fc[(fc["target"] == "hhs_care") & fc["model"].isin(models_sel) & (fc["h"] <= horizon)]
    pivot = tbl.pivot_table(index="date", columns="model", values="yhat").round(0)
    pivot.index = pivot.index.strftime("%a %d %b %Y")
    with st.expander("Forecast table (point forecasts)"):
        st.dataframe(pivot, height=320)
        st.download_button("Download these forecasts with 80%/95% intervals (CSV)", tbl.sort_values(["model", "h"]).to_csv(index=False).encode("utf-8"),
                           file_name=f"uac_care_load_forecast_{T_END:%Y%m%d}_h{horizon}.csv", mime="text/csv", on_click="ignore", key="dl_care_fc")
    st.markdown("**Capacity breach probability by day** (threshold from the sidebar)")
    probs = pd.DataFrame({m: breach_prob(model_path("hhs_care", m, horizon)["yhat"].to_numpy(), "hhs_care", m, capacity) for m in models_sel},
                         index=np.arange(1, horizon + 1))
    no_err = [m for m in models_sel if not np.isfinite(probs[m]).any()]
    if no_err:
        st.info(f"No walk-forward error history for {', '.join(no_err)}: breach probability unavailable for these models.")
    fig = go.Figure()
    for m in models_sel:
        fig.add_trace(go.Scatter(x=probs.index, y=probs[m] * 100, name=m, line=dict(color=MODEL_COLORS[m], width=2)))
    fig.update_xaxes(title="days ahead")
    fig.update_yaxes(range=[0, 100])        # fixed 0–100%: a flat 0% line no longer autoscales to duplicated '-1, -1, 0, 1, 1' ticks (QA D-UI-13)
    st.plotly_chart(style(fig, 300, yfmt=".0f", ytitle="probability (%)"), key="breach_chart")


# ============================================================================== Discharge demand
with tabs[2]:
    st.subheader("Discharge (placement) demand & intake balance")
    left, right = st.columns(2)
    for target, col in (("discharged", left), ("transferred", right)):
        fig = go.Figure()
        hist = daily[target].iloc[-history_days:]
        rep = daily.loc[hist.index, "is_reported"].astype(bool)
        fig.add_trace(go.Scatter(x=hist.index[rep], y=hist[rep], mode="markers", name="Reported day",
                                 marker=dict(color=ACTUAL, size=6)))
        fig.add_trace(go.Scatter(x=hist.index, y=hist.rolling(7, min_periods=1).mean(), name="7-day mean", line=dict(color=ACTUAL, width=2)))
        for m in models_sel:
            p = model_path(target, m, horizon)
            if lo_col:
                add_band(fig, p["date"], p[lo_col], p[hi_col], MODEL_COLORS[m], f"{m} {interval}", alpha=0.12)
            fig.add_trace(go.Scatter(x=p["date"], y=p["yhat"], name=m, line=dict(color=MODEL_COLORS[m], width=2)))
        mark_forecast_start(fig)
        col.markdown(f"**{LABELS[target]}**")
        col.plotly_chart(style(fig, 360, ytitle="children / day"), key=f"flow_{target}")


    weeks = pd.DataFrame({"Transfers in": fin, "Discharges out": fout}, index=model_path("discharged", "Ensemble", horizon)["date"])
    week_no = (np.arange(len(weeks)) // 7) + 1
    days_in_week = pd.Series(week_no).value_counts().sort_index()
    weekly = weeks.groupby(week_no).sum().round(0)
    weekly["Net change"] = weekly["Transfers in"] - weekly["Discharges out"] + days_in_week.to_numpy() * meta["recon_per_day"]
    weekly.index = [f"Week {i}" if n == 7 else f"Week {i} (partial: {n} day{'s' if n > 1 else ''})" for i, n in days_in_week.items()]
    c1, c2 = st.columns([1.3, 1])          # the table needs room for partial-week labels (QA D-UI-13)
    c1.markdown("**Weekly totals (Ensemble)**")
    c1.dataframe(weekly.style.format("{:+,.0f}", subset=["Net change"]).format("{:,.0f}", subset=["Transfers in", "Discharges out"]),
                 column_config={"_index": st.column_config.Column("Week", width="medium")})   # partial-week labels stay readable (QA D-UI-13)
    c1.caption(f"Net change adds the measured reconciliation term ({meta['recon_per_day']:+.1f}/day): care-load growth not explained by recorded transfers – discharges "
               "(non-CBP admissions, unreported weekend flows). The file has no 'other exits' column.")
    net = weeks["Transfers in"] - weeks["Discharges out"] + meta["recon_per_day"]
    fig = go.Figure(go.Bar(x=net.index, y=net, marker_color=[STATUS["serious"][0] if v > 0 else STATUS["good"][0] for v in net],
                           name="net pressure", hovertemplate="%{x|%d %b}: %{y:+.1f}/day<extra></extra>"))
    ticks = np.unique(np.round(np.linspace(min(net.min(), 0.0), max(net.max(), 0.0), 5), 1)) + 0.0   # "+ 0.0" turns -0.0 into 0.0
    fig.update_yaxes(tickvals=ticks, ticktext=[f"{v:,.1f}" for v in ticks])                      # no '-0' tick label (QA D-UI-13)
```

```
    c2.markdown("**Forecast net pressure (in − out, children/day)**: orange = filling, green = draining")
    c2.plotly_chart(style(fig, 280, yfmt=",.1f"), key="net_chart")   # '+.0f' is rejected by Plotly (QA D-UI-07); one decimal avoids a '-0' tick (QA D-UI-13)


# ============================================================================== Model comparison
with tabs[3]:
    st.subheader("Model selection & comparison (walk-forward backtest)")
    c1, c2 = st.columns(2)
    target = c1.selectbox("Series", list(LABELS), format_func=LABELS.get)
    period = c2.radio("Window", ["test", "calibration"], horizontal=True, help="Champions are chosen on the calibration window; the test window was never used for any choice.")
    board = A["overall"][(A["overall"]["target"] == target) & (A["overall"]["period"] == period)].drop(columns=["target", "period"])
    board = board.sort_values("MAE").set_index("model")
    st.markdown(f"Champion for **{LABELS[target]}**: **{CHAMP[target]}**")
    st.dataframe(board.style.format({"MAE": "{:,.2f}", "RMSE": "{:,.2f}", "MAPE %": "{:.1f}", "WAPE %": "{:.1f}", "Accuracy %": "{:.1f}", "Bias": "{:+.2f}", "n": "{:,.0f}"},
      na_rep="n/a")
                 .highlight_min(subset=["MAE", "RMSE"], color=rgba("#0ca30c", 0.18)))
    st.caption("MAE/RMSE in children (per day for flows). MAPE uses non-zero actuals only ('zero y excluded' counts the rest). "
               f"{meta.get('accuracy_definition', 'Accuracy % = 100 − WAPE')}. Bias = mean(forecast − actual): negative = under-forecast.")
    bh = A["by_h"][(A["by_h"]["target"] == target) & (A["by_h"]["period"] == period) & (A["by_h"]["h"] <= horizon)]
    fig = go.Figure()
    for m in MODELS:
        g = bh[bh["model"] == m]
        fig.add_trace(go.Scatter(x=g["h"], y=g["MAE"], name=m, visible=True if m in models_sel or m == CHAMP[target] else "legendonly",
                                 line=dict(color=MODEL_COLORS[m], width=3 if m == CHAMP[target] else 2)))
    fig.update_xaxes(title="horizon (days ahead)")
    st.markdown("**Horizon error**: MAE by days ahead (click legend items to toggle models)")
    st.plotly_chart(style(fig, 360, yfmt=",.1f", ytitle="MAE (children)"), key="horizon_chart")
    c1, c2 = st.columns(2)
    cov = A["coverage"][A["coverage"]["target"] == target].drop(columns="target").set_index("model")
    c1.markdown("**Interval coverage on test** (target: 0.80 / 0.95), fixed calibration window")
    c1.dataframe(cov.style.format({"coverage80": "{:.0%}", "coverage95": "{:.0%}", "width80": "{:,.1f}"}))
    rob = A["robust"][A["robust"]["target"] == target].drop(columns="target").set_index("model")
    c2.markdown("**Robustness scorecard**")
    c2.dataframe(rob.round(2))
    if A["cov_methods"] is not None:
        cm = A["cov_methods"][A["cov_methods"]["target"] == target].pivot_table(index="model", columns="method", values="coverage80")
        st.markdown("**80% interval coverage by calibration method (test window)**: the production forecast uses the rolling 26-week rule")
        st.dataframe(cm.style.format("{:.0%}", na_rep="n/a"))


# ============================================================================== Scenario lab
with tabs[4]:
    st.subheader("Scenario comparison view")
    st.caption("Only the **baseline** is a model forecast. Every other path is a **hypothetical what-if** built from an assumed change in flows; "
               "it is not a prediction. The threshold line is your input (the source data contain no bed capacity).")
    anchor = st.selectbox("Baseline care-load model", MODELS, index=MODELS.index("Ensemble") if "Ensemble" in MODELS else 0)
    base_care = model_path("hhs_care", anchor, horizon)["yhat"].to_numpy()
    dates = model_path("hhs_care", anchor, horizon)["date"]
    pre_in = meta["pre_break_transfers_per_day"]
    presets = {
        "Baseline (current trends)": dict(),
        "Discharge slowdown (−30%)": dict(out_mult=0.7),
        "Placement acceleration (+50% discharges)": dict(out_mult=1.5),
        "Intake surge (3× transfers from day 8)": dict(in_mult=3.0, start_day=8),
        "Combined stress (2× in, −25% out)": dict(in_mult=2.0, out_mult=0.75),
        f"Return to 2024 intake (~{pre_in:.0f}/day from day 8)": dict(extra_in=pre_in - fin.mean(), start_day=8),
    }
    chosen = st.multiselect("Preset scenarios", list(presets), default=list(presets)[:4])
    with st.expander("Build a custom scenario", expanded=True):
```

```
        k1, k2, k3, k4 = st.columns(4)
        in_mult = k1.slider("Transfers multiplier", 0.5, 5.0, 1.0, 0.1)
        out_mult = k2.slider("Discharges multiplier", 0.5, 2.0, 1.0, 0.05)
        extra = k3.slider("Extra daily intake", 0, 300, 0, 5)
        if horizon > 1:
            start = k4.slider("Shock starts on day", 1, horizon, 1)
        else:  # a one-day horizon leaves nothing to choose (a slider needs min < max)
            start = 1
            k4.caption("Shock starts on day 1 (one-day horizon)")
    scen = {name: run_scenario(base_care, fin, fout, **presets[name]) for name in chosen}
    if in_mult != 1 or out_mult != 1 or extra:
        scen["Custom scenario"] = run_scenario(base_care, fin, fout, in_mult, out_mult, start, extra)


    if not scen:  # empty state instead of a KeyError (QA D-UI-01)
        st.info("Select at least one preset scenario, or move a custom-scenario slider, to compare scenarios.")
    else:
        fig = go.Figure()
        for i, (name, path_s) in enumerate(scen.items()):
            fig.add_trace(go.Scatter(x=dates, y=path_s, name=name, line=dict(color=SCENARIO_COLORS[i % len(SCENARIO_COLORS)], width=2)))
        fig.add_hline(y=capacity, line=dict(color=STATUS["critical"][0], width=1), annotation_text=f"threshold {capacity:,}", annotation_position="top left")
        st.plotly_chart(style(fig, 440, ytitle="children in care"), key="scenario_chart")


        rows = []
        for name, path_s in scen.items():
            pb = breach_prob(path_s, "hhs_care", anchor, capacity)
            pb_max = np.nanmax(pb) if np.isfinite(pb).any() else np.nan
            first = next((i + 1 for i, v in enumerate(path_s) if v > capacity), None)
            colr, icon, label = risk_status(pb_max)
            rows.append({"Scenario": name, f"Day {horizon} care load": round(path_s[-1]), "Change vs today": round(path_s[-1] - current),
                         "Peak": round(path_s.max()),
                         "First day above threshold": f"Day {first}" if first else "Not reached",   # one dtype (QA D-UI-05)
                         "Breach probability (if the assumption holds)": pct(pb_max), "Status": f"{icon} {label}",
                         "Type": "model forecast" if name.startswith("Baseline") else "hypothetical"})
        st.dataframe(pd.DataFrame(rows).set_index("Scenario"),
                     column_config={"Change vs today": st.column_config.NumberColumn(format="%+d")})   # explicit sign (QA D-UI-13)


    if scen:  # the translator needs at least one scenario (QA D-UI-01)
        with st.expander("Staffing & bed translator (planning assumptions, edit to match your program)"):
            s1, s2, s3 = st.columns(3)
            ratio = s1.number_input("Children per case manager (assumption)", 1, 50, 10)
            buffer = s2.slider("Surge buffer on beds (%)", 0, 50, 10)
            sc_name = s3.selectbox("Scenario", list(scen))
            peak = scen[sc_name].max()
            e_h = errors_for("hhs_care", anchor, horizon)
            upper = peak + (np.quantile(e_h, 0.9) if len(e_h) else 0.0)
            st.write(f"Peak expected care load: **{peak:,.0f}** · planning upper bound (P90): **{upper:,.0f}**")
            st.write(f"Beds to have available incl. {buffer}% buffer: **{upper * (1 + buffer / 100):,.0f}** · "
                     f"case managers needed at 1:{ratio}: **{np.ceil(upper / ratio):,.0f}** (today: {np.ceil(current / ratio):,.0f})")
            st.caption("Ratios are user assumptions, not program policy. Use them to translate forecasts into staffing and bed conversations.")


# ============================================================================ Early warning & data quality
with tabs[5]:
    p_star = meta["alert_probability"]
    st.subheader("Early warning: historical surge replay (2023–2024)")
    st.caption(f"Alert rule: warn when the probability of exceeding a threshold within {H_MAX} days reaches **{p_star:.0%}** "
               "(the 'Elevated' status). Replayed in real time over the 2023–24 surges with a rolling ETS model.")
```

```
    rp, lead_t = A["replay"], A["lead"]
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=rp["origin"], y=rp["actual"], name="Actual care load", line=dict(color=ACTUAL, width=2)))
    fig.add_trace(go.Scatter(x=rp["origin"], y=rp["peak"], name=f"{H_MAX}-day alert level (P ≥ {p_star:.0%})", line=dict(color=MODEL_COLORS["ETS"], width=2)))
    for detected, name, colr in ((True, "Surge event: warned in advance", STATUS["good"][0]), (False, "Surge event: missed", STATUS["critical"][0])):
        ev = lead_t[lead_t["source"].str.startswith("replay") & (lead_t["detected"] == detected)]
        if len(ev):
            fig.add_trace(go.Scatter(x=ev["event_date"], y=ev["threshold"], mode="markers", name=name,
                                     marker=dict(color=colr, size=10, line=dict(color="white", width=2)),
                                     customdata=ev["lead_days"], hovertemplate="%{x|%d %b %Y} · threshold %{y:,}: %{customdata} days warning<extra></extra>"))
    st.plotly_chart(style(fig, 400, ytitle="children in care"), key="replay_chart")
    c1, c2 = st.columns([1, 1.2])
    c1.markdown(f"**Surge lead time at P ≥ {p_star:.0%}**")
    c1.dataframe(pd.DataFrame(meta["surge_lead"]).set_index("source").rename(columns={
        "events": "Events", "detection_rate": "Detection rate", "median_lead_days": "Median lead (days)",
        "mean_lead_days": "Mean lead (days)", "false_alarm_rate": "False alarms"}).style.format(
        {"Detection rate": "{:.0%}", "False alarms": "{:.0%}", "Median lead (days)": "{:.0f}", "Mean lead (days)": "{:.1f}"}))
    c2.markdown("**Operating curve**: earlier warnings cost more false alarms")
    c2.dataframe(A["oc"].set_index(["source", "alert at P(breach) ≥"]))


    st.subheader("Data quality: root-cause findings")
    q1, q2, q3, q4 = st.columns(4)
    q1.metric("Blank padding rows removed", meta["blank_rows"], border=True)
    q2.metric("Stale HHS reports masked", meta["stale_reports"], border=True)
    q3.metric("Copied CBP reports masked", meta["copied_cbp_reports"], border=True)
    q4.metric("Current regime since", pd.Timestamp(meta["regime_start"]).strftime("%d %b %Y"), border=True)
    if "validation_issues" in meta:
        st.caption(f"Input validation on the last run: **{meta['validation_issues']}** implausible value(s) quarantined or flagged; "
                   f"**{meta.get('fit_failures', 0)}** model fit/prediction failure(s) logged (a failing model falls back to naive persistence).")
    if A["validation"] is not None and len(A["validation"]):
        st.dataframe(A["validation"])
    st.markdown(
        f"""
* **Reporting cadence:** reports arrive Sunday–Thursday; Friday and Saturday are interpolated for continuity but **never scored**, and forecasts are only issued from reported days (no
      look-ahead).
* **Copied CBP block:** from **{meta['cbp_bad_start']}** to **{meta['cbp_bad_end']}** every CBP record (apprehended, in custody, transferred) duplicates a later date, and the stock-flow
      identity breaks. CBP columns are masked there; HHS columns reconcile and are kept.
* **Regime shift:** transfers broke on **{meta['break_transfers']}** and discharges on **{meta['break_discharges']}**. Statistical models train on the current regime; ML models use
      scale-free targets.
""")
    fig = go.Figure()
    for target, colr in (("transferred", "#eb6834"), ("discharged", "#1baf7a")):
        fig.add_trace(go.Scatter(x=daily.index, y=daily[target].rolling(7, min_periods=3).mean(), name=f"{LABELS[target]} (7-day mean)", line=dict(color=colr, width=2)))
    fig.add_vrect(x0=meta["cbp_bad_start"], x1=meta["cbp_bad_end"], fillcolor=rgba("#898781", 0.15), line_width=0,
                  annotation_text="CBP columns masked", annotation_position="top left")
    st.plotly_chart(style(fig, 340, ytitle="children / day"), key="dq_chart")



# ============================================================================ Pipeline analytics (descriptive, recomputed live)
PIPE_STATUS = {0: ("☑", "Normal"), 1: ("⚠", "Watch"), 2: ("🔘", "Alert")}



@st.cache_data(show_spinner=False)
def pipeline_reported(signature, _daily):
    """Reported days only (no interpolated Fri/Sat values), CBP masked where untrusted; cached per artifact version."""
    return reported_days(_daily)
```

```
@st.cache_data(show_spinner=False)   # spinner shown by the caller on every run (stable element positions, QA E2E-01)
def pipeline_flags(signature, th_values, _rep):
    """Indicators and alert levels over the FULL history, so trailing windows at a period's start use earlier reports."""
    return detect_bottlenecks(_rep, Thresholds(*th_values))



def num(v, spec, suffix=""):
    return f"{v:{spec}}{suffix}" if v is not None and np.isfinite(v) else "n/a"



def render_pipeline_tab():
    """Period filter → KPI cards → status banner → charts → episodes → summary table → data explorer and downloads."""
    try:
        rep = pipeline_reported(SIGNATURE, daily)
    except DataValidationError as e:
        st.error(f"The cleaned daily table cannot be used: {e}. Re-run the notebook to regenerate `artifacts/`.")
        return
    if rep.empty:
        st.info("The cleaned daily table contains no reported days.")
        return
    first, last = rep.index.min().date(), rep.index.max().date()
    regime = pd.Timestamp(meta["regime_start"]).date()
    pre_start = (pd.Timestamp(meta["cbp_bad_end"]) + pd.Timedelta(days=1)).date()
    pre_end = (pd.Timestamp(meta["break_transfers"]) - pd.Timedelta(days=1)).date()
    presets = {f"Current regime (since {regime:%d %b %Y})": (regime, last),
               "Last 90 days": (last - pd.Timedelta(days=89).to_pytimedelta(), last),
               "Last 12 months": (last - pd.Timedelta(days=364).to_pytimedelta(), last),
               f"Before the 2025 break ({pre_start:%d %b %Y} – {pre_end:%d %b %Y})": (pre_start, pre_end),
               "Full history": (first, last),
               "Custom range": None}


    c1, c2 = st.columns([1.3, 1])
    choice = c1.selectbox("Analysis period", list(presets), key="pa_period",
                          help="All KPIs, charts, alerts and tables below are recomputed for this period.")
    if presets[choice] is None:
        picked = c2.date_input("Custom date range", value=(max(first, last - pd.Timedelta(days=180).to_pytimedelta()), last),
                               min_value=first, max_value=last, format="DD/MM/YYYY", key="pa_range")
        if not isinstance(picked, (tuple, list)) or len(picked) != 2:   # also what a typed impossible or out-of-range date gives (QA3-UX-01)
            st.info(f"Select both a start and an end date to update the analytics. Dates must lie between {first:%d %b %Y} and "
                    f"{last:%d %b %Y}; type them as DD/MM/YYYY – DD/MM/YYYY or pick them in the calendar.")
            return
        start, end = picked
    else:
        start, end = presets[choice]
        c2.caption(f"**{pd.Timestamp(start):%d %b %Y} – {pd.Timestamp(end):%d %b %Y}** · data available {first:%d %b %Y} – {last:%d %b %Y}")


    with st.expander("Alert thresholds (documented defaults; adjust to your program's standards)"):
        t1, t2, t3 = st.columns(3)
        g_w, g_a = t1.slider("Census growth: Watch / Alert (% in 28 days)", 1.0, 50.0, (5.0, 10.0), 0.5, key="pa_growth")
        d_w, d_a = t2.slider("Discharge slowdown: Watch / Alert (% below 8-week median)", 5.0, 90.0, (15.0, 30.0), 1.0, key="pa_drop")
        h_w, h_a = t3.slider("CBP holding time: Watch / Alert (days)", 0.5, 10.0, (2.0, 3.0), 0.25, key="pa_hold")
        st.caption("Each indicator is evaluated on every reported day from trailing windows only (no look-ahead); a day's status is its worst indicator. "
```

```
              "The 3-day CBP default mirrors the 72-hour transfer standard for unaccompanied children (TVPRA 2008); the census and discharge "
              "defaults are operating choices, not program policy. Recorded throughput is shown but not used for alerts because recorded "
              "flows do not close the census balance.")
th = Thresholds(census_growth_watch_pct=g_w, census_growth_alert_pct=g_a, discharge_drop_watch_pct=d_w, discharge_drop_alert_pct=d_a,
                cbp_hold_watch_days=h_w, cbp_hold_alert_days=h_a)
try:
    th.validate()
    with st.spinner("Detecting bottlenecks…"):
        flags = pipeline_flags(SIGNATURE, th.as_tuple(), rep)
    p, fp = filter_period(rep, start, end), filter_period(flags, start, end)
except ValueError as e:                                      # reversed range or inconsistent thresholds
    st.warning(f"{e}. Adjust the selection to continue.")
    return
if p.empty:
    st.info(f"No reported days between {pd.Timestamp(start):%d %b %Y} and {pd.Timestamp(end):%d %b %Y} "
            "(reports are published Sunday–Thursday). Choose a wider range.")
    return


k = period_kpis(p)
span = (k["end"] - k["start"]).days + 1
lbl = {key: v[0] for key, v in KPI_DEFINITIONS.items()}
hlp = {key: v[1] for key, v in KPI_DEFINITIONS.items()}


st.markdown("##### Populations (stocks)")
m1, m2, m3, m4 = st.columns(4)
m1.metric(lbl["hhs_care_end"], num(k["hhs_care_end"], ",.0f"), border=True, delta_color="off",
          delta=f"{num(k['hhs_care_change'], '+,.0f')} ({num(k['hhs_care_change_pct'], '+.1f', '%')}) vs period start" if np.isfinite(k["hhs_care_change"]) else None,
          help=f"{hlp['hhs_care_end']}. Change: {hlp['hhs_care_change']}.", chart_data=p["hhs_care"].dropna().round().tolist()[-90:])
m2.metric(lbl["hhs_care_peak"], num(k["hhs_care_peak"], ",.0f"), border=True,
          help=f"{hlp['hhs_care_peak']}: {k['hhs_care_peak_date']:%d %b %Y}." if pd.notna(k["hhs_care_peak_date"]) else hlp["hhs_care_peak"])
cbp_note = "" if k["cbp_custody_days"] else " No trusted CBP reports in this period (copied records 12 Jan 2023 – 30 Jan 2024 are masked)."
m3.metric(lbl["cbp_custody_end"], num(k["cbp_custody_end"], ",.0f"), border=True, help=hlp["cbp_custody_end"] + "." + cbp_note,
          delta=f"{num(k['cbp_custody_change'], '+,.0f')} vs period start" if np.isfinite(k["cbp_custody_change"]) else None, delta_color="off")
m4.metric("Reported days in period", f"{k['reported_days']:,}", border=True,
          help=f"{span:,} calendar days from {k['start']:%d %b %Y} to {k['end']:%d %b %Y}; Friday/Saturday are not published and not imputed here.")


st.markdown("##### Flows and efficiency")
m1, m2, m3, m4 = st.columns(4)
m1.metric(lbl["transferred_per_day"], num(k["transferred_per_day"], ",.1f"), border=True, help=hlp["transferred_per_day"] + cbp_note)
m2.metric(lbl["discharged_per_day"], num(k["discharged_per_day"], ",.1f"), border=True, help=hlp["discharged_per_day"])
m3.metric(lbl["discharge_rate_pct"], num(k["discharge_rate_pct"], ".2f", "%"), border=True,
          help=f"{hlp['discharge_rate_pct']}. {lbl['implied_stay_days']}: {num(k['implied_stay_days'], ',.0f')} days ({hlp['implied_stay_days']}).")
m4.metric(lbl["implied_cbp_hold_days"], num(k["implied_cbp_hold_days"], ".2f", " days"), border=True, help=hlp["implied_cbp_hold_days"] + "." + cbp_note)
m1, m2, m3, m4 = st.columns(4)
m1.metric(lbl["apprehended_per_day"], num(k["apprehended_per_day"], ",.1f"), border=True, help=hlp["apprehended_per_day"] + cbp_note)
m2.metric(lbl["implied_stay_days"], num(k["implied_stay_days"], ",.0f", " days"), border=True,
          help=hlp["implied_stay_days"] + ". The research paper's 36 → 203-day figure (Section 3.3) averages a 28-day smoothed daily series over "
               "fixed windows, so it differs slightly; Appendix A of the paper's Appendix volume reconciles the two definitions.")
m3.metric(lbl["throughput_ratio"], num(k["throughput_ratio"], ".2f"), border=True, help=hlp["throughput_ratio"] + ".")
m4.metric(lbl["transfer_rate_pct"], num(k["transfer_rate_pct"], ".1f", "%"), border=True, help=hlp["transfer_rate_pct"] + "." + cbp_note)


latest = fp.iloc[-1]
if np.isfinite(latest["overall_level"]):
    icon, name = PIPE_STATUS[int(latest["overall_level"])]
else:
    icon, name = "❓", "Insufficient data"
```

```
st.markdown(f"#### {icon} Bottleneck status on {fp.index[-1]:%d %b %Y} (last report in period): **{name}**")
parts = []
for key, spec in PIPE_INDICATORS.items():
    lv, val, unit = latest[f"{key}_level"], latest[spec["value"]], spec["unit"]
    state = PIPE_STATUS[int(lv)][1] if np.isfinite(lv) else "insufficient data"
    if key == "discharge_slowdown" and np.isfinite(val) and val < 0:   # "-19 % below" read as a slowdown (QA E2E-04)
        val, unit = -val, unit.replace("below", "above")
    shown = "∞ (no transfers)" if np.isinf(val) else num(val, {"census_growth": "+.1f", "cbp_holding": ".2f"}.get(key, ".1f"))
    parts.append(f"{spec['label']}: **{state}** ({shown} {unit})")
st.caption(" · ".join(parts))
share = fp["overall_level"].value_counts(normalize=True)
st.caption("Share of reported days in this period: " + " · ".join(f"{PIPE_STATUS[lv][0]} {PIPE_STATUS[lv][1]} {share.get(float(lv), 0):.0%}" for lv in (0, 1, 2))
           + (f" · ❓ insufficient data {fp['overall_level'].isna().mean():.0%}" if fp["overall_level"].isna().any() else ""))


st.plotly_chart(pviz.population_figure(rep, start, end), key="pa_population")
st.plotly_chart(pviz.flows_figure(rep, start, end), key="pa_flows")
left, right = st.columns(2)
left.plotly_chart(pviz.rates_figure(flags, th, start, end), key="pa_rates")
right.plotly_chart(pviz.backlog_figure(flags, th, start, end), key="pa_backlog")


episodes = alert_episodes(fp, min_level=1)
st.markdown("**Detected pressure episodes** (consecutive reported days at Watch or Alert; episodes are cut at the period boundaries)")
if fp["overall_level"].isna().all():                    # nothing evaluated is not the same as "all clear" (QA E2E-05)
    st.info("No indicator could be evaluated in this period: the trailing windows need earlier reports. Choose a later or longer period.")
elif episodes.empty:
    st.success("No indicator reached Watch or Alert in this period.")
else:
    shown = episodes.assign(**{"Peak value": episodes["Peak value"].map(lambda v: "∞" if np.isinf(v) else f"{v:,.2f}")})
    st.dataframe(shown, hide_index=True, column_config={"Start": st.column_config.DateColumn(format="DD MMM YYYY"),
                                                        "End": st.column_config.DateColumn(format="DD MMM YYYY")})


gran = st.radio("Summary table", ["Daily", "Weekly", "Monthly"], index=1, horizontal=True, key="pa_gran",
                help="Weekly = Sunday–Saturday weeks labelled by the Sunday. Stocks show first/last/mean reports; flows show reported-day totals and means.")
table = aggregate(p, gran.lower())
fmt = {c: st.column_config.NumberColumn(format="%.2f") for c in ("Discharge rate %/day", "CBP holding time (days)", "Recorded throughput")}
fmt.update({c: st.column_config.NumberColumn(format="%.1f") for c in ("Transfers / reported day", "Discharges / reported day",
                                                                      "HHS census (mean)", "CBP custody (mean)")})
st.dataframe(table, height=320, column_config={"Period": st.column_config.DateColumn(format="DD MMM YYYY"), **fmt})


explorer = add_daily_metrics(p).join(fp.drop(columns=["indicators_available"]))
with st.expander(f"Data explorer: {len(explorer):,} reported days with derived metrics and alert levels"):
    st.dataframe(explorer, height=360, column_config={"_index": st.column_config.DateColumn("date", format="DD MMM YYYY")})
days_row = {"KPI": "Reported days in period", "Value": k["reported_days"],
            "Definition": f"Reports between {k['start']:%d %b %Y} and {k['end']:%d %b %Y}; Friday/Saturday are not published (QA E2E-06)"}
kpi_rows = [days_row] + [{"KPI": label, "Value": k[key], "Definition": definition} for key, (label, definition) in KPI_DEFINITIONS.items()]
stamp = f"{pd.Timestamp(start):%Y%m%d}_{pd.Timestamp(end):%Y%m%d}"
d1, d2, d3, d4 = st.columns(4)               # four distinct file names: "daily" summary and explorer used to collide (QA E2E-02)
d1.download_button("KPI summary (CSV)", pd.DataFrame(kpi_rows).to_csv(index=False).encode("utf-8"), f"uac_pipeline_kpis_{stamp}.csv",
                   "text/csv", on_click="ignore", key="pa_dl_kpis")
d2.download_button(f"{gran} summary (CSV)", table.to_csv().encode("utf-8"), f"uac_pipeline_summary_{gran.lower()}_{stamp}.csv",
                   "text/csv", on_click="ignore", key="pa_dl_table")
d3.download_button("Explorer data (CSV)", explorer.to_csv(index_label="date").encode("utf-8"), f"uac_pipeline_explorer_{stamp}.csv",
                   "text/csv", on_click="ignore", key="pa_dl_explorer")
d4.download_button("Alert episodes (CSV)", episodes.to_csv(index=False).encode("utf-8"), f"uac_pipeline_alerts_{stamp}.csv",
                   "text/csv", on_click="ignore", key="pa_dl_episodes")
```

```
with tabs[6]:
    st.subheader("Pipeline analytics: CBP custody → HHS care, transfers, discharges and backlog")
    st.caption("Descriptive analytics of the published reports, recomputed for the period you choose. **Reported days only**: Friday/Saturday are never "
               "published and are not imputed here. **Stocks** (children in custody or care) are point-in-time counts described by first, last, mean or "
               "peak values, never summed; **flows** (apprehended, transferred, discharged) are daily counts summed over reported days.")
    if PIPELINE_ERROR:
        st.error(PIPELINE_ERROR)
    else:
        render_pipeline_tab()
```

H.3 Data loading: src/data_loader.py

```
"""Load and validate the UAC dataset for descriptive pipeline analytics.


Two inputs, one contract:
* the raw HHS export (`HHS_Unaccompanied_Alien_Children_Program.csv`): validated here and used to check the cleaned table;
* the notebook's cleaned daily table (`artifacts/daily_clean.csv`): the analytical base, because it already carries the
  data-quality masks (5 stale HHS reports removed; CBP columns masked in the copied block 12 Jan 2023 - 30 Jan 2024).


Descriptive KPIs use reported days only. Friday/Saturday rows in the daily table are interpolated for the forecasting
models and never enter a descriptive KPI.
"""
from pathlib import Path


import numpy as np
import pandas as pd


# Raw header (position, normalised text) -> short name used everywhere (same names as the notebook)
RAW_HEADERS = [("date", "date"),
               ("children apprehended and placed in cbp custody", "apprehended"),
               ("children in cbp custody", "cbp_custody"),
               ("children transferred out of cbp custody", "transferred"),
               ("children in hhs care", "hhs_care"),
               ("children discharged from hhs care", "discharged")]
SERIES = [short for _, short in RAW_HEADERS[1:]]
STOCKS = ["cbp_custody", "hhs_care"]                  # point-in-time counts: never summed over time
FLOWS = ["apprehended", "transferred", "discharged"]   # daily counts: summed over reported days
CBP_COLS = ["apprehended", "cbp_custody", "transferred"]
HHS_COLS = ["hhs_care", "discharged"]
DAILY_REQUIRED = SERIES + ["is_reported", "cbp_trusted"]
MISSING_TOKENS = {"", "n/a", "na", "nan", "null", "-"}



class DataValidationError(ValueError):
    """Raised when an input cannot be used; the message names the column, file line and value."""



def _norm(text):
    return " ".join(str(text).replace("*", " ").lower().split())
```

```
def _examples(lines, values, k=5):
    return ", ".join(f"line {ln}: {v!r}" for ln, v in list(zip(lines, values))[:k])



def load_raw_csv(path):
    """Read and validate the raw HHS export.


    Returns (frame, report). `frame` is indexed by report date (ascending) with the five series as floats.
    `report` records what was repaired: blank rows dropped, ISO dates accepted, duplicate dates (first record in file
    order kept, as in the notebook), and negative counts quarantined to missing (never zero-filled).
    Raises FileNotFoundError or DataValidationError with a readable message.
    """
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError(f"Dataset not found: {path}")
    try:
        frame = pd.read_csv(path, dtype=str, keep_default_na=False, skip_blank_lines=False)
    except pd.errors.EmptyDataError as e:
        raise DataValidationError(f"{path.name} is empty") from e
    except (UnicodeDecodeError, pd.errors.ParserError) as e:   # a workbook, archive or non-UTF-8 export renamed .csv (QA3-ING-01)
        raise DataValidationError(f"{path.name} is not a readable UTF-8 CSV file ({type(e).__name__}). It may be an Excel workbook, "
                                  "a zip archive or a file saved in another encoding: export it again as 'CSV UTF-8'") from e
    if frame.shape[1] != len(RAW_HEADERS):
        raise DataValidationError(f"Expected {len(RAW_HEADERS)} columns, found {frame.shape[1]}: {list(frame.columns)}")
    for i, (expected, _) in enumerate(RAW_HEADERS):
        if _norm(frame.columns[i]) != expected:
            raise DataValidationError(f"Column {i + 1} is '{frame.columns[i]}'; expected '{expected}'")
    frame.columns = [short for _, short in RAW_HEADERS]
    frame["file_line"] = np.arange(len(frame)) + 2                       # header is line 1


    text = frame[[short for _, short in RAW_HEADERS]].apply(lambda s: s.str.strip())
    blank = (text == "").all(axis=1)
    frame, text = frame.loc[~blank], text.loc[~blank]
    report = {"blank_rows": int(blank.sum()), "iso_dates": 0, "duplicates": [], "quarantined": []}
    if frame.empty:
        raise DataValidationError("The file has a header but no data rows")


    dates = pd.to_datetime(text["date"], format="%B %d, %Y", errors="coerce")
    iso = dates.isna() & text["date"].ne("")
    dates[iso] = pd.to_datetime(text.loc[iso, "date"], format="%Y-%m-%d", errors="coerce")
    report["iso_dates"] = int((iso & dates.notna()).sum())
    if dates.isna().any():
        bad = dates.isna()
        raise DataValidationError(f"{int(bad.sum())} unparseable date(s): " + _examples(frame.loc[bad, "file_line"], text.loc[bad, "date"]))


    out = pd.DataFrame({"date": dates, "file_line": frame["file_line"]})
    for col in SERIES:
        raw = text[col].str.replace(",", "", regex=False).str.replace(" ", "", regex=False)
        missing = raw.str.lower().isin(MISSING_TOKENS)
        num = pd.to_numeric(raw.where(~missing), errors="coerce")
        bad = num.isna() & ~missing
        if bad.any():
            raise DataValidationError(f"Column '{col}' has {int(bad.sum())} non-numeric value(s): "
                                      + _examples(frame.loc[bad, "file_line"], text.loc[bad, col]))
        neg = num < 0
```

```
        for ln, d, v in zip(frame.loc[neg, "file_line"], dates[neg], num[neg]):
            report["quarantined"].append({"file_line": int(ln), "date": d, "column": col, "value": float(v)})
        out[col] = num.mask(neg).astype(float)

    dup = out["date"].duplicated(keep="first")              # file lists the newest publication first: keep that one
    for d in out.loc[dup, "date"].unique():
        rows = out[out["date"] == d]
        conflict = bool(rows[SERIES].nunique(dropna=False).gt(1).any())
        report["duplicates"].append({"date": d, "file_lines": rows["file_line"].tolist(), "conflicting": conflict})
    out = out.loc[~dup].sort_values("date", kind="stable").set_index("date").drop(columns="file_line")
    return out, report



def validate_daily(daily):
    """Check the cleaned daily table (DatetimeIndex + required columns); raise DataValidationError if unusable."""
    if not isinstance(daily.index, pd.DatetimeIndex):
        raise DataValidationError("The daily table must be indexed by date")
    missing = [c for c in DAILY_REQUIRED if c not in daily.columns]
    if missing:
        raise DataValidationError(f"The daily table lacks column(s): {', '.join(missing)}")
    if daily.index.has_duplicates:
        raise DataValidationError(f"The daily table has {int(daily.index.duplicated().sum())} duplicate date(s)")
    for col in SERIES:                                      # text in a count column would otherwise fail later with a bare ValueError
        bad = pd.to_numeric(daily[col], errors="coerce").isna() & daily[col].notna()
        if bad.any():
            raise DataValidationError(f"The daily table has non-numeric value(s) in '{col}', e.g. {daily.loc[bad, col].iloc[0]!r}")
    return daily



def load_daily_clean(path):
    """Read `artifacts/daily_clean.csv` (written by the notebook) and validate it."""
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError(f"Cleaned daily table not found: {path}. Run the notebook to regenerate artifacts/.")
    daily = pd.read_csv(path, parse_dates=["date"]).set_index("date").sort_index()
    return validate_daily(daily)



def reported_days(daily):
    """Rows with a genuine (non-stale) HHS report, the five series and the CBP trust flag; CBP values masked where untrusted."""
    validate_daily(daily)
    rep = daily.loc[daily["is_reported"].astype(bool), SERIES + ["cbp_trusted"]].copy()
    rep["cbp_trusted"] = rep["cbp_trusted"].astype(bool)
    rep.loc[~rep["cbp_trusted"], CBP_COLS] = np.nan       # defensive: the notebook already masks the copied block
    rep[SERIES] = rep[SERIES].astype(float)
    return rep.sort_index()



def reconcile(raw, rep):
    """Compare the cleaned reported days with the raw export.

    Returns a frame of cell mismatches (date, column, raw, clean). HHS columns must equal the raw values on every
    reported day; CBP columns must equal them where trusted and be missing where masked.
```

```
    """
    rows = []
    for d in rep.index:
        if d not in raw.index:
            rows.append({"date": d, "column": "(row)", "raw": np.nan, "clean": np.nan})
            continue
        for col in SERIES:
            r, c = raw.at[d, col], rep.at[d, col]
            expected_missing = col in CBP_COLS and not rep.at[d, "cbp_trusted"]
            same = np.isnan(c) if expected_missing else (np.isnan(r) and np.isnan(c)) or (not np.isnan(r) and not np.isnan(c) and np.isclose(r, c))
            if not same:
                rows.append({"date": d, "column": col, "raw": r, "clean": c})
    return pd.DataFrame(rows, columns=["date", "column", "raw", "clean"])
```

H.4 KPI calculations: src/kpi_calculations.py

```
"""Stock/flow-correct KPIs for a period of reported days, plus daily/weekly/monthly aggregation.


Rules that keep the numbers honest:
* Stocks (children in CBP custody, children in HHS care) are point-in-time counts. A period is described by its first,
  last, mean and peak report, and its change is last - first. Stocks are never summed over time.
* Flows (apprehended, transferred, discharged) are daily counts. Totals sum reported days only (Friday/Saturday are not
  published, so a "total" is a reported-day total, not a calendar total); per-day values are means over reported days.
* Ratios are ratios of sums over the days where both inputs are observed (robust to single zero days), and are NaN,
  never +/-inf, when the denominator is zero or missing.
"""
import numpy as np
import pandas as pd


from .data_loader import FLOWS, STOCKS


# One definition table, used by the dashboard tooltips, the README and the research paper's Appendix volume (Appendix A)
KPI_DEFINITIONS = {
    "hhs_care_end": ("HHS census at period end", "Children in HHS care on the last reported day of the period (stock)"),
    "hhs_care_change": ("HHS census change", "Last minus first reported HHS census in the period (stock change, not a sum)"),
    "hhs_care_peak": ("Peak HHS census", "Highest reported HHS census in the period, with its date"),
    "cbp_custody_end": ("CBP custody at period end", "Children in CBP custody on the last trusted report of the period (stock)"),
    "apprehended_per_day": ("Apprehensions per reported day", "Mean of daily CBP apprehensions over reported days"),
    "transferred_per_day": ("Transfers into HHS per reported day", "Mean of daily transfers out of CBP into HHS over reported days"),
    "discharged_per_day": ("Discharges per reported day", "Mean of daily HHS discharges (mostly to sponsors) over reported days"),
    "net_intake_per_day": ("Recorded net intake per reported day", "Mean of (transfers - discharges) on days with both reported. "
                           "Recorded flows do not close the census balance (about +3 children/day unexplained), so use the census change for backlog"),
    "transfer_rate_pct": ("CBP transfer rate (%/day)", "100 x sum(transfers) / sum(children in CBP custody): share of the CBP population moved to HHS per day"),
    "discharge_rate_pct": ("HHS discharge rate (%/day)", "100 x sum(discharges) / sum(children in HHS care): share of the HHS population discharged per day"),
    "throughput_ratio": ("Recorded throughput (exits per entry)", "sum(discharges) / sum(transfers): above 1 = recorded exits exceed entries. "
                         "Not used for alerts: it misses unreported Fri/Sat flows and non-CBP admissions"),
    "implied_stay_days": ("Implied average stay in HHS care (days)", "Little's law, ratio of period means: sum(children in HHS care) / sum(discharges) = 100 / discharge rate"),
    "implied_cbp_hold_days": ("Implied CBP holding time (days)", "Little's law: sum(children in CBP custody) / sum(transfers). An approximation of the average time in "
                              "CBP custody before transfer (an upper bound if children also leave CBP by other routes); compare with the 72-hour transfer standard"),
    "pass_through_ratio": ("CBP pass-through (transfers per apprehension)", "sum(transfers) / sum(apprehensions) on days with both reported"),
}



def safe_ratio(num, den):
    """num / den as a float, or NaN when either is missing or the denominator is zero (never +/-inf)."""
    num, den = float(num), float(den)
```

```
    return num / den if np.isfinite(num) and np.isfinite(den) and den != 0 else np.nan



def filter_period(frame, start, end):
    """Rows with start <= date <= end (both inclusive, whole days). Raises ValueError for invalid or reversed dates."""
    start, end = pd.Timestamp(start), pd.Timestamp(end)
    if pd.isna(start) or pd.isna(end):
        raise ValueError("Start and end dates are required")
    start, end = start.normalize(), end.normalize()
    if start > end:
        raise ValueError(f"The start date ({start:%d %b %Y}) is after the end date ({end:%d %b %Y})")
    days = frame.index.normalize()
    return frame.loc[(days >= start) & (days <= end)]



def _paired_sums(frame, num, den):
    both = frame[num].notna() & frame[den].notna()
    return frame.loc[both, num].sum(), frame.loc[both, den].sum(), int(both.sum())



def period_kpis(rep):
    """KPIs for one period of reported days (see KPI_DEFINITIONS). Every key is present; values are NaN when undefined."""
    out = {"reported_days": len(rep),
           "start": rep.index.min() if len(rep) else pd.NaT,
           "end": rep.index.max() if len(rep) else pd.NaT}
    for stock in STOCKS:
        s = rep[stock].dropna() if stock in rep else pd.Series(dtype=float)
        first, last = (s.iloc[0], s.iloc[-1]) if len(s) else (np.nan, np.nan)
        out.update({f"{stock}_start": first, f"{stock}_end": last, f"{stock}_change": last - first,
                    f"{stock}_change_pct": 100 * safe_ratio(last - first, first),
                    f"{stock}_mean": s.mean() if len(s) else np.nan,
                    f"{stock}_peak": s.max() if len(s) else np.nan,
                    f"{stock}_peak_date": s.idxmax() if len(s) else pd.NaT,
                    f"{stock}_days": len(s)})
    for flow in FLOWS:
        f = rep[flow].dropna() if flow in rep else pd.Series(dtype=float)
        out.update({f"{flow}_total": f.sum() if len(f) else np.nan,
                    f"{flow}_per_day": f.mean() if len(f) else np.nan,
                    f"{flow}_days": len(f)})

    both = rep["transferred"].notna() & rep["discharged"].notna()
    out["net_intake_per_day"] = (rep.loc[both, "transferred"] - rep.loc[both, "discharged"]).mean() if both.any() else np.nan
    tr, cbp, _ = _paired_sums(rep, "transferred", "cbp_custody")
    dis, care, _ = _paired_sums(rep, "discharged", "hhs_care")
    dis2, tr2, _ = _paired_sums(rep, "discharged", "transferred")
    tr3, app, _ = _paired_sums(rep, "transferred", "apprehended")
    out["transfer_rate_pct"] = 100 * safe_ratio(tr, cbp)
    out["discharge_rate_pct"] = 100 * safe_ratio(dis, care)
    out["throughput_ratio"] = safe_ratio(dis2, tr2)
    out["implied_stay_days"] = safe_ratio(care, dis)
    out["implied_cbp_hold_days"] = safe_ratio(cbp, tr)
    out["pass_through_ratio"] = safe_ratio(tr3, app)
    return out
```

```
def add_daily_metrics(rep):
    """Per-reported-day derived columns: net intake, rates, census change since the previous report and the gap in days."""
    out = rep.copy()
    out["net_intake"] = out["transferred"] - out["discharged"]
    out["discharge_rate_pct"] = 100 * out["discharged"] / out["hhs_care"].where(out["hhs_care"] > 0)
    out["transfer_rate_pct"] = 100 * out["transferred"] / out["cbp_custody"].where(out["cbp_custody"] > 0)
    out["hhs_care_change"] = out["hhs_care"].diff()
    out["days_since_prev_report"] = out.index.to_series().diff().dt.days.to_numpy()
    return out



AGG_COLUMNS = {  # period_kpis key -> table column
    "reported_days": "Reported days",
    "hhs_care_start": "HHS census (first)", "hhs_care_end": "HHS census (last)", "hhs_care_mean": "HHS census (mean)",
    "hhs_care_change": "HHS census change",
    "cbp_custody_end": "CBP custody (last)", "cbp_custody_mean": "CBP custody (mean)",
    "apprehended_total": "Apprehended (reported-day total)", "transferred_total": "Transferred (reported-day total)",
    "discharged_total": "Discharged (reported-day total)",
    "transferred_per_day": "Transfers / reported day", "discharged_per_day": "Discharges / reported day",
    "discharge_rate_pct": "Discharge rate %/day", "implied_stay_days": "Implied stay (days)",
    "implied_cbp_hold_days": "CBP holding time (days)", "throughput_ratio": "Recorded throughput",
}



def aggregate(rep, freq):
    """Summarise reported days by 'daily', 'weekly' (Sunday-Saturday, labelled by the Sunday) or 'monthly'.


    Vectorised, with exactly the period_kpis definitions (tests/test_kpi_calculations.py checks the two agree):
    stocks use their first/last/mean report, flows their reported-day sums and means, ratios the sums over days where
    both inputs are observed. A period with fewer reported days than usual simply reports fewer days.
    """
    if freq not in ("daily", "weekly", "monthly"):
        raise ValueError(f"Unknown frequency '{freq}'; use daily, weekly or monthly")
    if rep.empty:
        return pd.DataFrame(columns=list(AGG_COLUMNS.values()), index=pd.DatetimeIndex([], name="Period"))
    key = rep.index if freq == "daily" else rep.index.to_period("W-SAT" if freq == "weekly" else "M").start_time
    g = rep.groupby(key, sort=True)
    paired = {}
    for num_col, den_col in (("discharged", "hhs_care"), ("cbp_custody", "transferred"), ("discharged", "transferred")):
        both = rep[num_col].notna() & rep[den_col].notna()
        sums = rep[[num_col, den_col]].where(both, axis=0).groupby(key, sort=True).sum(min_count=1)
        paired[(num_col, den_col)] = (sums[num_col], sums[den_col])


    def ratio(num, den):
        return num / den.where(den != 0)


    care_first, care_last = g["hhs_care"].first(), g["hhs_care"].last()
    out = pd.DataFrame({
        "reported_days": g.size(),
        "hhs_care_start": care_first, "hhs_care_end": care_last, "hhs_care_mean": g["hhs_care"].mean(),
        "hhs_care_change": care_last - care_first,
        "cbp_custody_end": g["cbp_custody"].last(), "cbp_custody_mean": g["cbp_custody"].mean(),
        "apprehended_total": g["apprehended"].sum(min_count=1), "transferred_total": g["transferred"].sum(min_count=1),
```

```
        "discharged_total": g["discharged"].sum(min_count=1),
        "transferred_per_day": g["transferred"].mean(), "discharged_per_day": g["discharged"].mean(),
        "discharge_rate_pct": 100 * ratio(*paired[("discharged", "hhs_care")]),
        "implied_stay_days": ratio(*reversed(paired[("discharged", "hhs_care")])),
        "implied_cbp_hold_days": ratio(*paired[("cbp_custody", "transferred")]),
        "throughput_ratio": ratio(*paired[("discharged", "transferred")]),
    })
    out.index = pd.DatetimeIndex(out.index, name="Period")
    return out[list(AGG_COLUMNS)].rename(columns=AGG_COLUMNS)
```

H.5 Bottleneck detection: src/bottleneck_detection.py

```
"""Rule-based backlog and bottleneck detection on reported days, with documented, adjustable thresholds.


Three indicators are evaluated on every reported day using trailing windows only (no look-ahead), so a historical
alert is exactly what an analyst would have seen on that day:


1. census_growth       Backlog building: % change in children in HHS care versus the last report at least 28 days earlier.
                       The census (a stock) is used because recorded flows do not close the population balance.
2. discharge_slowdown  Placement bottleneck: the 7-day discharge rate (sum discharges / sum census) falls below its
                       median over the previous 8 weeks.
3. cbp_holding         Transfer bottleneck: implied CBP holding time = sum(CBP custody) / sum(transfers) over 7 days.
                       The 3-day alert level mirrors the 72-hour transfer standard for unaccompanied children
                       (TVPRA 2008, 8 U.S.C. 1232(b)(3)); the stock/flow ratio approximates an average, it does not
                       measure any individual child's custody time.


Levels: 0 = Normal, 1 = Watch, 2 = Alert. A day's overall level is the worst indicator level ("worst wins").
Indicators with too few reports in their window are NaN (insufficient data) and are ignored for the overall level.


Recorded throughput (discharges / transfers) was tested as a backlog signal and rejected: over the trusted period
(31 Jan 2024 - 25 Jan 2025) it averaged 0.93, i.e. recorded entries exceeded exits, yet the census fell by about 3,500.
Its month-to-month direction often agrees with the census, but its level is biased because Friday/Saturday flows are
unpublished and recorded flows leave about 3 children a day unexplained (research paper, Section 2.3; Appendix B.6 has the detail).
"""
from dataclasses import asdict, dataclass


import numpy as np
import pandas as pd


LEVELS = {0: "Normal", 1: "Watch", 2: "Alert"}
INDICATORS = {
    "census_growth": {"label": "HHS census growth (backlog building)", "value": "census_growth_pct", "unit": "% vs 28 days earlier"},
    "discharge_slowdown": {"label": "Discharge slowdown (placement bottleneck)", "value": "discharge_drop_pct", "unit": "% below 8-week median rate"},
    "cbp_holding": {"label": "CBP holding time (transfer bottleneck)", "value": "cbp_hold_days", "unit": "days (custody / transfers, 7-day)"},
}



@dataclass(frozen=True)
class Thresholds:
    """Alert rules. Defaults are operational choices documented in the README and research paper, not fitted to the data."""
    census_growth_watch_pct: float = 5.0      # census up 5% in four weeks
    census_growth_alert_pct: float = 10.0     # census up 10% in four weeks
    growth_window_days: int = 28
    discharge_drop_watch_pct: float = 15.0    # 7-day discharge rate 15% below its recent norm
```

```
    discharge_drop_alert_pct: float = 30.0
    rate_window_days: int = 7
    baseline_days: int = 56                  # 8-week trailing baseline, ending where the 7-day window starts
    cbp_hold_watch_days: float = 2.0         # 48 hours
    cbp_hold_alert_days: float = 3.0         # 72-hour transfer standard
    min_reports: int = 3                     # minimum reports in a 7-day window
    min_baseline_reports: int = 15           # minimum 7-day rates in the 8-week baseline


    def validate(self):
        pairs = [("census growth", self.census_growth_watch_pct, self.census_growth_alert_pct),
                 ("discharge slowdown", self.discharge_drop_watch_pct, self.discharge_drop_alert_pct),
                 ("CBP holding time", self.cbp_hold_watch_days, self.cbp_hold_alert_days)]
        for name, watch, alert in pairs:
            if not (np.isfinite(watch) and np.isfinite(alert)) or watch > alert:
                raise ValueError(f"{name}: the Watch threshold ({watch}) must be finite and not above the Alert threshold ({alert})")
        if self.cbp_hold_watch_days <= 0 or self.discharge_drop_watch_pct <= 0:
            raise ValueError("CBP holding and discharge-slowdown thresholds must be positive")
        for name in ("growth_window_days", "rate_window_days", "baseline_days", "min_reports", "min_baseline_reports"):
            if int(getattr(self, name)) < 1:
                raise ValueError(f"{name} must be at least 1")
        return self


    def as_tuple(self):
        return tuple(asdict(self).values())



def _window_sums(rep, num, den, days, min_reports):
    """Trailing time-window sums of num and den over reported days where both are observed; count of such days."""
    both = rep[num].notna() & rep[den].notna()
    roll = lambda s: s.rolling(f"{days}D", min_periods=1).sum()
    n, d, k = roll(rep[num].where(both, 0.0)), roll(rep[den].where(both, 0.0)), roll(both.astype(float))
    return n.where(k >= min_reports), d.where(k >= min_reports)



def compute_indicators(rep, th=Thresholds()):
    """Indicator values per reported day (rep must be sorted reported days with the five series)."""
    th.validate()
    out = pd.DataFrame(index=rep.index)
    if rep.empty:
        for c in ("census_growth_pct", "discharge_rate_7d_pct", "discharge_baseline_pct", "discharge_drop_pct", "cbp_hold_days"):
            out[c] = pd.Series(dtype=float)
        return out


    # 1. census growth: last report at or before t - window, accepted only if within 7 days of that date (not stale)
    care = rep["hhs_care"].dropna()
    left = pd.DataFrame({"date": rep.index, "lag_date": rep.index - pd.Timedelta(days=th.growth_window_days)})
    right = pd.DataFrame({"lag_date": care.index, "lag_care": care.to_numpy()})
    lagged = pd.merge_asof(left, right, on="lag_date", direction="backward", tolerance=pd.Timedelta(days=7))
    lag_care = pd.Series(lagged["lag_care"].to_numpy(), index=rep.index)
    out["census_growth_pct"] = 100 * (rep["hhs_care"] / lag_care.where(lag_care > 0) - 1)


    # 2. discharge slowdown: 7-day rate vs the median 7-day rate over the 8 weeks before the current window
    dis, care_sum = _window_sums(rep, "discharged", "hhs_care", th.rate_window_days, th.min_reports)
    rate = 100 * dis / care_sum.where(care_sum > 0)
```

```
    out["discharge_rate_7d_pct"] = rate
    shifted = pd.Series(rate.to_numpy(), index=rate.index + pd.Timedelta(days=th.rate_window_days)).dropna()
    base = shifted.rolling(f"{th.baseline_days}D", min_periods=th.min_baseline_reports).median()
    base_at = pd.merge_asof(pd.DataFrame({"date": rep.index}), pd.DataFrame({"date": base.index, "b": base.to_numpy()}),
                            on="date", direction="backward")["b"].to_numpy()
    out["discharge_baseline_pct"] = base_at
    out["discharge_drop_pct"] = 100 * (1 - rate / out["discharge_baseline_pct"].where(out["discharge_baseline_pct"] > 0))


    # 3. CBP holding time: zero transfers while children are held is an unbounded holding time (inf), i.e. an alert
    cbp, tr = _window_sums(rep, "cbp_custody", "transferred", th.rate_window_days, th.min_reports)
    hold = cbp / tr.where(tr > 0)
    out["cbp_hold_days"] = hold.mask((tr == 0) & (cbp > 0), np.inf)
    return out



def _level(values, watch, alert):
    lv = np.where(values >= alert, 2.0, np.where(values >= watch, 1.0, 0.0))
    return pd.Series(np.where(np.isnan(values.to_numpy(dtype=float)), np.nan, lv), index=values.index)



def detect_bottlenecks(rep, th=Thresholds()):
    """Indicator values, per-indicator levels (0/1/2, NaN = insufficient data) and the overall level/status per reported day."""
    ind = compute_indicators(rep, th)
    ind["census_growth_level"] = _level(ind["census_growth_pct"], th.census_growth_watch_pct, th.census_growth_alert_pct)
    ind["discharge_slowdown_level"] = _level(ind["discharge_drop_pct"], th.discharge_drop_watch_pct, th.discharge_drop_alert_pct)
    ind["cbp_holding_level"] = _level(ind["cbp_hold_days"], th.cbp_hold_watch_days, th.cbp_hold_alert_days)
    levels = ind[[f"{k}_level" for k in INDICATORS]]
    ind["indicators_available"] = levels.notna().sum(axis=1)
    ind["overall_level"] = levels.max(axis=1, skipna=True)          # NaN only when no indicator is available
    ind["overall_status"] = ind["overall_level"].map(LEVELS).fillna("Insufficient data")
    return ind



def alert_episodes(flags, min_level=1, max_gap_days=7):
    """Contiguous runs of reported days at or above min_level, per indicator.


    Consecutive reports belong to one episode unless they are more than max_gap_days apart (so the regular
    Thursday -> Sunday gap never splits an episode). Returns one row per episode, longest first within an indicator.
    """
    cols = ["Indicator", "Start", "End", "Reported days", "Calendar days", "Highest level", "Peak value", "Unit"]
    rows = []
    for key, spec in INDICATORS.items():
        lvl = flags.get(f"{key}_level")
        if lvl is None or lvl.empty:
            continue
        on = (lvl >= min_level).to_numpy()
        dates = flags.index
        gap = np.r_[np.inf, np.diff(dates.values).astype("timedelta64[D]").astype(float)]
        start_new = on & (~np.r_[False, on[:-1]] | (gap > max_gap_days))
        episode_id = np.cumsum(start_new) * on
        for eid in np.unique(episode_id[episode_id > 0]):
            idx = np.flatnonzero(episode_id == eid)
            vals = flags[spec["value"]].iloc[idx]
            peak = np.inf if np.isinf(vals).any() else vals.max()
```

```
            rows.append({"Indicator": spec["label"], "Start": dates[idx[0]], "End": dates[idx[-1]], "Reported days": len(idx),
                         "Calendar days": int((dates[idx[-1]] - dates[idx[0]]).days) + 1,
                         "Highest level": LEVELS[int(lvl.iloc[idx].max())], "Peak value": peak, "Unit": spec["unit"]})
    return pd.DataFrame(rows, columns=cols)
```

H.6 Chart helpers: src/visualizations.py

```
"""Plotly figures for the dashboard's "Pipeline analytics" tab.

Each builder takes the full reported-day history plus the selected period, so rolling windows at the start of a
period still use the preceding reports, and returns a figure containing only dates inside the period.
Colours follow the dashboard's validated palette: one fixed colour per flow (categorical slots 1-3 in pipeline order),
grey for observed stocks, and the reserved status colours (always paired with an icon and a label) for alert levels.
Different units never share an axis: they get separate panels.
"""
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

from .bottleneck_detection import LEVELS
from .kpi_calculations import filter_period

OBSERVED = "#898781"
FLOW_COLORS = {"apprehended": "#2a78d6", "transferred": "#eb6834", "discharged": "#1baf7a"}
FLOW_LABELS = {"apprehended": "Apprehended (CBP intake)", "transferred": "Transferred into HHS", "discharged": "Discharged from HHS"}
LEVEL_STYLE = {0: ("#0ca30c", "☑"), 1: ("#fab219", "⚠"), 2: ("#d03b3b", "🔘")}
INSUFFICIENT = "#c3c2b7"



def _layout(fig, height, legend=True):
    fig.update_layout(height=height, margin=dict(l=8, r=8, t=36, b=8), hovermode="x unified", showlegend=legend,
                      legend=dict(orientation="h", yanchor="top", y=-0.12, x=0, title=None))
    fig.update_xaxes(showgrid=False)
    fig.update_yaxes(gridwidth=1, zeroline=False)
    return fig



def _threshold(fig, y, level, text, row=None):
    color, icon = LEVEL_STYLE[level]
    kw = dict(row=row, col=1) if row else {}
    fig.add_hline(y=y, line=dict(color=color, width=1, dash="dash"), annotation_text=f"{icon} {text}",
                  annotation_position="top left", annotation_font=dict(size=11, color="#52514e"), **kw)



def population_figure(rep, start, end):
    """Children in HHS care and in CBP custody (stocks) on reported days: two panels, because the scales differ ~100x."""
    p = filter_period(rep, start, end)
    fig = make_subplots(rows=2, cols=1, shared_xaxes=True, vertical_spacing=0.12,
                        subplot_titles=("Children in HHS care (reported days)", "Children in CBP custody (reported days)"))
    marker = len(p) <= 120
    for row, col in ((1, "hhs_care"), (2, "cbp_custody")):
        fig.add_trace(go.Scatter(x=p.index, y=p[col], mode="lines+markers" if marker else "lines", name=fig.layout.annotations[row - 1].text,
```

```
                                line=dict(color=OBSERVED, width=2), marker=dict(size=8, color=OBSERVED, line=dict(color="white", width=2)),
                                connectgaps=False, hovertemplate="%{y:,.0f}<extra></extra>"), row=row, col=1)
        fig.update_yaxes(tickformat=",.0f", title="children", row=row, col=1)
    if len(p) and p["cbp_custody"].isna().all():
        fig.add_annotation(text="CBP values masked in this period (copied records, 12 Jan 2023 - 30 Jan 2024)", xref="x2 domain", yref="y2 domain",
                           x=0.5, y=0.5, showarrow=False, font=dict(color="#52514e"))
    return _layout(fig, 520, legend=False)



def flows_figure(rep, start, end, window_days=7):
    """Daily flows as trailing 7-day means per reported day (computed on the full history, then cut to the period)."""
    fig = go.Figure()
    for col in ("apprehended", "transferred", "discharged"):
        s = rep[col].rolling(f"{window_days}D", min_periods=3).mean()
        s = filter_period(s.to_frame(), start, end)[col]
        fig.add_trace(go.Scatter(x=s.index, y=s, name=FLOW_LABELS[col], line=dict(color=FLOW_COLORS[col], width=2),
                                 connectgaps=False, hovertemplate="%{y:,.1f}/day<extra>" + FLOW_LABELS[col] + "</extra>"))
    fig.update_yaxes(tickformat=",.0f", title="children per reported day")
    fig.update_layout(title=dict(text=f"Flows: {window_days}-day mean per reported day", font=dict(size=14)))
    return _layout(fig, 380)



def rates_figure(flags, th, start, end):
    """Discharge rate vs its 8-week baseline, and implied CBP holding time vs the Watch/Alert thresholds (two panels)."""
    p = filter_period(flags, start, end)
    fig = make_subplots(rows=2, cols=1, shared_xaxes=True, vertical_spacing=0.14,
                        subplot_titles=("HHS discharge rate, 7-day (% of census discharged per day)",
                                        "Implied CBP holding time, 7-day (custody ÷ transfers, days)"))
    fig.add_trace(go.Scatter(x=p.index, y=p["discharge_rate_7d_pct"], name="Discharge rate (7-day)", line=dict(color=FLOW_COLORS["discharged"], width=2),
                             hovertemplate="%{y:.2f}%/day<extra>7-day rate</extra>"), row=1, col=1)
    fig.add_trace(go.Scatter(x=p.index, y=p["discharge_baseline_pct"], name="Baseline (median of previous 8 weeks)",
                             line=dict(color=OBSERVED, width=2, dash="dot"), hovertemplate="%{y:.2f}%/day<extra>baseline</extra>"), row=1, col=1)
    hold = p["cbp_hold_days"].replace(np.inf, np.nan)
    fig.add_trace(go.Scatter(x=p.index, y=hold, name="CBP holding time (7-day)", line=dict(color=FLOW_COLORS["transferred"], width=2),
                             connectgaps=False, hovertemplate="%{y:.1f} days<extra>holding time</extra>"), row=2, col=1)
    _threshold(fig, th.cbp_hold_watch_days, 1, f"Watch ≥ {th.cbp_hold_watch_days:g} days", row=2)
    _threshold(fig, th.cbp_hold_alert_days, 2, f"Alert ≥ {th.cbp_hold_alert_days:g} days (72-hour standard)", row=2)
    fig.update_yaxes(tickformat=".2f", ticksuffix="%", row=1, col=1)
    fig.update_yaxes(tickformat=".1f", title="days", row=2, col=1)
    if np.isinf(p["cbp_hold_days"]).any():
        fig.add_annotation(text=f"{int(np.isinf(p['cbp_hold_days']).sum())} day(s) with no transfers in the window (unbounded holding time) are not drawn",
                           xref="x2 domain", yref="y2 domain", x=0, y=1, showarrow=False, xanchor="left", font=dict(size=11, color="#52514e"))
    return _layout(fig, 560)



def backlog_figure(flags, th, start, end):
    """Census growth vs the Watch/Alert thresholds, with a strip of the daily overall alert status underneath."""
    p = filter_period(flags, start, end)
    fig = make_subplots(rows=2, cols=1, shared_xaxes=True, vertical_spacing=0.1, row_heights=[0.78, 0.22],
                        subplot_titles=(f"HHS census growth: % change vs {th.growth_window_days} days earlier", "Overall alert status (worst indicator)"))
    fig.add_trace(go.Scatter(x=p.index, y=p["census_growth_pct"], name="Census growth", line=dict(color=OBSERVED, width=2),
                             connectgaps=False, hovertemplate="%{y:+.1f}%<extra>census growth</extra>"), row=1, col=1)
    _threshold(fig, th.census_growth_watch_pct, 1, f"Watch ≥ +{th.census_growth_watch_pct:g}%", row=1)
    _threshold(fig, th.census_growth_alert_pct, 2, f"Alert ≥ +{th.census_growth_alert_pct:g}%", row=1)
    fig.update_yaxes(tickformat=",.0f", ticksuffix="%", row=1, col=1)
```

```
    for level, name in list(LEVELS.items()) + [(None, "Insufficient data")]:
        mask = p["overall_level"].isna() if level is None else p["overall_level"] == level
        if not mask.any():
            continue
        color, icon = (INSUFFICIENT, "❓") if level is None else LEVEL_STYLE[level]
        fig.add_trace(go.Bar(x=p.index[mask], y=np.ones(int(mask.sum())), name=f"{icon} {name}", marker=dict(color=color, line=dict(width=0)),
                             hovertemplate=f"{icon} {name}<extra></extra>"), row=2, col=1)
    fig.update_yaxes(visible=False, row=2, col=1)
    fig.update_layout(barmode="overlay", bargap=0)
    return _layout(fig, 480)
```

H.7 Pipeline analysis script: tools/pipeline_analysis.py

```
"""Reproduce every number in the research paper's Section 8 (pipeline analytics) from the same src/ functions the dashboard uses.


    python tools/pipeline_analysis.py


Reads artifacts/daily_clean.csv and artifacts/metadata.json. Writes:
    reports/pipeline_kpis.csv              KPIs per analysis period (reported days only)
    reports/pipeline_alert_episodes.csv    every Watch/Alert episode over the full history (default thresholds)
    reports/pipeline_indicators_by_period.csv  per period and indicator: % of reports at Watch/Alert, median and maximum value
    reports/pipeline_status_by_year.csv    share of reported days by overall status and year
    reports/pipeline_los_reconciliation.csv  implied length of stay under the notebook and dashboard definitions
    reports/pipeline_analysis.md           the tables above as Markdown (the paper quotes these values)
    figures/19_pipeline_monitoring.png     notebook-size chart for the Appendix volume (tools/paper_figures.py draws the print-size version)
"""
import json
import sys
from pathlib import Path


import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.bottleneck_detection import INDICATORS, LEVELS, Thresholds, alert_episodes, detect_bottlenecks   # noqa: E402
from src.data_loader import load_daily_clean, reported_days                                             # noqa: E402
from src.kpi_calculations import KPI_DEFINITIONS, filter_period, period_kpis                            # noqa: E402


INK2, MUTED, GRID, BASE, SURFACE = "#52514e", "#898781", "#e1e0d9", "#c3c2b7", "#fcfcfb"   # notebook figure style
COLORS = {"hhs_care": "#2a78d6", "transferred": "#eb6834", "discharged": "#1baf7a"}
LEVEL_COLORS = {1: "#fab219", 2: "#d03b3b"}



def periods(meta, rep):
    last = rep.index.max()
    return {
        "2023 (CBP masked)": (rep.index.min(), pd.Timestamp(meta["cbp_bad_end"])),
        "Before the break": (pd.Timestamp(meta["cbp_bad_end"]) + pd.Timedelta(days=1), pd.Timestamp(meta["break_transfers"]) - pd.Timedelta(days=1)),
        "Current regime": (pd.Timestamp(meta["regime_start"]), last),
        "Last 90 days": (last - pd.Timedelta(days=89), last),
```

```
    }


def kpi_table(rep, per):
    keys = ["reported_days", "hhs_care_start", "hhs_care_end", "hhs_care_change", "hhs_care_change_pct", "hhs_care_mean", "hhs_care_peak",
            "hhs_care_peak_date", "cbp_custody_mean", "cbp_custody_end", "apprehended_per_day", "transferred_per_day", "discharged_per_day",
            "net_intake_per_day", "discharge_rate_pct", "implied_stay_days", "implied_cbp_hold_days", "transfer_rate_pct", "throughput_ratio",
            "pass_through_ratio"]
    cols = {}
    for name, (a, b) in per.items():
        k = period_kpis(filter_period(rep, a, b))
        cols[f"{name} ({a:%d %b %Y} - {b:%d %b %Y})"] = [k[x] for x in keys]
    return pd.DataFrame(cols, index=keys)



def los_reconciliation(daily, rep, meta):
    """Implied stay: notebook (28-day smoothed calendar grid, paper Section 3.3) vs dashboard (ratio of reported-day means)."""
    los = daily["hhs_care"].rolling(28).mean() / daily["discharged"].rolling(28).mean()
    brk_dis, regime = pd.Timestamp(meta["break_discharges"]), pd.Timestamp(meta["regime_start"])
    nb_pre = los.loc[:brk_dis].tail(180)
    windows = {"Paper window before the break (last 180 days to the discharge break)": (nb_pre.index.min(), brk_dis),
               "Current regime": (regime, rep.index.max())}
    rows = []
    for name, (a, b) in windows.items():
        k = period_kpis(filter_period(rep, a, b))
        rows.append({"Window": f"{name}: {a:%d %b %Y} - {b:%d %b %Y}", "Notebook definition (days)": los.loc[a:b].mean(),
                     "Dashboard definition (days)": k["implied_stay_days"]})
    out = pd.DataFrame(rows)
    out["Difference %"] = 100 * (out["Dashboard definition (days)"] / out["Notebook definition (days)"] - 1)
    return out



def status_by_year(flags):
    s = flags["overall_status"].groupby(flags.index.year).value_counts(normalize=True).unstack(fill_value=0) * 100
    cols = [c for c in ["Normal", "Watch", "Alert", "Insufficient data"] if c in s]
    per_ind = {}
    for k in INDICATORS:                                    # share among reports where the indicator has a complete window
        lv = flags[f"{k}_level"].dropna()
        per_ind[INDICATORS[k]["label"] + " at Watch or Alert"] = (lv >= 1).groupby(lv.index.year).mean() * 100
    return s[cols].join(pd.DataFrame(per_ind)).round(1)



def indicator_table(flags, per):
    """Per period and indicator: reports with a complete window, % at Watch or above, % at Alert, median and maximum value."""
    rows = []
    for name, (a, b) in per.items():
        f = filter_period(flags, a, b)
        for k, spec in INDICATORS.items():
            lv, val = f[f"{k}_level"].dropna(), f[spec["value"]].replace(np.inf, np.nan).dropna()
            rows.append({"Period": name, "Indicator": spec["label"], "Reports evaluated": len(lv),
                         "% Watch or Alert": 100 * (lv >= 1).mean() if len(lv) else np.nan, "% Alert": 100 * (lv >= 2).mean() if len(lv) else np.nan,
                         "Median value": val.median() if len(val) else np.nan, "Max value": val.max() if len(val) else np.nan, "Unit": spec["unit"]})
    return pd.DataFrame(rows).set_index("Period")
```

```
def figure(rep, flags, th, meta, path):
    plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "axes.edgecolor": BASE, "axes.labelcolor": INK2,
                         "xtick.color": INK2, "ytick.color": INK2, "axes.grid": True, "grid.color": GRID, "axes.spines.top": False,
                         "axes.spines.right": False, "font.size": 10})
    start = pd.Timestamp(meta["cbp_bad_end"]) + pd.Timedelta(days=1)
    r, f = rep.loc[start:], flags.loc[start:]
    fig, axes = plt.subplots(3, 1, figsize=(12, 9), sharex=True, constrained_layout=True)
    ax = axes[0]
    ax.plot(r.index, r["hhs_care"], color=COLORS["hhs_care"], lw=1.8)
    for lv in (1, 2):                                       # shade days whose census-growth indicator is at Watch / Alert
        on = f["census_growth_level"] == lv
        ax.fill_between(f.index, 0, 1, where=on, transform=ax.get_xaxis_transform(), color=LEVEL_COLORS[lv], alpha=0.18, lw=0,
                        label=f"census growth {LEVELS[lv]} (≥ +{th.census_growth_watch_pct if lv == 1 else th.census_growth_alert_pct:g}% in 28 days)")
    ax.set_title("(a) Children in HHS care, reported days", loc="left"); ax.set_ylabel("children"); ax.legend(loc="upper right", frameon=False)
    ax = axes[1]
    ax.plot(f.index, f["discharge_rate_7d_pct"], color=COLORS["discharged"], lw=1.8, label="7-day discharge rate")
    ax.plot(f.index, f["discharge_baseline_pct"], color=MUTED, lw=1.4, ls=":", label="baseline (median of previous 8 weeks)")
    for lv in (1, 2):
        on = f["discharge_slowdown_level"] == lv
        ax.fill_between(f.index, 0, 1, where=on, transform=ax.get_xaxis_transform(), color=LEVEL_COLORS[lv], alpha=0.18, lw=0,
                        label=f"discharge slowdown {LEVELS[lv]}")
    ax.set_title("(b) HHS discharge rate: % of the census discharged per day", loc="left"); ax.set_ylabel("% per day")
    ax.legend(loc="lower left", frameon=False, ncol=2)
    ax = axes[2]
    hold = f["cbp_hold_days"].replace(np.inf, np.nan)
    ax.plot(f.index, hold, color=COLORS["transferred"], lw=1.8, label="implied CBP holding time (custody ÷ transfers, 7-day)")
    ax.axhline(th.cbp_hold_watch_days, color=LEVEL_COLORS[1], lw=1.2, ls="--", label=f"Watch ≥ {th.cbp_hold_watch_days:g} days")
    ax.axhline(th.cbp_hold_alert_days, color=LEVEL_COLORS[2], lw=1.2, ls="--", label=f"Alert ≥ {th.cbp_hold_alert_days:g} days (72-hour standard)")
    for d, lab, side in ((meta["break_transfers"], "transfer break ", "right"), (meta["regime_start"], " current regime", "left")):
        for a in axes:
            a.axvline(pd.Timestamp(d), color=MUTED, lw=1)
        axes[0].text(pd.Timestamp(d), axes[0].get_ylim()[1] * 0.97, lab, color=INK2, fontsize=9, va="top", ha=side)   # labels on opposite sides
    ax.set_title("(c) Implied CBP holding time (days)", loc="left"); ax.set_ylabel("days"); ax.legend(loc="upper left", frameon=False)
    fig.savefig(path, dpi=110)
    plt.close(fig)



def md(df, floatfmt="{:,.2f}"):
    def cell(v):
        if isinstance(v, pd.Timestamp):
            return f"{v:%d %b %Y}"
        if isinstance(v, (float, np.floating)):
            return "n/a" if not np.isfinite(v) else floatfmt.format(v)
        return str(v)
    head = "| " + " | ".join([df.index.name or ""] + [str(c) for c in df.columns]) + " |"
    rows = ["| " + " | ".join([str(i)] + [cell(v) for v in r]) + " |" for i, r in zip(df.index, df.itertuples(index=False))]
    return "\n".join([head, "|" + "---|" * (len(df.columns) + 1)] + rows)



def main():
    meta = json.loads((ROOT / "artifacts" / "metadata.json").read_text())
    daily = load_daily_clean(ROOT / "artifacts" / "daily_clean.csv")
    rep, th = reported_days(daily), Thresholds()
```

```
    flags = detect_bottlenecks(rep, th)
    out = ROOT / "reports"


    per = periods(meta, rep)
    kpis = kpi_table(rep, per)
    ind = indicator_table(flags, {k: v for k, v in per.items() if k != "2023 (CBP masked)"})
    ind.to_csv(out / "pipeline_indicators_by_period.csv")
    kpis.index = [KPI_DEFINITIONS[k][0] if k in KPI_DEFINITIONS else k for k in kpis.index]
    kpis.index.name = "KPI"
    kpis.to_csv(out / "pipeline_kpis.csv")
    episodes = alert_episodes(flags, min_level=1)
    episodes.to_csv(out / "pipeline_alert_episodes.csv", index=False)
    years = status_by_year(flags)
    years.index.name = "Year"
    years.to_csv(out / "pipeline_status_by_year.csv")
    los = los_reconciliation(daily, rep, meta).set_index("Window")
    los.to_csv(out / "pipeline_los_reconciliation.csv")
    figure(rep, flags, th, meta, ROOT / "figures" / "19_pipeline_monitoring.png")


    alerts = episodes[episodes["Highest level"] == "Alert"].sort_values(["Indicator", "Start"]).reset_index(drop=True)
    alerts.index = alerts.index + 1
    alerts.index.name = "#"
    latest = flags.iloc[-1]
    text = [f"# Pipeline analytics: generated values (do not edit; rerun `python tools/pipeline_analysis.py`)\n",
            f"Data through {rep.index.max():%d %b %Y}; {len(rep)} reported days; default thresholds {th}.\n",
            "## KPIs by period (reported days only)\n", md(kpis), "\n## Indicators by period (reports with a complete window)\n", md(ind),
            "\n## Share of reported days by overall status (%)\n", md(years, "{:.1f}"),
            "\n## Alert-level episodes (full history)\n", md(alerts[["Indicator", "Start", "End", "Reported days", "Calendar days", "Peak value"]]),
            "\n## Implied length of stay: definition reconciliation\n", md(los, "{:,.1f}"),
            f"\n## Latest report ({flags.index[-1]:%d %b %Y})\n",
            f"Overall: {latest['overall_status']}; census growth {latest['census_growth_pct']:+.1f}% ({LEVELS[int(latest['census_growth_level'])]}); "
            f"discharge rate {latest['discharge_drop_pct']:+.1f}% vs baseline ({LEVELS[int(latest['discharge_slowdown_level'])]}); "
            f"CBP holding time {latest['cbp_hold_days']:.2f} days ({LEVELS[int(latest['cbp_holding_level'])]}).\n"]
    (out / "pipeline_analysis.md").write_text("\n".join(text), encoding="utf-8")
    print("\n".join(text))



if __name__ == "__main__":
    main()
```

H.8 Paper figure script: tools/paper_figures.py

```
"""Draw the research paper's main-body figures at their exact print size, so that axis labels and legends stay readable on the page.


    python tools/paper_figures.py            # writes figures/paper/figNN_*.png, or stops if the data no longer match the paper


Why this script exists: the notebook charts are drawn 12-15 inches wide. Placed on a 6.4-inch text column their labels shrink to 4-5 pt.
These versions use the same data at 6.4 inches and 8 pt type. The notebook charts stay in figures/ and go to the Appendix volume.


Every figure first runs check(): the data it is about to plot must reproduce the numbers printed in the paper. A chart can therefore
never silently disagree with the tables beside it.
"""
import json
import sys
from pathlib import Path
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.ticker import FuncFormatter


ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.bottleneck_detection import Thresholds, detect_bottlenecks   # noqa: E402
from src.data_loader import load_daily_clean, reported_days           # noqa: E402


OUT = ROOT / "figures" / "paper"
WIDTH = 6.4                                                           # inches: the paper's text column is 6.57 inches wide
DPI = 300
INK, INK2, MUTED, GRID, BASE = "#1f2328", "#52514e", "#898781", "#e6e5df", "#a9a89f"
FLOW = {"hhs_care": "#2a78d6", "transferred": "#eb6834", "discharged": "#1baf7a"}
MODEL = {"Naive": "#2a78d6", "Moving Avg (7d)": "#eb6834", "SARIMA": "#1baf7a", "ETS": "#eda100", "Gradient Boosting": "#008300", "Ensemble": "#4a3aa7"}
STYLE = {"Naive": (0, (4, 2)), "Moving Avg (7d)": (0, (1, 1)), "SARIMA": "-", "ETS": (0, (5, 1, 1, 1)), "Gradient Boosting": (0, (2, 1)), "Ensemble": "-"}
SCENARIO = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300"]
WATCH, ALERT, GOOD = "#fab219", "#d03b3b", "#0ca30c"
COMMA = FuncFormatter(lambda v, _: f"{v:,.0f}")
DATE = mdates.DateFormatter("%b %Y")

plt.rcParams.update({
    "font.family": "DejaVu Sans", "font.size": 8, "axes.titlesize": 8, "axes.titleweight": "bold", "axes.titlecolor": INK,
    "axes.labelsize": 8, "axes.labelcolor": INK2, "axes.edgecolor": BASE, "axes.linewidth": 0.6, "axes.spines.top": False,
    "axes.spines.right": False, "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.5, "axes.axisbelow": True,
    "xtick.color": INK2, "ytick.color": INK2, "xtick.labelsize": 7.5, "ytick.labelsize": 7.5, "xtick.major.size": 2.5, "ytick.major.size": 0,
    "legend.fontsize": 7.5, "legend.frameon": False, "legend.handlelength": 2.2, "legend.columnspacing": 1.2, "legend.labelcolor": INK,
    "lines.linewidth": 1.2, "figure.facecolor": "white", "axes.facecolor": "white", "savefig.dpi": DPI, "figure.dpi": DPI,
})



def check(name, actual, expected, tol):
    """Stop the build if plotted data do not reproduce a number the paper prints."""
    if abs(float(actual) - float(expected)) > tol:
        raise SystemExit(f"Figure data do not match the paper: {name} = {actual} (paper says {expected}, tolerance {tol})")



def save(fig, name):
    OUT.mkdir(parents=True, exist_ok=True)
    fig.savefig(OUT / f"{name}.png", facecolor="white")
    plt.close(fig)
    print("wrote", (OUT / f"{name}.png").relative_to(ROOT))



def raw_reports():
    """The source file exactly as reported (CBP values unmasked): the stock-flow test needs the suspect values themselves."""
    d = pd.read_csv(ROOT / "HHS_Unaccompanied_Alien_Children_Program.csv", encoding="utf-8-sig").dropna(how="all")
    d.columns = ["date", "apprehended", "cbp_custody", "transferred", "hhs_care", "discharged"]
```

```
    d["date"] = pd.to_datetime(d["date"].astype(str).str.strip(), format="%B %d, %Y")
    for c in d.columns[1:]:
        d[c] = pd.to_numeric(d[c].astype(str).str.replace(",", "", regex=False), errors="coerce")
    d = d.sort_values("date", kind="stable").drop_duplicates("date", keep="first").set_index("date")
    stale = (d["hhs_care"].diff() == 0) & (d["discharged"].diff() == 0)   # the five stale HHS reports are masked before the test, as in the notebook
    d.loc[stale, ["hhs_care", "discharged"]] = np.nan
    return d



def fig1_stock_flow(meta):
    df = raw_reports()
    bad_end = pd.Timestamp(meta["cbp_bad_end"])
    gap = df.index.to_series().diff().dt.days
    d = pd.DataFrame({"change": df["hhs_care"].diff(), "net": df["transferred"] - df["discharged"]})
    d["residual"] = d["change"] - d["net"]
    pairs = d[(gap == 1) & d["residual"].notna()]
    groups = [("Copied CBP block (2023 – Jan 2024)", pairs[pairs.index <= bad_end], 192, 0.51, 190, 1), ("Trusted (Feb 2024 onward)", pairs[pairs.index > bad_end], 357, 0.90, 2.6, 0.05)]
    fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 2.2), constrained_layout=True)
    for ax, (title, g, n, corr, resid, tol) in zip(axes, groups):
        r = g["change"].corr(g["net"])
        check(f"{title} pairs", len(g), n, 0)
        check(f"{title} correlation", r, corr, 0.005)
        check(f"{title} mean residual", g["residual"].mean(), resid, tol)
        ax.scatter(g["net"], g["change"], s=7, color=FLOW["hhs_care"], alpha=0.55, linewidths=0)
        lim = np.nanpercentile(np.abs(np.r_[g["net"], g["change"]]), 99.5) * 1.05
        ax.plot([-lim, lim], [-lim, lim], color=INK2, lw=0.9, label="change = net flow")
        ax.set_xlim(-lim, lim)
        ax.set_ylim(-lim, lim)
        ax.set_title(f"{title}\nr = {r:.2f}, {len(g)} day pairs", loc="left")
        ax.set_xlabel("recorded transfers – discharges (children)")
        ax.set_ylabel("change in care load (children)")
        ax.legend(loc="lower right", handlelength=1.5)
    save(fig, "fig01_stock_flow")



def fig2_history(daily, meta):
    pre = daily.loc[pd.Timestamp(meta["cbp_bad_end"]):pd.Timestamp(meta["break_transfers"]) - pd.Timedelta(days=1)]
    post = daily.loc[pd.Timestamp(meta["regime_start"]):]
    for col, a, b in (("transferred", 209.9, 9.6), ("discharged", 203.2, 11.7), ("hhs_care", 6751, 2262)):
        tol = 0.5 if col == "hhs_care" else 0.05
        check(f"pre-break mean {col}", pre[col].mean(), a, tol)
        check(f"regime mean {col}", post[col].mean(), b, tol)
    fig, axes = plt.subplots(2, 1, figsize=(WIDTH, 2.3), sharex=True, constrained_layout=True, gridspec_kw={"height_ratios": [1, 1]})
    axes[0].plot(daily.index, daily["hhs_care"], color=FLOW["hhs_care"])
    axes[0].set_title("(a) Children in HHS care", loc="left")
    axes[0].set_ylabel("children")
    axes[0].yaxis.set_major_formatter(COMMA)
    axes[0].set_ylim(bottom=0)
    for c, lab in (("transferred", "Transfers into HHS"), ("discharged", "Discharges from HHS")):
        axes[1].plot(daily.index, daily[c].rolling(7, min_periods=3).mean(), color=FLOW[c], label=lab, ls="-" if c == "transferred" else (0, (4, 1.5)))
    axes[1].axvspan(daily.index.min(), pd.Timestamp(meta["cbp_bad_end"]), color="#efeee8", lw=0)
    axes[1].text(pd.Timestamp("2023-07-20"), 30, "transfers masked: copied CBP block", color=INK2, fontsize=7.5, ha="center")
    axes[1].set_title("(b) Daily flows, 7-day mean", loc="left")
    axes[1].set_ylabel("children per day")
    axes[1].set_ylim(bottom=0)
    axes[1].legend(loc="lower right", bbox_to_anchor=(1.0, 0.98), ncol=2, borderaxespad=0, fontsize=7)
```

```
    for ax in axes:
        ax.axvline(pd.Timestamp(meta["break_transfers"]), color=INK2, lw=0.8)
    axes[0].text(pd.Timestamp(meta["break_transfers"]) + pd.Timedelta(days=8), axes[0].get_ylim()[1] * 0.86, "transfer break\n26 Jan 2025", fontsize=7.5, color=INK2, va="top")
    axes[1].xaxis.set_major_formatter(DATE)
    axes[1].xaxis.set_major_locator(mdates.MonthLocator(bymonth=(1, 7)))
    save(fig, "fig02_history")



def fig3_length_of_stay(daily, meta):
    los = daily["hhs_care"].rolling(28).mean() / daily["discharged"].rolling(28).mean()
    fig, ax = plt.subplots(figsize=(WIDTH, 1.6), constrained_layout=True)
    ax.plot(los.index, los, color=FLOW["discharged"])
    brk = pd.Timestamp(meta["break_discharges"])
    ax.axvline(brk, color=INK2, lw=0.8)
    ax.text(brk - pd.Timedelta(days=8), los.max() * 0.92, "discharge break\n17 Feb 2025", ha="right", va="top", color=INK2, fontsize=7.5)
    ax.set_ylabel("days (28-day smoothing)")
    ax.set_ylim(bottom=0)
    ax.xaxis.set_major_formatter(DATE)
    ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=(1, 7)))
    save(fig, "fig03_length_of_stay")



def fig4_design(daily, meta):
    t_end, cal, test, regime = daily.index.max(), pd.Timestamp(meta["cal_start"]), pd.Timestamp(meta["test_start"]), pd.Timestamp(meta["regime_start"])
    ml_start = pd.Timestamp("2024-02-14")                           # first date with a complete feature window after the copied block (Table 7)
    bars = [("HHS history (exploration, surge replay)", daily.index.min(), t_end, "#a9c9f0", INK),
            ("ML training: 14 Feb 2024 to origin", ml_start, t_end, "#5b9be6", INK),
            ("Statistical training: 17 Mar 2025 to origin", regime, t_end, "#2a78d6", INK),
            ("Calibration: 22 Jun – 20 Sep 2025", cal, test, "#eda100", INK),
            ("Test: 21 Sep – 21 Dec 2025", test, t_end, "#eb6834", INK)]
    fig, ax = plt.subplots(figsize=(WIDTH, 1.3), constrained_layout=True)
    for i, (name, a, b, col, _) in enumerate(bars[::-1]):
        ax.barh(i, (b - a).days, left=mdates.date2num(a), height=0.68, color=col, lw=0)
        ax.text(mdates.date2num(a) - 6, i, name, va="center", ha="right", color=INK, fontsize=7.5) if (a > pd.Timestamp("2024-01-01")) else ax.text(mdates.date2num(a) + 6, i, name,
      va="center", color=INK, fontsize=7.5)
    ax.xaxis_date()
    ax.xaxis.set_major_formatter(DATE)
    ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=(1, 7)))
    ax.set_yticks([])
    ax.grid(axis="y", visible=False)
    ax.spines["left"].set_visible(False)
    ax.set_xlim(mdates.date2num(daily.index.min()) - 5, mdates.date2num(t_end) + 8)
    save(fig, "fig04_design")



def scored_errors():
    s = pd.read_csv(ROOT / "artifacts" / "backtest.csv.gz", parse_dates=["origin", "date"])
    return s.assign(error=s["y"] - s["yhat"])



def fig5_horizon(s):
    test = s[s["period"] == "test"]
    by_h = test.groupby(["target", "model", "h"])["error"].apply(lambda e: e.abs().mean()).reset_index(name="MAE")
    care = test[test["target"] == "hhs_care"].assign(week=lambda x: (x["h"] - 1) // 7)
```

```
    paper = {"Naive": (17.9, 44.9, 71.7, 100.5), "SARIMA": (13.4, 28.4, 39.9, 52.0), "ETS": (13.1, 28.0, 38.8, 51.1), "Gradient Boosting": (19.7, 23.0, 32.3, 53.3), "Ensemble": (13.0,
      24.0, 33.7, 48.4)}
    for m, vals in paper.items():
        got = care[care["model"] == m].groupby("week")["error"].apply(lambda e: e.abs().mean())
        for w, v in enumerate(vals):
            check(f"Table 10 {m} week {w + 1}", got[w], v, 0.05)
    panels = [("hhs_care", "(a) Children in care", ["Naive", "SARIMA", "ETS", "Gradient Boosting", "Ensemble"]),
              ("discharged", "(b) Discharges", ["Naive", "SARIMA", "Ensemble"]),
              ("transferred", "(c) Transfers", ["Naive", "Moving Avg (7d)", "Ensemble"])]
    fig, axes = plt.subplots(1, 3, figsize=(WIDTH, 2.2), constrained_layout=True, gridspec_kw={"width_ratios": [1.25, 1, 1]})
    seen = {}
    for ax, (target, title, models) in zip(axes, panels):
        for m in models:
            g = by_h[(by_h["target"] == target) & (by_h["model"] == m)]
            line, = ax.plot(g["h"], g["MAE"], color=MODEL[m], ls=STYLE[m], lw=1.3)
            seen[m] = line
        ax.set_title(title, loc="left")
        ax.set_xticks([1, 7, 14, 21, 28])
        ax.set_xlabel("days ahead")
        ax.set_ylim(bottom=0)
    axes[0].set_ylabel("mean absolute error (children)")
    order = [m for m in MODEL if m in seen]
    fig.legend([seen[m] for m in order], [m.replace("Avg (7d)", "average (7d)") for m in order], loc="outside lower center", ncol=len(order))
    save(fig, "fig05_horizon_error")



def fig6_fan(s, daily, meta):
    cal = s[(s["period"] == "calibration") & (s["target"] == "hhs_care") & (s["model"] == "SARIMA")]
    test = s[(s["period"] == "test") & (s["target"] == "hhs_care") & (s["model"] == "SARIMA")].copy()
    q = {k: {h: cal.loc[(cal["h"] - h).abs() <= 2, "error"].quantile(p) for h in range(1, 29)} for k, p in {"lo95": .025, "lo80": .10, "hi80": .90, "hi95": .975}.items()}
    for k in q:
        test[k] = test["yhat"] + test["h"].map(q[k])
    check("Table 12 SARIMA fixed-calibration coverage %", test["y"].between(test["lo80"], test["hi80"]).mean() * 100, 91.6, 0.05)
    test_start, end = pd.Timestamp(meta["test_start"]), daily.index.max()
    origins = daily.index[(daily.index >= test_start) & (daily.index < end) & daily["is_reported"]][::18][:4]
    hist = daily.loc[test_start - pd.Timedelta(days=21):]
    hist = hist[hist["is_reported"]]["hhs_care"]
    fig, ax = plt.subplots(figsize=(WIDTH, 2.25), constrained_layout=True)
    col = MODEL["SARIMA"]
    for o in origins:
        g = test[test["origin"] == o].sort_values("date")
        ax.fill_between(g["date"], g["lo95"], g["hi95"], color=col, alpha=0.13, lw=0, label="95% interval" if o == origins[0] else None)
        ax.fill_between(g["date"], g["lo80"], g["hi80"], color=col, alpha=0.26, lw=0, label="80% interval" if o == origins[0] else None)
        ax.plot(g["date"], g["yhat"], color=col, lw=1.3, label="SARIMA forecast" if o == origins[0] else None)
        ax.scatter([o], [daily.at[o, "hhs_care"]], s=13, color=col, edgecolor="white", linewidth=0.6, zorder=4, label="forecast origin" if o == origins[0] else None)
    ax.plot(hist.index, hist, color=INK, lw=1.1, label="actual care load", zorder=3)
    ax.yaxis.set_major_formatter(COMMA)
    ax.set_ylabel("children in HHS care")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
    ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=mdates.SU, interval=2))
    ax.legend(loc="upper left", ncol=3)
    save(fig, "fig06_fan_chart")



def fig7_replay():
    paths = pd.read_csv(ROOT / "artifacts" / "surge_replay_paths.csv", parse_dates=["origin"], encoding="utf-8").set_index("origin")
```

```
    lead = pd.read_csv(ROOT / "artifacts" / "surge_lead_times.csv", parse_dates=["event_date"], encoding="utf-8")
    lead = lead[lead["source"].str.startswith("replay")]
    check("replay surge events", len(lead), 25, 0)
    check("replay events detected", lead["detected"].sum(), 25, 0)
    check("replay median lead days", lead["lead_days"].median(), 14, 0)
    fig, ax = plt.subplots(figsize=(WIDTH, 2.25), constrained_layout=True)
    ax.plot(paths.index, paths["peak"], color=MODEL["ETS"], lw=1.1, label="28-day alert level (P(breach) ≥ 30%)")
    ax.plot(paths.index, paths["actual"], color=INK, lw=1.2, label="actual care load")
    ax.scatter(lead["event_date"], lead["threshold"], s=15, marker="o", color=GOOD, edgecolor="white", linewidth=0.5, zorder=4, label="surge event: warned in advance")
    ax.set_ylim(top=14600)
    top2 = lead[lead["event_date"].dt.year == 2023].nlargest(2, "threshold").sort_values("event_date")
    for r, side in zip(top2.itertuples(), ("right", "left")):
        ax.annotate(f"{r.lead_days}-day warning", (r.event_date, r.threshold), xytext=(r.event_date, 14050), ha=side, va="center", fontsize=7.5, color=INK2,
                    arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.6, shrinkB=2))
    ax.yaxis.set_major_formatter(COMMA)
    ax.set_ylabel("children in HHS care")
    ax.xaxis.set_major_formatter(DATE)
    ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=(1, 4, 7, 10)))
    fig.legend(*ax.get_legend_handles_labels(), loc="outside upper center", ncol=3, fontsize=7.5)
    save(fig, "fig07_surge_replay")



def run_scenario(care, flow_in, flow_out, in_mult=1.0, out_mult=1.0, start_day=1, extra_in=0.0):
    """The stock-flow identity, as in the notebook and the dashboard."""
    days = np.arange(1, len(care) + 1)
    active = (days >= start_day).astype(float)
    return care + np.cumsum(((flow_in * (in_mult - 1) + extra_in) - flow_out * (out_mult - 1)) * active)



def fig8_scenarios(meta):
    fc = pd.read_csv(ROOT / "artifacts" / "forecast_future.csv", parse_dates=["date"])
    pick = lambda target, model: fc[(fc["target"] == target) & (fc["model"] == model)].sort_values("h")
    care = pick("hhs_care", "Ensemble")
    dates, base = care["date"].to_numpy(), care["yhat"].to_numpy()
    fin = pick("transferred", "Ensemble")["yhat"].to_numpy()
    fout = pick("discharged", "Ensemble")["yhat"].to_numpy()
    pre = meta["pre_break_transfers_per_day"]
    spec = [("Baseline (current trends)", {}, (2533, 2571)),
            ("Discharge slowdown (−30%)", {"out_mult": .7}, (2586, 2677)),
            ("Placement acceleration (+50%)", {"out_mult": 1.5}, (2444, 2393)),
            ("Intake surge (3× from day 8)", {"in_mult": 3.0, "start_day": 8}, (2650, 2917)),
            ("Combined stress (2× in, −25% out)", {"in_mult": 2.0, "out_mult": .75}, (2695, 2892)),
            (f"Return to 2024 intake (~{pre:.0f}/day from day 8)", {"extra_in": pre - fin.mean(), "start_day": 8}, (3913, 6710))]
    fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 2.5), constrained_layout=True, gridspec_kw={"width_ratios": [1.5, 1]})
    handles = {}
    for (name, kw, (d14, d28)), colr in zip(spec, SCENARIO):
        path = run_scenario(base, fin, fout, **kw)
        check(f"Table 17 {name} day 14", path[13], d14, 0.5)
        check(f"Table 17 {name} day 28", path[27], d28, 0.5)
        ax = axes[1] if name.startswith("Return") else axes[0]
        handles[name], = ax.plot(dates, path, color=colr, lw=1.4, ls="-" if name.startswith(("Baseline", "Return")) else (0, (4, 1.2)) if name.startswith(("Discharge", "Combined")) else
      (0, (1.2, 1)))
    axes[1].plot(dates, base, color=SCENARIO[0], lw=1.4)
    for ax in axes:
        ax.axhline(2650, color=INK2, lw=0.8, ls=(0, (1, 1.5)))
        ax.yaxis.set_major_formatter(COMMA)
```

```
        ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=mdates.SU, interval=2))
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
    axes[0].text(dates[0], 2650, "threshold 2,650", color=INK2, fontsize=7.5, va="bottom")
    axes[1].text(dates[-1], 2690, "threshold 2,650", color=INK2, fontsize=7.5, va="bottom", ha="right")
    axes[0].set_title("(a) Operational scenarios", loc="left")
    axes[1].set_title("(b) Return to 2024 intake", loc="left")
    axes[0].set_ylabel("children in HHS care")
    fig.legend(list(handles.values()), list(handles.keys()), loc="outside lower center", ncol=2)
    save(fig, "fig08_scenarios")



def fig9_pipeline(daily, meta):
    rep, th = reported_days(daily), Thresholds()
    flags = detect_bottlenecks(rep, th)
    last = flags.iloc[-1]
    check("CBP holding time at last report", last["cbp_hold_days"], 3.75, 0.005)
    check("census growth at last report", last["census_growth_pct"], 4.6, 0.05)
    start = pd.Timestamp(meta["cbp_bad_end"]) + pd.Timedelta(days=1)
    r, f = rep.loc[start:], flags.loc[start:]
    fig, axes = plt.subplots(3, 1, figsize=(WIDTH, 3.95), sharex=True, constrained_layout=True)


    def shade(ax, col, label=""):
        for lv, colr, name in ((1, WATCH, "Watch"), (2, ALERT, "Alert")):
            ax.fill_between(f.index, 0, 1, where=f[col] == lv, transform=ax.get_xaxis_transform(), color=colr, alpha=0.22, lw=0, label=f"{label}: {name}" if label else name)


    ax = axes[0]
    ax.plot(r.index, r["hhs_care"], color=FLOW["hhs_care"], label="care load")
    shade(ax, "census_growth_level", "")
    ax.set_title("(a) Children in HHS care", loc="left")
    ax.set_ylabel("children")
    ax.yaxis.set_major_formatter(COMMA)
    ax.set_ylim(bottom=0)
    ax = axes[1]
    ax.plot(f.index, f["discharge_rate_7d_pct"], color=FLOW["discharged"], label="7-day rate")
    ax.plot(f.index, f["discharge_baseline_pct"], color=INK2, lw=1, ls=(0, (1, 1)), label="baseline (prior 8 weeks)")
    shade(ax, "discharge_slowdown_level", "")
    ax.set_title("(b) HHS discharge rate", loc="left")
    ax.set_ylabel("% of census per day")
    ax.set_ylim(bottom=0)
    ax = axes[2]
    ax.plot(f.index, f["cbp_hold_days"].replace(np.inf, np.nan), color=FLOW["transferred"], label="custody ÷ transfers (7-day)")
    ax.axhline(th.cbp_hold_watch_days, color="#b77f00", lw=0.9, ls=(0, (4, 2)), label="Watch ≥ 2 days")
    ax.axhline(th.cbp_hold_alert_days, color=ALERT, lw=0.9, ls=(0, (4, 2)), label="Alert ≥ 3 days")
    ax.set_title("(c) Implied CBP holding time", loc="left")
    ax.set_ylabel("days")
    ax.set_ylim(bottom=0)
    for a in axes:
        a.axvline(pd.Timestamp(meta["break_transfers"]), color=INK2, lw=0.7)
        a.axvline(pd.Timestamp(meta["regime_start"]), color=INK2, lw=0.7, ls=(0, (2, 1)))
        a.legend(loc="lower right", bbox_to_anchor=(1.0, 0.98), ncol=4, borderaxespad=0, handlelength=1.8, columnspacing=1.0, fontsize=7)
    axes[0].text(pd.Timestamp(meta["break_transfers"]) - pd.Timedelta(days=6), 1000, "transfer\nbreak", ha="right", va="center", color=INK2, fontsize=7.5)
    axes[0].text(pd.Timestamp(meta["regime_start"]) + pd.Timedelta(days=6), 5400, "current\nregime", ha="left", va="center", color=INK2, fontsize=7.5)
    axes[2].xaxis.set_major_formatter(DATE)
    axes[2].xaxis.set_major_locator(mdates.MonthLocator(bymonth=(1, 4, 7, 10)))
    save(fig, "fig09_pipeline")
```

```
def main():
    meta = json.loads((ROOT / "artifacts" / "metadata.json").read_text(encoding="utf-8"))
    daily = load_daily_clean(ROOT / "artifacts" / "daily_clean.csv")
    s = scored_errors()
    fig1_stock_flow(meta)
    fig2_history(daily, meta)
    fig3_length_of_stay(daily, meta)
    fig4_design(daily, meta)
    fig5_horizon(s)
    fig6_fan(s, daily, meta)
    fig7_replay()
    fig8_scenarios(meta)
    fig9_pipeline(daily, meta)



if __name__ == "__main__":
    main()
```